



Introduzione alla sicurezza Informatica

Docente Stefano Bistarelli

-
-

DMI - Informatica

Introduzione alla Sicurezza Informatica

CAPITOLO 1

CONCETTI FONDAMENTALI

Sicurezza informatica: protezione offerta a un sistema informatico automatizzato al fine di raggiungere gli obiettivi di preservare l'integrità, la disponibilità e la confidenzialità delle risorse del sistema

Confidenzialità: capacità di garantire che informazioni private o confidenziali non vengano divulgate a persone non autorizzate a trattarle. Ogni individuo deve poter controllare o influenzare ogni propria informazione che deve essere memorizzata e decidere chi può gestirla

Integrità: si riferisce alla capacità di un sistema di garantire che l'informazione non venga alterata da persone non autorizzate e seguendo degli specifici protocolli

Accessibilità: assicura che i sistemi contenenti informazioni lavorino propriamente garantendo una continuità del servizio, in modo tale da rendere l'informazione disponibile a chi di dovere

Una perdita di Confidenzialità è intesa come la divulgazione non autorizzata di informazioni

Una perdita di Integrità è una modifica o una cancellazione dell'informazione non autorizzata

Una perdita di Disponibilità è l'interruzione dell'accesso o dell'utilizzo delle informazioni di un sistema informativo

A questi obiettivi se ne aggiungono altri 4 addizionali: **IAAA**

Identificazione: fase in cui si dimostra la propria identità associando l'identità al login (utente); questa fase viene eseguita una sola volta;

Autenticazione: fase in cui mi assicuro tramite il login che effettivamente l'utente è chi dice di essere; questa fase viene eseguita ogni volta.

Autorizzazione: dopo aver effettuato l'autenticazione con successo permetto di fare determinate attività.

Accountability: condizione che si raggiunge quando si è in grado di monitorare, garantire e dimostrare le attività di un account/utente. (si utilizzano, per fare questo, i file di Log)

Vengono usati tre diversi livelli di impatto sulle organizzazioni o sugli individui in caso di violazione della sicurezza (breach of security):

- 
- **Low:** effetto negativo limitato sulle operazioni dell'organizzazione, sui bene sugli individui
 - **Moderate:** grave effetto negativo
 - **High:** la perdita possa aver un effetto grave negativo sull'organizzazione o sugli individui

ARCHITETTURA OSI

Un modo sistematico che aiuta il manager responsabile della sicurezza dei computer e delle reti a valutare le esigenze di sicurezza e scegliere i vari prodotti e politiche di sicurezza, è la raccomandazione **X.800** (architettura di sicurezza per OSI). Essa si concentra sugli **attacchi**, i **meccanismi** e i **servizi**. Definiti brevemente come:

- **Meccanismi di sicurezza:** un processo che permette di rilevare, prevenire o fare recovering in caso di attacco alla sicurezza
- **Servizi di sicurezza:** hanno lo scopo di contrastare gli attacchi alla sicurezza avvalendosi di uno o più meccanismi di sicurezza per fornire il servizio
- **Attacchi alla sicurezza:** qualsiasi azione che compromette la sicurezza delle informazioni di un'organizzazione

ATTACCHI ALLA SICUREZZA

ATTACCHI PASSIVI: tenta di apprendere o utilizzare informazioni senza influire sulle risorse del sistema. Consistono nell'intercettazione o nel monitoraggio delle trasmissioni con l'obiettivo di ottenere le informazioni trasmesse. Due tipi:

- **Recupero del contenuto dei messaggi:** Una conversazione telefonica, una mail o il trasferimento di un file possono contenere informazioni sensibili o confidenziali. Vorremmo prevenire l'acquisizione di tali informazioni da parte di un malintenzionato
- **Analisi del traffico:** Anche avendo a disposizione tecniche come la crittografia per mascherare il contenuto dei messaggi, i malintenzionati potrebbero venire a conoscenza di informazioni sulla lunghezza del messaggio e su chi sono i protagonisti, risalendo in modo completamente nascosto alla natura della comunicazione in corso e alla sua struttura. Ottenendo potenziali informazioni rilevanti.

Gli attacchi passivi sono molto difficili da rilevare perché non comportano alcuna alterazione dei dati. Né il mittente né il destinatario sono a conoscenza del fatto che ci sia una terza persona che legga i messaggi o che osservi lo schema del traffico. Tuttavia, è possibile impedire il

successo di questi attacchi, di solito attraverso la **crittografia**. L'enfasi di questi tipi di attacchi è sulla **prevenzione** piuttosto che sulla rilevazione.

ATTACCHI ATTIVI: tentano di alterare le risorse di sistema o di influire sul loro funzionamento; comportano modifiche del flusso di dati o la creazione di un flusso falso; divisi in 4 categorie:

- **masquerade:** hanno luogo quando un'entità finge di essere un'entità diversa con privilegi aggiuntivi e superiori.
- **replay:** comporta l'acquisizione passiva di un'unità di dati e la sua ritrasmissione per produrre un effetto non autorizzato.
- **la modifica dei messaggi:** una parte del messaggio legittimo viene alterata, oppure che i messaggi vengano ritardati o riordinati per produrre un effetto non autorizzato.
- **denial of service:** impedisce o inibisce il normale utilizzo o la gestione di strutture di comunicazione (es. interruzione di un'intera rete). Esso può avere un obiettivo specifico; ad esempio, un'unità può sopprimere tutti i messaggi diretti a una particolare destinazione.

Gli attacchi passivi presentano **caratteristiche opposte a quelli attivi**. Mentre gli attacchi passivi sono difficili da rilevare tramite misure di prevenzione, gli **attivi sono molto più complessi da prevenire** a causa dell'ampia varietà di potenziale vulnerabilità fisiche, software e di rete. L'obiettivo è invece quello di **rilevare** gli attacchi attivi e di recuperare le interruzioni o ritardi da essi causati.

SERVIZI DI SICUREZZA

Un servizio di sicurezza è un servizio di elaborazione o comunicazione offerto da un sistema per fornire un tipo specifico di protezione alle risorse del sistema stesso. I servizi di sicurezza implementano le politiche di sicurezza e sono implementati dai meccanismi di sicurezza.

X.800 suddivide questi servizi in **5** categorie e **14** servizi specifici:

1. **Autenticazione:** si occupa di garantire che l'entità comunicante è quella che afferma di essere. (Nel caso di un singolo messaggio, ad esempio un allarme, la sua funzione è quella di assicurare al destinatario che l'informazione provenga dalla fonte dichiarata. Nel caso di un'interazione continua, si assicura, in primo luogo, che le due entità siano autentiche, in secondo luogo, che la comunicazione non subisca interferenze tali da consentire a una terza persona di mascherarsi). Due servizi:
 - **Peer entity autenticazione:** fornisce la conferma dell'identità di un'entità peer in un'associazione (evita masquerade)
 - **Autenticazione dell'origine dei dati:** fornisce la conferma dell'origine di un'unità dei dati

2. **Controllo degli accessi:** capacità di limitare e controllare l'accesso ai sistemi e alle applicazioni host attraverso i collegamenti di comunicazione; vale a dire che questo servizio controlla chi può avere accesso a una risorsa. Ogni entità che cerca di accedere deve essere prima identificata, o autenticata, in modo che i diritti di accesso possano essere adatti all'individuo.

3. **Confidenzialità dei dati - Riservatezza:** protezione dei dati trasmessi da attacchi passivi; ovvero la protezione da divulgazione non autorizzata. 4 sottocategorie:
 - connection confidentiality → protezione dati in una connessione

 - connectionless confidentiality → protezione dati in un blocco di dati

 - selective-field confidentiality → protezione di alcuni dati selezionati su una connessione o su un blocco di dati

 - traffic-flow confidentiality → protezione da analisi del traffico.

4. **Integrità dei dati:** servizio che assicura che i dati ricevuti siano esattamente come inviati da un'entità autorizzata; ovvero non contengono alterazioni, cancellazioni, riordini o riproduzioni. 5 sottocategorie:
 - integrità senza connessione → fornisce l'integrità di un singolo blocco di dati senza connessione e assume la forma di 'rilevamento di modifica di dati'
 - integrità senza connessione a campo selettivo → stessa cosa di prima ma solo su un insieme di dati selezionati
 - integrità della connessione con ripristino → rileva qualsiasi tipo di modifica di dati su una connessione e tenta il ripristino
 - integrità della connessione senza ripristino → come prima ma senza ripristino
 - integrità della connessione a campo selettivo → rileva modifiche solo su un insieme di dati selezionati su una connessione.

5. **Non ripudio:** servizio che fornisce protezione contro il **RIFIUTO**, da parte di una delle entità coinvolte in una comunicazione, di aver partecipato a tutta o parte della comunicazione; ovvero impedisce a destinatario o mittente di negare l'invio di un messaggio trasmesso. (NON RIPUDIO ORIGINE e NON RIPUDIO DESTINAZIONE)

6. **Servizio di disponibilità:** una serie di attacchi può causare la perdita o la riduzione della disponibilità. Alcuni di questi attacchi sono suscettibili di contromisure automatiche, altri necessitano di azioni fisiche. Un servizio di disponibilità affronta i problemi di sicurezza sollevati dagli attacchi denial-of-service. Dipende dalla corretta gestione e dal controllo delle risorse del sistema e quindi dal servizio di controllo degli accessi e da altri servizi di sicurezza.

MECCANISMI DI SICUREZZA

Sono suddivisi in:

1 MECCANISMI DI SICUREZZA SPECIFICI

2 MECCANISMI DI SICUREZZA PERVASIVI

1 sono quelli che possono essere incorporati nel livello appropriato di protocollo :

- Cifratura: uso di algoritmi matematici per trasformare i dati in una forma non facilmente leggibile
- Firma digitale: dati aggiunti o trasformazioni crittografiche di un'unità di dati che consentono al ricevente di provare l'origine e l'integrità proteggendolo da falsificazioni
- Controllo degli accessi: impone diritti di accesso a risorse
- Integrità dei dati: meccanismi utilizzati per garantire l'integrità
- Scambio di autenticazione: certifica identità tramite scambio di informazioni
- Traffic padding (imbottitura): Inserimento di bit negli spazi vuoti di un flusso di dati per vanificare i tentativi di analisi del traffico
- Controllo dell'instradamento: selezione di percorsi particolarmente sicuri per i dati e permette di modificare il percorso
- Autenticazione notarile: una terza parte garantisce correttezza nello scambio di dati.

2 non sono specifici di un particolare servizio di sicurezza OSI o di un livello di protocollo:

- Funzioni di affidabilità: ciò che viene percepito come corretto con rispetto di alcuni criteri
- Etichette di sicurezza: marcatura legata a una risorsa che nomina o designa gli attributi di sicurezza di tale risorsa
- Event Detection: rilevamento di eventi rilevanti per la sicurezza
- Traccia di controllo della sicurezza: dati raccolti e potenzialmente utilizzati per facilitare un audit di sicurezza, ovvero una revisione o un esame indipendente dei registri e delle attività del sistemico
- Security Recovery: gestisce le richieste dei meccanismi, come le funzioni di gestione degli eventi, e intraprende azioni di recupero

Per servizio si intende: **ciò che deve essere garantito**

Per meccanismo: **mezzo con il quale si garantisce il servizio**

SERVIZI

Autenticazione dell'entità peer

Autenticazione dell'origine dei dati

Controllo degli accessi

Confidenzialità

Confidenzialità del flusso di traffico

Data integrity

Non ripudio

Disponibilità

MECCANISMI

Cifratura, firma digitale, scambio di autenticazione

Cifratura, firma digitale

Controllo degli accessi

Cifratura, controllo dell'instradamento

Cifratura, traffic padding, controllo instradamento

Cifratura, firma digitale, integrità dei dati

Firma digitale, integrità dei dati, notarizzazione

Integrità dei dati, scambio di autenticazione

UN MODELLO PER LA SICUREZZA DELLA RETE

Un messaggio deve essere trasmesso da una parte ad un'altra attraverso una sorta di servizio internet. Le due parti, chiamate committenti (**principals**), devono cooperare affinché lo scambio avvenga. Viene stabilito un canale logico di informazione definendo un percorso attraverso internet dalla sorgente alla destinazione e tramite l'uso di protocolli come tcp/ip da parte dei due committenti.

Mittente → Security related transformation → Info segreta → messaggio sicuro → **INFORMATION CHANNEL** → messaggio sicuro → Info Segreta → Security related transformation → Destinatario

Si ha la necessità di proteggere le informazioni trasmesse da un eventuale opponent che potrebbe risultare una minaccia per confidenzialità, integrità e autenticità.

Tutte le tecniche per fornire sicurezza hanno due componenti:

1. Security-related transformation dell'informazione da inviare, include crittografia del messaggio che lo rende incomprensibile e l'aggiunta di codice basato sul contenuto del messaggio per verificare l'identità del Mittente.
2. Aggiunta di alcune informazioni segrete che devono essere condivise tra le due parti e sconosciute agli altri; esempio Chiavi crittografiche condivise.

Potrebbe essere necessaria una 3' parte per la distribuzione delle informazioni segrete tra le due parti, o per gestire le dispute relative all'autenticità del messaggio trasmesso.

In generale ci sono 4 attività di base nella progettazione di un particolare servizio di sicurezza:

1. progettare un algoritmo per eseguire la security-related transformation
2. generare le informazioni segrete da usare con l'algoritmo
3. sviluppare metodi per distribuire le informazioni segrete
4. specificare un protocollo usato da entrambe le parti che fa uso dell'algoritmo di sicurezza e delle informazioni segrete per raggiungere un particolare livello di sicurezza

CAPITOLO 3.6

FIRME DIGITALI

Cifratura Simmetrica: è più veloce rispetto alla asimmetrica, le parti devono scambiarsi la chiave simmetrica utilizzata per cifrare i dati prima di poter scambiare i dati stessi.

Cifratura Asimmetrica: in questo caso ognuno ha due coppie di chiavi, una pubblica e una privata, il mittente cifra il messaggio con la chiave pubblica del destinatario e il destinatario decifra il messaggio con la sua chiave privata che solo lui conosce e possiede.

La **firma digitale** è una trasformazione crittografica su un insieme di dati, che risulta essere un meccanismo per verificare l'autenticità, l'integrità e il non ripudio del firmatario. È un modello di bit dipendente dai dati generato da un agente in funzione di un file, un messaggio o un blocco di dati. Un altro agente può accedere al blocco e verificare che 1) esso sia stato firmato dal presunto firmatario (autenticità) 2) sia integro, cioè non abbia subito modifiche da parte di persone non autorizzate. Inoltre, 3) il firmatario non può ripudiare la firma.

Può essere utilizzato uno di questi 3 algoritmi di firma digitale:

- **DSA:** algoritmo originale approvato dal NIST, si basa sui logaritmi discreti
- **RSA:** basato sull'algoritmo di chiave pubblica

- **ECDSA**: crittografia a curva ellittica

GENERAZIONE E VERIFICA DELLA FIRMA DIGITALE

Per ottenere la **FIRMA DIGITALE** dobbiamo generare due chiavi una **PUBBLICA** e una **PRIVATA** in maniera autonoma, dopodichè dobbiamo recarci da un ente certificatore con la chiave pubblica, il documento di identità e ci facciamo fare il certificato.

Certificato = atto che attesta che la nostra identità è associata a quella chiave pubblica, infatti il certificato al suo interno riporta: nome e firma dell'ente certificatore, nome e cognome nostro e chiave pubblica nostra.

Ente certificatore = è colui che può emettere il certificato.

Il **mittente** invia:

1. il messaggio in chiaro;
2. l'hash del messaggio cifrato con la sua chiave privata;
3. il certificato;
4. il certificato cifrato con la chiave pubblica dell'ente certificatore.

Il **destinatario** esegue **2** controlli:

1. prende il messaggio in chiaro e gli applica l'hash, poi prende l'hash cifrato con la chiave privata del mittente e lo decifra con la chiave pubblica del mittente. Se i due valori hash sono uguali allora il messaggio non ha subito alterazioni.
2. deve accertarsi che il certificato sia valido quindi va sul sito dell'ente certificatore e cerca, tramite l'identità del mittente, il certificato a lui associato e la chiave pubblica dell'ente certificatore; a questo punto applica la chiave pubblica dell'ente certificatore al certificato e verifica che sia uguale al certificato cifrato con la chiave pubblica dell'ente certificatore inviato dal mittente.

Effettuando questi due controlli si garantisce **l'INTEGRITA'** e **AUTENTICITA'**, ma non **CONFIDENZIALITA'**.

Per garantire la **CONFIDENZIALITA'** dobbiamo:

1. cifrare il messaggio con una chiave simmetrica e mandarlo cifrato
2. inviare la chiave simmetrica cifrata con la chiave pubblica del destinatario.
3. il destinatario con la sua chiave privata decifra la chiave simmetrica e dopodiché con la chiave simmetrica decifra il messaggio.

RSA (da rivedere)

L'essenza dell'algoritmo di firma digitale RSA consiste nel criptare l'hash del messaggio da firmare utilizzando RSA. Esistono diversi approcci uno dei quali è lo schema di firma probabilistica RSA-PSS. Processo di generazione della firma RSA-PSS:

1. Per prima cosa viene generato un hash value, o message digest, mHash dal messaggio M da firmare
2. Viene generato un blocco chiamato M' unendo mHash con una costante padding1 e un numero pseudo randomico 'salt'
3. Si applica la funzione hash a M' generando il valore H
4. Viene generato un blocco chiamato DB costituito da una costante padding2 e un altro 'salt'
5. Utilizzare una funzione di generazione della maschera MGF, che produce un output randomizzato della stessa lunghezza di DB da H
6. Creare l'encoded message EM block concatenando H con lo XOR tra l'output della MGF e DB, e con una costante esadecimale BC
7. Crittografare EM con la chiave privata del firmatario

Poiché il 'salt' cambia ad ogni utilizzo, firmando due volte lo stesso messaggio con la stessa chiave privata, si otterranno due firme diverse.

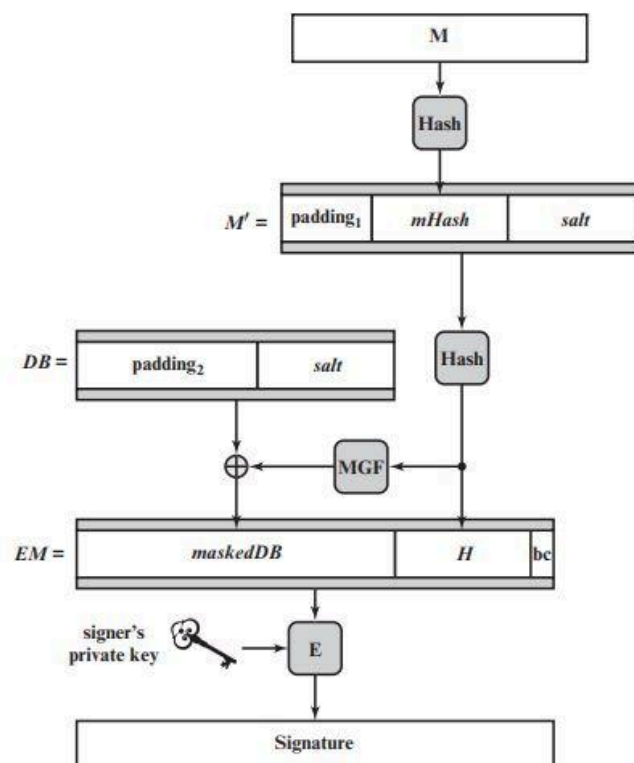


Figure 3.16 RSA-PSS Encoding and Signature Generation

CAPITOLO 4

PRINCIPI DI AUTENTICAZIONE DELL'UTENTE REMOTO

Nella maggior parte dei contesti di sicurezza informatica, l'**autenticazione utente** è l'elemento fondamentale e la principale linea di difesa. **RFC 4949** definisce l'autenticazione come il processo di verifica di un'identità rivendicata da o per un'entità di sistema. Questo processo si compone di due parti:

1. **Fase di Identificazione**: presentazione di un identificatore (identifier ID) al sistema di sicurezza;
2. **Fase di Verifica**: presentazione o generazione di informazioni di autenticazione che confermano l'associazione tra l'entità e l'identificatore.

Ad esempio, **Alice Toklas** potrebbe avere l'ID utente **ABTOKLAS**.

Queste informazioni devono essere memorizzate su qualsiasi server o sistema informatico che Alice desideri utilizzare e potrebbero essere note agli amministratori di sistema e ad altri utenti. Un elemento tipico di informazioni di autenticazione associate a questo ID utente è una **password**, che deve essere tenuta segreta. Se nessuno è in grado di indovinare la password di Alice, la **combinazione ID-password** consente agli amministratori di configurare le autorizzazioni di accesso di Alice e controllare la sua attività. Poiché la password è segreta, nessuno può fingersi Alice.

In sostanza, l'**identificazione** è il mezzo con cui un utente fornisce al sistema un'identità; la **verifica** è il mezzo per stabilire la validità dell'affermazione.

MEZZI DI AUTENTICAZIONE

Ci sono quattro metodi generali per autenticare l'identità di un utente, che possono essere utilizzati da soli o in combinazione:

- **Something the individual knows**: può essere una *password*, un *PIN* o risposte a una serie di domande prestabilite;
- **Something the individual possesses**: chiavi crittografiche, chiavi fisiche, smart card;
- **Something the individual is (biometria statica)**: impronta digitale, retina e volto;
- **Something the individual does (biometria dinamica)**: pattern vocale, caratteristiche della scrittura a mano, ritmo di battitura.

Tutti questi metodi, opportunamente implementati e utilizzati, possono fornire un'autenticazione sicura dell'utente; ma ognuno può presentare particolari complicazioni. Per l'autenticazione utente basata sulla rete, i metodi più importanti coinvolgono chiavi crittografiche e qualcosa che

l'individuo conosce, come una password.

DISTRIBUZIONE DI UNA CHIAVE SIMMETRICA UTILIZZANDO LA CIFRATURA SIMMETRICA

Affinché la crittografia simmetrica funzioni, le due parti di uno scambio devono condividere la stessa chiave e tale chiave deve essere protetta dall'accesso da parte di altri. Pertanto, la forza di qualsiasi sistema crittografico risiede nella "*tecnica di distribuzione delle chiavi*", termine che si riferisce ai mezzi per consegnare una chiave a due parti che desiderano scambiare dati, senza consentire ad altri di vedere la chiave. Essa può essere ottenuta in diversi modi per due parti A e B:

1. Una chiave potrebbe essere scelta da A e **consegnata fisicamente** a B
2. Una **terza parte potrebbe selezionare la chiave e consegnarla fisicamente** ad A e B
3. Se A e B hanno usato di recente una chiave, **una delle due parti potrebbe trasmettere la nuova chiave all'altra, utilizzando la vecchia come chiave per crittografare la nuova chiave**
4. Se A e B hanno ciascuno una connessione crittografata a una terza parte C, **essa potrebbe fornire una chiave sui collegamenti crittografati** ad A e B

Opzione 1 e 2: per la crittografia a collegamento diretto (link encryption), solo fra due estremità, le opzioni di scambio fisico sono ragionevoli. Tuttavia, in un sistema distribuito (end to end encryption), in cui sono presenti diversi host e ogni dispositivo necessita di un numero di chiavi fornite dinamicamente, è molto impraticabile.

Opzione 3: è una possibilità sia per la link encryption che per la end to end encryption, ma se un malintenzionato riesce ad ottenere l'accesso ad una chiave, vengono rivelate tutte le chiavi successive.

Opzione 4: vengono utilizzati due tipi di chiavi:

1. chiave di sessione: quando due sistemi terminali desiderano comunicare, stabiliscono una connessione logica (un circuito virtuale). Per la durata di quella connessione logica, chiamata **sessione**, tutti i dati utente vengono crittografati con una chiave di sessione monouso. Al termine della sessione la chiave viene distrutta.
2. chiave permanente: una chiave permanente è una chiave utilizzata tra entità per scambiare chiavi di sessione.

Un elemento fondamentale per l'**opzione 4** è il centro di distribuzione chiavi (**KDC**). Il **KDC** determina quali sistemi sono autorizzati a comunicare tra loro e fornisce ad essi una chiave di sessione monouso per quella connessione. In generale il funzionamento di KDC procede così:

1. Quando un host A desidera stabilire una connessione con B, trasmette una richiesta al KDC. La connessione tra A e il KDC è crittografata usando la chiave permanente condivisa solo tra A e KDC.
2. Se il KDC approva la richiesta di connessione, genera una chiave di sessione monouso che trasmette, cifrata con la rispettiva chiave permanente, ad A e B.
3. A e B adesso possono stabilire una connessione logica crittografata usando la chiave di sessione monouso che hanno ricevuto dal KDC.

L'applicazione più utilizzata che implementa questo approccio è **Kerberos**.

KERBEROS

Kerberos è in servizio di distribuzione di chiavi e di autenticazione utente. Il problema che affronta è questo: supponiamo un ambiente aperto e distribuito in cui gli utenti delle workstation desiderano accedere ai servizi offerti dai server distribuiti in tutta la rete. Vorremmo che i server fossero in grado di limitare l'accesso agli utenti autorizzati e di poter autenticare le richieste di servizio.

Un utente però potrebbe accedere ad una postazione e fingersi un altro, potrebbe modificare l'indirizzo IP di una workstation così da impersonificarsi qualcun'altro oppure potrebbe intercettare scambi e fare replay così da ottenere l'accesso o interrompere il servizio; è facile comprendere che un utente non autorizzato può essere in grado di accedere a servizi al quale non è autorizzato ad accedere.

Kerberos fornisce un server di autenticazione centralizzato la cui funzione è autenticare gli utenti sui server e i server sugli utenti. Si basa esclusivamente sulla crittografia simmetrica, senza utilizzare la crittografia a chiave pubblica.

KERBEROS V4

UN SEMPLICE DIALOGO DI AUTENTICAZIONE

In un ambiente di rete non protetto, qualsiasi client può richiedere un servizio a qualsiasi server. Un avversario può fingersi un altro client ed ottenere privilegi non autorizzati sulle macchine server. Per contrastare questa minaccia, i server devono essere in grado di confermare l'identità dei client che richiedono il servizio. Un modo potrebbe essere quello di richiedere ad ogni server di svolgere questa attività per ogni client, ma in un ambiente aperto questo pone un onere sostanziale su ciascun server. Un'alternativa è usare un **server di autenticazione (AS)**, che conosca le password di tutti gli utenti e le memorizzi in un database centralizzato. Inoltre, l'AS condivide una chiave segreta univoca con ciascun server.

Esempio dialogo tra un client C e un server V tramite un AS:

1. $C \rightarrow AS: ID_c || P_c || ID_v$
 2. $AS \rightarrow C: Ticket$
 3. $C \rightarrow V: ID_c || Ticket$
- $$Ticket = E(K_v, [ID_c || AD_c || ID_v])$$

Dove:

$C \rightarrow$ client

$AS \rightarrow$ server di autenticazione

$V \rightarrow$ server

$ID_c \rightarrow$ Identificatore ID di C

$ID_v \rightarrow$ Identificatore ID di V

$P_c \rightarrow$ password utente C

$AD_c \rightarrow$ indirizzo di rete di C

$K_v \rightarrow$ chiave crittografata segreta condivisa da AS e V

In questo scenario cosa succede: l'utente accede ad una postazione e richiede l'accesso al server **V**. Il modulo **C** nella postazione dell'utente richiede all'utente di inserire la password, a questo punto **C** invia all'**AS** un messaggio che contiene **[ID_c , ID_v , P_c]**. L'AS controlla il proprio database e verifica che: P_c sia la password corretta per ID_c e che ID_c abbia l'autorizzazione ad accedere al server V. Se i 2 controlli sono OK l'AS considera come autenticato l'utente e deve convincere adesso il server V. Per fare ciò, l'AS crea un **ticket** contenente **[ID_c , AD_c , ID_v]** e lo cripta con la chiave segreta **K_v** che AS e V condividono tra loro. L'AS a questo punto spedisce questo ticket a **C**. Così facendo **C** o un **opponent** non potranno modificare il ticket poiché criptato. A questo punto **C** invia il **[ticket , ID_c]** al server **V** richiedendo l'accesso. **V** decripta il ticket e verifica che **ID_c** nel ticket sia uguale a quello presente nel messaggio in chiaro. Se i controlli sono okay allora C è autenticato sul server V.

Osservazioni:

- Il **ticket** viene criptato per evitare alterazioni o contraffazioni.
- L'**ID_v** è inserito nel ticket affinché il server V possa verificare di aver eseguito correttamente la decifrazione.
- **ID_c** è inserito nel ticket per indicare che il ticket è stato emesso per conto di C
- Un opponent potrebbe prendere il ticket e inviarlo da un'altra postazione, il server **V** verificherebbe il ticket come autentico. L'**AS** inserisce **AD_c** nel ticket per evitare

questo, infatti il ticket sarà valido solo se trasmesso dalla stessa postazione che inizialmente ha richiesto il ticket all'AS.

UN DIALOGO DI AUTENTICAZIONE PIU' SICURO

Nonostante lo scenario visto precedentemente risolva alcuni dei problemi di autenticazione, rimangono ancora **due** problemi evidenti. Vorremmo **ridurre al minimo il numero di volte che l'utente inserisce la password**. Possiamo rendere i ticket riutilizzabili; quindi per una singola sessione di accesso, la workstation può archiviare il ticket di un server dopo che è stato ricevuto e utilizzarlo per conto dell'utente per più accessi al server. Rimane un problema nel caso in cui l'utente voglia accedere a più servizi poichè avrebbe bisogno di un nuovo ticket per ogni servizio.

Inoltre, **lo scenario precedente prevede l'invio della password in chiaro!**

Per risolvere questi problemi aggiuntivi viene introdotto un nuovo server chiamato **ticket-granting server (TGS)** e uno **schema per evitare trasmissione di password in chiaro**. TGS fornisce i ticket agli utenti autenticati presso un AS.

Una volta per sessione di accesso utente/Once per user logon session:

1. $C \rightarrow AS:$ $ID_c \parallel ID_{tgs}$
2. $AS \rightarrow C:$ $E(K_c, Ticket_{tgs})$
 $Ticket_{tgs} = E(K_{tgs}, [ID_c \parallel AD_c \parallel ID_{tgs} \parallel TimeStamp1 \parallel Lifetime1])$

Una volta per tipo di servizio/Once per type of service:

3. $C \rightarrow TGS:$ $ID_c \parallel ID_v \parallel Ticket_{tgs}$
4. $TGS \rightarrow C:$ $Ticket_v$
 $Ticket_v = E(K_v, [ID_c \parallel AD_c \parallel ID_v \parallel TimeStamp2 \parallel Lifetime2])$

Una volta per sessione di servizio/Once per service session:

5. $C \rightarrow V:$ $ID_c \parallel Ticket_v$

1. Il client **C** richiede un ticket di concessione del ticket per conto dell'utente inviando all'**AS** il proprio **ID_c** insieme all'**ID_{tgs}**, indicando una richiesta di utilizzo del servizio **TGS**. Il modulo client nella postazione dell'utente, una volta ricevuto il ticket, salva questo **ticket_{tgs}**, e ogni volta che l'utente richiede l'accesso a un nuovo servizio, **C** si rivolge al **TGS** utilizzando il **ticket_{tgs}** per autenticarsi.

2. L'**AS** risponde con **ticket_{tgs}** crittografato con una chiave derivata dalla password dell'utente **K_c**, già memorizzata presso l'**AS**. Quando questa risposta arriva a **C**, esso richiede l'inserimento della password all'utente, genera così la chiave e tenta di decriptare il messaggio in arrivo. Se la password è corretta, il **ticket_{tgs}** viene recuperato con successo. Poichè solo l'utente dovrebbe conoscere la propria password, solo l'utente corretto può recuperare il ticket. Così facendo abbiamo evitato di trasmettere la password in chiaro. Il **Ticket_{tgs}** è criptato con una chiave conosciuta solo dall'**AS** e dal **TGS**, **K_{tgs}**, in modo tale che l'utente non possa modificarne il contenuto. Ogni ticket ha ID e indirizzo di rete dell'utente e ID del TGS. L'idea è che il client possa utilizzare il **Ticket_{tgs}** per richiedere più Ticket di concessione del servizio; quindi **Ticket_{tgs}** deve essere riutilizzabile. Vogliamo poi che un opponent non sia in grado di utilizzare il nostro **Ticket_{tgs}** falsificando quindi il TGS; per fare questo il **Ticket_{tgs}** include un **timestamp** che indica data e ora di creazione del ticket e un **lifetime** che indica il periodo di tempo per il quale il ticket è valido.
3. Il client **C** richiede un ticket di concessione del servizio **Ticket_v** per conto dell'utente. A tale scopo **C** trasmette al **TGS** un messaggio contenente l'ID utente **ID_c**, l'ID del servizio richiesto **ID_v** e il **Ticket_{tgs}**.
4. Il **TGS** decifra il **Ticket_{tgs}** con la chiave privata condivisa solo dall'**AS** e il **TGS**, **K_{tgs}**, e verifica l'esito della decrittazione tramite la presenza del proprio **ID_{tgs}** (deve essere corretto). Controlla il lifetime per assicurarsi che la sessione non sia scaduta. Quindi confronta l'**ID_c** e il **AD_c** presenti nel **Ticket_{tgs}** con quelle del **C** che lo ha contattato per autenticare l'utente. Se all'utente è consentito l'accesso al server **V**, il **TGS** emette un nuovo ticket **Ticket_v** da consegnare a **V**. Il **Ticket_v** ha la stessa struttura del **Ticket_{tgs}**. Si noti che **Ticket_v** è criptato con una chiave segreta **K_v** nota solo al **TGS** e al server **V**.
5. Infine il client **C** richiede l'accesso a un servizio per conto dell'utente. A tale scopo, il client **C** trasmette al server **V** un messaggio contenente l'**ID_c** e il **Ticket_v**. Il server **V** esegue l'autenticazione utilizzando il contenuto del **Ticket_v**.

Questo nuovo scenario soddisfa i due requisiti di una sola richiesta di password per sessione utente e di protezione della password utente.

AUTHENTICATION DIALOGUE DI KERBEROS VERSIONE 4 (ARCHITETTURA KERBEROS VERA E PROPRIA)

Sebbene lo scenario appena visto migliora la sicurezza rispetto al primo scenario, rimangono ancora 2 problemi/requisiti. **1** Il primo problema è legato alla durata del ticket di concessione del ticket: se il ticket dura troppo poco allora l'utente è costretto ad inserire molto spesso la sua password, se, invece, dura troppo, allora un avversario ha una probabilità maggiore di fare replay attack. Esso potrebbe ottenere una copia del **ticket_{tgs}** e aspettare che l'utente legittimo si disconnetta. Quindi potrebbe falsificare l'indirizzo di

rete dell'utente legittimo e inviare il messaggio della fase 3. al TGS. Ciò darebbe all'opponent un accesso illimitato alle risorse e ai file disponibili per l'utente legittimo.

Arriviamo così ad un ulteriore requisito: Un servizio di rete (il TGS o un servizio applicativo) deve essere in grado di dimostrare che la persona che utilizza un ticket è la stessa persona a cui quel biglietto è stato emesso.

Un secondo requisito è che i server si autenticino a loro volta agli utenti. Senza tale autenticazione, un avversario potrebbe sabotare la configurazione in modo che i messaggi per un server vengano indirizzati a un'altra posizione. Il falso server quindi sarebbe in grado di agire come vero server, catturando qualsiasi informazione dall'utente e negare l'accesso al vero servizio all'utente.

Si considera in primo luogo la necessità di determinare che il presentatore del ticket sia lo stesso client per il quale è stato emesso. La minaccia è che un avversario rubi il ticket e lo utilizzi prima che scada. Per aggirare questo problema, facciamo in modo che l'**AS fornisca in modo sicuro un'informazione segreta sia al client che al TGS**. Quindi il client può provare la sua identità al TGS rivelando tale informazione. Un modo efficiente per fare ciò è **utilizzare una chiave di crittografia come informazione sicura**, questo è indicato come una chiave di sessione in Kerberos.

Riepilogo scambi di messaggi in Kerberos V4:

1. Il client invia un messaggio all'**AS** contenente l'**ID_c**, **ID_{tgs}** e **TimeStamp1** richiedendo l'accesso al **TGS**.
2. L'**AS** risponde con un messaggio cifrato con una chiave derivata dalla password dell'utente **K_c**, contenente il **Ticket_{tgs}** e altre informazioni. Il messaggio cifrato al suo interno contiene anche una copia della chiave di sessione **K_{c,tgs}**, ovvero la chiave di sessione per **C** e **TGS**. Poiché **K_{c,tgs}** si trova all'interno del messaggio crittografato con **K_c**, solo il client dell'utente può leggerlo. La stessa chiave di sessione **K_{c,tgs}** è inclusa nel **Ticket_{tgs}**, che può essere letto solo dal **TGS**. Pertanto, la chiave di sessione **K_{c,tgs}** è stata consegnata in modo sicuro sia a **C** che a **TGS**.

Osservazioni: rispetto al dialogo precedente, sono state aggiunte diverse informazioni. Il messaggio 1. include un TimeStamp, in modo che l'**AS sappia che il messaggio è tempestivo**. Il messaggio 2. contiene altri elementi del ticket in una forma accessibile dal client. Questo consente a C di confermare che questo biglietto è per il TGS e di conoscere l'ora di scadenza.

3. Il client **C** adesso invia al **TGS** un messaggio che include l'**ID_v** e il **Ticket_{tgs}**. Insieme, trasmette anche un autenticatore **Authenticator_{c1}**, che include l'**ID_c** e **AD_c** ed un TimeStamp3, il tutto criptato con la chiave di sessione che condividono **K_{c,tgs}**. **Authenticator_{c1}** è concepito per essere utilizzato solo una volta ed ha una durata molto breve.
4. Il **TGS**, a questo punto, decifra il **Ticket_{tgs}** con la chiave **K_{tgs}** che condivide con l'**AS**, e utilizza la chiave di sessione **K_{c,tgs}** per decifrare **Authenticator_{c1}**. Il **TGS** verifica che l'**ID_c** e **AD_c** del client corrispondano a quelli del **Ticket_{tgs}** e **AD_c** con l'indirizzo di rete del messaggio in arrivo. *Se tutti corrispondono, il TGS ha la certezza che il mittente del ticket è effettivamente il vero proprietario del ticket.* A questo punto il **TGS** invia un messaggio a **C** cifrato con la chiave di sessione che condividono, **K_{c,tgs}**. Tale messaggio contiene il **Ticket_v**, l'**ID_v**, **TimeStamp4** e **Lifetime4**, e la chiave di sessione tra **C** e il server **V** ovvero **K_{c,v}**. Poiché **K_{c,v}** si trova all'interno del messaggio crittografato con **K_{c,tgs}**, solo il client dell'utente può leggerlo. La stessa chiave di sessione **K_{c,v}** è inclusa nel **Ticket_v**, che è criptato con la chiave che condividono **TGS** e **V** **K_v**, per cui **K_{c,v}** può essere letto solo da **V**. Pertanto, la chiave di sessione **K_{c,v}** è stata consegnata in modo sicuro sia a **C** che a **V**.
5. A questo punto **C** ha il **Ticket_v** riutilizzabile per **V**. **C** invia a **V** il **Ticket_v** e un autenticatore **Authenticator_{c2}**. Il server **V** decifra il **Ticket_v** con **K_v** e ottiene la chiave di sessione **K_{c,v}** tra se stesso e **C**; dopodichè decifra **Authenticator_{c2}**. Infine **V** verifica che l'**ID_c** e **AD_c** del client corrispondano a quelli del **Ticket_v** e **AD_c** con l'indirizzo di rete del messaggio in arrivo. *Se tutti corrispondono, V ha la certezza che il mittente del ticket è effettivamente il vero proprietario del ticket.*
6. Se è richiesta l'autenticazione reciproca, **V** restituisce il valore del timestamp dell'autenticatore incrementato di 1 e crittografato con la chiave di sessione condivisa tra **C** e **V**, **K_{c,v}**. **C** decifra il messaggio per recuperare il timestamp incrementato; poiché il messaggio è cifrato con la chiave di sessione **K_{c,v}**, **C** ha la certezza che sia stato creato da **V**. Il contenuto del messaggio ovvero l'incremento del timestamp assicura a **C** che non si tratta di una replica della risposta precedente. Infine, al termine di questo processo, **C** e **V** condividono una chiave segreta; questa chiave potrà essere utilizzata per crittografare i messaggi futuri tra i due o per scambiarsi una nuova chiave di sessione.

Osservazione: E' l'autenticatore che prova l'identità del client, perchè può essere utilizzato solo una volta e ha una durata breve; la minaccia di un avversario che ruba sia Ticket che l'autenticatore per la presentazione successiva viene contrastato.

Authentication service exchange to obtain ticket-granting ticket (AUTENTICAZIONE):

1. $C \rightarrow AS:$ $ID_c \parallel ID_{tgs} \parallel TimeStamp1$
2. $AS \rightarrow C:$ $E(K_c, [K_{c,tgs} \parallel ID_{tgs} \parallel TimeStamp2 \parallel Lifetime2 \parallel \textit{Ticket}_{tgs}])$
 $\textit{Ticket}_{tgs} = E(K_{tgs}, [K_{c,tgs} \parallel ID_c \parallel AD_c \parallel ID_{tgs} \parallel TimeStamp2 \parallel Lifetime2])$

Ticket granting service exchange to obtain service-granting ticket (AUTORIZZAZIONE):

3. $C \rightarrow TGS:$ $ID_v \parallel \textit{Ticket}_{tgs} \parallel \textit{Authenticator}_{c1}$
4. $TGS \rightarrow C:$ $E(K_{c,tgs}, [K_{c,v} \parallel ID_v \parallel TimeStamp4 \parallel Lifetime4 \parallel \textit{Ticket}_v])$
 $\textit{Ticket}_v = E(K_v, [K_{c,v} \parallel ID_c \parallel AD_c \parallel ID_v \parallel TimeStamp4 \parallel Lifetime4])$
 $\textit{Authenticator}_{c1} = E(K_{c,tgs}, [ID_c \parallel AD_c \parallel TimeStamp3])$

Client/Server auth exchange to obtain service (SERVIZIO):

5. $C \rightarrow V:$ $\textit{Ticket}_v \parallel \textit{Authenticator}_{c2}$
6. $V \rightarrow C:$ $E(K_{c,v}, [TimeStamp5 + 1])$ //per autenticazione reciproca
 $\textit{Authenticator}_{c2} = E(K_{c,v}, [ID_c \parallel AD_c \parallel TimeStamp5])$

$K_{c,tgs}$ = AuthKey (chiave di sessione condivisa tra C e TGS)

$K_{c,v}$ = ServKey (chiave di sessione condivisa tra C e V)

$\textit{Ticket}_{tgs}$ = TicketTGS

$\textit{Ticket}_v$ = TicketServizio

L'**AS** genera AuthKey al tempo TimeStamp2

Il **TGS** genera ServKey al tempo TimeStamp4

Validità:

- Di AuthKey (cioè di TS2) in ore, diciamo LA
- Di ServKey (cioè di TS4) in minuti, diciamo LS
- Di un autenticatore (ossia TS1, TS3, TS5) in sec

TGS può generare ServKey solo qualora sia: (*) **TS4 + LS <= TS2 + LA**

Altrimenti potrebbe verificarsi il problema di cascata dell'attacco (attacchi consequenziali):

Supponiamo che un opponente (B) abbia violato una chiave di sessione di autorizzazione scaduta AuthKey che AS condivide con C. Con semplice decodifica ottiene ServKey ancora valide se non si impone (*). B può ora accedere a V per la durata residua di LS.

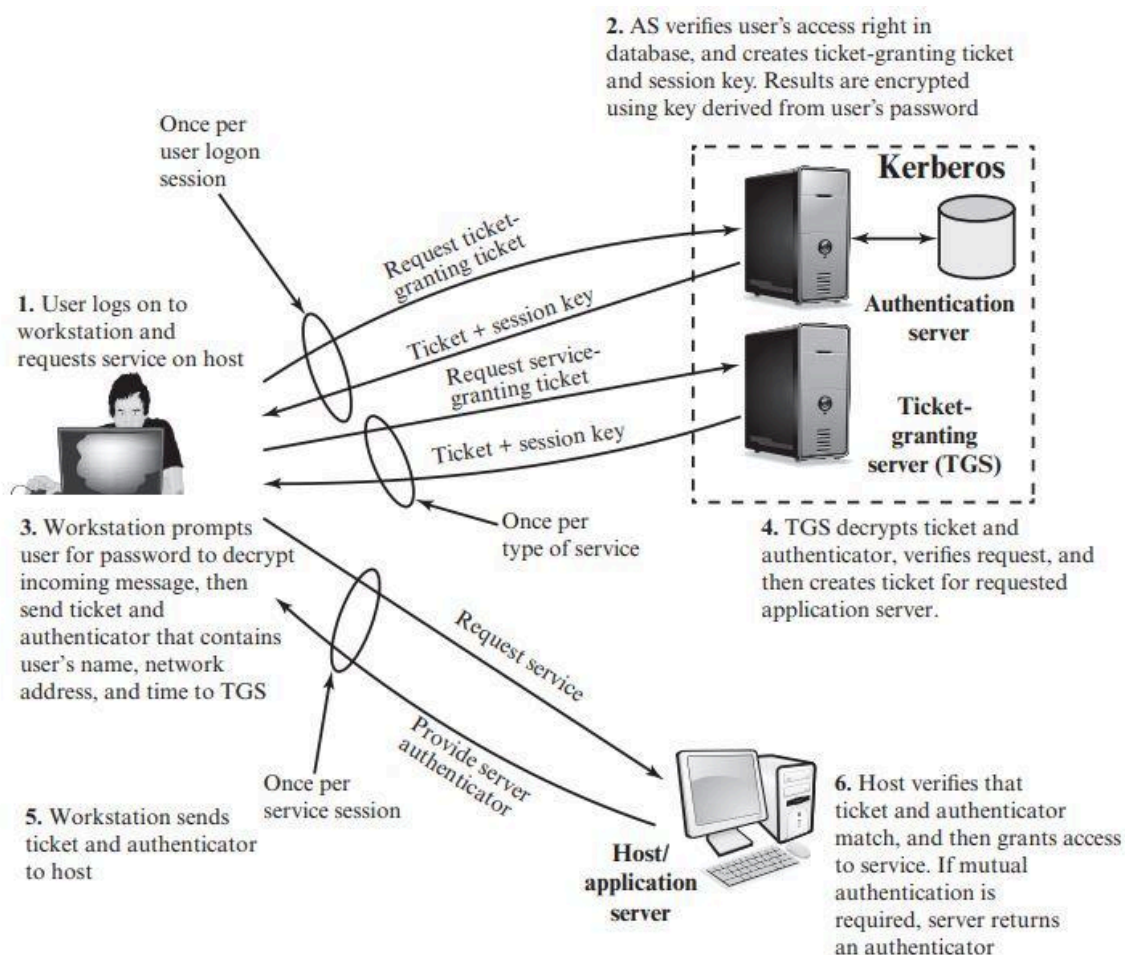


Figure 4.2 Overview of Kerberos

KERBEROS REALMS AND MULTIPLE KERBERI

Un ambiente Kerberos completo è costituito da un server Kerberos, una serie di clients e una serie di application servers che richiedono quanto segue:

1. Il server Kerberos deve avere l'ID utente e le password di tutti gli utenti nel suo database. Tutti gli utenti sono registrati con il server Kerberos.
2. Il server Kerberos deve condividere una chiave segreta con ciascun server. Tutti i server sono registrati con il server Kerberos.

Tale ambiente è definito come **Kerberos Realm**. Esso, Realm = Regno, è un insieme di nodi gestiti che condividono lo stesso database Kerberos. Il database risiede sul sistema informativo principale Kerberos, che deve essere conservato in una stanza fisicamente sicura. Una copia di sola lettura potrebbe risiedere anche in altri sistemi informatici Kerberos, tuttavia, tutte le modifiche devono essere effettuate sul sistema principale.

Un concetto correlato è quello di **Kerberos principal**, ovvero un servizio o un utente noto al sistema Kerberos. Ogni **Kerberos principal** è identificato dal suo principal name che si compone di 3 parti: service name o user name, nome di istanza e nome del regno.

Le reti di client e server sotto diverse organizzazioni amministrative in genere costituiscono regni diversi. Tuttavia, utenti in un regno potrebbero aver bisogno di accedere ai server in altri regni e alcuni potrebbero essere disposti a fornire servizi a utenti di altri regni, a condizione che tali utenti siano autenticati.

KERBEROS V5

La versione 5 di Kerberos ha lo scopo di affrontare i limiti della versione 4 e apportare dei miglioramenti in due aree: carenze ambientali e carenze tecniche.

Carenze ambientali:

1. Dipendenze dal sistema di crittografia: La v4 obbliga l'uso di DES. Nella v5 si può utilizzare qualsiasi tecnica di crittografia.
2. Dipendenza del protocollo Internet: La v4 richiede l'uso di indirizzi IP. Nella v5 si può utilizzare qualsiasi tipo di indirizzo di rete.
3. Ordinamento dei byte del messaggio: nella v4 l'utente utilizza un ordinamento dei byte di sua scelta e contrassegna il messaggio per indicare il byte meno significativo nell'indirizzo più basso o il byte più significativo nell'indirizzo più basso. Questa tecnica non segue le convenzioni stabilite. Nella v5 le strutture dei messaggi vengono definite seguendo ASN.1 e BER che forniscono un ordinamento dei byte non ambiguo.
4. Durata del ticket: nella v4 durata massima di circa 21 ore. Per alcune applicazioni potrebbe non essere adeguata. Nella v5, i ticket includono un'ora di inizio e di fine esplicite, consentendo ticket con durate arbitrarie.
5. Inoltro di autenticazione: la v4 non consente che le credenziali rilasciate a un client vengano inoltrate a un altro host e utilizzate da un altro client. Ad esempio, un client invia una richiesta a un server di stampa che quindi accede al file del client da un file server, utilizzando le credenziali del client per l'accesso. La v5 fornisce questa funzionalità.

6. Autenticazione Interrealm: Nella v4, l'interoperabilità tra N realms richiede nell'ordine di N^2 relazioni Kerberos-Kerberos. La versione 5 supporta un metodo che richiede meno relazioni.

Carenze tecniche:

1. Doppia crittografia: I ticket forniti ai client vengono crittografati 2 volte (2., 4.): una volta con la chiave segreta nota al server di destinazione e poi di nuovo con una nota al client. La seconda crittografia non è necessaria ed è computazionalmente dispendiosa.
2. Crittografia PCBC: La crittografia nella v4 utilizza una modalità non standard di DES nota come PCBC (propagating cipher block chaining). E' stato dimostrato che questa modalità è vulnerabile ad un attacco che comporta l'interscambio di blocchi di testo cifrato. La v5 consente l'utilizzo della modalità CBC standard per la crittografia.
3. Chiavi di sessione: Le chiavi di sessione possono essere utilizzate da client e server per autenticare il client al server e per proteggere i messaggi passati durante quella sessione. Tuttavia, poiché lo stesso ticket può essere utilizzato ripetutamente per ottenere il servizio da un particolare server, esiste il rischio che un avversario riproduca i messaggi di una vecchia sessione al client o al server. Nella v5, è possibile per un client e un server negoziare una chiave di sottosessione che deve essere utilizzata solo per quella connessione.
4. Attacchi alle password: Entrambe le versioni sono vulnerabili a un attacco alle password. Il messaggio dall'AS al cliente include materiale crittografato con una chiave basata sulla password del client. Un avversario può catturare questo messaggio e tentare di decifrarlo provando varie password. La v5 fornisce un meccanismo noto come pre-autenticazione che dovrebbe rendere più difficili attacchi di questo tipo, ma non li impedisce.

UN DIALOGO DI AUTENTICAZIONE KERBEROS 5

Authentication service exchange to obtain ticket-granting ticket (AUTENTICAZIONE):

1. $C \rightarrow AS$: Options || ID_c || Realmc || ID_{tgs} || Times || Nonce₁
2. $AS \rightarrow C$: Realmc || ID_c || **Ticket_{tgs}** || E(K_c, [K_{c,tgs} || Times || Nonce₁ || Realmtgs || ID_{tgs}])

Ticket_{tgs} = E(K_{tgs}, [Flags || K_{c,tgs} || Realmc || ID_c || AD_c || Times])

Ticket granting service exchange to obtain service-granting ticket (AUTORIZZAZIONE):

3. $C \rightarrow TGS$: Options || ID_v || Times || Nonce₂ || **Ticket_{tgs}** || Authenticator_{c1}

4. TGS $\rightarrow$ C: Realmc || IDc || **Ticket_v** || E(K_{c,tgs}, [K_{c,v} || Times || Nonce2 || Realmv || IDv])

Ticket_{tgs} = E(K_{tgs}, [Flags || K_{c,tgs} || Realmc || IDc || ADc || Times])

Ticket_v = E(K_v, [Flags || K_{c,v} || Realmc || IDc || ADc || Times])

Authenticator_{c1} = E(K_{c,tgs}, [IDc || Realmc || TimeStamp1])

Client/Server auth exchange to obtain service (SERVIZIO):

5. C $\rightarrow$ V: Options || **Ticket_v** || **Authenticator_{c2}**

6. V $\rightarrow$ C: E(K_{c,v}, [TimeStamp2 || Subkey || Seq#])

Ticket_v = E(K_v, [Flags || K_{c,v} || Realmc || IDc || ADc || Times])

Authenticator_{c2} = E(K_{c,v}, [IDc || Realmc || TimeStamp2 || Subkey || Seq#])

[\(VEDERE SPIEGAZIONE SU LIBRO CAP.4\)](#)

DISTRIBUZIONE DELLE CHIAVI USANDO LA CRITTOGRAFIA ASIMMETRICA

Un aspetto fondamentale della crittografia a chiave pubblica è affrontare il problema della distribuzione delle chiavi. Ci sono in realtà due aspetti distinti nell'uso della crittografia a chiave pubblica come mezzo per distribuire chiavi:

1. La distribuzione delle chiavi pubbliche
2. L'uso della crittografia a chiave pubblica per distribuire chiavi segrete

CERTIFICATI A CHIAVE PUBBLICA

Con la crittografia a chiave pubblica qualsiasi partecipante può trasmettere la propria chiave pubblica a qualsiasi altro partecipante o trasmetterla alla comunità in generale. Sebbene questo approccio sia conveniente, ha un grosso punto debole: un utente può fingere di essere un altro utente A, e inviare una chiave pubblica FALSA ad un altro partecipante. Fin tanto che l'utente A non scopre la contraffazione e avvisa gli altri partecipanti, il falsario è in grado di leggere tutti i messaggi, teoricamente diretti ad A, utilizzando la chiave falsa per decifrare.

La soluzione a questo problema è il **certificato a chiave pubblica**. In sostanza un certificato è costituito da una chiave pubblica più un ID utente del proprietario della chiave, e firmato da una terza parte fidata. In generale la terza parte è un'autorità di certificazione (CA) attendibile dalla comunità di utenti. Un utente può presentare, in modo sicuro, la sua chiave pubblica alla CA e richiedere il certificato. Chiunque necessiti della chiave pubblica di

un utente può ottenere il certificato e verificarne la validità tramite la firma attendibile allegata. Lo standard X.509 è diventato uno schema universalmente accettato per la formattazione dei certificati a chiave pubblica. (utilizzato per sicurezza IP, SSL, S/MIME)

DISTRIBUZIONE A CHIAVE PUBBLICA DI CHIAVI SEGRETE

Nella crittografia convenzionale (a chiave privata) c'è il problema dello scambio di quest'unica chiave tra le due entità comunicanti. Servirebbe un'altra chiave segreta per criptare la precedente, che a sua volta non si sa come scambiare. Un approccio ampiamente utilizzato è l'uso dello scambio di chiavi Diffie-Hellman; tuttavia, presenta lo svantaggio che, nella sua forma più semplice, non fornisce l'autenticazione dei due partner che comunicano. Una potente alternativa è l'uso di **certificati a chiave pubblica**. Quando Bob desidera comunicare con Alice, può fare quanto segue:

1. Prepara il messaggio, lo cripta con la crittografia convenzionale usando una chiave di sessione una tantum.
2. Cripta la chiave di sessione, utilizzando la crittografia a chiave pubblica, usando la chiave pubblica di Alice.
3. Allega la chiave di sessione criptata e invia tutto ad Alice.

Solo Alice è in grado di decifrare il messaggio recuperando la chiave di sessione tramite la sua chiave privata. Se Bob ha ottenuto la chiave pubblica di Alice tramite certificato, allora Bob è sicuro che si tratta di una chiave valida.

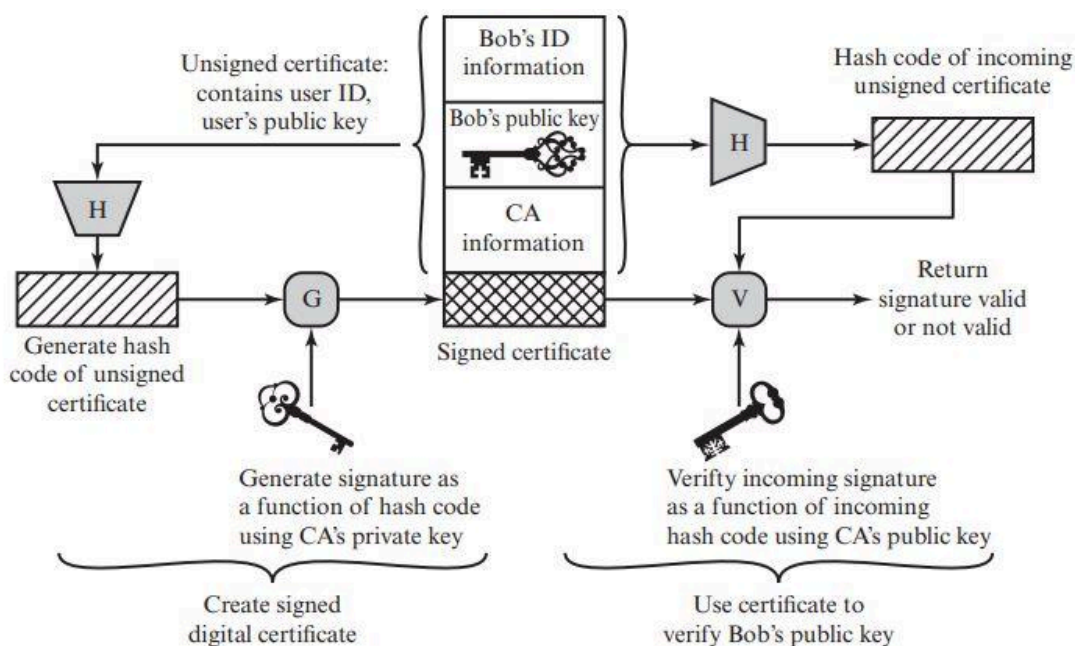


Figure 4.4 Public-Key Certificate Use

CERTIFICATI X.509

La raccomandazione ITU-T X.509 fa parte della serie di raccomandazioni X.500 che definiscono un servizio di directory. La directory è un server o un insieme distribuito di server che mantengono un database di informazioni sugli utenti. Esse includono una mappatura dal nome utente all'indirizzo di rete e altri attributi.

X.509 definisce un framework per la fornitura di servizi di autenticazione ai suoi utenti. La directory può fungere anche da repository di certificati a chiave pubblica. Ogni certificato contiene la chiave pubblica di un utente ed è firmato con la chiave privata dell'autorità certificante attendibile. Inoltre, X.509 definisce protocolli di autenticazione alternativi basati sull'uso di certificati a chiave pubblica. X.509 è importante poiché la struttura del certificato e i protocolli di autenticazione definiti in X.509 sono utilizzabili in una varietà di contesti quali S/MIME, Sicurezza IP e SSL/TLS.

X.509 si basa sull'uso della crittografia a chiave pubblica e delle firme digitali. Lo standard non impone l'uso di uno specifico algoritmo di firma digitale né di una funzione di hash specifica. Il certificato per la chiave pubblica di Bob include: informazioni di identificazione univoche per Bob, la sua chiave pubblica, e informazioni di identificazione della CA, oltre ad altre informazioni descritte nella figura sottostante. Queste informazioni vengono quindi firmate calcolando un valore di hash e generando successivamente una firma digitale utilizzando la chiave privata della CA.

CERTIFICATI

Il cuore di X.509 è il certificato a chiave pubblica associato a ciascun utente. Si presuppone che questi certificati siano creati da una **CA** attendibile e inseriti nella directory dalla CA o dall'utente. Il server di directory non è responsabile della creazione di chiavi pubbliche o della funzione di certificazione; fornisce semplicemente una posizione facilmente accessibile agli utenti per ottenere i certificati.

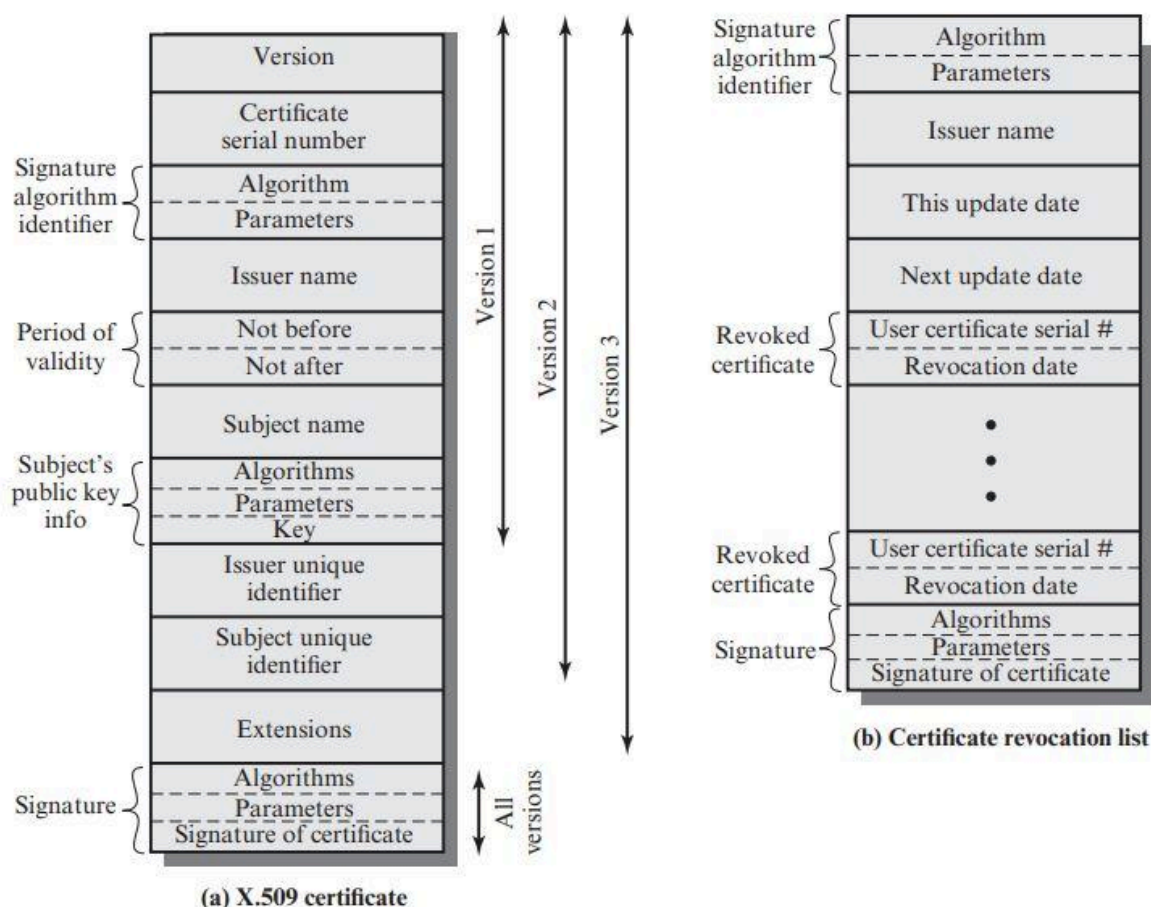


Figure 4.5 X.509 Formats

OTTENERE UN CERTIFICATO UTENTE

I certificati utente emessi da un ente certificatore CA hanno due caratteristiche: **1** qualsiasi utente con accesso alla chiave pubblica di CA può verificare la validità della chiave pubblica dell'utente che è stata certificata da CA; **2** nessuna parte diversa da CA può modificare il certificato senza che questo venga rilevato. Se è presente una vasta comunità di utenti potrebbe essere più pratico che vi siano più CA, ciascuna delle quali fornisce in modo sicuro la propria chiave pubblica ad una frazione di utenti.

REVOCA DEI CERTIFICATI

Ricordiamo dalla Figura 4.5 che ogni certificato include un periodo di validità, proprio come una carta di credito. In genere, viene emesso un nuovo certificato solo prima della scadenza del vecchio. Inoltre, può essere auspicabile occasionalmente revocare un certificato prima che scada per uno dei seguenti motivi:

1. Si presume che la chiave privata dell'utente sia compromessa.
2. L'utente non è più certificato da questa CA. Le ragioni di ciò includono che il nome del soggetto è cambiato, il certificato è stato sostituito o il certificato era stato emesso in conformità con le politiche della CA.
3. Si presume che il certificato della CA sia compromesso.

INFRASTRUTTURA A CHIAVE PUBBLICA

RFC 4949 definisce l'infrastruttura a chiave pubblica (**PKI** Public-Key Infrastructure) come l'insieme di hardware, software, persone, politiche e procedure necessarie per creare, gestire, archiviare, distribuire e revocare certificati digitali basati sulla crittografia asimmetrica. L'obiettivo principale per lo sviluppo di una PKI è consentire un accesso sicuro, convenienza ed efficienza nell'acquisizione di chiavi pubbliche.

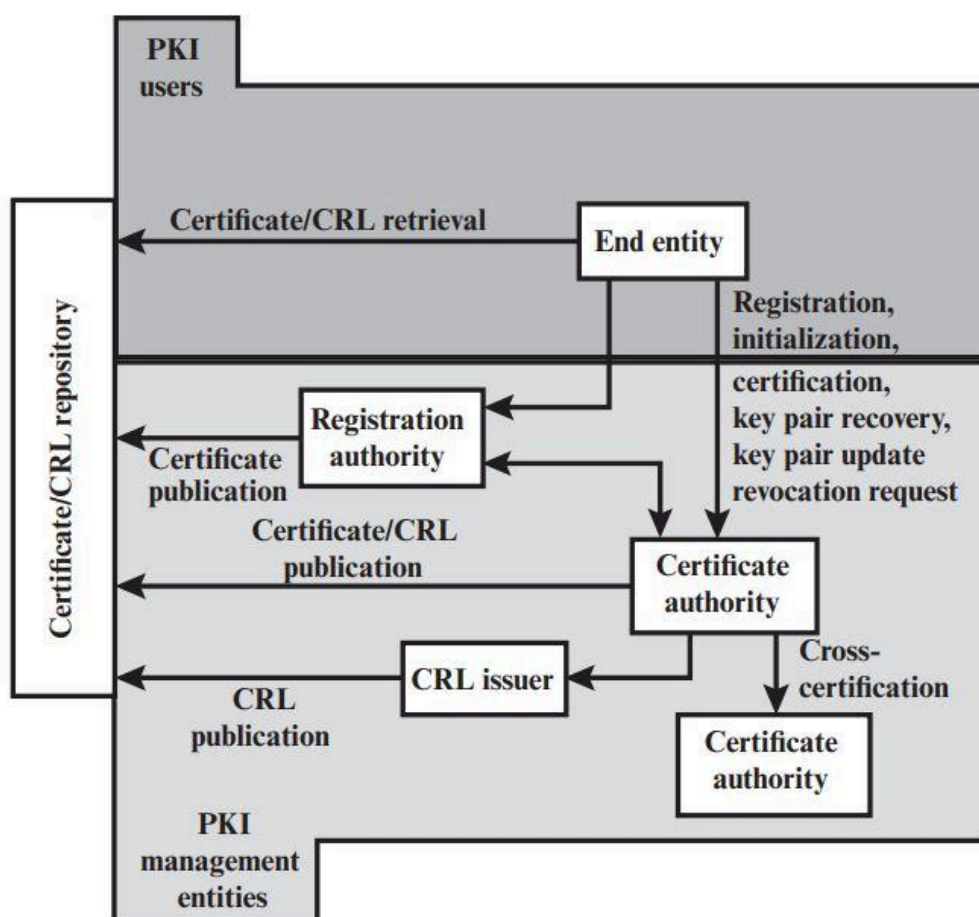


Figure 4.7 PKIX Architectural Model

PKIX model = public key infrastructure with X.509.

La figura precedente mostra l'interrelazione tra gli elementi chiave del modello PKIX. Questi elementi sono:

- End entity = termine generico per indicare un utente finale, un dispositivo o qualsiasi altra entità che possa essere identificata nel campo dell'oggetto di un certificato a chiave pubblica;
- Certificate authority CA = ente certificatore;
- Registration authority RA = componente facoltativo che può assumere diverse funzioni amministrative dalla CA;
- CRL issuer = componente facoltativo a cui la CA può delegare la pubblicazione dei certificati;
- Repository = termine generico utilizzato per indicare qualsiasi tecnica per l'archiviazione dei certificati in modo che possano essere recuperati dalle entità finali.

FUNZIONI DI GESTIONE PKIX

- **Registrazione:** processo con cui un utente si rende noto ad una CA.
- **Inizializzazione:** installazione dei key materials che abbiano le giuste relazioni con le chiavi archiviate altrove nell'infrastruttura.
- **Certificazione:** processo mediante il quale un CA emette un certificato per la chiave pubblica dell'utente e restituisce il certificato al sistema client dell'utente e/o inserisce tale certificato in un repository.
- **Recupero coppia di chiavi:** fornire un meccanismo per recuperare le chiavi di decrittazione quando non è più possibile il normale accesso al materiale di codifica, altrimenti non sarà più possibile recuperare i dati crittografati.
- **Aggiornamento delle coppie di chiavi:** le coppie necessitano di aggiornamenti regolari.
- **Richiesta di revoca:** una persona autorizzata avvisa la CA di una situazione anomala che richiede la revoca del certificato.
- **Certificazione incrociata:** due CA si scambiano le informazioni utilizzate per stabilire una certificazione incrociata. Un certificato incrociato è un certificato emesso da una CA ad un'altra CA che contiene una chiave di firma CA utilizzata per l'emissione di certificati.

CAPITOLO 6

CONSIDERAZIONI SULLA SICUREZZA WEB

Ci sono alcune caratteristiche del Web che suggeriscono la necessità di strumenti di sicurezza su misura, e queste sono:

1. Anche se i Browser Web sono facili da usare, i server Web sono relativamente facili da configurare e i contenuti Web sono facili da sviluppare, il software sottostante è straordinariamente complesso. Questo software può nascondere molti difetti di sicurezza.
2. Un server Web può essere sfruttato da un avversario come trampolino di lancio all'intero complesso informatico di un'azienda o di un'agenzia; così da ottenere l'accesso a risorse che non fanno parte del Web ma sono connesse al server Web nel sistema informatico aziendale locale.
3. Utenti standard non sono coscienti dei rischi esistenti per la sicurezza e non hanno conoscenze e strumenti per adottare contromisure efficaci.

MINACCE ALLA SICUREZZA WEB

La tabella seguente mostra a quali tipologie di minacce si può incorrere durante l'utilizzo del Web. Queste minacce si possono distinguere in: attacchi attivi e attacchi passivi. Un'altra classificazione delle minacce Web può essere fatta tenendo conto della posizione della minaccia: server Web, browser Web e traffico di rete tra browser e server.

Table 6.1 A Comparison of Threats on the Web

	Threats	Consequences	Countermeasures
Integrity	• Modification of user data • Trojan horse browser • Modification of memory • Modification of message traffic in transit	• Loss of information • Compromise of machine • Vulnerability to all other threats	Cryptographic checksums
Confidentiality	• Eavesdropping on the net • Theft of info from server • Theft of data from client • Info about network configuration • Info about which client talks to server	• Loss of information • Loss of privacy	Encryption, Web proxies
Denial of Service	• Killing of user threads • Flooding machine with bogus requests • Filling up disk or memory • Isolating machine by DNS attacks	• Disruptive • Annoying • Prevent user from getting work done	Difficult to prevent
Authentication	• Impersonation of legitimate users • Data forgery	• Misrepresentation of user • Belief that false information is valid	Cryptographic techniques

APPROCCI ALLA SICUREZZA DEL TRAFFICO WEB

Sono possibili diversi approcci per fornire sicurezza Web; questi approcci sono simili nei servizi che offrono ma si differenziano rispetto al loro ambito di applicabilità e alla loro posizione all'interno dello stack TCP/IP.

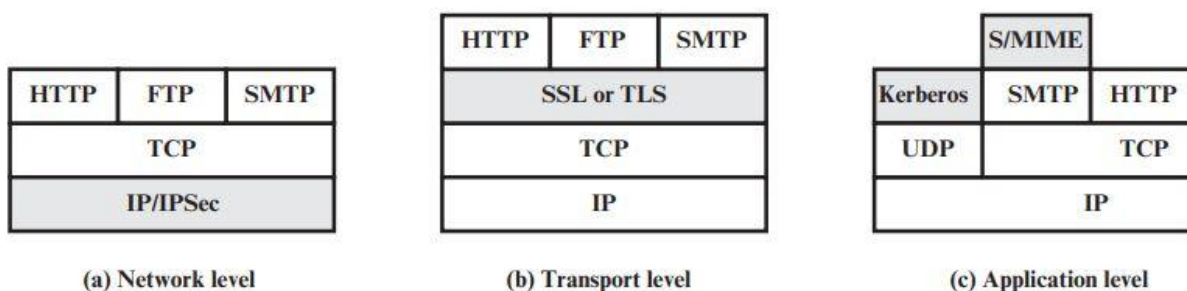


Figure 6.1 Relative Location of Security Facilities in the TCP/IP Protocol Stack

Questa figura illustra la differenza sopracitata. Un modo per fornire sicurezza Web è utilizzando IPsec, un'altra soluzione è implementare la sicurezza appena sopra il TCP e l'esempio principale di tale approccio è l'SSL (Secure Socket Layer) e lo standard Internet successivo noto come TLS (Transport Layer Security), infine un altro approccio è quello di servizi di sicurezza specifici incorporati direttamente nell'applicazione.

SICUREZZA A LIVELLO DI TRASPORTO

Uno dei servizi di sicurezza più utilizzati è **Transport Layer Security (TLS)**; TLS è lo standard Internet che rappresenta l'evoluzione del protocollo commerciale (SSL). TLS è un servizio generico **implementato come un insieme di protocolli che si basano su TCP**. Riguardo TLS ci possono essere due scelte implementative: **una** è quella che TLS venga fornito come parte della suite di protocolli sottostante e quindi risultare trasparente per le applicazioni, **l'altra** è quella di incorporare TLS in pacchetti specifici, ad esempio molti browser sono dotati di TLS e la maggior parte dei server Web implementa TLS.

ARCHITETTURA TLS

TLS è progettato per utilizzare TCP per fornire un servizio end-to-end sicuro e affidabile. **TLS non è un singolo protocollo ma piuttosto due strati di protocolli**, come mostrato nella seguente figura.

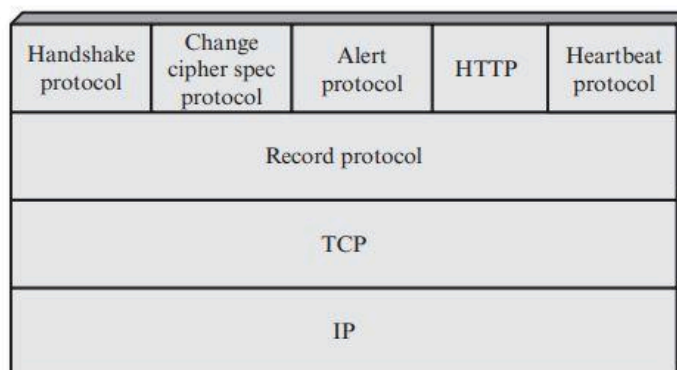


Figure 6.2 TLS Protocol Stack

Il protocollo TLS **Record Protocol** fornisce servizi di sicurezza di base a vari protocolli di livello superiore, in particolare all'**HTTP** (HyperText Transfer Protocol) che fornisce il servizio di trasferimento per l'interazione client/server Web.

Il protocollo **Handshake protocol**, **Change cipher spec protocol** protocollo di modifica delle specifiche di cifratura e **Alert protocol** protocollo di allerta; sono definiti come parte di TLS. Questi ultimi sono usati nella gestione degli scambi TLS. Un quarto protocollo è l'**HeartBeat protocol** definito in modo separato.

Due concetti importanti del TLS sono TLS Connection e TLS Session:

1. **Connessione:** una connessione è un trasporto che fornisce un tipo di servizio adeguato. Per TLS le connessioni sono relazioni peer-to-peer e sono transitorie. Ogni connessione è associata ad una Sessione.
2. **Sessione:** è un'associazione tra client e server. Vengono create dal protocollo Handshake. Definiscono un insieme di parametri di sicurezza crittografici che possono essere condivisi tra più connessioni. Sono create per evitare la costosa negoziazione di nuovi parametri di sicurezza per ciascuna connessione.

Ci sono un certo numero di stati associati a ciascuna sessione. Una volta stabilita una sessione, esiste uno stato operativo corrente sia per la lettura che per la scrittura (ovvero, ricezione e invio). Inoltre, durante il protocollo Handshake, in attesa di lettura e scrittura si creano gli stati. Dopo la conclusione positiva del protocollo Handshake, gli stati pendenti diventano gli stati attuali.

Uno stato della sessione è definito dai seguenti parametri (riporto i più importanti):

1. Session Identifier: una sequenza di byte arbitraria scelta dal server per identificare uno stato di sessione attivo o ripristinabile.

2. Peer certificate: un certificato X509.v3 del peer; questo elemento dello stato può essere nullo.
3. Compression method: l'algoritmo utilizzato per comprimere i dati prima della crittografia.
4. Cipher spec: specifica l'algoritmo di crittografia dei dati in blocco e un algoritmo hash utilizzato per il calcolo MAC; definisce anche attributi crittografici come hash_size.
5. Master secret: segreto di 48 byte condiviso tra il client e il server.
6. Is resumable: un flag che indica se la sessione può essere utilizzata per avviare nuove connessioni.

TLS RECORD PROTOCOL

Il protocollo Record Protocol fornisce 2 servizi per le connessioni TLS:

1. Confidenzialità:
2. Integrità del messaggio:

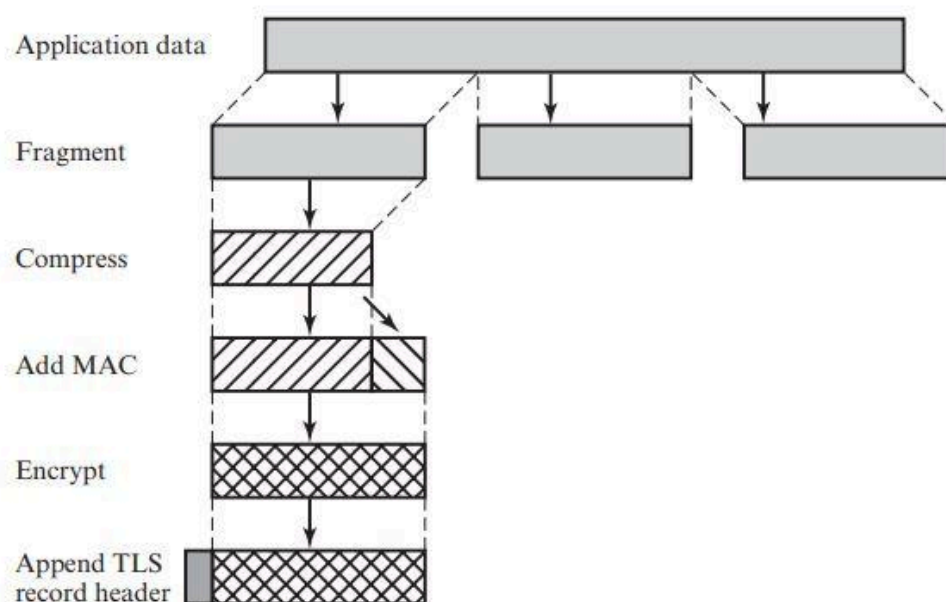


Figure 6.3 TLS Record Protocol Operation

Il **Record Protocol** funziona come descritto nella figura precedente, prende un messaggio applicativo da trasmettere, frammenta i dati in blocchi gestibili, facoltativamente comprime i dati, applica un MAC, esegue la crittografia, aggiunge un'intestazione e trasmette l'unità risultante in un segmento TCP.

Il Record Protocol funziona come descritto nella figura precedente, il primo passo è la **frammentazione**; ogni messaggio di livello superiore è frammentato in blocchi di 2^{14} byte

(16384 byte) o meno. Successivamente, viene applicata facoltativamente la **compressione**. La compressione deve essere senza perdita di dati e non può aumentare la lunghezza del contenuto di oltre 1024 byte. Il passo successivo consiste nel calcolare un **Message Authentication Code** sui dati compressi. TLS utilizza l'algoritmo HMAC definito come:

$$HMAC_K(M) = H[(K^+ + opad) \parallel H[(K^+ + ipad) \parallel M]]$$

dove:

H → funzione hash incorporata (in TLS, MD5 o SHA-1)

M → messaggio inviato a HMAC

K⁺ → chiave segreta riempita di zeri a sinistra in modo che il risultato sia uguale alla lunghezza del blocco del codice hash (per MD5 e SHA-1, lunghezza del blocco = 512 bit)

ipad → 00110110 (36 in esadecimale) ripetuto 64 volte (512 bit)

opad → 01011100 (5C in esadecimale) ripetuto 64 volte (512 bit)

+ → XOR (credo)

Per TLS, il calcolo MAC comprende i campi indicati nella seguente espressione:

$$HMAC_hash(MAC_write_secret, seq_num \parallel TLSCompressed.type \parallel TLSCompressed.version \parallel TLSCompressed.length \parallel TLSCompressed.fragment)$$

Il calcolo MAC copre tutti i campi XXX, più il campo TLSCompressed.version, che è la versione del protocollo utilizzato.

Dopodiché, il messaggio compresso più il MAC vengono **crittografati** utilizzando la crittografia simmetrica. La crittografia non può aumentare la lunghezza del contenuto di oltre 1024 byte, in modo che la lunghezza totale non possa superare $2^{14} + 2048$. Sono consentiti AES e 3DES come algoritmi di cifratura a blocchi e RC4-128 per la cifratura a flusso. Il passaggio finale dell'elaborazione di Record Protocol TLS consiste nell'**anteporre un'intestazione** composta dai seguenti campi:

1. **Content Type (8 bits)**: il protocollo di livello superiore utilizzato per elaborare il frammento racchiuso.
2. **Major Version (8 bits)**: indica la versione principale di TLS in uso. Per TLSv2, il valore è 3.
3. **Minor Version (8 bits)**: indica la versione secondaria in uso. Per TLSv2, il valore è 1.
4. **Compressed Length (16 bits)**: la lunghezza in byte del testo in chiaro (o del frammento compresso se viene utilizzata la compressione). Il valore massimo è $2^{14} + 2018$ byte.

I tipi di contenuto di questi Record TLS sono quelli precedentemente citati: **change_cipher_spec**, **alert**, **handshake**, and **application_data**. I primi tre sono i protocolli

specifici TLS. Mentre, per quanto riguarda **application_data**, non viene fatta nessuna distinzione tra le varie applicazioni che potrebbero utilizzare TLS; il contenuto dei dati creati da tali applicazioni è opaco per TLS.

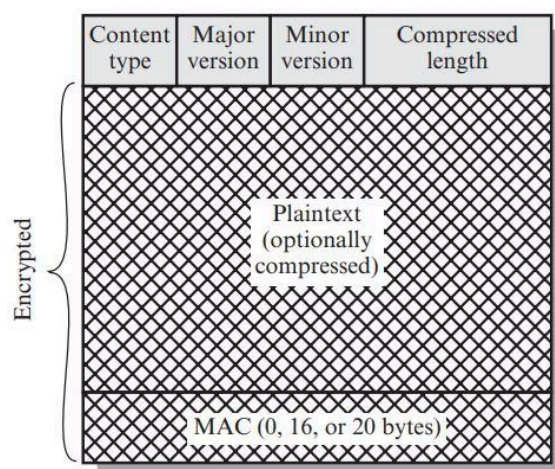


Figure 6.4 TLS Record Format

CHANGE CIPHER SPEC PROTOCOL

Il protocollo **Change Cipher Spec** è uno dei protocolli specifici di TLS che utilizzano il Record Protocol TLS, ed è anche il più semplice. Questo protocollo è costituito da un singolo messaggio di un singolo byte con valore a 1. L'unico scopo di questo messaggio è quello di far sì che lo stato in sospeso (pendente) venga copiato nello stato corrente, che aggiorna la suite di cifratura da utilizzare su questa connessione. (a) Fig 6.5

ALERT PROTOCOL

L'**Alert Protocol** viene utilizzato per trasmettere avvisi relativi a TLS alle entità peer. Come con altre applicazioni che utilizzano TLS, i messaggi di avviso vengono compressi e crittografati. Ogni messaggio dell'Alert Protocol consiste di due byte: il primo assume il valore warning (1) o fatal (2) per trasmettere la gravità del messaggio. Se è fatal, TLS interrompe immediatamente la connessione. Altre connessioni nella stessa sessione possono continuare, ma non possono essere stabilite nuove connessioni in questa sessione. Il secondo byte contiene un codice che indica l'avviso specifico. (b) Fig

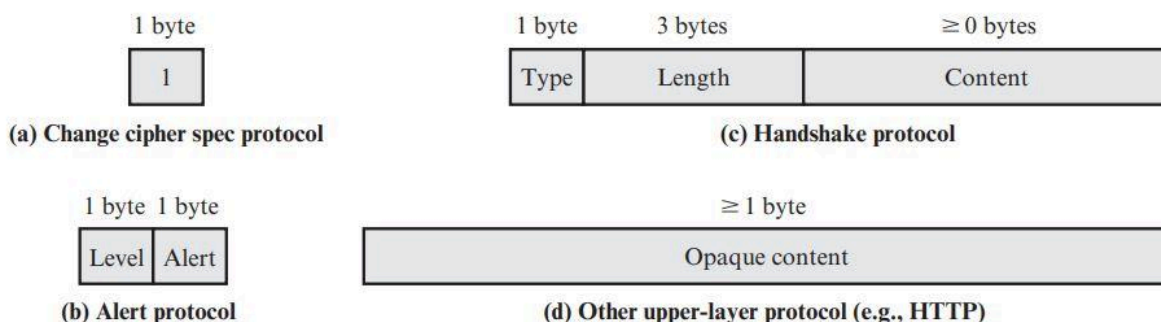


Figure 6.5 TLS Record Protocol Payload

HANDSHAKE PROTOCOL

Handshake Protocol è la parte più complessa di TLS, consente al client e al server di autenticarsi a vicenda, negoziare un algoritmo di crittografia e un algoritmo MAC, negoziare le chiavi crittografiche da utilizzare per proteggere i dati inviati in un record TLS. Handshake viene utilizzato prima della trasmissione di qualsiasi dato dell'applicazione.

Handshake Protocol consiste in una serie di messaggi scambiati tra client e server ((c) Fig 6.5), ognuno di essi contiene:

1. Type (1 byte): indica uno dei 10 possibili tipi di messaggi (Fig 6.2 'Message Type')
2. Length (3 byte): la lunghezza del messaggio in byte
3. Content (≥ 0 byte): i parametri associati al tipo di messaggio inviato (Fig 6.2 'Parameters')

Table 6.2 TLS Handshake Protocol Message Types

Message Type	Parameters
hello_request	null
client_hello	version, random, session id, cipher suite, compression method
server_hello	version, random, session id, cipher suite, compression method
certificate	chain of X.509v3 certificates
server_key_exchange	parameters, signature
certificate_request	type, authorities
server_done	null
certificate_verify	signature
client_key_exchange	parameters, signature
finished	hash value

Lo scambio iniziale necessario per stabilire una connessione logica tra client e server tramite l'**Handshake Protocol** lo si può vedere come suddiviso in **4 fasi**:

FASE 1. Establish Security Capabilities/Stabilire le capacità di sicurezza:

La fase 1 avvia una connessione logica e stabilisce le capacità di sicurezza che saranno associate ad essa. Lo scambio viene avviato dal client, che invia un **messaggio client_hello** con i seguenti parametri:

- **Version**: versione TLS più alta compresa dal client;
- **Random**: una struttura casuale generata dal client costituita da un timestamp a 32 bit e 28 byte generati da un generatore di numeri casuali sicuro. Questi valori fungono da nonce e vengono utilizzati durante lo scambio di chiavi per prevenire attacchi di riproduzione;
- **Session ID**: un identificatore di sessione di lunghezza variabile. Un valore diverso da zero indica che il client desidera aggiornare i parametri di una connessione esistente o creare una nuova connessione su questa sessione. Un valore zero indica che il client desidera stabilire una nuova connessione in una nuova sessione;
- **CipherSuite**: è un elenco che contiene le combinazioni di algoritmi di crittografia supportati dal client, in ordine decrescente di preferenza. Ogni elemento dell'elenco (ogni suite di cifratura) definisce sia un algoritmo di scambio di chiavi che un CipherSpec; questi sono discussi successivamente;
- **Compression method**: è un elenco dei metodi di compressione che il client supporta.

Il client dopo aver inviato il **messaggio client_hello** attende il **messaggio server_hello** che contiene gli stessi parametri di client_hello.

Per il **messaggio server_hello** si applicano le seguenti convenzioni:

1. Version contiene la versione più bassa tra le versioni suggerite dal client e la versione più alta supportata dal server.
2. Random è generato dal server ed è indipendente dal campo random del client.
3. Se Session ID del client era diverso da zero, allora lo stesso valore viene utilizzato dal server; altrimenti il campo Session ID deve contenere il valore per una nuova sessione.
4. CipherSuite contiene la singola suite di cifratura selezionata dal server tra quelle proposte dal client.
5. Compression method contiene il metodo di compressione selezionato dal server tra quelli proposti dal client.

Il primo elemento del parametro Ciphersuite è il **metodo di scambio delle chiavi** (vale a dire, i mezzi con cui le chiavi crittografiche per la crittografia convenzionale e MAC vengono scambiati). Sono supportati i seguenti metodi di scambio di chiavi:

1. **RSA**: la chiave segreta è crittografata con la chiave pubblica RSA del destinatario. Deve essere reso disponibile un certificato di chiave pubblica per la chiave del destinatario.
2. **Fixed Diffie-Hellman**: è uno scambio di chiavi Diffie-Hellman in cui il certificato del server contiene i parametri pubblici Diffie-Hellman firmati dall'ente certificatore (CA). Cioè, il certificato a chiave pubblica contiene i parametri della chiave pubblica Diffie-Hellman. Il cliente fornisce i suoi parametri di chiave pubblica Diffie-Hellman in un certificato, se l'autenticazione del client è richiesta, oppure in un messaggio di scambio di chiavi. Questo metodo si traduce in una chiave segreta fissa tra due peer basata sul calcolo Diffie-Hellman utilizzando le chiavi pubbliche fisse.
3. **Ephemeral Diffie-Hellman**: questa tecnica è usata per creare chiavi segrete effimere ovvero temporanee. In questo caso, le chiavi pubbliche Diffie-Hellman sono scambiate e firmate utilizzando la chiave privata RSA o DSS del mittente. Il destinatario può utilizzare la chiave pubblica corrispondente per verificare la firma. Vengono utilizzati i certificati per autenticare le chiavi pubbliche. Questo sembrerebbe essere il più sicuro delle tre opzioni Diffie-Hellman perché si traduce in una chiave autenticata temporanea.
4. **Anonymous Diffie-Hellman**: viene utilizzato l'algoritmo di base Diffie-Hellman senza autenticazione. Cioè, ciascuna parte invia i propri parametri pubblici Diffie-Hellman all'altra senza autenticazione. Questo approccio è vulnerabile ad attacchi main-in-the-middle, in cui l'attaccante conduce anonymous Diffie-Hellman con entrambe le parti.

Il secondo elemento del parametro CipherSuite è il **CipherSpec** che include:

1. **CipherAlgorithm**: qualsiasi algoritmo di cifratura .. DES, 3DES, IDEA, RC4 ...
2. **MACAlgorithm**: MD5 or SHA-1
3. **CipherType**: flusso o blocco
4. **IsExportable**: vero o falso
5. **HashSize**: 0, 16 (for MD5), or 20 (for SHA-1) bytes
6. **Key Material**: una sequenza di byte che contiene i dati utilizzati per generare le chiavi di scrittura
7. **IV Size**: la dimensione del valore di inizializzazione per la crittografia Cipher Block Chaining (CBC).

FASE 2. Server Authentication and Key Exchange/Autenticazione del server e scambio di chiavi:

Il server inizia questa fase inviando il proprio certificato, tramite il **messaggio certificate**, se necessita di essere autenticato; il messaggio contiene uno o una catena di certificati X.509. Il messaggio **certificate** è necessario per qualsiasi metodo di scambio di chiavi concordato tranne che per anonymous Diffie-Hellman. Se viene utilizzato il fixed Diffie-Hellman allora questo messaggio **certificate** funziona come messaggio di scambio di chiavi del server perché contiene i parametri pubblici Diffie-Hellman del server.

Successivamente, se necessario, invia un messaggio **server_key_exchange**, tale messaggio non è richiesto in due casi: (1) il server ha inviato un certificato con parametri Diffie-Hellman fissi; oppure (2) viene utilizzato lo scambio di chiavi RSA. Il messaggio **server_key_exchange** è necessario per quanto segue:

- anonymous Diffie-Hellman
- ephemeral Diffie-Hellman
- scambio di chiavi RSA (in cui il server utilizza RSA ma dispone di una chiave RSA di sola firma)

Successivamente, un server non anonimo (cioè un server che non utilizza anonymous Diffie-Hellman) può richiedere un certificato al client tramite il messaggio **certificate_request** che include 2 parametri: certificate_type e certificate_authorities. certificate_type indica l'algoritmo a chiave pubblica in uso (RSA, DSS etc.), mentre, il secondo, è una lista dei nomi di enti certificatori accettati.

Il messaggio finale nella fase 2, sempre necessario, è il messaggio **server_done** che viene inviato dal server per indicare la fine dell'hello del server e dei messaggi associati.

FASE 3. Client Authentication and Key Exchange/Autenticazione del client e scambio di chiavi:

Alla ricezione del messaggio **server_done**, il client deve verificare che il server abbia fornito un certificato valido (se richiesto) e verificare che i parametri di **server_hello** siano accettabili. Se ciò che ha ricevuto lo soddisfa, allora il client procede ad inviare uno o più messaggi.

Se il server ha richiesto un certificato, il client inizia questa Fase 3 inviando un **messaggio certificate**. Se non è disponibile alcun certificato adatto, il client invia invece un avviso **no_certificate**.

Dopodiché il client invia il **messaggio client_key_exchange**, che deve essere inviato necessariamente dal client; Il contenuto del messaggio dipende dal tipo di scambio di chiavi, come segue:

1. RSA: il client genera un segreto pre-master di 48 byte e lo cripta con la chiave pubblica dal certificato del server o la chiave RSA temporanea ricevuta da un ***messaggio server_key_exchange***.
2. ephemeral o anonymous Diffie-Hellman: vengono inviati i parametri Diffie-Hellman pubblici del client.
3. fixed Diffie-Hellman: i parametri pubblici Diffie-Hellman del client sono stati inviati in un ***messaggio certificate***, quindi il contenuto di questo messaggio è nullo.

Infine, in questa fase, il client può inviare un ***messaggio certificate_verify*** per fornire una verifica esplicita di un certificato client. Questo messaggio viene inviato solo dopo qualsiasi certificato client con capacità di firma.

FASE 4. Finish/Fine:

La fase 4 completa la configurazione di una connessione sicura. Il client invia un ***messaggio change_cipher_spec*** e copia il CipherSpec in sospeso nel CipherSpec corrente. Si noti che questo messaggio non è considerato parte dell'Handshake Protocol ma viene inviato utilizzando il ***protocollo Change Cipher Spec***. Il client quindi invia immediatamente il ***messaggio finished*** con i nuovi algoritmi, chiavi e segreti. Il messaggio finished verifica che lo scambio di chiavi e i processi di autenticazione abbiano avuto successo.

In risposta a questi due messaggi, il server invia il proprio ***messaggio change_cipher_spec***, trasferisce il messaggio in sospeso al CipherSpec corrente e invia il ***messaggio finished***. A questo punto, l'handshake è completo, il client e il server potrebbero iniziare a scambiare dati a livello di applicazione.

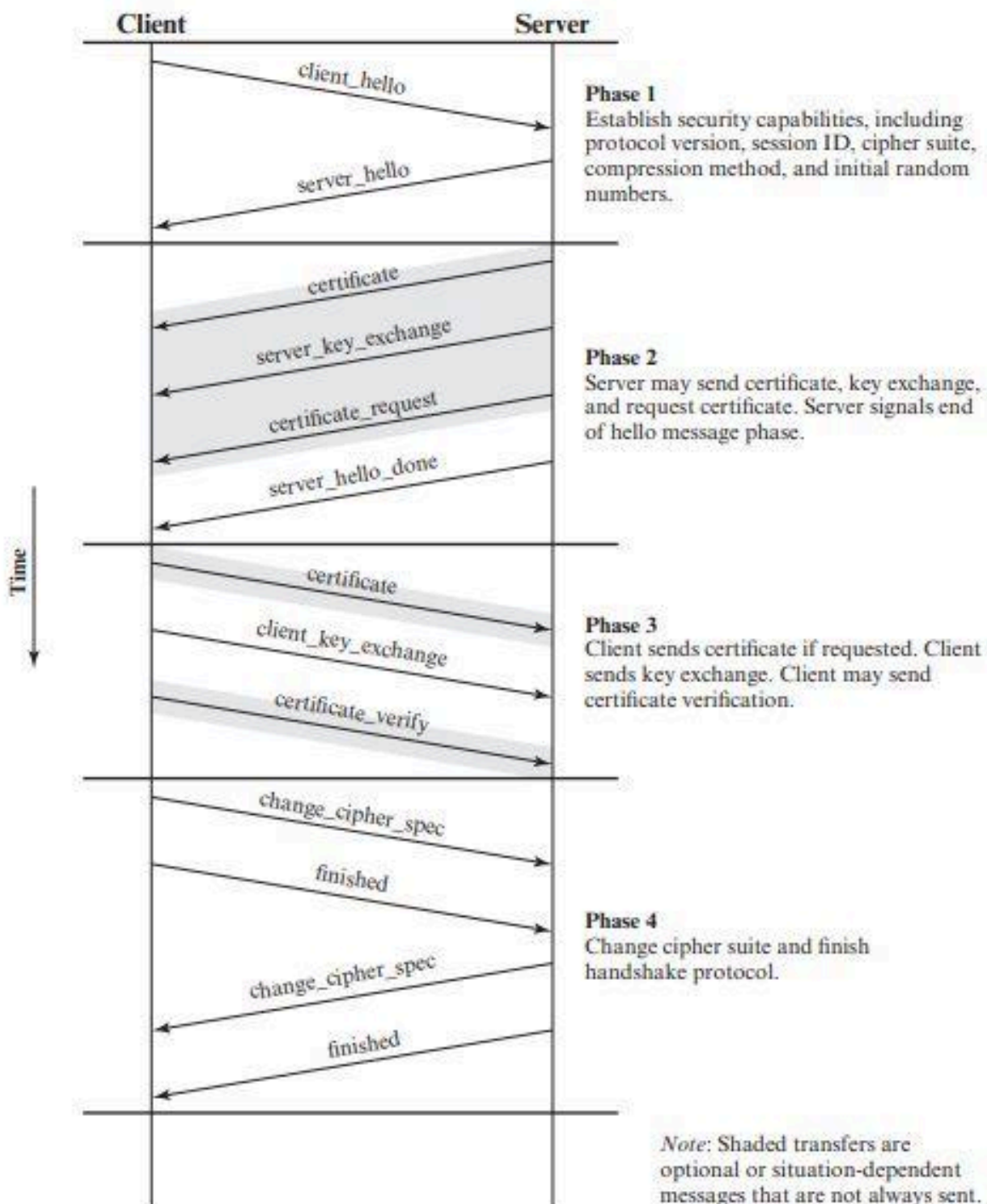


Figure 6.6 Handshake Protocol Action

HEARTBEAT PROTOCOL

Nel contesto delle reti di computer, l'heartbeat è un segnale periodico generato da hardware o software per indicare il normale funzionamento o per sincronizzare altre parti di un sistema. Un protocollo heartbeat viene tipicamente utilizzato per monitorare la disponibilità di un'entità di un protocollo.

L'**HeartBeat Protocol** viene eseguito al di sopra del **Record Protocol TLS** ed è costituito da due tipi di messaggio: **heartbeat_request** e **heartbeat_response**.

L'utilizzo di Heartbeat Protocol viene stabilito durante la Fase 1 dell'Handshake Protocol (Fig 6.6). Ogni peer indica se supporta gli heartbeat. Se gli heartbeat sono supportati, il peer indica se è disposto a ricevere messaggi heartbeat_request e rispondere con messaggi heartbeat_response oppure è soltanto disposto a inviare messaggi heartbeat_request.

L'Heartbeat ha due scopi. In primo luogo, assicura al mittente che il destinatario è ancora vivo, anche se potrebbe non esserci stata alcuna attività sulla connessione TCP sottostante per un po' di tempo. In secondo luogo, l'Heartbeat genera attività attraverso la connessione durante i periodi di inattività, evitando la chiusura da parte di un firewall che non tollera le connessioni inattive.

ATTACCHI SSL/TLS

Dall'introduzione di SSL fino ad arrivare alla standardizzazione di TLS, sono stati ideati numerosi attacchi contro questi protocolli. La comparsa di ciascuno di questi attacchi ha richiesto modifiche al protocollo, agli strumenti di crittografia utilizzati o ad alcuni aspetti dell'implementazione di SSL e TLS per contrastare queste minacce.

Possiamo raggruppare questi attacchi in 4 categorie:

1. Attacchi al Handshake Protocol
2. Attacchi al Record Protocol e agli Application Data Protocol
3. Attacchi alla Public Key Infrastructure
4. Altri attacchi

HTTPS

HTTPS si riferisce alla combinazione di HTTP e SSL per implementare la comunicazione sicura tra un browser Web e un server Web. La funzionalità HTTPS è integrata in tutti i browser Web moderni. Il suo utilizzo dipende dal server Web che supporta la comunicazione HTTPS. Ad esempio, alcuni motori di ricerca non supportano HTTPS.

La principale differenza riscontrata da un utente di un browser Web è che gli indirizzi URL (uniform resource locator) iniziano con **https://** anziché con **http://**. Una normale connessione **HTTP** utilizza la porta **80**. Se viene specificato **HTTPS**, viene utilizzata la porta **443**, che richiama SSL. Quando viene utilizzato HTTPS, i seguenti elementi della comunicazione sono criptati:

- URL della risorsa richiesta
- Contenuto della risorsa
- Contenuto delle form browser (compilati dall'utente del browser)
- Cookie inviati tra browser e server
- Contenuto dell'intestazione HTTP (header HTTP)

HTTP può essere utilizzato sia su SSL che su TLS ed entrambe le implementazioni sono denominate HTTPS.

AVVIO DELLA CONNESSIONE HTTPS

Per HTTPS, l'agente che funge da client HTTP funge anche da client TLS. Il client avvia una connessione al server sulla porta appropriata e quindi invia **client_hello** TLS per iniziare l'**Handshake Protocol** TLS. Quando l'Handshake è terminato, il client può inviare la prima request HTTP. Tutti i dati HTTP devono essere inviati come application data TLS. Dovrebbe essere seguito il normale comportamento HTTP, comprese le connessioni mantenute. Esistono tre livelli di consapevolezza di una connessione in HTTPS. A livello HTTP, un client HTTP richiede una connessione a un server HTTP inviando una richiesta di connessione al successivo livello più basso. In genere, il livello inferiore successivo è TCP, ma potrebbe anche essere TLS/SSL. A livello di TLS, viene stabilita una sessione tra un client TLS e un server TLS. Questa sessione può supportare una o più connessioni in qualsiasi momento. Come abbiamo visto, una richiesta TLS per stabilire una connessione inizia con la creazione di una connessione TCP tra l'entità TCP sul lato client e l'entità TCP sul lato server.

CHIUSURA DELLA CONNESSIONE HTTPS

Un client o un server HTTP possono richiedere la chiusura di una connessione includendo la seguente riga in un record HTTP: 'Connection: close' → ciò indica che la connessione verrà chiusa dopo la consegna di questo record. La chiusura di una connessione HTTPS richiede che TLS chiuda la connessione con l'entità TLS peer sul lato remoto, il che comporterà la chiusura della connessione TCP sottostante. A livello TLS, il modo corretto per chiudere una connessione è che ciascuna parte utilizzi l'Alert Protocol TLS per inviare un **close_notify** alert.

L'implementazione TLS deve avviare uno scambio di avvisi di chiusura prima di chiudere una connessione. Un'implementazione TLS può, dopo aver inviato un avviso di chiusura, chiudere la connessione senza attendere che il peer invii l'avviso di chiusura, generando

una "chiusura incompleta". Si noti che un'implementazione che esegue questa operazione può scegliere di riutilizzare la sessione. Questo dovrebbe essere fatto solo quando l'applicazione sa (in genere attraverso il rilevamento dei limiti dei messaggi HTTP) di aver ricevuto tutti i dati del messaggio che le interessano. I client HTTP devono anche essere in grado di far fronte a una situazione in cui la connessione TCP sottostante viene terminata senza un precedente avviso *close_notify* e senza un indicatore 'Connection: close'. Tale situazione potrebbe essere dovuta a un errore di programmazione sul server o a un errore di comunicazione che causa l'interruzione della connessione TCP. Tuttavia, la chiusura non annunciata del TCP potrebbe essere la prova di una sorta di attacco. Quindi il client HTTPS dovrebbe emettere una sorta di avviso di sicurezza quando ciò si verifica.

SECURE SHELL SSH

SSH Secure Shell è un protocollo per comunicazioni di rete sicure progettato per essere relativamente semplice e poco costoso da implementare. Tale protocollo permette di stabilire una sessione remota cifrata tramite interfaccia a riga di comando con un altro host di una rete informatica. SSH è il protocollo che ha sostituito Telnet.

Le applicazioni client e server SSH sono ampiamente disponibili per la maggior parte dei sistemi operativi. SSH è diventato il metodo preferito per l'accesso remoto e l'X tunneling e sta rapidamente diventando una delle applicazioni più diffuse per la tecnologia di crittografia al di fuori dei sistemi embedded.

SSH è organizzato come [tre protocolli](#) che in genere vengono eseguiti su TCP:

1. **Transport Layer Protocol:** fornisce l'autenticazione del server, la riservatezza dei dati e l'integrità dei dati con forward secrecy (ovvero, se una chiave viene compromessa durante una sessione, la conoscenza non influisce sulla sicurezza delle sessioni precedenti). Lo strato di trasporto può facoltativamente fornire compressione.
2. **User Authentication Protocol:** autentica l'utente sul server.
3. **Connection Protocol:** esegue il multiplexing di più canali di comunicazione logici su una singola connessione SSH sottostante.

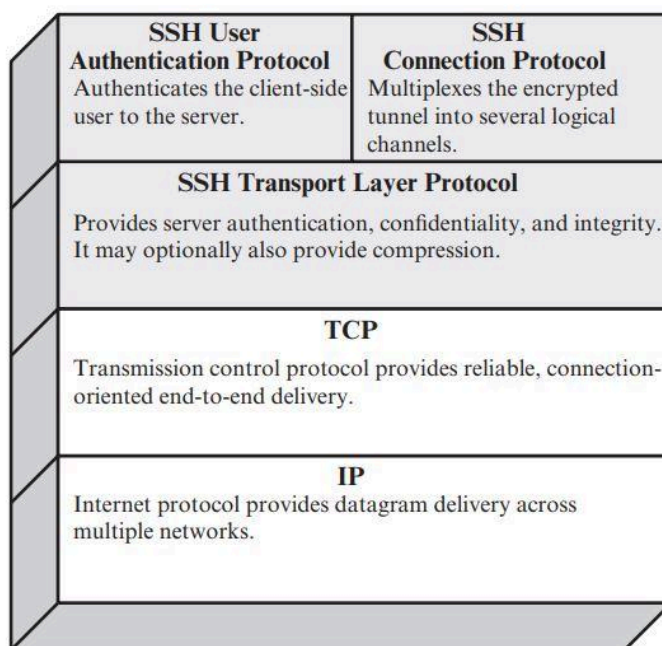


Figure 6.8 SSH Protocol Stack

TRANSPORT LAYER PROTOCOL

HOST KEYS L'autenticazione del server avviene a livello di trasporto, in base alla coppia di chiavi pubblica/privata in possesso dal server. Un server può avere più chiavi host che utilizzano più algoritmi di crittografia asimmetrica diversi. Più host possono condividere la stessa chiave host. In ogni caso, la chiave host del server viene utilizzata durante lo scambio di chiavi per autenticare l'identità dell'host. Perché ciò sia possibile, il client deve conoscere a priori la chiave host pubblica del server. RFC 4251 impone due modelli di fiducia alternativi che possono essere utilizzati:

1. Il client dispone di un database locale che associa ciascun nome host (come digitato dall'utente) alla public host key corrispondente. Questo metodo non richiede alcuna infrastruttura amministrata centralmente e nessun coordinamento di terze parti. Lo svantaggio è che il database delle associazioni nome-key può diventare gravoso da mantenere.
2. L'associazione host name - public key è certificata da un'autorità di certificazione (CA) attendibile. Il client conosce solo la chiave root della CA e può verificare la validità di tutte le chiavi host certificate da CA accettate. Questa alternativa facilita il problema della manutenzione, poiché idealmente, solo una singola chiave CA deve essere archiviata in modo sicuro sul client. D'altra parte, ogni chiave host deve essere opportunamente certificata da un'autorità centrale prima che sia possibile l'autorizzazione.

PACKET EXCHANGE La Fig 6.9 illustra la sequenza di eventi nel Transport Layer Protocol SSH. Innanzitutto, il client stabilisce una connessione TCP al server. Questo viene fatto tramite il protocollo TCP e non fa parte del Transport Layer Protocol. Una volta stabilita la connessione, il client e il server si scambiano dati, denominati *pacchetti*, nel campo dati di un segmento TCP. Ogni pacchetto è nel seguente formato (Figura 6.10):

1. **Packet Length:** lunghezza del pacchetto in byte, esclusi la lunghezza del pacchetto e i campi MAC.
2. **Padding Length:** lunghezza del campo di riempimento casuale.
3. **Payload:** contenuto utile del pacchetto. Prima della negoziazione dell'algoritmo, questo campo non è compresso. Se la compressione viene negoziata, nei pacchetti successivi questo campo viene compresso.
4. **Random Padding:** una volta negoziato un algoritmo di crittografia, questo campo viene aggiunto. Contiene byte casuali di riempimento in modo che la lunghezza totale del pacchetto (escluso il campo MAC) sia un multiplo della dimensione del blocco di cifratura o 8 byte per una cifratura a flusso.
5. **Message Authentication Code:** se l'autenticazione del messaggio è stata negoziata, questo campo contiene il valore MAC. Il valore MAC viene calcolato sull'intero pacchetto più un numero di sequenza, escluso il campo MAC. Il numero di sequenza è una sequenza di pacchetto implicita a 32 bit che viene inizializzata a zero per il primo pacchetto e incrementata per ogni pacchetto. Il numero di sequenza non è incluso nel pacchetto inviato tramite la connessione TCP.

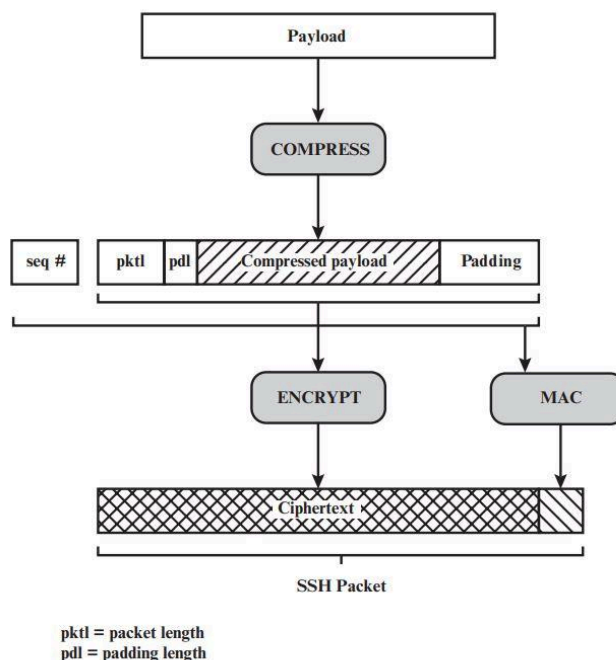


Figure 6.10 SSH Transport Layer Protocol Packet Formation

Una volta negoziato un algoritmo di crittografia, l'intero pacchetto (escluso il campo MAC) viene crittografato dopo il calcolo del valore MAC. Lo scambio di pacchetti SSH Transport Layer consiste in una sequenza di passaggi (Figura 6.9).

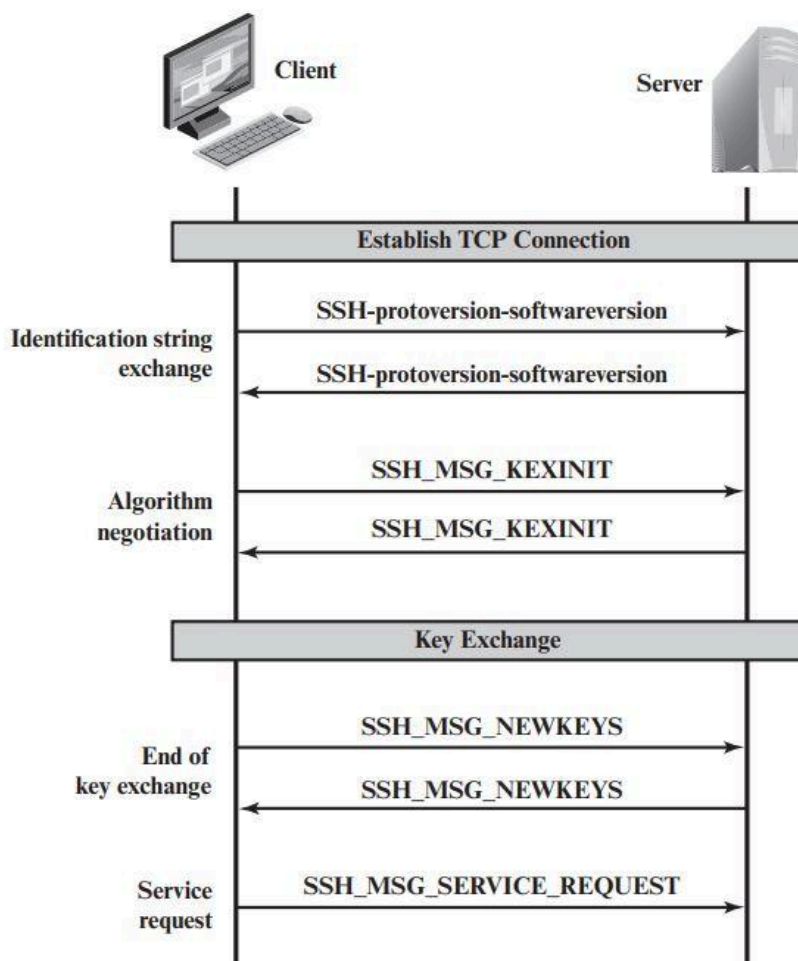


Figure 6.9 SSH Transport Layer Protocol Packet Exchanges

Lo scambio di pacchetti del SSH Transport Layer consiste in una sequenza di passaggi. Il primo passaggio, l' **identification string exchange**, inizia con l'invio da parte del client di un pacchetto con una stringa di identificazione della forma:

SSH-protoversion-softwareversion SP comments CR LF

dove SP, CR e LF sono rispettivamente spazio, ritorno a capo e avanzamento riga.

Il server risponde con la propria stringa di identificazione. Queste stringhe sono utilizzate nello scambio di chiavi Diffie-Hellman.

Lo scambio di pacchetti prosegue con la **negoiazione dell'algoritmo**. Entrambe le parti coinvolte nella comunicazione (client e server), inviano un *SSH_MSG_KEXINIT* contenente un elenco degli algoritmi supportati, in ordine di preferenza, per il mittente del pacchetto. Esiste un elenco per ogni tipo di algoritmo crittografico e gli algoritmi includono: scambio di chiavi, crittografia, MAC algorithm e compression algorithm.

Table 6.3 SSH Transport Layer Cryptographic Algorithms

Cipher		MAC algorithm	
3des-cbc*	Three-key 3DES in CBC mode	hmac-sha1*	HMAC-SHA1; digest length = key length = 20
blowfish-cbc	Blowfish in CBC mode	hmac-sha1-96**	First 96 bits of HMAC-SHA1; digest length = 12; key length = 20
twofish256-cbc	Twofish in CBC mode with a 256-bit key	hmac-md5	HMAC-MD5; digest length = key length = 16
twofish192-cbc	Twofish with a 192-bit key	hmac-md5-96	First 96 bits of HMAC-MD5; digest length = 12; key length = 16
twofish128-cbc	Twofish with a 128-bit key	Compression algorithm	
aes256-cbc	AES in CBC mode with a 256-bit key		
aes192-cbc	AES with a 192-bit key	none*	No compression
aes128-cbc**	AES with a 128-bit key	zlib	Defined in RFC 1950 and RFC 1951
Serpent256-cbc	Serpent in CBC mode with a 256-bit key		
Serpent192-cbc	Serpent with a 192-bit key		
Serpent128-cbc	Serpent with a 128-bit key		
arcfour	RC4 with a 128-bit key		
cast128-cbc	CAST-128 in CBC mode		

* = Required

** = Recommended

Il passo successivo è il **key exchange**. La specifica consente metodi alternativi di scambio di chiavi, ma al momento sono specificate solo due versioni di Diffie-hellman. Entrambe le versioni sono definite nel RFC 2409 e richiedono l'invio di un solo pacchetto in ciascuna direzione. Il risultato dello scambio di chiavi tramite Diffie-Hellman è che le due parti condividono la stessa chiave master K. Inoltre, il server è stato autenticato presso il client, perché il server, durante lo scambio, ha usato la sua chiave privata per firmare la sua metà dello scambio Diffie-Hellman.

La **fine dello scambio di chiavi** è segnalata dall'invio del pacchetto *SSH_MSG_NEWKEYS_REQUEST*. A questo punto entrambe le parti possono iniziare ad usare le chiavi generate a partire da K.

Il passaggio finale è la **richiesta di servizio**. Il client invia un pacchetto `SSH_MSG_SERVICE_REQUEST` per richiedere l'autenticazione utente o il protocollo di connessione. Successivamente, tutti i dati vengono scambiati come payload di un SSH transport layer packet, protetto da cifratura e da MAC.

USER AUTHENTICATION PROTOCOL

Il protocollo di autenticazione utente fornisce i mezzi con il quale il client viene autenticato al server.

TIPI E FORMATI DEI MESSAGGI Nel protocollo di autenticazione utente vengono utilizzati tre tipi di messaggi:

- **SSH_MSG_USERAUTH_REQUEST (50)** con il seguente formato:

<i>byte</i>	<i>SSH_MSG_USERAUTH_REQUEST (50)</i>
<i>string</i>	<i>user name</i>
<i>string</i>	<i>service name</i>
<i>string</i>	<i>method name</i>
<i>...</i>	<i>method specific fields</i>

dove *user name* è l'identità di autorizzazione che il client sta rivendicando, *service name* è la struttura a cui il client sta richiedendo l'accesso (in genere il protocollo di connessione SSH) e *method name* è il metodo di autenticazione utilizzato in questa richiesta. Il primo byte ha valore decimale 50, che viene interpretato come `SSH_MSG_USERAUTH_REQUEST`.

- Se il server rifiuta la richiesta o accetta la richiesta ma richiede uno o più metodi di autenticazione aggiuntivi, il server invia un messaggio **SSH_MSG_USERAUTH_FAILURE (51)** con il seguente formato:

<i>byte</i>	<i>SSH_MSG_USERAUTH_FAILURE (51)</i>
<i>name-list</i>	<i>authentications that can continue</i>
<i>boolean</i>	<i>partial success</i>

dove *name-list* è un elenco di metodi che possono continuare in modo produttivo il dialogo.

- Se il server accetta l'autenticazione, invia un messaggio a byte singolo: **SSH_MSG_USERAUTH_SUCCESS (52)**.

SCAMBIO DI MESSAGGI

1. Il client invia `SSH_MSG_USERAUTH_REQUEST` con un metodo richiesto 'none' (nessuno).
2. Il server verifica se il nome utente è valido. In caso contrario, il server restituisce `SSH_MSG_USERAUTH_FAILURE` con il valore di `partial success` a `false`. Se il nome utente è valido, il server procede al passaggio 3.
3. Il server restituisce `SSH_MSG_USERAUTH_FAILURE (51)` con una lista di uno o più metodi di autenticazione da utilizzare.
4. Il client seleziona uno dei metodi di autenticazione accettabili e invia un `SSH_MSG_USERAUTH_REQUEST` con quel nome di metodo e i campi specifici del metodo richiesti. A questo punto, potrebbe esserci una sequenza di scambi per eseguire il metodo.
5. Se l'autenticazione ha esito positivo e sono richiesti più metodi di autenticazione, il server procede al passaggio 3, utilizzando un valore di `partial success` pari a `true`. Se l'autenticazione fallisce, il server procede al passaggio 3, utilizzando un valore di successo parziale di `false`.
6. Quando tutti i metodi di autenticazione richiesti hanno esito positivo, il server invia un messaggio `SSH_MSG_USERAUTH_SUCCESS` e il protocollo di autenticazione è terminato.

METODI DI AUTENTICAZIONE Il server potrebbe richiedere uno o più metodi di autenticazione tra i seguenti:

1. **publickey:** I dettagli di questo metodo dipendono dall'algoritmo a chiave pubblica scelto. In sostanza, il client invia un messaggio al server che contiene la chiave pubblica del client e il messaggio firmato con la chiave privata del client. Quando il server riceve questo messaggio, controlla se la chiave fornita è accettabile per l'autenticazione e, in tal caso, controlla se la firma è corretta.
2. **password:** il client invia un messaggio contenente una password in chiaro, che è protetta dalla crittografia del Transport Layer Protocol.
3. **hostbased:** L'autenticazione viene eseguita sull'host del client piuttosto che sul client stesso. Pertanto, un host che supporta più client fornirebbe l'autenticazione per tutti i suoi client. Questo metodo funziona chiedendo al client di inviare una firma creata con la chiave privata dell'host del client. Pertanto, invece di verificare direttamente l'identità dell'utente, il server SSH verifica l'identità dell'host client e quindi crede all'host quando afferma che l'utente si è già autenticato sul lato client.

CONNECTION PROTOCOL

Connection Protocol viene eseguito sopra il Transport Layer Protocol e assume che sia in uso una connessione di autenticazione sicura. Tale connessione sicura, denominata **tunnel**, è usata dal Connection Protocol per moltiplicare un numero di canali logici.

MECCANISMO DEL CANALE Tutti i tipi di comunicazione che utilizzano SSH, come una sessione terminale, sono supportati utilizzando canali separati. Entrambe le parti possono aprire un canale. Per ogni canale, ciascuna parte associa un numero di canale univoco, che non deve necessariamente essere lo stesso su entrambe le estremità. I canali sono a flusso controllato utilizzando un meccanismo a finestra. Nessun dato può essere inviato a un canale finché non viene ricevuto un messaggio per indicare che lo spazio della finestra è disponibile.

La vita di un canale procede attraverso **tre fasi**: apertura di un canale, trasferimento dei dati e chiusura di un canale. Quando una delle 2 parti vuole **aprire un canale**, assegna un numero locale al canale e invia un messaggio del tipo:

```
byte          SSH_MSG_CHANNEL_OPEN
string        channel type
uint32        sender channel
uint32        initial window size
uint32        maximum packet size
....          channel type specific data follows
```

dove *uint32* significa intero senza segno a 32 bit, *channel type* identifica l'applicazione per questo canale, come descritto di seguito, *sender channel* è il numero del canale locale, *initial window size* specifica quanti byte di dati del canale possono essere inviati al mittente di questo messaggio senza regolare la finestra, *maximum packet size* specifica la dimensione massima di un singolo pacchetto di dati che può essere inviato al mittente. Ad esempio, si potrebbe voler utilizzare pacchetti più piccoli per connessioni interattive per ottenere una migliore risposta interattiva su collegamenti lenti.

Se il lato remoto è in grado di aprire il canale, restituisce *SSH_MSG_CHANNEL_OPEN_CONFIRMATION*, che include il numero del canale del mittente, il numero del canale del destinatario e i valori delle dimensioni della finestra e del pacchetto per il traffico in entrata.

In caso contrario, il lato remoto restituisce un messaggio *SSH_MSG_CHANNEL_OPEN_FAILURE* con un codice di errore che indica il motivo dell'errore.

Una volta aperto un canale, il **trasferimento dei dati** viene eseguito utilizzando un messaggio *SSH_MSG_CHANNEL_DATA*, che include il numero del canale del destinatario e un blocco di dati. Questi messaggi, in entrambe le direzioni, possono continuare finché il canale è aperto. Quando una delle parti desidera **chiudere un canale**, invia un *SSH_MSG_CHANNEL_CLOSE*, che include il numero del canale del destinatario. La figura 6.11 fornisce un esempio di scambio di messaggi del Connection Protocol.

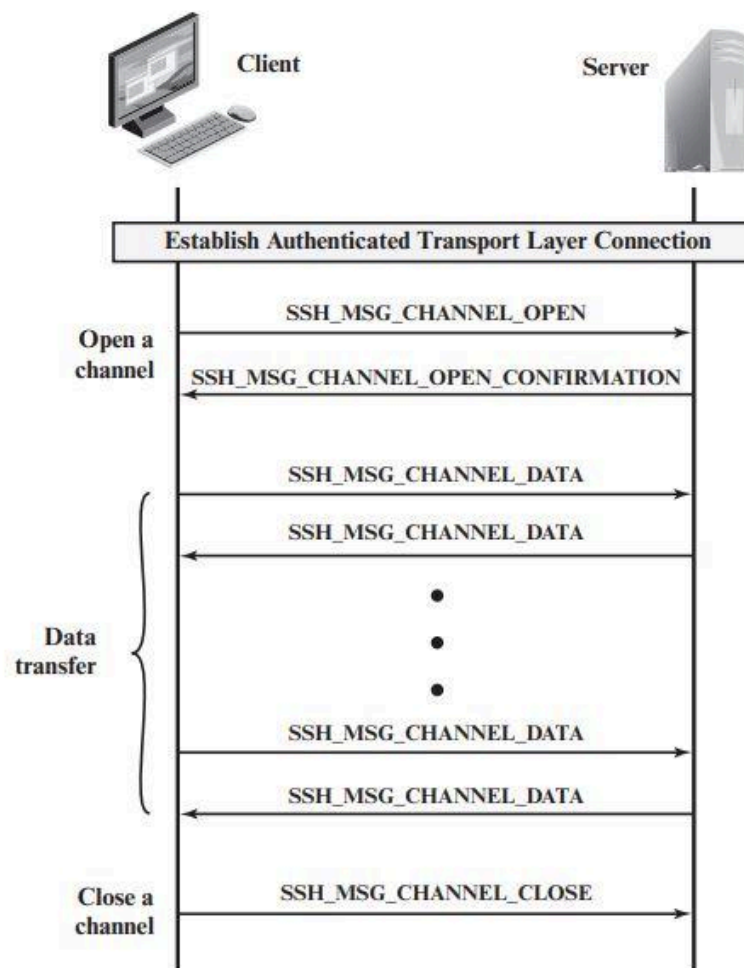


Figure 6.11 Example of SSH Connection Protocol Message Exchange

TIPI DI CANALE Nella specifica del Connection Protocol SSH sono riconosciuti **4** tipi di canali:

1. **session**: l'esecuzione remota di un programma. Il programma può essere una shell, un'applicazione come il trasferimento di file o la posta elettronica, un comando di sistema o un sottosistema integrato. Una volta aperto un canale di sessione, le richieste successive vengono utilizzate per avviare il programma remoto.
2. **x11**: questo si riferisce al sistema X Windows, un sistema software per computer e un protocollo di rete che fornisce un'interfaccia utente grafica (GUI) per i computer in rete. X consente alle applicazioni di essere eseguite su un server di rete ma di essere visualizzate su un computer desktop.
3. **forwarded-tcpip**: questo è il port forwarding remoto, come spiegato nella sottosezione successiva.
4. **direct-tcpip**: questo è il port forwarding locale, come spiegato nella sottosezione successiva.

PORT FORWARDING Una delle funzionalità più utili di SSH è il port forwarding. In sostanza, il port forwarding offre la possibilità di convertire qualsiasi connessione TCP non sicura in una connessione SSH sicura. Questo è anche noto come **tunneling SSH**. Dobbiamo sapere cos'è una **port** in questo contesto. Una **port** è un identificatore di un utente di TCP. Quindi, qualsiasi applicazione che gira su TCP ha un numero di porta. Il traffico TCP in entrata viene consegnato all'applicazione appropriata in base al numero di porta. Un'applicazione può utilizzare più numeri di porta.

Ad esempio, per il protocollo SMTP (Simple Mail Transfer Protocol), il lato server è generalmente in ascolto sulla porta 25, quindi una richiesta SMTP in entrata utilizza TCP e indirizza i dati alla porta di destinazione 25. TCP riconosce che questo è l'indirizzo del server SMTP e instrada il dati all'applicazione del server SMTP.

La **Fig 6.12** illustra il concetto alla base del port forwarding. Abbiamo un'applicazione client identificata dal numero di porta **x** e un'applicazione server identificata dal numero di porta **y**. Ad un certo punto, l'applicazione client richiama l'entità TCP locale e richiede una connessione al server remoto sulla porta **y**. L'entità TCP locale negozia una connessione TCP con l'entità TCP remota, in modo tale che la connessione colleghi la porta locale **x** alla porta remota **y**. **Per proteggere questa connessione**, SSH è configurato in modo che il **Transport Layer Protocol SSH** stabilisca una connessione TCP tra il *client SSH* e le entità server, rispettivamente con i numeri di porta TCP **a** e **b**.

Su questa connessione TCP viene stabilito un **tunnel SSH sicuro**. Il traffico dal client sulla porta **x** viene reindirizzato all'entità SSH locale e viaggia attraverso il tunnel raggiungendo l'entità SSH remota che consegna i dati all'applicazione server sulla porta **y**. Il traffico nell'altra direzione viene reindirizzato in modo simile.

SSH supporta due tipi di port forwarding: **local forwarding** e **remote forwarding**. **LOCAL FORWARDING** consente al client di impostare un processo di "hijacker" (dirottatore). Questo intercetterà il traffico selezionato a livello di applicazione e lo indirizzerà da una connessione TCP non protetta a un tunnel SSH sicuro. SSH è configurato per l'ascolto su porte selezionate. SSH afferra tutto il traffico utilizzando una porta selezionata e lo invia attraverso un tunnel SSH. Dall'altra parte, il server SSH invia il traffico in entrata alla porta di destinazione dettata dall'applicazione client.

L'esempio seguente dovrebbe aiutare a chiarire l'inoltro locale. Supponiamo di avere un client di posta elettronica sul desktop e di utilizzarlo per ricevere la posta dal server di posta tramite Post Office Protocol (POP). Il numero di porta assegnato per POP3 è la porta 110. Possiamo proteggere questo traffico nel modo seguente:

1. Il client SSH stabilisce una connessione al server remoto.
2. Selezionare un numero di porta locale inutilizzato, ad esempio 9999, e configurare SSH in modo che accetti il traffico da questa porta destinato alla porta 110 sul server.

3. Il client SSH informa il server SSH di creare una connessione alla destinazione, in questo caso la porta del server di posta 110.
4. Il client prende tutti i bit inviati alla porta locale 9999 e li invia al server all'interno della sessione SSH crittografata. Il server SSH decifra i bit in entrata e invia il testo in chiaro alla porta 110.
5. Nella direzione opposta, il server SSH prende tutti i bit ricevuti sulla porta 110 e li invia all'interno della sessione SSH al client, che li decrittografa e li invia al processo connesso alla porta 9999.

Con il [REMOTE FORWARDING](#), il client SSH dell'utente agisce per conto del server. Il client riceve il traffico con un determinato numero di porta di destinazione, colloca il traffico sulla porta corretta e lo invia alla destinazione scelta dall'utente. Un tipico esempio di inoltrato remoto è il seguente. Desideri accedere a un server al lavoro dal tuo computer di casa. Poiché il server di lavoro è protetto da un firewall, non accetterà una richiesta SSH dal tuo computer di casa. Tuttavia, dal lavoro puoi configurare un tunnel SSH utilizzando l'inoltrato remoto. Ciò comporta i seguenti passaggi:

1. Dal computer di lavoro, imposta una connessione SSH al tuo computer di casa. Il firewall lo consentirà, poiché si tratta di una connessione in uscita protetta.
2. Configurare il server SSH per l'ascolto su una porta locale, ad esempio 22, e per fornire i dati attraverso la connessione SSH indirizzata alla porta remota, ad esempio 2222.
3. È ora possibile accedere al computer di casa e configurare SSH per accettare il traffico su porta 2222.
4. Ora si dispone di un tunnel SSH che può essere utilizzato per l'accesso remoto al server di lavoro.

CAPITOLO 8

In tutti gli ambienti distribuiti, la posta elettronica è divenuta l'applicazione di rete più utilizzata e questo ha fatto aumentare significativamente la domanda di servizi di autenticazione e riservatezza.

ARCHITETTURA DELLA POSTA ELETTRONICA

Per comprendere gli argomenti di questo capitolo è bene prima capire l'architettura della posta elettronica, attualmente definita in RFC 5598.

Al suo livello più fondamentale, l'architettura della posta Internet è costituita da un **mondo utente** sotto forma di Message User Agents (MUA) e il **mondo di trasferimento**, sotto forma di Message Handling Service (MHS), che è composto da Message Transfer Agents (MTA). L'**MHS** accetta un messaggio da un utente e lo consegna a uno o più utenti, creando un ambiente di scambio virtuale da MUA-to-MUA.

COMPONENTI DELLA POSTA ELETTRONICA:

1. **Message User Agent (MUA)**: MUA è il rappresentante dell'utente all'interno del servizio di posta elettronica, ed opera per conto dell'utente o applicazioni utente. In genere questa funzione è ospitata nel computer dell'utente e viene definita programma di posta elettronica client o server di posta elettronica di rete locale. L'autore MUA formatta un messaggio ed esegue l'invio iniziale all'MHS tramite un MSA. Il MUA del destinatario elabora la posta ricevuta per l'archiviazione e/o la visualizzazione all'utente del destinatario.
2. **Mail Submission Agent (MSA)**: MSA accetta il messaggio inviato da un MUA e applica le politiche del dominio di hosting e i requisiti degli standard Internet. Questa funzione può trovarsi insieme al MUA o come modello funzionale separato. In quest'ultimo caso, viene utilizzato il Simple Mail Transfer Protocol (**SMTP**) tra MUA e MSA.
3. **Message Transfer Agent (MTA)**: MTA inoltra e riceve la posta tra MTA. È come uno switch o un router IP, in quanto il suo compito è effettuare valutazioni di instradamento e spostare il messaggio più vicino ai destinatari. L'inoltro viene eseguito da una sequenza di MTA finché il messaggio non raggiunge un MDA di destinazione. Un MTA aggiunge anche informazioni di traccia all'intestazione del messaggio. L'**SMTP** viene utilizzato tra MTA e tra un MTA e un MSA o MDA.
4. **Mail Delivery Agent (MDA)**: MDA è responsabile del trasferimento del messaggio dal MHS al MS.
5. **Message Store (MS)**: un MUA può impiegare un MS a lungo termine. Un MS può trovarsi su un server remoto o sulla stessa macchina del MUA. In genere, un MUA recupera i messaggi da un server remoto utilizzando **POP** (Post Office Protocol) o **IMAP** (Internet Message Access Protocol).

Altri due concetti devono essere definiti:

1. Un **Administrative Management Domain (ADMD)** è un provider di posta elettronica Internet. Gli esempi includono un reparto che gestisce un inoltro di posta locale (MTA), un reparto IT che gestisce un inoltro di posta aziendale e un ISP che gestisce un servizio di posta elettronica condiviso pubblico. Ogni ADMD può avere diverse politiche operative e processi decisionali basati sulla fiducia. Un esempio ovvio è la distinzione tra posta scambiata all'interno di un'organizzazione e posta scambiata tra organizzazioni indipendenti. Le regole per la gestione dei due tipi di traffico tendono ad essere piuttosto diverse.
2. Il **Domain Name System (DNS)** è un servizio di ricerca di directory che fornisce una mappatura tra il nome di un host su Internet e il suo indirizzo numerico IP. Il DNS viene discusso successivamente in questo capitolo.

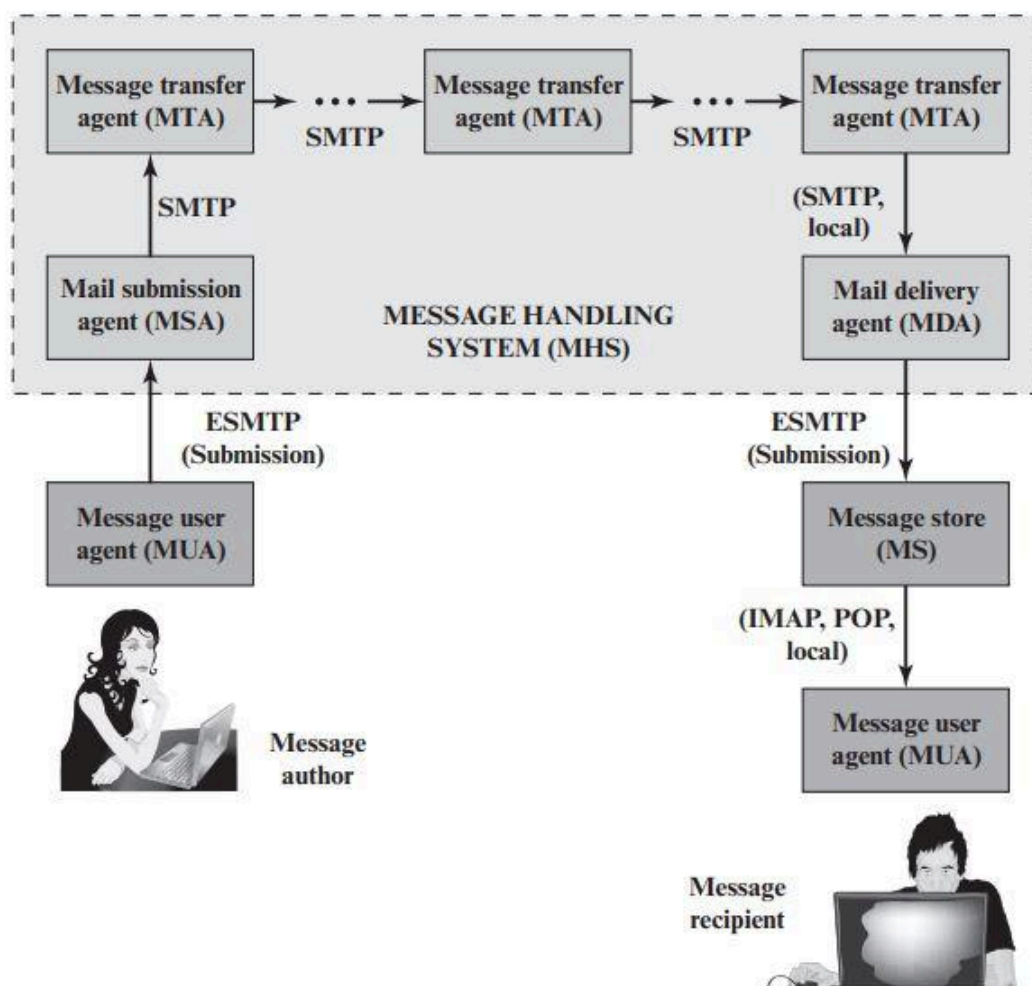


Figure 8.1 Function Modules and Standardized Protocols Used between them in the Internet Mail Architecture

PROTOCOLLI DI POSTA ELETTRONICA

Per il trasferimento della posta elettronica vengono utilizzati due tipi di protocolli. Il primo tipo viene utilizzato per spostare i messaggi attraverso Internet dall'origine alla destinazione. Il protocollo utilizzato a tale scopo è **SMTP**, con varie estensioni e in alcuni casi restrizioni. Il secondo tipo è costituito da protocolli utilizzati per trasferire messaggi tra server di posta, ovvero protocolli di accesso alla posta, di cui **IMAP** e **POP** sono i più comunemente utilizzati.

Processo invio posta elettronica (da Wikipedia):

Una mail è inviata da un client di posta (MUA) ad un server mail (MSA), usando SMTP attraverso TCP sulla porta 587. La maggior parte dei provider di posta tuttavia permettono ancora l'invio sulla tradizionale porta 25. L'MSA consegna la mail al MTA. Spesso MSA e MTA sono istanze dello stesso software, avviato con opzioni diverse sulla stessa macchina. Oltre che con un classico client, un account di posta in uscita con protocollo SMTP può essere configurato in qualsiasi applicazione software che possa inviare posta elettronica.

Il MTA utilizza il DNS per trovare il record MX di uno specifico dominio (la parte dell'indirizzo dopo il carattere @). Il record MX contiene il nome dell'host target. Basandosi sul nome dell'host e su altre informazioni, il MTA sceglie un server e si connette come SMTP client.

Il trasferimento di messaggi può avvenire in una singola connessione tra due MTA o attraverso una serie di salti attraverso diversi sistemi intermediari. Un server SMTP può essere destinatario del messaggio o intermediario (compiendo operazioni di store and forward) o gateway (può trasmettere il messaggio utilizzando anche altri protocolli, oltre a SMTP). Ogni salto è un trasferimento formale di responsabilità sul messaggio, per cui ciascun server che riceve il messaggio deve consegnarlo o segnalare un errore.

Una volta che il server destinatario accetta il messaggio in arrivo, lo consegna ad un mail delivery agent (MDA), che lo salva in una mailbox per la consegna locale.

Una volta consegnato al mail server, il messaggio è memorizzato per poter esser recuperato da un determinato mail client autenticato (MUAs). La mail è recuperata attraverso applicazioni sul dispositivo dell'utente, chiamate **mail client**, usando il protocollo **Internet Message Access Protocol** (IMAP), che facilita sia l'accesso alle mail sia gestisce le mail immagazzinate, o **Post Office Protocol** (POP) che tipicamente usa il tradizionale formato mbox per le mail, o un sistema proprietario come per esempio Microsoft Exchange/Outlook o Lotus Notes/Domino.

SIMPLE MAIL TRANSFER PROTOCOL (SMTP)

SMTP incapsula un messaggio di posta elettronica in una busta e viene utilizzato per inoltrare tali messaggi incapsulati dall'origine alla destinazione tramite più MTA. SMTP è stato originariamente specificato nel 1982 come RFC 821 e ha subito diverse revisioni, la più recente delle quali è RFC 5321 (ottobre 2008). Queste revisioni hanno aggiunto ulteriori comandi e introdotto estensioni. Il termine Extended SMTP (ESMTP) viene spesso utilizzato per fare riferimento a queste versioni successive di SMTP.

SMTP è un protocollo **client-server** basato su testo in cui il client (mittente di posta elettronica) contatta il server (destinatario al passo successivo) ed emette una serie di comandi per comunicare al server il messaggio da inviare, poi invia il messaggio stesso. La maggior parte di questi comandi sono **messaggi di testo ASCII** inviati dal client e un **codice di ritorno risultante** (e testo ASCII aggiuntivo) restituito dal server.

Il trasferimento di un messaggio da un'origine alla sua destinazione finale può avvenire tramite una singola conversazione client/server SMTP su una singola connessione TCP. In alternativa, un server SMTP può essere un **relay intermedio** che assume il ruolo di un client SMTP dopo aver ricevuto un messaggio e quindi lo inoltra a un server SMTP lungo un percorso verso la destinazione finale.

Il funzionamento di SMTP consiste in una serie di comandi e risposte scambiate tra il mittente e il destinatario SMTP. L'iniziativa è del mittente SMTP, che stabilisce la connessione TCP. Una volta stabilita la connessione, il mittente SMTP invia i comandi tramite la connessione al destinatario. Ogni comando è costituito da una singola riga di testo, che inizia con un **codice di comando di quattro lettere** seguito in alcuni casi da un campo di argomento. Ogni comando genera esattamente una risposta dal destinatario SMTP. La maggior parte delle risposte sono su una sola riga, anche se sono possibili risposte su più righe. Ogni risposta inizia con un **codice a tre cifre** e può essere seguita da informazioni aggiuntive.

```

S: 220 foo.com Simple Mail Transfer Service Ready
C: HELO bar.com
S: 250 OK
C: MAIL FROM:<Smith@bar.com>
S: 250 OK
C: RCPT TO:<Jones@foo.com>
S: 250 OK
C: RCPT TO:<Green@foo.com>
S: 550 No such user here
C: RCPT TO:<Brown@foo.com>
S: 250 OK
C: DATA
S: 354 Start mail input; end with <crlf>.<crlf>
C: Blah blah blah . . .
C: . . . etc. etc. etc.
C: <crlf><crlf>
S: 250 OK
C: QUIT
S: 221 foo.com Service closing transmission channel

```

Figure 8.2 Example SMTP Transaction Scenario

La figura precedente illustra lo scambio SMTP tra un client (C) e un server (S). Lo scambio inizia con il client che stabilisce una connessione TCP alla porta TCP 25 sul server (non mostrato in figura). Ciò fa sì che il server attivi SMTP e invii una risposta **220** al client. Il comando **HELO** identifica il dominio di invio, che il server riconosce e accetta con una risposta **250**. Il mittente SMTP sta trasmettendo la posta originata dall'utente Smith@bar.com. Il comando **MAIL** identifica l'originatore del messaggio. Il messaggio è indirizzato a tre utenti sulla macchina foo.com, ovvero Jones, Green e Brown. Il client identifica ognuno di questi in un comando **RCPT** separato. Il ricevitore SMTP indica di avere caselle postali per Jones e Brown ma non ha informazioni su Green. Poiché almeno uno dei destinatari previsti è stato verificato, il client procede all'invio del messaggio di testo, inviando prima un comando **DATA** per assicurarsi che il server sia pronto per i dati. Dopo che il server ha riconosciuto la ricezione di tutti i dati, emette un messaggio **250 OK**. Quindi il client emette un comando **QUIT** e il server chiude la connessione.

Un'importante estensione relativa alla sicurezza per SMTP, denominata **STARTTLS**, è definita in RFC 3207 (SMTP Service Extension for Secure SMTP over Transport Layer Security, febbraio 2002). STARTTLS consente l'aggiunta di riservatezza e autenticazione nello scambio tra agenti SMTP. Ciò offre agli agenti SMTP la possibilità di proteggere alcune o tutte le loro comunicazioni da intercettazioni e aggressori. Se il client avvia la connessione su una porta abilitata per TLS (ad esempio, la porta 465 è stata precedentemente utilizzata per SMTP su SSL), il server potrebbe richiedere un messaggio che indica che l'opzione STARTTLS è disponibile. Il client può quindi emettere il comando STARTTLS nel flusso di

comandi SMTP e le due parti procedono a stabilire una connessione TLS sicura. Un vantaggio dell'utilizzo di STARTTLS è che il server può offrire il servizio SMTP su una singola porta, piuttosto che richiedere numeri di porta separati per operazioni sicure e in chiaro. Meccanismi simili sono disponibili per l'esecuzione di TLS su protocolli IMAP e POP.

Storicamente, i trasferimenti di messaggi MUA/MSA hanno utilizzato SMTP. Lo standard attualmente preferito è **SUBMISSION**, definito in RFC 6409 (Message Submission for Mail, novembre 2011). Sebbene SUBMISSION derivi da SMTP, utilizza una porta TCP separata e impone requisiti distinti, come l'autorizzazione all'accesso.

MAIL ACCESS PROTOCOLS (POP, IMAP)

Post Office Protocol (POP3) consente a un client di posta elettronica (MUA) di scaricare un'e-mail da un server di posta elettronica (MTA). Gli agenti utente POP3 si connettono tramite TCP al server (in genere la porta **110**). L'agente utente inserisce un nome utente e una password (memorizzati internamente per comodità o inseriti ogni volta dall'utente per una maggiore sicurezza). Dopo l'autorizzazione, il MUA può emettere comandi POP3 per recuperare ed eliminare la posta.

Come con POP3, anche IMAP (Internet Mail Access Protocol) consente a un client di posta elettronica di accedere alla posta su un server di posta elettronica. IMAP utilizza anche TCP, con la porta TCP del server **143**. IMAP è più complesso di POP3, fornisce un'autenticazione più forte rispetto a POP3 e fornisce altre funzioni non supportate da POP3.

FORMATI E-MAIL

Per comprendere **S/MIME**, dobbiamo prima avere una comprensione generale del formato di posta elettronica sottostante che utilizza, vale a dire **MIME**. Ma per comprendere il significato di MIME, dobbiamo tornare allo standard del formato di posta elettronica tradizionale, **RFC 822**, che è ancora di uso comune. La versione più recente di questa specifica di formato è **RFC 5322** (Internet Message Format, ottobre 2008).

RFC 5322

RFC 5322 definisce un formato per i messaggi di testo inviati tramite posta elettronica. È stato lo standard per i messaggi di posta di testo basati su Internet e rimane di uso comune. Nel contesto RFC 5322, i messaggi vengono visualizzati come aventi una **busta** e un **contenuto**. La busta contiene tutte le informazioni necessarie per eseguire la trasmissione e la consegna. Il contenuto compone l'oggetto da consegnare al destinatario. Lo standard RFC 5322 si applica solo ai contenuti. Tuttavia, lo standard di contenuto

include un insieme di campi di intestazione che possono essere utilizzati dal sistema di posta per creare la busta e lo standard ha lo scopo di facilitare l'acquisizione di tali informazioni da parte dei programmi.

La struttura generale di un messaggio conforme a RFC 5322 è molto semplice. Un messaggio è costituito da un certo numero di righe di intestazione (**header**) seguite da testo senza restrizioni (**body**). L'intestazione è separata dal corpo da una riga vuota. In altre parole, un messaggio è un testo ASCII e si presume che tutte le righe fino alla prima riga vuota siano righe di intestazione utilizzate dalla parte dell'agente utente del sistema di posta.

Una riga di intestazione di solito consiste in **una parola chiave**, seguita da **due punti**, seguita dagli **argomenti della parola chiave**; il formato consente di suddividere una lunga riga in più righe. Le parole chiave utilizzate più di frequente sono *From*, *To*, *Subject* e *Date*.

Date: October 8, 2009 2:15:49 PM EDT

From: "William Stallings"

Subject: The Syntax in RFC 5322

To: Smith@Other-host.com

Cc: Jones@Yet-Another-Host.com

//riga vuota per dividere header dal body

Hello. This section begins the actual message body, which is delimited from the message heading by a blank line.

Un altro campo che si trova comunemente nelle intestazioni RFC 5322 è *Message-ID*. Questo campo contiene un identificatore univoco associato a questo messaggio.

MULTIPURPOSE INTERNET MAIL EXTENSIONS (MIME)

Multipurpose Internet Mail Extension (MIME) è un'estensione di RFC 5322 che ha lo scopo di risolvere alcuni dei problemi e delle limitazioni dell'uso di **SMTP** (o di qualche altro protocollo di trasferimento della posta) e **RFC 5322** per la posta elettronica. Gli RFC da 2045 a 2049 definiscono MIME e da allora ci sono stati numerosi documenti di aggiornamento.

Limitazioni dello schema SMTP/RFC 5322:

1. SMTP non può trasmettere file eseguibili o altri oggetti binari.

2. SMTP non può trasmettere dati di testo che includono caratteri della lingua nazionale, poiché questi sono rappresentati da codici a 8 bit con valori di 128 decimali o superiori e SMTP è limitato a ASCII a 7 bit.
3. I server SMTP possono rifiutare messaggi di posta elettronica superiori a una certa dimensione.
4. I gateway SMTP che convertono tra ASCII e il codice carattere EBCDIC non utilizzano un insieme coerente di mappature, con conseguenti problemi di traduzione.
5. I gateway SMTP verso le reti di posta elettronica X.400 non possono gestire i dati non testuali inclusi nei messaggi X.400
6. Alcune implementazioni SMTP non aderiscono completamente agli standard SMTP definiti in RFC 821. I problemi comuni includono:
 - Eliminazione, aggiunta o riordino di ritorno a capo e avanzamento riga;
 - Troncamento o ritorno a capo di righe più lunghe di 76 caratteri;
 - Rimozione dello spazio bianco finale (caratteri di tabulazione e spazio);
 - Imbottitura di righe in un messaggio della stessa lunghezza;
 - Conversione di caratteri di tabulazione in più caratteri di spazio.

MIME ha lo scopo di risolvere questi problemi in modo compatibile con le implementazioni RFC 5322 esistenti.

Panoramica:

La specifica MIME include i seguenti elementi:

1. Vengono definiti ***cinque nuovi campi di intestazione del messaggio***, che possono essere inclusi in un'intestazione RFC 5322. Questi campi forniscono informazioni sul corpo del messaggio.
2. Vengono definiti ***numerosi formati di contenuto***, standardizzando così le rappresentazioni che supportano la posta elettronica multimediale.
3. Vengono definite ***codifiche di trasferimento*** che consentono la conversione di qualsiasi formato di contenuto in una forma protetta dall'alterazione da parte del sistema di posta.

Campi di intestazione definiti in MIME:

I 5 campi di intestazione definiti in MIME sono:

- ***MIME-Version***: deve avere il valore del parametro 1 o 0; questo campo indica che il messaggio è conforme alle RFC 2045 e 2046.
- ***Content-Type***: descrive i dati contenuti nel body con dettagli sufficienti affinché l'agente utente ricevente possa scegliere un agente o un meccanismo appropriato per rappresentare i dati all'utente o altrimenti trattare i dati in modo appropriato.

- **Content-Transfer-Encoding:** indica il tipo di trasformazione utilizzato per rappresentare il body del messaggio in un modo accettabile per il trasporto della posta.
- **Content-ID:** utilizzato per identificare entità MIME in modo univoco in più contesti.
- **Content-Description:** una descrizione testuale dell'oggetto con il body; questo è utile quando l'oggetto non è leggibile (ad esempio, dati audio).

Uno o tutti questi campi possono apparire in una normale intestazione RFC 5322. Un'implementazione conforme deve supportare i campi MIME-Version, Content-Type e Content-Transfer-Encoding; i campi Content-ID e Content-Description sono facoltativi e possono essere ignorati dall'implementazione del destinatario.

Tipi di contenuti MIME:

La maggior parte delle specifiche MIME riguarda la definizione di una varietà di tipi di contenuto. Ciò riflette la necessità di fornire *modalità standardizzate per trattare un'ampia varietà di rappresentazioni di informazioni in un ambiente multimediale*. La Tabella sottostante elenca i tipi di contenuto specificati in RFC 2046. Esistono 7 diversi tipi principali di contenuto e un totale di 15 sottotipi.

In generale, un **tipo** di contenuto dichiara il tipo generale di dati e il **sottotipo** specifica un formato particolare per quel tipo di dati.

TIPO	SOTTOTIPO	DESCRIZIONE
Text	<i>Plain</i>	Testo non formattato, può essere ASCII o ISO 8859
	<i>Enriched</i>	Fornisce una maggiore flessibilità di formato.
Multipart	<i>Mixed</i>	Le diverse parti sono indipendenti ma devono essere trasmesse insieme. Devono essere presentate al destinatario nell'ordine in cui appaiono nel messaggio di posta.
	<i>Parallel</i>	Differisce da Mixed solo per il fatto che non è definito alcun ordine per la consegna delle parti al destinatario.
	<i>Alternative</i>	Le diverse parti sono versioni alternative delle stesse informazioni. Sono ordinati in modo sempre più fedele all'originale e il sistema di posta del destinatario dovrebbe mostrare all'utente la versione "migliore".

	<i>Digest</i>	Simile a Mixed, ma il tipo/sottotipo predefinito di ciascuna parte è message/rfc822.
Message	<i>rfc822</i>	Il body è esso stesso un messaggio incapsulato conforme a RFC 822.
	<i>Partial</i>	Utilizzato per consentire la frammentazione di elementi di posta di grandi dimensioni, in modo trasparente per il destinatario.
	<i>External-body</i>	Contiene un puntatore a un oggetto che esiste altrove.
Image	<i>jpeg</i>	L'immagine è in formato JPEG, codifica JFIF.
	<i>gif</i>	L'immagine è in formato GIF.
Video	<i>mpeg</i>	Formato MPEG.
Audio	<i>Basic</i>	Codifica m-law ISDN a 8 bit a canale singolo a una frequenza di campionamento di 8 kHz.
Application	<i>PostScript</i>	Formato Adobe Postscript.
	<i>octet-stream</i>	Dati binari generali costituiti da byte a 8 bit.

(per vedere specifiche dettagliate da pagina 262 a 264 [non necessario credo])

Codifiche di trasferimento MIME:

L'obiettivo è fornire una consegna affidabile nella più ampia gamma di ambienti.

Lo standard MIME definisce due metodi di codifica dei dati.

Il campo **Content-Transfer-Encoding** può effettivamente assumere **sei** valori, come elencato nella successiva tabella. Tuttavia, tre di questi valori (**7 bit**, **8 bit** e **binary**) indicano che non è stata eseguita alcuna codifica ma forniscono alcune informazioni sulla natura dei dati. Per il trasferimento SMTP, è sicuro utilizzare il modulo a 7 bit. Le forme a **8 bit** e **binary** possono essere utilizzate in altri contesti di trasporto della posta. Un altro valore Content-Transfer-Encoding è **x-token**, che indica che viene utilizzato un altro schema di codifica per il quale deve essere fornito un nome. Questo potrebbe essere uno schema specifico del fornitore o dell'applicazione. I due schemi di codifica effettivi definiti sono

quoted-printable e *base64*. Sono definiti due schemi per fornire una scelta tra una tecnica di trasferimento che sia essenzialmente leggibile dall'uomo e una che sia sicura per tutti i tipi di dati in modo ragionevolmente compatto.

La codifica di trasferimento *quoted-printable* è utile quando i dati consistono in gran parte di ottetti che corrispondono a caratteri ASCII stampabili. In sostanza, rappresenta i caratteri non sicuri mediante la rappresentazione esadecimale del loro codice e introduce interruzioni di riga reversibili (soft) per limitare le righe del messaggio a 76 caratteri.

La codifica di trasferimento *base64*, nota anche come codifica *radix-64*, è comune per la codifica di dati binari arbitrari in modo tale da essere invulnerabile all'elaborazione da parte dei programmi di trasporto della posta.

Table 8.2 MIME Transfer Encodings

7 bit	The data are all represented by short lines of ASCII characters.
8 bit	The lines are short, but there may be non-ASCII characters (octets with the high-order bit set).
binary	Not only may non-ASCII characters be present but the lines are not necessarily short enough for SMTP transport.
quoted-printable	Encodes the data in such a way that if the data being encoded are mostly ASCII text, the encoded form of the data remains largely recognizable by humans.
base64	Encodes data by mapping 6-bit blocks of input to 8-bit blocks of output, all of which are printable ASCII characters.
x-token	A named nonstandard encoding.

A multipart example:

La Figura 8.3, tratta da RFC 2045, è lo schema di un complesso messaggio multiparte. Il messaggio ha **5** parti da visualizzare in serie: 2 parti introduttive di testo normale, 1 messaggio Multipart incorporato, 1 parte richtext e 1 messaggio di testo di chiusura incapsulato in un set di caratteri non ASCII. Il messaggio Multipart incorporato ha 2 parti da visualizzare in parallelo: un'immagine e un frammento audio.

```

MIME-Version: 1.0
From: Nathaniel Borenstein <nsb@bellcore.com>
To: Ned Freed <ned@innosoft.com>
Subject: A multipart example
Content-Type: multipart/mixed;
    boundary=unique-boundary-1
This is the preamble area of a multipart message. Mail readers that
understand multipart format should ignore this preamble. If you are reading
this text, you might want to consider changing to a mail reader that
understands how to properly display multipart messages.

--unique-boundary-1
    . . . Some text appears here . . .
[Note that the preceding blank line means no header fields were given and
this is text, with charset US ASCII. It could have been done with explicit
typing as in the next part.]

--unique-boundary-1
Content-type: text/plain; charset=US-ASCII
This could have been part of the previous part, but illustrates explicit
versus implicit typing of body parts.

--unique-boundary-1
Content-Type: multipart/parallel; boundary=unique-boundary-2

--unique-boundary-2
Content-Type: audio/basic
Content-Transfer-Encoding: base64
    . . . base64-encoded 8000 Hz single-channel mu-law-format audio data goes
here . . . .

--unique-boundary-2
Content-Type: image/jpeg
Content-Transfer-Encoding: base64
    . . . base64-encoded image data goes here . . . .

--unique-boundary-2-
--unique-boundary-1
Content-type: text/enriched

This is richtext. as defined in RFC 1896

    Isn't it cool?

--unique-boundary-1
Content-Type: message/rfc822

From: (mailbox in US-ASCII)
To: (address in US-ASCII)
Subject: (subject in US-ASCII)
Content-Type: Text/plain; charset=ISO-8859-1
Content-Transfer-Encoding: Quoted-printable

    . . . Additional text in ISO-8859-1 goes here . . .

--unique-boundary-1-

```

Canonical Form:

Un concetto importante in MIME e S/MIME è quello di *forma canonica*. La forma canonica è un formato, appropriato al tipo di contenuto, standardizzato per l'uso tra sistemi. Ciò è in contrasto con la forma nativa, che è un formato che può essere peculiare di un particolare sistema. RFC 2049 definisce queste due forme come segue:

- *Forma nativa*: il corpo da trasmettere viene creato nel formato nativo del sistema. Viene utilizzato il set di caratteri nativo e, ove appropriato, vengono utilizzate anche le convenzioni di fine riga locali. Il corpo può essere qualsiasi formato che corrisponda al modello locale per la rappresentazione di qualche forma di informazione. Gli esempi includono un file di testo in stile UNIX, un'immagine raster Sun o un file indicizzato VMS e dati audio in un formato dipendente dal sistema archiviato solo nella memoria. In sostanza, i dati vengono creati nella forma nativa che corrisponde al tipo specificato dal tipo di supporto.
- *Forma canonica*: l'intero corpo, comprese le informazioni fuori banda come le lunghezze dei record e possibilmente le informazioni sugli attributi dei file, viene convertito in una forma canonica universale. Il tipo di supporto specifico del body così come i suoi attributi associati determinano la natura della forma canonica utilizzata. La conversione nella forma canonica corretta può comportare la conversione del set di caratteri, la trasformazione di dati audio, la compressione o varie altre operazioni specifiche per i vari tipi di media.

E-MAIL THREATS E PROTEZIONE COMPLETA DELLA POSTA ELETTRONICA

Sia per le organizzazioni che per gli individui, la posta elettronica è molto diffusa e particolarmente vulnerabile a un'ampia gamma di minacce alla sicurezza. In termini generali, le minacce alla sicurezza della posta elettronica possono essere classificate come segue:

1. **Minacce relative all'autenticità**: potrebbero causare l'accesso non autorizzato al sistema di posta elettronica di un'azienda. (Authenticity-related threats)
2. **Minacce relative all'integrità**: potrebbero comportare la modifica non autorizzata del contenuto della posta elettronica. (Integrity-related threats)
3. **Minacce relative alla riservatezza**: potrebbero comportare la divulgazione non autorizzata di informazioni sensibili. (Confidentiality-related threats)
4. **Minacce relative alla disponibilità**: potrebbero impedire agli utenti finali di inviare o ricevere e-mail. (Availability-related threats)

Un utile elenco di minacce e-mail specifiche, insieme agli approcci alla mitigazione, è fornito in SP 800-177 (Trustworthy E-mail, settembre 2015) ed è mostrato nella **Tabella 8.3**.

SP 800-177 raccomanda l'uso di una varietà di protocolli standardizzati come mezzo per contrastare queste minacce. Questi includono:

- **STARTTLS**: un'estensione di sicurezza SMTP che fornisce autenticazione, integrità, non ripudio (tramite firme digitali) e riservatezza (tramite crittografia) per l'intero messaggio SMTP eseguendo SMTP su TLS.
- **S/MIME**: fornisce autenticazione, integrità, non ripudio (tramite firme digitali) e riservatezza (tramite crittografia) del corpo del messaggio contenuto nei messaggi SMTP.
- **Estensioni di sicurezza DNS (DNSSEC)**: fornisce l'autenticazione e la protezione dell'integrità dei dati DNS ed è uno strumento sottostante utilizzato da vari protocolli di sicurezza della posta elettronica.
- **Autenticazione delle entità denominate basata su DNS (DANE)**: è progettato per superare i problemi nel sistema dell'autorità di certificazione (CA) fornendo un canale alternativo per l'autenticazione delle chiavi pubbliche basato su DNSSEC, con il risultato che le stesse relazioni di fiducia utilizzate per certificare gli indirizzi IP vengono utilizzate per certificare i server che operano su tali indirizzi .
- **Sender Policy Framework (SPF)**: utilizza il Domain Name System (DNS) per consentire ai proprietari di domini di creare record che associano il nome di dominio a uno specifico intervallo di indirizzi IP di mittenti di messaggi autorizzati. È una cosa semplice per i destinatari controllare il record SPF TXT nel DNS per confermare che il presunto mittente di un messaggio è autorizzato a utilizzare quell'indirizzo di origine e rifiutare la posta che non proviene da un indirizzo IP autorizzato.
- **DomainKeys Identified Mail (DKIM)**: consente a un MTA di firmare le intestazioni selezionate e il corpo di un messaggio. Ciò convalida il dominio di origine della posta e fornisce l'integrità del corpo del messaggio.
- **Domain-based Message Authentication, Reporting, and Conformance (DMARC)**: consente ai mittenti di conoscere l'efficacia proporzionata dei loro criteri SPF e DKIM e segnala ai destinatari quale azione deve essere intrapresa in vari scenari di attacco individuale e di massa.

Tabella 8.3

Tabella 8.3 Minacce e mitigazioni della posta elettronica

Minaccia	Impatto su presunto Mittente	Impatto sul ricevitore	Mitigazione
E-mail inviata da MTA non autorizzato in azienda (ad es. botnet malware)	Perdita di reputazione, e-mail valide dall'azienda possono essere bloccate come possibile attacco di spam/phishing.	UBE e/o e-mail contenenti collegamenti dannosi possono essere recapitati nelle caselle di posta degli utenti.	Implementazione di tecniche di autenticazione basate sul dominio. Uso delle firme digitali tramite posta elettronica.
Messaggio di posta elettronica inviato utilizzando un dominio di invio contraffatto o non registrato	Perdita di reputazione, e-mail valide dall'azienda possono essere bloccate come possibile attacco di spam/phishing.	UBE e/o e-mail contenenti collegamenti dannosi possono essere recapitati nelle caselle di posta degli utenti.	Implementazione dell'autenticazione basata sul dominio. Uso delle firme digitali tramite posta elettronica.
Messaggio di posta elettronica inviato utilizzando un indirizzo di invio o un indirizzo di posta elettronica contraffatto (ad es. phishing, spear phishing)	Perdita di reputazione, e-mail valide dall'azienda possono essere bloccate come possibile attacco di spam/phishing.	Potrebbero essere recapitati UBE e/o e-mail contenenti collegamenti dannosi. Gli utenti possono divulgare inavvertitamente informazioni sensibili o PII.	Implementazione di tecniche di autenticazione basate sul dominio. Uso delle firme digitali tramite posta elettronica.
E-mail modificata in transito	Fuga di informazioni sensibili o PII.	Perdita di informazioni sensibili, il messaggio alterato può contenere informazioni dannose.	Utilizzo di TLS per crittografare il trasferimento di posta elettronica tra server. Utilizzo della crittografia e-mail end-to-end.
Divulgazione di informazioni sensibili (ad es. PII) tramite il monitoraggio e l'acquisizione del traffico di posta elettronica	Fuga di informazioni sensibili o PII.	Perdita di informazioni sensibili, il messaggio alterato può contenere informazioni dannose.	Utilizzo di TLS per crittografare il trasferimento di posta elettronica tra server. Utilizzo della crittografia e-mail end-to-end.
E-mail di massa non richieste (UBE) (ovvero spam)	Nessuno, a meno che il presunto mittente non sia falsificato.	UBE e/o e-mail contenenti collegamenti dannosi possono essere recapitati nelle caselle di posta degli utenti.	Tecniche da affrontare UBE.
Attacco DoS/DDoS contro i server di posta elettronica di un'azienda	Impossibilità di inviare e-mail.	Impossibilità di ricevere e-mail.	Più server di posta, utilizzo di provider di posta elettronica basati su cloud.



Tale figura mostra come questi componenti interagiscono per fornire l'autenticità e l'integrità del messaggio. Non mostrato, per semplicità, è che S/MIME fornisce anche la riservatezza dei messaggi crittografando i messaggi.

Tale figura mostra come questi componenti interagiscono per fornire l'autenticità e l'integrità del messaggio. Non mostrato, per semplicità, è che S/MIME fornisce anche la riservatezza dei messaggi crittografando i messaggi.

S/MIME

Secure/Multipurpose Internet Mail Extension (S/MIME) è un miglioramento della sicurezza dello standard del formato di posta elettronica Internet MIME; basato sulla tecnologia di RSA Data Security. S/MIME è una funzionalità complessa definita in numerosi documenti.

Descrizione operativa

S/MIME fornisce quattro servizi relativi ai messaggi: **autenticazione**, **confidenzialità**, **compressione** e **compatibilità e-mail** (Tabella successiva).

FUNZIONE	ALGORITMO TIPICO	TIPICA AZIONE
<u>Firma digitale</u> (Autenticazione)	<i>RSA/SHA-256</i>	Viene creato un codice hash di un messaggio utilizzando SHA-256. Questo digest del messaggio viene crittografato utilizzando SHA-256 con la chiave privata del mittente e incluso nel messaggio.
<u>Crittografia dei messaggi</u> (Confidenzialità)	<i>AES-128 with CBC</i>	Un messaggio viene crittografato utilizzando AES-128 con CBC con una chiave di sessione monouso generata dal mittente. La chiave di sessione viene crittografata tramite RSA con la chiave pubblica del destinatario e inclusa nel messaggio.
<u>Compressione</u>	<i>non specificato</i>	Un messaggio può essere compresso per l'archiviazione o la trasmissione.
<u>Compatibilità e-mail</u>	<i>Radix-64 conversion</i>	Per fornire trasparenza alle applicazioni di posta elettronica, un messaggio crittografato può essere convertito in una stringa ASCII utilizzando la conversione radix-64.

Esaminiamo più in dettaglio queste capacità:

AUTENTICAZIONE

L'autenticazione è fornita per mezzo della *firma digitale*, utilizzando lo schema generale discusso nel Capitolo 3. Più comunemente viene utilizzato *RSA con SHA-256*. La sequenza è la seguente:

1. Il mittente crea il messaggio;
2. Viene utilizzato *SHA-256* per generare un messaggio *digest a 256 bit* del messaggio.
3. Il *digest* del messaggio viene crittografato con RSA utilizzando la chiave privata del mittente e il risultato viene aggiunto al messaggio. Sono inoltre aggiunte informazioni di identificazione per il firmatario, che consentiranno al destinatario di recuperare la chiave pubblica del firmatario.
4. Il destinatario utilizza RSA con la chiave pubblica del mittente per decrittografare e recuperare il digest del messaggio.
5. Il destinatario genera un nuovo messaggio digest per il messaggio e lo confronta con il codice hash decrittografato. Se i due corrispondono, il messaggio viene accettato come autentico.

La combinazione di SHA-256 e RSA fornisce un efficace schema di firma digitale. A causa della forza di **RSA** → il destinatario ha la certezza che solo il possessore della chiave privata corrispondente può generare la firma. Grazie alla forza di **SHA-256** → il destinatario ha la certezza che nessun altro potrebbe generare un nuovo messaggio che corrisponda al codice hash e, quindi, alla firma del messaggio originale.

CONFIDENZIALITA'

S/MIME fornisce riservatezza/confidenzialità *crittografando i messaggi*. Più comunemente viene utilizzato *AES con una chiave a 128 bit*, con la modalità Cipher Block Chaining (CBC). Anche la chiave stessa è crittografata, in genere con RSA, come spiegato di seguito. Come sempre, bisogna affrontare il problema della distribuzione delle chiavi. In S/MIME, ciascuna chiave simmetrica, denominata *content-encryption key*, viene utilizzata una sola volta. In altre parole, viene generata una nuova chiave come numero casuale per ogni messaggio. Poiché deve essere utilizzata solo una volta, la content-encryption key è associata al messaggio e trasmessa con esso. Per proteggere la chiave, viene crittografata con la chiave pubblica del destinatario. La sequenza può essere descritta come segue:

1. Il mittente genera il messaggio e un numero casuale a 128 bit da utilizzare come content-encryption key solo per questo messaggio.
2. Il messaggio viene crittografato utilizzando la content-encryption key.
3. La content-encryption key viene crittografata con RSA utilizzando la chiave pubblica del destinatario ed è allegata al messaggio.
4. Il destinatario utilizza RSA con la sua chiave privata per decrittografare e recuperare la content-encryption key.
5. La content-encryption key viene utilizzata per decrittografare il messaggio.

CONFIDENZIALITA' E AUTENTICAZIONE

Per lo stesso messaggio possono essere utilizzate sia la confidenzialità (crittografia) e autenticazione (firma digitale). La figura mostra una sequenza in cui viene generata una firma per il messaggio in chiaro e allegata al messaggio. Quindi il messaggio in chiaro e la firma vengono crittografati come un unico blocco utilizzando la crittografia simmetrica e la chiave di crittografia simmetrica viene crittografata utilizzando la crittografia a chiave pubblica. S/MIME consente di eseguire le operazioni di firma e crittografia dei messaggi in entrambi gli ordini. Se la firma digitale (autenticazione) viene eseguita per prima, l'identità del firmatario viene nascosta dalla crittografia. Se la crittografia (confidenzialità) viene eseguita prima, è possibile verificare una firma senza esporre il contenuto del messaggio. Tuttavia, in questo caso il destinatario non può determinare alcuna relazione tra il firmatario e il contenuto non crittografato del messaggio.

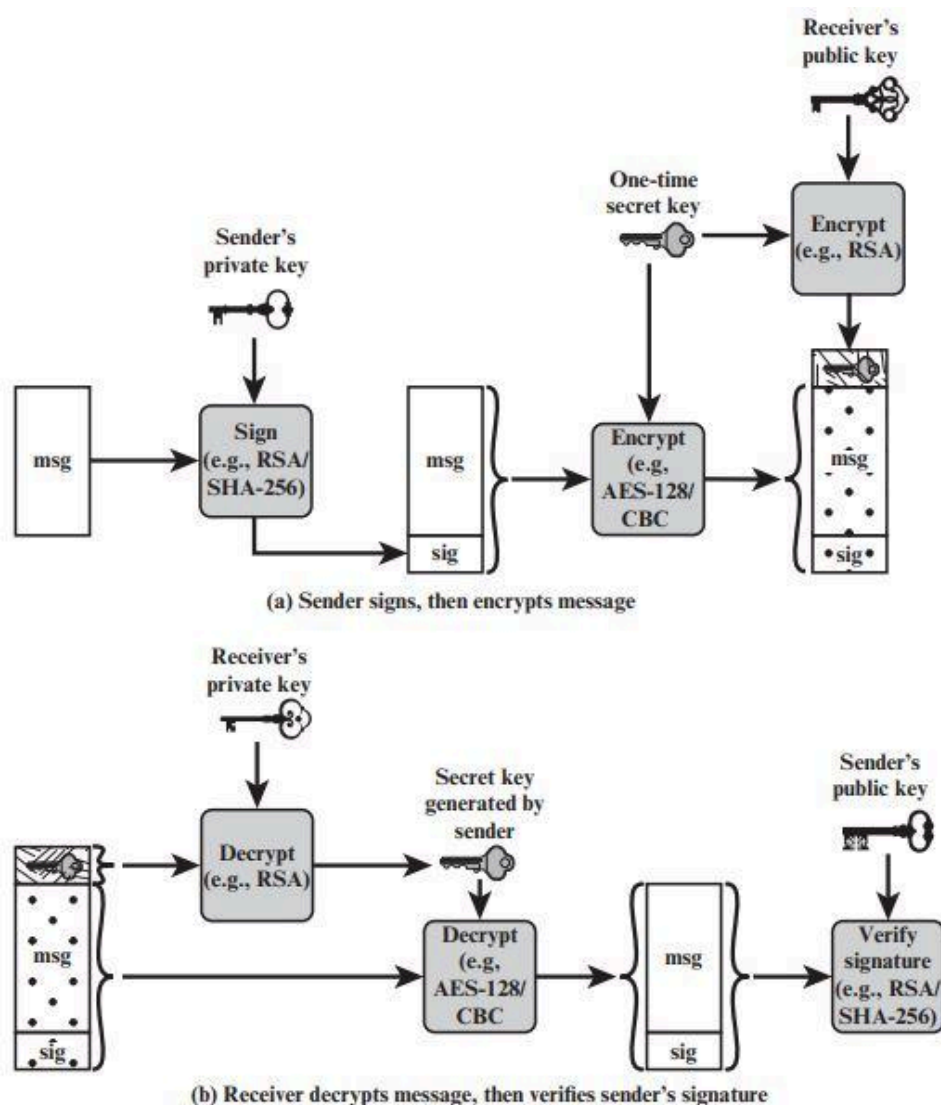


Figure 8.5 Simplified S/MIME Functional Flow

COMPATIBILITA' CON LA POSTA ELETTRONICA

Quando si utilizza S/MIME, almeno una parte del blocco da trasmettere è crittografata. Se viene utilizzato solo il servizio di firma digitale (autenticazione), il *digest* del messaggio viene crittografato (con la chiave privata del mittente). Se si utilizza il servizio di crittografia (confidenzialità), il messaggio più la firma (se presente) sono crittografati (con chiave simmetrica monouso). Pertanto, parte o tutto il blocco risultante è costituito da un flusso di ottetti arbitrari a 8 bit. Tuttavia, molti sistemi di posta elettronica consentono solo l'uso di blocchi costituiti da testo ASCII. Per soddisfare questa restrizione, S/MIME fornisce il servizio di conversione del flusso binario grezzo a 8 bit in un flusso di caratteri ASCII stampabili, un processo denominato *codifica a 7 bit*.

Lo schema tipicamente utilizzato per questo scopo è la conversione **Base64**. Ogni gruppo di tre ottetti di dati binari è mappato in quattro caratteri ASCII.

Un aspetto degno di nota dell'algoritmo Base64 è che *converte ciecamente il flusso di input nel formato Base64 indipendentemente dal contenuto*, anche se l'input è testo ASCII. Pertanto, se un messaggio è firmato ma non crittografato e la conversione viene applicata all'intero blocco, l'output sarà illeggibile per l'osservatore casuale, il che fornisce un certo livello di riservatezza.

COMPRESSIONE

S/MIME offre anche la possibilità di comprimere un messaggio. Ciò ha il vantaggio di risparmiare spazio sia per la trasmissione della posta elettronica che per l'archiviazione dei file. La compressione può essere applicata in qualsiasi ordine rispetto alle operazioni di firma (*autenticazione*) e crittografia dei messaggi (*confidenzialità*). RFC 5751 fornisce le seguenti linee guida:

1. La compressione di dati crittografati con codifica *binary* è sconsigliata, poiché non produrrà una compressione significativa. Tuttavia, i dati crittografati Base64 potrebbero trarne molto vantaggio.
2. Se viene utilizzato un algoritmo di compressione con perdita con la firma digitale, sarà necessario prima comprimere e poi firmare.

Tipi di contenuto dei messaggi S/MIME

S/MIME utilizza i seguenti tipi di contenuto dei messaggi, definiti in RFC 5652, Cryptographic Message Syntax:

1. **Data**: fa riferimento al contenuto interno del messaggio con codifica MIME, che può quindi essere incapsulato in un tipo di contenuto SignedData, EnvelopedData o CompressedData.

2. **SignedData**: utilizzato per applicare una firma digitale a un messaggio.
3. **EnvelopedData**: consiste nel contenuto crittografato di qualsiasi tipo e chiavi di crittografia del contenuto crittografato per uno o più destinatari.
4. **CompressedData**: utilizzato per applicare la compressione dei dati a un messaggio.

Algoritmi crittografici approvati

S/MIME utilizza la seguente terminologia tratta da RFC 2119 (Key Words for use in RFCs to Indicate Requirement Levels, marzo 1997) per specificare il livello dei requisiti:

- **MUST (DEVE)**: la definizione è un requisito assoluto della specifica. Un'implementazione deve includere questa caratteristica o funzione per essere conforme alle specifiche.
- **SHOULD (DOVREBBE)**: potrebbero esistere ragioni valide in circostanze particolari per ignorare questa caratteristica o funzione, ma si consiglia che un'implementazione includa la caratteristica o la funzione.

La specifica S/MIME include una discussione sulla procedura per decidere quale algoritmo di crittografia del contenuto utilizzare. In sostanza, un agente mittente deve prendere due decisioni. Innanzitutto deve determinare se l'agente ricevente è in grado di decrittografare utilizzando un determinato algoritmo di crittografia. In secondo luogo, se l'agente ricevente è in grado di accettare solo contenuti crittografati in modo debole, deve decidere se è accettabile inviare utilizzando una crittografia debole. Per supportare questo processo decisionale, un agente mittente può annunciare le sue capacità di decrittazione in ordine di preferenza per qualsiasi messaggio che invia. Un agente ricevente può memorizzare tali informazioni per un uso futuro.

Table 8.5 Cryptographic Algorithms Used in S/MIME

Function	Requirement
Create a message digest to be used in forming a digital signature.	MUST support SHA-256 SHOULD support SHA-1 Receiver SHOULD support MD5 for backward compatibility
Use message digest to form a digital signature.	MUST support RSA with SHA-256 SHOULD support — DSA with SHA-256 — RSASSA-PSS with SHA-256 — RSA with SHA-1 — DSA with SHA-1 — RSA with MD5
Encrypt session key for transmission with a message.	MUST support RSA encryption SHOULD support — RSAES-OAEP — Diffie-Hellman ephemeral-static mode
Encrypt message for transmission with a one-time session key.	MUST support AES-128 with CBC SHOULD support — AES-192 CBC and AES-256 CBC — Triple DES CBC

Messaggi S/MIME

S/MIME fa uso di una serie di nuovi tipi di contenuto MIME. Tutti i nuovi tipi di applicazione utilizzano la designazione **PKCS**. Questa si riferisce a un insieme di specifiche di crittografia a chiave pubblica emesse da *RSA Laboratories* e rese disponibili per lo sforzo **S/MIME**. Esaminiamo ognuno di questi tipi successivamente, dopo aver esaminato le procedure generali per la preparazione dei messaggi S/MIME.

Protezione di un'entità MIME:

S/MIME protegge un'entità MIME con una firma, crittografia o entrambe. Un'entità MIME può essere un intero messaggio (ad eccezione delle intestazioni RFC 5322) oppure, se il tipo di contenuto MIME è multiparte, un'entità MIME è una o più delle sottoparti del messaggio. L'entità MIME è preparata secondo le normali regole per la preparazione dei messaggi MIME. Quindi l'entità MIME più alcuni dati relativi alla sicurezza, come identificatori di algoritmi e certificati, vengono elaborati da S/MIME per produrre ciò che è noto come **oggetto PKCS**. Un oggetto PKCS viene quindi trattato come contenuto del messaggio e racchiuso in MIME (fornito di intestazioni MIME appropriate). (Questo processo dovrebbe diventare chiaro quando esaminiamo oggetti specifici e forniamo esempi)

In tutti i casi, il messaggio da inviare è convertito in forma canonica. In particolare, per un dato tipo e sottotipo, viene utilizzata la forma canonica appropriata per il contenuto del messaggio. Per un messaggio multiparte, viene utilizzata la forma canonica appropriata per ogni sottoparte.

Ora esaminiamo ciascuno dei tipi di contenuto S/MIME.

Un sottotipo **application/pkcs7-mime** viene utilizzato per una delle quattro categorie di elaborazione S/MIME, ciascuna con un parametro **smime-type** univoco. In tutti i casi, l'entità risultante (indicata come oggetto) è rappresentata in una forma nota come Basic Encoding Rules (BER), definita nella Raccomandazione ITU-T X.209. Il formato BER è costituito da stringhe di ottetti arbitrarie ed è quindi un dato binario. Tale oggetto dovrebbe essere trasferito e codificato con **base64** nel messaggio MIME esterno.

EnvelopedData

Per prima cosa guardiamo *EnvelopedData*. I passaggi per la preparazione di un'entità MIME *EnvelopedData* sono:

1. Genera una chiave di sessione pseudo-casuale per un particolare algoritmo di crittografia simmetrica.
2. Per ogni destinatario, crittografa la chiave di sessione con la rispettiva chiave RSA pubblica del destinatario.

3. Per ogni destinatario, viene preparato un blocco noto come *RecipientInfo* che contiene un identificatore del certificato a chiave pubblica del destinatario, un identificatore dell'algoritmo utilizzato per crittografare la chiave di sessione e la chiave di sessione crittografata.
4. Viene crittografato il contenuto del messaggio con la chiave di sessione.

I blocchi *RecipientInfo* seguiti dal contenuto crittografato costituiscono i dati avvolti/incapsulati. Queste informazioni vengono poi codificate in *base64*. Di seguito viene fornito un messaggio di esempio (escluse le intestazioni RFC 5322).

```
Content-Type: application/pkcs7-mime; smime-type=enveloped-
data; name=smime.p7m
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename=smime.p7m

rfvbnj756tbBghyHhHUujhJhjH77n8HHGT9HG4VQpfyF467GhIGfHfYT6
7n8HHGghyHhHUujhJh4VQpfyF467GhIGfHfYGTTrfvbnjT6jH7756tbB9H
f8HHGTTrfvhJhjH776tbB9HG4VQbnj7567GhIGfHfYT6ghyHhHUujpfyF4
0GhIGfHfQbnj756YT64V
```

Per recuperare il messaggio crittografato, il destinatario rimuove prima la codifica *base64*. Dopodiché il destinatario utilizza la propria chiave privata per recuperare la chiave di sessione. Infine, il contenuto del messaggio viene decifrato con la chiave di sessione.

SignedData

Il smime-type *SignedData* può essere utilizzato con uno o più firmatari. Per chiarezza, limitiamo la nostra descrizione al caso di una singola firma digitale. I passaggi per la preparazione di un'entità MIME *SignedData* sono i seguenti:

1. Seleziona un algoritmo digest del messaggio (SHA o MD5).
2. Calcola il digest del messaggio (**funzione hash**) del contenuto da firmare.
3. Crittografa il digest del messaggio con la chiave privata del firmatario.
4. Prepara un blocco noto come *SignerInfo* che contiene il certificato a chiave pubblica del firmatario, un identificatore dell'algoritmo digest del messaggio, un identificatore dell'algoritmo utilizzato per crittografare il digest del messaggio e il digest del messaggio crittografato.

Queste informazioni vengono poi codificate in *base64*. Un messaggio di esempio è il seguente (escluse le intestazioni RFC 5322):

```

Content-Type: application/pkcs7-mime; smime-type=signed-
data; name=smime.p7m
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename=smime.p7m
567GhIGfHfYT6ghyHhHUujpfyF4f8HHGTrfvhJhjH776tbB9HG4VQbnj7
77n8HHGT9HG4VQpfyF467GhIGfHfYT6rfvbnj756tbBghyHhHUujhJhjH
HUujhJh4VQpfyF467GhIGfHfYGTrfvbnjT6jH7756tbB9H7n8HHGghyHh
6YT64V0GhIGfHfQbnj75

```

Per recuperare il messaggio firmato e verificare la firma, il destinatario rimuove prima la codifica *base64*. Poi utilizza la chiave pubblica del firmatario per decrittografare il *digest* del messaggio. Il destinatario, infine, calcola autonomamente il *digest* del messaggio e lo confronta con il digest del messaggio decifrato per verificare la firma.

Richiesta di Registrazione

In genere, un'applicazione o un utente si rivolgerà a un'autorità di certificazione per un certificato a chiave pubblica. L'entità S/MIME application/pkcs10 viene utilizzata per trasferire una richiesta di certificazione. La richiesta di certificazione include il blocco *CertificationRequestInfo*, seguito da un identificatore dell'algoritmo di crittografia a chiave pubblica, seguito dalla firma del blocco *CertificationRequestInfo*, effettuata utilizzando la chiave privata del mittente. Il blocco *CertificationRequestInfo* include un nome del soggetto del certificato (l'entità la cui chiave pubblica deve essere certificata) e una rappresentazione in formato bit-string della chiave pubblica dell'utente.

Elaborazione del certificato S/MIME

S/MIME utilizza certificati a chiave pubblica conformi alla versione 3 di X.509. I gestori e/o gli utenti S/MIME devono configurare ciascun client con un elenco di chiavi attendibili e con elenchi di revoche di certificati. Cioè, la responsabilità è locale per il mantenimento dei certificati necessari per verificare le firme in entrata e per crittografare i messaggi in uscita. I certificati invece vengono firmati dalle autorità di certificazione.

Ruolo dell'agente utente

Un utente S/MIME deve eseguire diverse funzioni di gestione delle chiavi:

1. Key Generation: DEVE essere in grado di generare coppie di chiavi Diffie-Hellman e DSS separate e DOVREBBE essere in grado di generare coppie di chiavi RSA.

Ciascuna coppia di chiavi DEVE essere generata da una buona fonte di input casuale non deterministico ed essere protetta in modo sicuro.

2. Registration: la chiave pubblica di un utente deve essere registrata presso una CA per ricevere un certificato di chiave pubblica X.509.
3. Archiviazione e recupero dei certificati: un utente richiede l'accesso a un elenco locale di certificati per verificare le firme in entrata e crittografare i messaggi in uscita.

PGP - Pretty Good Privacy

Un protocollo di sicurezza e-mail alternativo è **Pretty Good Privacy** (PGP), che ha essenzialmente le stesse funzionalità di S/MIME. PGP è stato creato da Phil Zimmerman e implementato come prodotto rilasciato per la prima volta nel 1991. È stato reso disponibile gratuitamente ed è diventato molto popolare per uso personale. Il protocollo **PGP** iniziale era proprietario e utilizzava alcuni algoritmi di crittografia con restrizioni sulla proprietà intellettuale. Nel 1996, la versione **5.x** di PGP è stata definita in IETF RFC 1991, PGP Message Exchange Formats. Successivamente, **OpenPGP** è stato sviluppato come un nuovo protocollo standard basato sulla versione 5.x di PGP.

Ci sono **2** differenze significative tra **S/MIME** e **OpenPGP**:

1. Certificazione chiave: S/MIME utilizza certificati X.509 emessi da autorità di certificazione. In OpenPGP, gli utenti generano le proprie chiavi pubbliche e private OpenPGP, e quindi richiedono firme per le loro chiavi pubbliche da individui o organizzazioni a cui sono conosciuti. Mentre i certificati X.509 sono attendibili se esiste una catena PKIX valida a una radice attendibile, una chiave pubblica OpenPGP è attendibile se è firmata da un'altra chiave pubblica OpenPGP considerata attendibile dal destinatario. Questo è chiamato **Web-of-Trust**.
2. Distribuzione della chiave: OpenPGP non include la chiave pubblica del mittente in ogni messaggio, quindi è necessario che i destinatari dei messaggi OpenPGP ottengano separatamente la chiave pubblica del mittente per verificare il messaggio. Molte organizzazioni pubblicano chiavi OpenPGP su siti Web protetti da TLS: le persone che desiderano verificare le firme digitali o inviare a queste organizzazioni posta crittografata devono scaricare manualmente queste chiavi e aggiungerle ai propri client OpenPGP. Le chiavi possono anche essere registrate con i server a chiave pubblica OpenPGP, che sono server che mantengono un database di chiavi pubbliche PGP organizzate per indirizzo e-mail. Chiunque può inviare una chiave pubblica ai server delle chiavi OpenPGP e quella chiave pubblica può contenere qualsiasi indirizzo e-mail. Non esiste alcun controllo delle chiavi OpenPGP, quindi gli utenti devono utilizzare il Web-of-Trust per decidere se fidarsi di una determinata chiave pubblica.

SP 800-177 raccomanda l'uso di **S/MIME** piuttosto che **PGP** a causa della maggiore fiducia nel sistema CA di verifica delle chiavi pubbliche.

DNSSEC

Le estensioni di sicurezza DNS (**DNSSEC**) vengono utilizzate da diversi protocolli che forniscono la sicurezza della posta elettronica. Questa sezione fornisce una breve panoramica del *Domain Name System* (**DNS**) e dopodichè passa ad esaminare **DNSSEC**.

Domain Name System (DNS)

DNS è un servizio di ricerca di directory che fornisce una mappatura tra il nome di un host su Internet e il suo indirizzo IP numerico. Il DNS è essenziale per il funzionamento di Internet. Il DNS viene utilizzato da MUA e MTA per trovare l'indirizzo del server hop successivo per la consegna della posta. Gli MTA di invio interrogano il DNS per il *Mail Exchange Resource Record* (**MX RR**) del dominio del destinatario (il lato destro del simbolo "@") per trovare l'MTA ricevente da contattare.

Sono 4 gli elementi che compongono/comprendono il DNS:

1. **Domain name space**: il DNS utilizza uno spazio dei nomi strutturato ad albero per identificare le risorse su Internet.
2. **Database DNS**: concettualmente, ogni nodo e foglia nella struttura ad albero dello spazio dei nomi denomina un insieme di informazioni (ad esempio, indirizzo IP, server dei nomi per questo nome di dominio) che è contenuto nel record di risorse. La raccolta di tutti gli RR è organizzata in un database distribuito.
3. **Name servers**: si tratta di programmi server che contengono informazioni su una parte della struttura ad albero dei nomi di dominio e sugli RR associati.
4. **Resolvers**: si tratta di programmi che estraggono informazioni dai name servers in risposta alle richieste dei client. Una tipica richiesta del client riguarda un indirizzo IP corrispondente a un determinato nome di dominio.

Il DNS database

Il DNS si basa su un *database gerarchico* contenente record di risorse (**RR**) che includono il nome, l'indirizzo IP e altre informazioni sugli host. Le principali caratteristiche del database sono le seguenti:

- **Gerarchia a profondità variabile per i nomi**: DNS consente essenzialmente livelli illimitati e utilizza il punto (.) come delimitatore di livello impresso sui nomi, come descritto in precedenza.
- **Database distribuito**: il database risiede in server DNS sparsi in Internet.

- **Distribuzione controllata dal database:** il database DNS è suddiviso in migliaia di zone gestite separatamente, ovvero gestite da amministratori separati. La distribuzione e l'aggiornamento dei record è controllato dal software del database.

Utilizzando questo database, i server DNS forniscono un servizio di directory **nome-indirizzo** per le applicazioni di rete che devono individuare server specifici. Ad esempio, ogni volta che viene inviato un messaggio di posta elettronica o si accede a una pagina Web, deve essere eseguita una ricerca del nome DNS per determinare l'indirizzo IP del server di posta elettronica o del server Web.

Tipo	Descrizione
A	Un indirizzo host. Questo tipo RR associa il nome di un sistema al suo indirizzo IPv4. Alcuni sistemi (ad es. router) hanno più indirizzi e per ciascuno esiste un RR separato.
AAAA	Simile al tipo A, ma per gli indirizzi IPv6.
CNAME	Nome canonico. Specifica un nome alias per un host e lo associa al nome canonico (vero).
HINFO	Informazioni sull'host. Designa il processore e il sistema operativo utilizzato dall'host.
MINFO	Informazioni sulla casella di posta o sull'elenco di posta. Associa il nome di una casella di posta o di un elenco di posta a un nome host.
MX	Mail exchange. Indica a quali server debba essere inviata la posta elettronica per un certo dominio.
NS	Utilizzato per indicare quali siano i server DNS autorevoli per un certo dominio
PTR	Il DNS viene utilizzato anche per realizzare la risoluzione inversa, ovvero per far corrispondere a un indirizzo IP il corrispondente nome di dominio. Per questo si usano i record di tipo "PTR" (e una apposita zona dello spazio dei nomi in-addr.arpa).
SOA	Start of a zone of authority (quale parte della gerarchia dei nomi è implementata). Include i parametri relativi a questa zona.
SRV	Identificano il server (o i server) per un determinato servizio all'interno di un dominio. Possono essere considerati una generalizzazione dei record MX.
TXT	Testo arbitrario. Fornisce un modo per aggiungere commenti di testo al database.
WKS	Servizi noti. Può elencare i servizi applicativi disponibili sull'host indicato.

Funzionamento DNS

Il funzionamento del DNS in genere include i seguenti passaggi:

1. Un programma utente richiede un indirizzo IP per un *hostname*.
2. Un modulo *resolver* nell'host locale **interroga**, ovvero invia una **query**, il *name server locale* nello stesso dominio del resolver.
3. Il name server locale verifica se il nome si trova nel suo database locale o nella cache e, in tal caso, restituisce l'indirizzo IP al richiedente. In caso contrario, il name server interroga altri name server disponibili, se necessario andando al *root server*, come spiegato successivamente.
4. Quando viene ricevuta una risposta sul name server locale, memorizza la mappatura *hostname/indirizzoIP* nella sua *cache locale* e può mantenere questa voce per il periodo di tempo specificato nel campo *time-to-live* del RR recuperato.
5. Al programma utente, infine, viene fornito l'indirizzo IP o un messaggio di errore.

Graficamente:

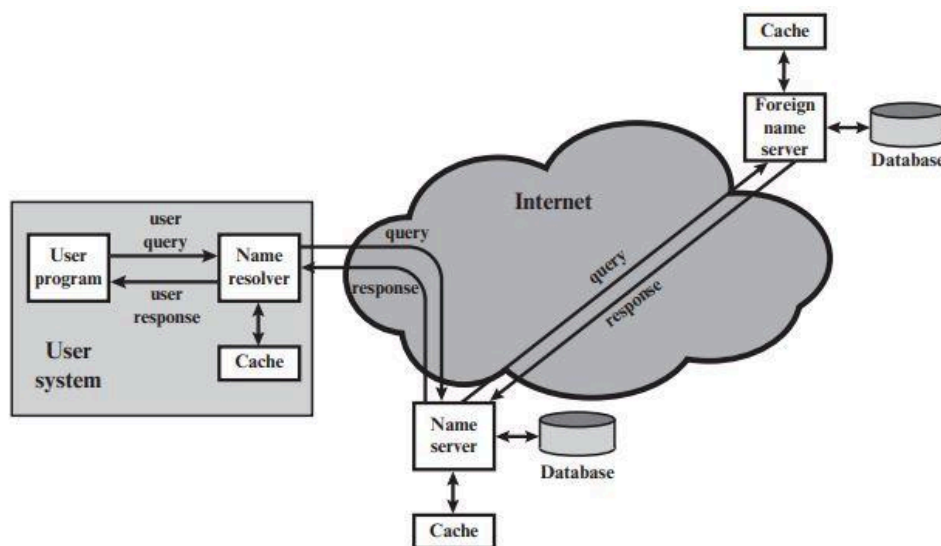


Figure 8.6 DNS Name Resolution

(Per DNS leggere anche appunti quaderno)

DNS Security Extensions

DNSSEC è un'implementazione sicura del sistema DNS, che garantisce integrità e fiducia firmando tutti i record DNS con chiavi di sicurezza per creare firme crittografiche. Queste firme vengono archiviate insieme ai record DNS tradizionali e, controllando la firma, si garantisce che i record restituiti dal server DNS provengano dal name server autorizzato per la zona specifica. I sistemi DNS tradizionali sono vulnerabili sia agli attacchi

man-in-the-middle, in cui i record DNS vengono alterati nel percorso verso l'utente, sia agli attacchi *DNS poisoning*, in cui il risolutore DNS locale (molto probabilmente il tuo computer) riceve un'ondata di risoluzioni DNS errate che dicono al risolutore locale di risolvere un dato dominio nel sito dell'aggressore, piuttosto che in quello legittimo. DNSSEC consiste in un insieme di nuovi tipi di record di risorse e modifiche al protocollo DNS esistente.

Funzionamento DNSSEC

In sostanza, DNSSEC è progettato per proteggere i client DNS dall'accettazione di record di risorse DNS contraffatti o alterati. Lo fa utilizzando le firme digitali per fornire:

- ❖ **Autenticazione dell'origine dei dati:** garantisce che i dati provengano dalla fonte corretta.
- ❖ **Verifica dell'integrità dei dati:** assicura che il contenuto di un RR non sia stato modificato.

L'amministratore di zona DNS firma digitalmente ogni set di record di risorse (*RRset*) nella zona e pubblica questa raccolta di firme digitali, insieme alla chiave pubblica dell'amministratore di zona, nel DNS stesso. In DNSSEC, la fiducia nella chiave pubblica (per la verifica della firma) dell'origine non viene stabilita rivolgendosi a una terza parte o a una catena di terze parti (come nel concatenamento dell'infrastruttura a chiave pubblica [PKI]), ma partendo da una zona attendibile (*come la zona root*) e stabilendo la catena di fiducia fino all'attuale fonte di risposta attraverso verifiche successive della firma della chiave pubblica di un figlio da parte del suo genitore. La chiave pubblica della zona attendibile è chiamata **trust anchor**.

Record di Risorse per DNSSEC

RFC 4034 definisce 4 nuovi record di risorse DNS:

1. **DNSKEY:** record contiene la chiave di firma pubblica
2. **RRSIG:** record che contiene la firma crittografica per un set di record associato
3. **NSEC:** record autenticato di negazione dell'esistenza
4. **DS:** contiene l'hash di DNSKEY

RRSET

Il primo passo nella configurazione di un sistema DNSSEC è raggruppare tutti i record di risorse (RR) in un **RRset**. L'RRset raggruppa tutti i record dello stesso tipo ed etichetta in un unico bundle che può essere successivamente firmato. Questo raggruppamento riduce notevolmente il traffico necessario per verificare e considerare attendibile ciascun record,

poiché i dati crittografici devono essere estratti solo una volta per tutti i record dello stesso tipo.

Zone-signing key (ZSK)

Ciascuna zona in DNSSEC è munita di una coppia di chiavi, denominate **Zone Signing Keys (ZSK)**: la parte *privata* della chiave firma digitalmente ogni RRset nella zona, mentre la parte *pubblica* ha il compito di verificare la firma. Per abilitare DNSSEC, un operatore di zona crea delle firme digitali per ciascun RRset utilizzando la **ZSK privata**, e le archivia nel proprio nameserver come **record RRSIG**. Ciò equivale a dire: "Questi sono i miei record DNS, provengono dal mio server e dovrebbero apparire così".

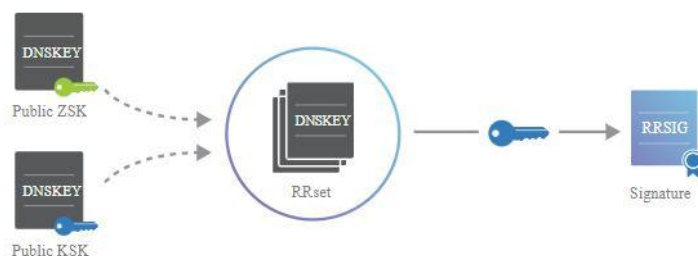
Tuttavia, questi record RRSIG sono inutili a meno che i resolver DNS non dispongano di un metodo per verificare le firme. L'operatore di zona deve inoltre rendere disponibile la propria **ZSK pubblica** aggiungendola al proprio nameserver in un **record DNSKEY**.

Quando un resolver DNSSEC richiede un particolare tipo di record (ad es. AAAA), il nameserver restituisce anche il corrispondente **RRSIG**. Il resolver può quindi estrarre il record **DNSKEY** contenente la ZSK pubblica dal nameserver. Insieme, **RRSet**, **RRSIG** e la **ZSK pubblica** possono convalidare la risposta.

Se ci fidiamo della **ZKS** nel record **DNSKEY**, allora possiamo fidarci di tutti i record nella zona. E se invece questa chiave fosse compromessa? Abbiamo bisogno di trovare un modo per convalidare la ZSK pubblica.

Key-signing key (KSK)

Oltre a una ZSK, i name server DNSSEC dispongono anche di una **Key Signing Key (KSK)**. La chiave **KSK** convalida il record **DNSKEY**, esattamente nello stesso modo in cui la nostra chiave ZSK ha protetto il resto dei nostri RRSet nella sezione precedente, ovvero firma la chiave **ZSK pubblica** (memorizzata in un record **DNSKEY**) e crea un **RRSIG** per il **DNSKEY**.



Proprio come la ZSK pubblica, il nameserver pubblica la **KSK pubblica** in un altro record **DNSKEY**, che ci dà il RRSet DNSKEY mostrato sopra. Sia la **KSK** che la **ZSK pubbliche** sono firmate dalla **KSK privata**. I resolver possono quindi utilizzare la KSK pubblica per convalidare la ZSK pubblica.

La convalida per i resolver ora si configura in questo modo:

1. Il resolver richiede un record al DNS (ex. un record A).
2. Il DNS restituisce l'RRSet contenente il record desiderato e restituisce anche il **record RRSIG** contenente la firma dell'RRset firmata dalla **ZSK**.
3. Il resolver interroga nuovamente il DNS per il **record DNSKEY** per recuperare la **ZSK pubblica** per convalidare la firma.
4. Il DNS restituisce l'**RRset DNSKEY** contenente sia **ZSK** che **KSK** pubbliche, e restituisce anche il **record RRSIG** per il **DNSKEY RRSet**.
5. Il resolver verifica il **RRSIG** per la richiesta RRset (i record A) con la **ZSK pubblica**.
6. Se il punto 5 è andato a buon fine il resolver passa a convalidare la **ZSK** confrontando il **RRSIG del RRSet DNSKEY** con la **KSK pubblica**.

Naturalmente, il **RRSet DNSKEY** e i corrispondenti **record RRSIG** possono essere memorizzati nella cache, in modo che i name server DNS non vengano costantemente bombardati da richieste superflue.

Ora abbiamo stabilito la fiducia all'interno della nostra zona, ma il DNS è un sistema gerarchico, e le zone raramente operano in modo indipendente. A complicare ulteriormente le cose, la chiave di key-signing si firma da sola, e ciò non apporta alcuna fiducia aggiuntiva. *Abbiamo pertanto bisogno di un modo per collegare la fiducia nella nostra zona con la sua zona genitore.*

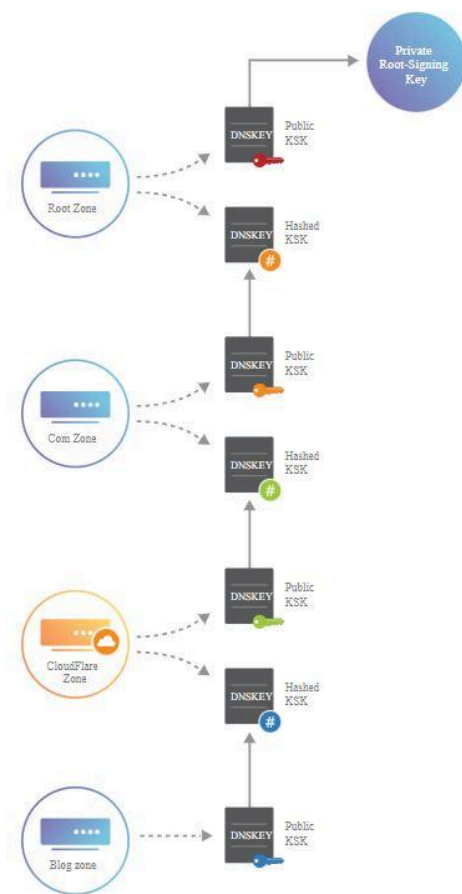
Record di delegazione del firmatario

DNSSEC introduce un record di delegazione del firmatario (**DS**) per consentire il trasferimento di fiducia da una zona genitore a una zona figlio. Un operatore di zona esegue l'**hash del record DNSKEY contenente la KSK pubblica** e lo assegna alla zona genitore per la pubblicazione come record **DS**.

Ogni volta che un resolver viene ridirezionato a una zona figlio, la zona genitore fornisce a sua volta un **record DS**. Questo **record DS** è il *modo in cui i resolver sanno che la zona figlio è abilitata al DNSSEC*. Per verificare la validità della **KSK pubblica della zona figlio**, il resolver esegue l'**HASH** e lo *confronta con il record DS del genitore*. Se corrispondono, il resolver

può presumere che la **KSK pubblica non sia stata manomessa**, il che significa che può considerare attendibili tutti i record nella zona figlio. Questo è il modo in cui viene stabilita una catena della fiducia in DNSSEC.

Ora disponiamo di un metodo per stabilire la fiducia all'interno di una zona e collegarla alla sua zona genitore, ma **come possiamo fidarci del record DS?** Il **record DS viene firmato proprio come qualsiasi altro RRset, il che significa che ha un RRSIG corrispondente nel genitore**.



DNS-Based Authentication of Named Entities

DANE (Autenticazione basata su DNS di entità denominate) è un protocollo di sicurezza Internet, che consente ai certificati digitali X.509, comunemente utilizzati per Transport Layer Security (TLS), di essere associati ai nomi di dominio utilizzando [Domain Name System Security Extensions \(DNSSEC\)](#). Viene proposto nella **RFC 6698** come un modo per autenticare le entità client e server TLS senza un'autorità di certificazione (CA).

La crittografia TLS/SSL è attualmente basata su certificati emessi dalle autorità di certificazione (CA). Negli ultimi anni diverse (CA) hanno subito gravi violazioni della

sicurezza, consentendo l'emissione di certificati per domini noti a coloro che non possiedono tali domini. Affidarsi a un gran numero di CA potrebbe essere un problema perché qualsiasi CA violata potrebbe emettere un certificato per qualsiasi nome di dominio. DANE consente all'amministratore di un nome di dominio di certificare le chiavi utilizzate nei client o nei server TLS di quel dominio memorizzandole nel Domain Name System (DNS). DANE ha bisogno che i record DNS siano firmati con DNSSEC affinché il suo modello di sicurezza funzioni.

Inoltre DANE consente al proprietario di un dominio di specificare quale CA è autorizzata a emettere certificati per una particolare risorsa, il che risolve il problema di qualsiasi CA che è in grado di emettere certificati per qualsiasi dominio.

Lo scopo di DANE è quello di sostituire l'affidamento sulla sicurezza del sistema CA con l'affidamento sulla sicurezza fornita da DNSSEC.

TLSA Record

DANE definisce un nuovo tipo di record DNS, il **record TLSA**, che può essere utilizzato per un metodo sicuro di autenticazione dei certificati SSL/TLS. Il TLSA prevede:

1. Specificare i vincoli su quale CA può garantire un certificato o quale specifico certificato di entità finale PKIX è valido.
2. Specificando che un certificato di servizio o una CA possono essere autenticati direttamente nel DNS stesso.

Il TLSA RR consente di vincolare l'emissione e la consegna del certificato a un determinato dominio. Il proprietario di un dominio server crea un record di risorse TLSA che identifica il certificato e la relativa chiave pubblica. Quando un client riceve un certificato X.509 nella negoziazione TLS, cerca il TLSA RR per quel dominio e confronta i dati TLSA con il certificato come parte della procedura di convalida del certificato del client.

La figura mostra il formato di un **TLSA RR** così come viene trasmesso a un'entità richiedente. Contiene quattro campi. Il campo **Certificate Usage** definisce quattro diversi modelli di utilizzo, per soddisfare gli utenti che richiedono diverse forme di autenticazione. I modelli di utilizzo sono: **PKIX-TA**, **PKIX-EE**, **DANE-TA** e **DANE-EE**. I primi due modelli di utilizzo sono progettati per coesistere e rafforzare il sistema di CA pubblica. Gli ultimi due modelli di utilizzo funzionano senza l'uso della CA pubbliche. Il campo **Selector** indica se verrà abbinato il certificato completo o solo il valore della chiave pubblica. La corrispondenza viene effettuata tra il certificato presentato nella negoziazione TLS e il certificato presente nel TLSA RR. Il campo **Matching Type** indica come viene effettuata la corrispondenza del certificato. Le opzioni sono corrispondenza esatta, corrispondenza

hash SHA-256 o corrispondenza hash SHA-512. I **Certificate Association Data** sono i dati grezzi del certificato in formato esadecimale.

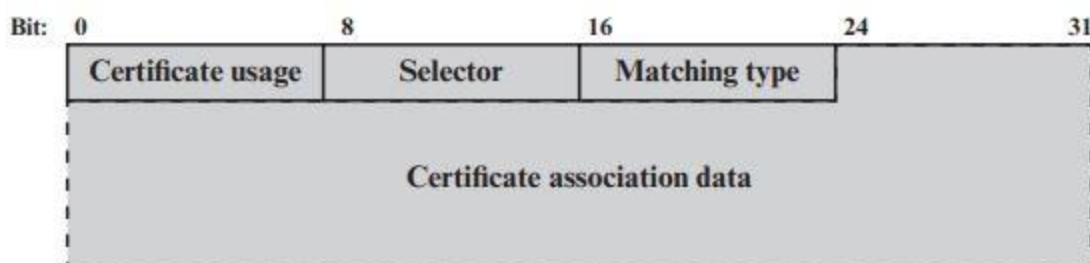


Figure 8.7 TLSA RR Transmission Format

Utilizzo di DANE per SMTP

DANE può essere utilizzato insieme a **SMTP** su **TLS**, come fornito da **STARTTLS**, per garantire una consegna della posta elettronica più sicura. DANE può autenticare il certificato del server di invio SMTP con cui comunica il client di posta dell'utente (MUA). Può anche autenticare le connessioni TLS tra server SMTP (MTA). SMTP può utilizzare l'estensione STARTTLS per eseguire SMTP su TLS, in modo che l'intero messaggio di posta elettronica più la busta SMTP siano crittografati. Ciò avviene in modo opportunistico, cioè se entrambe le parti supportano STARTTLS. Anche quando viene utilizzato per garantire la riservatezza, **TLS** è vulnerabile agli attacchi nei seguenti modi:

- ☐ Gli aggressori possono eliminare l'annuncio della funzionalità TLS e eseguire il downgrade della connessione per non utilizzare TLS.
- ☐ Le connessioni TLS sono spesso non autenticate (ad esempio, è comune l'uso di certificati autofirmati e di certificati non corrispondenti).

DANE può affrontare entrambe queste vulnerabilità. Un dominio può utilizzare la presenza del TLSA RR come indicatore della necessità di eseguire la crittografia, impedendo così downgrade dannosi. Un dominio può autenticare il certificato utilizzato nella configurazione della connessione TLS utilizzando un TLSA RR firmato DNSSEC.

Utilizzo di DNSSEC per S/MIME

DNSSEC può essere utilizzato insieme a S/MIME per proteggere in modo più completo la consegna della posta elettronica, in modo simile alla funzionalità DANE. Questo utilizzo è documentato in una bozza Internet, che propone un nuovo **SMIMEA DNS RR**. Lo scopo dello SMIMEA RR è associare i certificati ai nomi di dominio DNS. I messaggi S/MIME spesso contengono certificati che possono aiutare ad autenticare il mittente del messaggio e possono essere utilizzati per crittografare i messaggi inviati in risposta. Questa funzionalità richiede che il MUA ricevente convalidi il certificato associato al presunto mittente. Gli RR SMIMEA possono fornire un mezzo sicuro per eseguire questa convalida. In sostanza, lo

SMIMEA RR avrà lo stesso formato e contenuto del TLSA RR, con le stesse funzionalità. La differenza è che è adattato alle esigenze dei MUA nel gestire i nomi di dominio specificati negli indirizzi e-mail nel corpo del messaggio, piuttosto che i nomi di dominio specificati nella busta SMTP esterna.

Sender Policy Framework

SPF (Sender Policy Framework) è il modo standardizzato con cui un dominio di invio identifica e afferma i mittenti di posta autorizzati per quel determinato dominio e impedire a mittenti non autorizzati di inviare messaggi a nome di tale dominio; infine permette di rilevare lo spoofing delle e-mail.

e-mail spoofing → creazione di mail con indirizzo mittente contraffatto.

ADMD → Administrative Management Domain (dominio di gestione amministrativa)

SPF è definito nel **RFC 7208**. RFC 7208 fornisce un protocollo mediante il quale gli ADMD possono autorizzare gli host a utilizzare i propri nomi di dominio nelle identità "MAIL FROM" o "HELO". Gli ADMD conformi pubblicano i **record Sender Policy Framework** (SPF) nel **DNS** specificando quali host sono autorizzati a utilizzare i loro nomi, mentre i ricevitori di posta conformi utilizzano i **record SPF** pubblicati per verificare l'autorizzazione all'invio di MTA utilizzando un determinato "HELO" o l'identità "MAIL FROM" durante una transazione di posta.

SPF funziona controllando l'**indirizzo IP del mittente** rispetto alla policy codificata in qualsiasi **record SPF** trovato nel dominio di invio. Il dominio di invio è il dominio utilizzato nella connessione SMTP, non il dominio indicato nell'intestazione del messaggio come visualizzato nel MUA. Ciò significa che i controlli SPF possono essere applicati prima che il contenuto del messaggio venga ricevuto dal mittente.

La figura sottostante è un esempio in cui entrerebbe in gioco l'**SPF**. Supponiamo che l'indirizzo IP del mittente sia **192.168.0.1**. Il messaggio arriva dall'MTA con dominio **mta.example.net**. Il mittente utilizza il tag MAIL FROM di **alice@example.org**, indicando che il messaggio ha origine nel dominio **example.org**. Ma l'intestazione del messaggio specifica **alice.sender@example.net**. Il destinatario utilizza SPF per richiedere l'**SPF RR** che corrisponde a **example.com** per verificare se l'indirizzo IP 192.168.0.1 è elencato come mittente valido, quindi intraprende l'azione appropriata in base ai risultati del controllo dell'RR.

```

S: 220 foo.com Simple Mail Transfer Service Ready
C: HELO mta.example.net
S: 250 OK
C: MAIL FROM:<alice@example.org>
S: 250 OK
C: RCPT TO:<Jones@foo.com>
S: 250 OK
C: DATA
S: 354 Start mail input; end with <crLf>.<crLf>
C: To: bob@foo.com
C: From: alice.sender@example.net
C: Date: Today
C: Subject: Meeting Today
. . .

```

Figure 8.8 Example in which SMTP Envelope Header Does Not Match Message Header

SPF sul lato mittente

Un dominio di invio deve identificare tutti i mittenti per un determinato dominio e aggiungere tali informazioni nel DNS come record di risorse separato. Successivamente, il dominio di invio codifica la policy appropriata per ciascun mittente utilizzando la sintassi SPF. La codifica viene eseguita in un **record DNS TXT** come elenco di **meccanismi** e modificatori. I meccanismi vengono utilizzati per definire un indirizzo IP o un intervallo di indirizzi da abbinare e i **modificatori** indicano la politica per una determinata corrispondenza. Le Tabelle sottostanti elencano i meccanismi e i modificatori più importanti utilizzati nell'SPF. La sintassi SPF è piuttosto complessa e può esprimere relazioni complesse tra i mittenti.

Tag	Descrizione
ip4	Specifica un indirizzo IPv4 o un intervallo di indirizzi che sono mittenti autorizzati per un dominio.
ip6	Specifica un indirizzo IPv6 o un intervallo di indirizzi che sono mittenti autorizzati per un dominio.
mx	Afferma che gli host elencati per i RR di Mail Exchange sono anche mittenti validi per il dominio.
include	Elenca un altro dominio in cui il destinatario dovrebbe cercare un SPF RR per

	ulteriori mittenti. Ciò può essere utile per le organizzazioni di grandi dimensioni con molti domini o sottodomini che dispongono di un unico set di mittenti condivisi. Il meccanismo di inclusione è ricorsivo, nel senso che il controllo SPF del record trovato viene testato nella sua interezza prima di procedere. Non si tratta semplicemente di una concatenazione dei controlli.
all	Corrisponde a ogni indirizzo IP che non è stato altrimenti abbinato.

Meccanismi SPF

<u>Modifier</u>	<u>Descrizione</u>
+	Il controllo del meccanismo indicato deve essere superato. Questo è il meccanismo predefinito e non è necessario elencarlo esplicitamente.
-	Il meccanismo indicato non consente l'invio di e-mail per conto del dominio.
~	Il meccanismo indicato è in fase di transizione e se un'e-mail viene vista dall'host/indirizzo IP elencato, dovrebbe essere accettata ma contrassegnata per un controllo più attento.
?	L'SPF RR non dice esplicitamente nulla riguardo al meccanismo. In questo caso, il comportamento predefinito è accettare l'e-mail. (Ciò lo rende equivalente a '+' a meno che non venga condotta una sorta di revisione discreta o aggregata del messaggio.)

Modificatori dei meccanismi SPF

SPF sul lato ricevitore

Se SPF è implementato presso un destinatario, l'entità SPF utilizza il dominio dell'indirizzo MAIL FROM: della busta SMTP e l'indirizzo IP del mittente per interrogare un SPF TXT RR. I controlli SPF possono essere avviati prima della ricezione del corpo del messaggio di posta elettronica, il che potrebbe comportare il blocco della trasmissione del contenuto dell'e-mail. In alternativa, l'intero messaggio può essere assorbito e bufferizzato fino al completamento di tutti i controlli. In entrambi i casi, i controlli devono essere completati prima che il messaggio di posta venga inviato alla casella di posta dell'utente finale.

Il controllo prevede le seguenti regole:

1. Se non viene restituito alcun SPF TXT RR, il comportamento predefinito è accettare il messaggio.
2. Se SPF TXT RR presenta errori di formattazione, il comportamento predefinito è accettare il messaggio.

3. Altrimenti i meccanismi e i modificatori nel RR vengono utilizzati per determinare la disposizione del messaggio di posta elettronica.

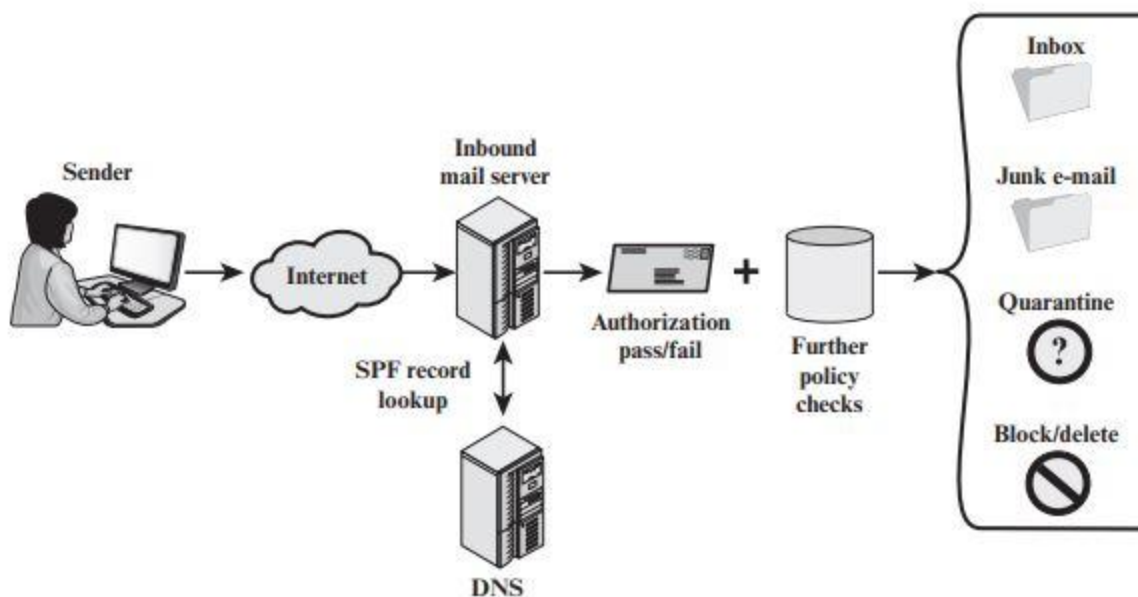


Figure 8.9 Sender Policy Framework Operation

Posta identificata da DomainKeys

DomainKeys Identified Mail (**DKIM**) è una specifica per la firma crittografica dei messaggi di posta elettronica, che consente a un dominio firmatario di rivendicare la responsabilità di un messaggio nel flusso di posta. I destinatari del messaggio (o gli agenti che agiscono per loro conto) possono verificare la firma interrogando direttamente il dominio del firmatario per recuperare la chiave pubblica appropriata e quindi confermare che il messaggio è stato autenticato da una parte in possesso della chiave privata per il dominio della firma. DKIM è uno standard Internet definito nel RFC 6376.

Strategia DKIM

DKIM è progettato per fornire una tecnica di autenticazione della posta elettronica trasparente per l'utente finale. In sostanza, il messaggio di posta elettronica di un utente è firmato da una chiave privata del dominio amministrativo da cui proviene la posta elettronica. **La firma copre tutto il contenuto del messaggio e alcune intestazioni del messaggio RFC 5322**. Dal lato ricevente, l'MDA può accedere alla chiave pubblica corrispondente tramite un DNS e verificare la firma, autenticando così che il messaggio proviene dal dominio amministrativo richiesto. Pertanto, la posta che proviene da qualche altro posto ma dichiara di provenire da un determinato dominio non supererà il test di autenticazione e potrà essere rifiutata. Questo approccio differisce da quello di S/MIME e

PGP, che *utilizzano la chiave privata dell'originatore per firmare il contenuto del messaggio*. La motivazione per DKIM si basa sul seguente ragionamento:

1. S/MIME dipende dall'utilizzo di S/MIME sia da parte degli utenti di invio che da quelli riceventi. Per quasi tutti gli utenti, la maggior parte della posta in arrivo non utilizza S/MIME e la maggior parte della posta che l'utente desidera inviare è indirizzata a destinatari che non utilizzano S/MIME.
2. S/MIME firma solo il contenuto del messaggio. Pertanto, le informazioni dell'intestazione RFC 5322 relative all'origine possono essere compromesse.
3. DKIM non è implementato nei programmi client (MUA) ed è quindi trasparente per l'utente; l'utente non deve intraprendere alcuna azione.
4. DKIM si applica a tutta la posta proveniente dai domini cooperanti.
5. DKIM consente ai mittenti validi di dimostrare di aver inviato un determinato messaggio e di impedire ai falsari di mascherarsi da mittenti validi.

In sostanza, il mittente definisce quali campi desidera includere nella propria firma del record DKIM. I campi possono essere ad esempio l'indirizzo del mittente "from", il corpo della mail, l'oggetto e molti altri. Essi devono rimanere invariati durante la trasmissione, altrimenti l'autenticazione DKIM non andrà a buon fine. Successivamente, la piattaforma di posta elettronica del mittente genera un **HASH** dei campi di testo inclusi nella firma DKIM. Una volta generata la stringa hash, essa viene crittografata utilizzando una chiave privata accessibile soltanto al mittente. Infine, una volta che l'email viene inviata, sarà compito del MDA del destinatario convalidare la firma DKIM; recuperando la chiave pubblica dal DNS e usandola per **decifrare la firma DKIM** nella sua stringa **hash originale**. Dopo di che, il destinatario *genera il proprio hash dei campi inclusi nella firma DKIM* e lo **confronta** con la stringa hash appena decifrata. Se corrispondono significa due cose: primo, che i campi della firma DKIM non hanno subito modifiche durante la trasmissione e secondo, che l'email proviene veramente dal mittente in questione.

DMARC Domain-Based Message Authentication, Reporting, and Conformance

DMARC (Autenticazione, reporting e conformità dei messaggi basati sul dominio) consente ai mittenti di posta elettronica di specificare i criteri su come gestire la posta, i tipi di report che i destinatari possono inviare e la frequenza con cui tali report devono essere inviati. È definito nella **RFC 7489**. DMARC funziona con **SPF** e **DKIM**. SPF e DKIM consentono ai mittenti di avvisare i destinatari, tramite DNS, se la posta che sembra provenire dal mittente è valida e se deve essere consegnata, contrassegnata o scartata. Tuttavia, né SPF né DKIM includono un meccanismo per comunicare ai destinatari se SPF o DKIM sono in uso, né dispongono di un meccanismo di feedback per informare i mittenti dell'efficacia delle tecniche anti-spam. DMARC affronta questi problemi essenzialmente standardizzando il modo in cui i destinatari della posta elettronica eseguono l'autenticazione della posta elettronica utilizzando i meccanismi SPF e DKIM.

Identifier Alignment

DKIM, **SPF** e **DMARC** autenticano vari aspetti di un singolo messaggio. **DKIM** autentica il dominio che ha apposto una firma al messaggio. **SPF** si concentra sulla busta SMTP, definita nella RFC 5321. Può autenticare il dominio che appare nella parte **MAIL FROM** della busta SMTP o il dominio HELO, o entrambi. Potrebbero trattarsi di domini diversi e in genere non sono visibili all'utente finale. L'autenticazione **DMARC** si occupa del dominio **'From'** nell'intestazione del messaggio, come definito nella RFC 5322. Questo campo viene utilizzato come identità centrale del meccanismo DMARC perché è un campo di intestazione del messaggio obbligatorio e pertanto è garantito che sia presente nei messaggi conformi e nella maggior parte dei casi i MUA rappresentano il campo RFC 5322 **'From'** come originatore del messaggio e restituiscono parte o tutto il contenuto del campo di intestazione agli utenti finali. L'indirizzo e-mail in questo campo è quello utilizzato dagli utenti finali per identificare la fonte del messaggio e quindi è un obiettivo primario per gli abusi. **DMARC** richiede che l'indirizzo del mittente corrisponda (*sia allineato con*) un identificatore autenticato da DKIM o SPF. Nel caso di DKIM, la corrispondenza viene effettuata tra il dominio di firma DKIM e il dominio **'From'**. Nel caso di SPF, la corrispondenza è tra il dominio autenticato da SPF e il dominio **'From'**.

DMARC sul lato mittente

Un mittente di posta che utilizza DMARC deve utilizzare anche SPF o DKIM o entrambi. Il mittente pubblica una policy DMARC nel DNS che consiglia ai destinatari come trattare i messaggi che pretendono/affermano di provenire dal dominio del mittente. La policy è sotto forma di record di risorse **TXT DNS**. Il mittente deve inoltre *stabilire indirizzi di posta elettronica per ricevere* **report aggregati e forensi**. Poiché questi indirizzi e-mail vengono pubblicati non crittografati nel **TXT DNS**, vengono facilmente scoperti, lasciando l'autore dell'invio soggetto a messaggi di posta elettronica in massa non richiesti. Pertanto, chi ha pubblicato il **TXT DNS** deve adottare qualche tipo di contromisura contro gli abusi. Similmente a SPF e DKIM, la policy DMARC nel TXT RR è codificata in una serie di coppie *tag=value* separate da punto e virgola. Una volta pubblicato il **DMARC RR**, i messaggi provenienti dal mittente vengono generalmente elaborati come segue:

1. Il proprietario del dominio crea una politica SPF e la pubblica nel suo database DNS. Il proprietario del dominio configura inoltre il proprio sistema per la firma DKIM. Infine, il proprietario del dominio pubblica tramite DNS una policy di gestione dei messaggi DMARC.
2. L'autore (mittente) genera un messaggio e consegna il messaggio al servizio di invio della posta designato dal proprietario del dominio.

3. Il servizio di invio passa i dettagli rilevanti al modulo di firma DKIM per generare una firma DKIM da applicare al messaggio.
4. Il servizio di invio inoltra il messaggio ora firmato al servizio di trasporto designato per l'instradamento ai destinatari previsti.

DMARC sul lato ricevitore

Un messaggio generato dal lato mittente può passare attraverso altri server intermedi ma alla fine arriva al servizio di trasporto del destinatario. Il tipico ordine di elaborazione per DMARC sul lato ricevente è il seguente:

1. Il destinatario esegue test di convalida standard, come il controllo delle blocklist IP e degli elenchi di reputazione dei domini, nonché l'applicazione dei limiti di velocità da una particolare fonte.
2. Il destinatario estrae l'indirizzo RFC 5322 '**From**' dal messaggio. Questo deve contenere un unico indirizzo valido altrimenti la mail viene rifiutata come errore.
3. Il destinatario richiede il **record DNS DMARC** in base al dominio di invio. Se non esiste, termina l'elaborazione DMARC.
4. Il destinatario esegue il controllo della **firma DKIM**. Se nel messaggio è presente più di una firma DKIM, è necessario verificarla.
5. Il destinatario richiede il **record SPF** del dominio mittente ed esegue controlli di convalida SPF.
6. Il destinatario effettua controlli di allineamento dell'identificatore tra il RFC 5321 '**From**' e i risultati dei **record SPF** e **DKIM** (se presenti).
7. I risultati di questi passaggi vengono passati al modulo DMARC insieme al dominio dell'autore. Il modulo DMARC tenta di recuperare una policy dal DNS per quel dominio. Se non ne viene trovato nessuno, il modulo DMARC determina il dominio organizzativo e ripete il tentativo di recuperare una policy dal DNS.
8. Se viene trovata una policy, questa viene combinata con il dominio dell'autore e i risultati SPF e DKIM per produrre un risultato della policy DMARC (un "**pass**" o "**fail**") e può facoltativamente causare la generazione di uno dei due tipi di report.
9. Il servizio di trasporto dei destinatari recapita il messaggio alla casella di posta del destinatario oppure esegue altre azioni di policy locali in base al risultato DMARC.
10. Quando richiesto, il servizio di trasporto del destinatario raccoglie i dati dalla sessione di consegna del messaggio da utilizzare per fornire feedback DMARC.

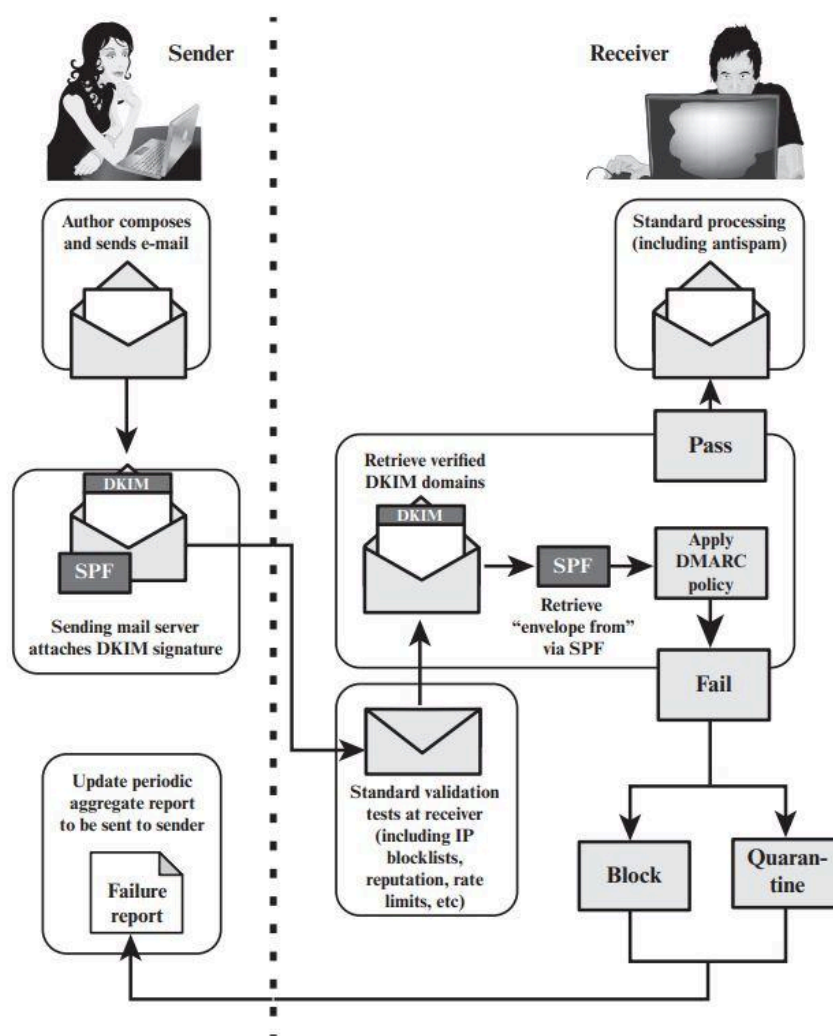


Figure 8.12 DMARC Functional Flow

CAPITOLO 9

Esistono meccanismi di sicurezza specifici dell'applicazione per una serie di aree applicative, tra cui posta elettronica (S/MIME, PGP), client/server (Kerberos), accesso Web (SSL) e altri. Tuttavia, gli utenti hanno problemi di sicurezza che riguardano tutti i livelli del protocollo. Ad esempio, un'azienda può gestire una rete IP privata e sicura impedendo collegamenti a siti non attendibili, crittografando i pacchetti in uscita e autenticando i pacchetti in entrata. Implementando la sicurezza a livello IP, un'organizzazione può garantire una rete sicura non solo per le applicazioni dotate di meccanismi di sicurezza ma anche per le numerose applicazioni che ignorano la sicurezza. La sicurezza a livello IP comprende tre aree funzionali: **autenticazione**, **confidenzialità** e **gestione delle chiavi**. Il meccanismo di autenticazione garantisce che il pacchetto ricevuto sia stato effettivamente

trasmissione dalla parte identificata come sorgente nell'intestazione del pacchetto. Inoltre, questo meccanismo garantisce che il pacchetto non sia stato alterato durante il transito. La funzione di confidenzialità consente ai nodi comunicanti di crittografare i messaggi per impedire l'intercettazione da parte di terzi. La funzione di gestione delle chiavi riguarda lo scambio sicuro delle chiavi.

Panoramica sulla sicurezza IP

Nel 1994, l'*Internet Architecture Board (IAB)* ha pubblicato un rapporto intitolato "Security in the Internet Architecture" (RFC 1636). Il rapporto ha individuato le aree chiave per i meccanismi di sicurezza. Tra questi c'era la necessità di proteggere l'infrastruttura di rete dal monitoraggio e controllo non autorizzati del traffico di rete e la necessità di proteggere il traffico da utente finale a utente finale utilizzando meccanismi di autenticazione e crittografia. Per garantire la sicurezza, lo IAB ha incluso l'autenticazione e la crittografia come funzionalità di sicurezza necessarie nell'IP di prossima generazione IPv6. Fortunatamente, queste funzionalità di sicurezza sono state progettate per essere utilizzabili con IPv4. Ciò significa che i fornitori possono iniziare a offrire queste funzionalità IPsec nei loro prodotti. La specifica *IPsec* ora esiste come insieme di standard Internet.

Applicazioni di IPsec

IPsec offre la possibilità di proteggere le comunicazioni su una LAN, su WAN pubbliche e private e su Internet. Esempi del suo utilizzo includono:

- ❖ **Connettività sicura delle filiali su Internet:** un'azienda può creare una rete privata virtuale sicura su Internet o su una WAN pubblica. Ciò consente a un'azienda di fare molto affidamento su Internet e di ridurre la necessità di reti private, risparmiando sui costi e sulle spese generali di gestione della rete.
- ❖ **Accesso remoto sicuro su Internet:** un utente finale il cui sistema è dotato di protocolli di sicurezza IP può effettuare una chiamata locale a un provider di servizi Internet (ISP) e ottenere un accesso sicuro a una rete aziendale. Ciò riduce il costo dei pedaggi per i dipendenti in viaggio e i telelavoratori.
- ❖ **Stabilire connettività extranet e intranet con i partner:** IPsec può essere utilizzato per proteggere la comunicazione con altre organizzazioni, garantendo autenticazione e riservatezza e fornendo un meccanismo di scambio di chiavi.
- ❖ **Miglioramento della sicurezza del commercio elettronico:** anche se alcune applicazioni Web e di commercio elettronico dispongono di protocolli di sicurezza integrati, l'uso di IPsec migliora tale sicurezza. IPsec garantisce che tutto il traffico

designato dall'amministratore di rete sia crittografato e autenticato, aggiungendo un ulteriore livello di sicurezza a tutto ciò che viene fornito a livello di applicazione.

La caratteristica principale di IPsec, che gli consente di supportare queste diverse applicazioni, è che **può crittografare e/o autenticare tutto il traffico a livello IP**. Pertanto, tutte le applicazioni distribuite possono essere protette.

La figura seguente è uno scenario tipico di utilizzo di IPsec. Un'organizzazione mantiene le LAN in località disperse. Su ciascuna LAN viene condotto traffico IP non protetto. Per il traffico fuori sede, attraverso una sorta di WAN privata o pubblica, vengono utilizzati i protocolli IPsec. Questi protocolli operano in dispositivi di rete, come *router o firewall*, che collegano ciascuna LAN al mondo esterno. Il **dispositivo di rete IPsec** in genere crittografa tutto il traffico in ingresso nella WAN e decripta il traffico proveniente dalla WAN; queste operazioni sono trasparenti per le workstation e i server sulla LAN. La trasmissione sicura è possibile anche con singoli utenti che si collegano alla WAN. Tali workstation utente devono implementare i protocolli IPsec per garantire la sicurezza.

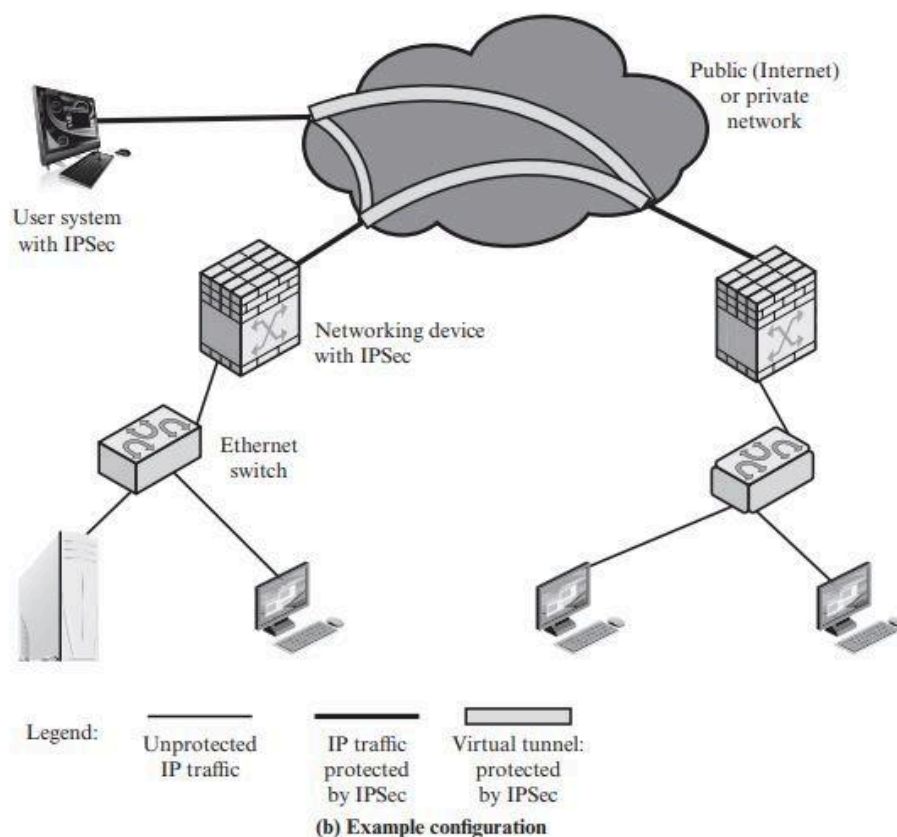


Figure 9.1 An IPsec VPN Scenario

Benefici dell'utilizzo di IPsec

Alcuni dei vantaggi di IPsec:

- Quando IPsec viene implementato in un firewall o router, fornisce una protezione avanzata che può essere applicata a tutto il traffico che attraversa il perimetro. Il traffico all'interno di un'azienda o di un gruppo di lavoro non comporta il sovraccarico dell'elaborazione relativa alla sicurezza.
- IPsec in un firewall è resistente al **bypass** se tutto il traffico dall'esterno deve utilizzare IP e il firewall è l'unico mezzo di ingresso da Internet nell'organizzazione.
- IPsec si trova al di sotto del livello di trasporto (TCP, UDP) e quindi è trasparente per le applicazioni. Non è necessario modificare il software su un sistema utente o server quando IPsec è implementato nel firewall o nel router. Anche se IPsec viene implementato nei sistemi finali, il software di livello superiore, comprese le applicazioni, non viene interessato.
- IPsec può essere trasparente per gli utenti finali. Non è necessario formare gli utenti sui meccanismi di sicurezza, rilasciare materiale per le chiavi in base al singolo utente o revocare il materiale per le chiavi quando gli utenti lasciano l'organizzazione.
- IPsec può fornire sicurezza ai singoli utenti, se necessario. Ciò è utile per i lavoratori fuori sede e per configurare una sottorete virtuale sicura all'interno di un'organizzazione per applicazioni sensibili.

Routing Applications

Oltre a supportare gli utenti finali e proteggere i sistemi e le reti locali, IPsec può svolgere un ruolo vitale nell'architettura di routing richiesta per l'internetworking. [HUIT98] elenca i seguenti esempi di utilizzo di IPsec. IPsec può assicurare che:

1. Un annuncio router (cioè un nuovo router pubblica la propria presenza) proviene da un router autorizzato.
2. Un annuncio di un neighbor (un router cerca di stabilire o mantenere una relazione di prossimità con un router in un altro dominio di instradamento) proviene da un router autorizzato.
3. Un messaggio di reindirizzamento proviene dal router a cui è stato inviato il pacchetto IP iniziale.
4. Un aggiornamento del routing non è contraffatto.

Senza tali misure di sicurezza, un avversario può interrompere le comunicazioni o deviare parte del traffico. I protocolli di routing come Open Shortest Path First (OSPF) devono essere eseguiti sulle associazioni di sicurezza tra router definite da IPsec.

Documenti IPsec

IPsec comprende tre aree funzionali: autenticazione, confidenzialità e gestione delle chiavi. La totalità delle specifiche IPsec è distribuita in dozzine di RFC e bozze di documenti IETF, rendendola molto complessa. Il modo migliore per comprendere IPsec è consultare l'ultima versione della roadmap del documento IPsec che è RFC 6071. I documenti possono essere classificati nei seguenti gruppi:

1. **Architettura**: copre i concetti generali, i requisiti di sicurezza, le definizioni e i meccanismi che definiscono la tecnologia IPsec (RFC 4301).
2. **Authentication Header**: AH è un'intestazione di estensione per fornire l'autenticazione del messaggio (RFC 4302). Poiché l'autenticazione dei messaggi è fornita da **ESP**, l'uso di AH è deprecato. È incluso in IPsecv3 per compatibilità con le versioni precedenti ma non deve essere utilizzato in nuove applicazioni. Non discuteremo AH in questo capitolo.
3. **Encapsulating Security Payload (ESP)**: ESP è costituito da un'intestazione e un trailer di incapsulamento utilizzati per fornire crittografia o crittografia/autenticazione combinata (RFC 4303).
4. **Internet Key Exchange (IKE)**: si tratta di una raccolta di documenti che descrivono gli schemi di gestione delle chiavi da utilizzare con IPsec (RFC 7296) ma esistono numerose RFC correlate.
5. **Algoritmi crittografici**: questa categoria comprende un ampio insieme di documenti che definiscono e descrivono algoritmi crittografici per la crittografia, l'autenticazione dei messaggi, le funzioni pseudocasuali (PRF) e lo scambio di chiavi crittografiche.
6. **Altri**

Servizi IPsec

IPsec fornisce servizi di sicurezza a livello IP consentendo a un sistema di selezionare i protocolli di sicurezza richiesti, determinare gli algoritmi da utilizzare per i servizi e mettere in atto eventuali chiavi crittografiche necessarie per fornire i servizi richiesti. Per garantire la sicurezza vengono utilizzati due protocolli: un protocollo di autenticazione designato dall'intestazione del protocollo, **Authentication Header (AH)**; e un protocollo combinato di crittografia/autenticazione designato dal formato del pacchetto per quel protocollo, **Encapsulating Security Payload (ESP)**. RFC 4301 elenca i seguenti servizi:

- Controllo degli accessi
- Autenticazione dell'origine dei dati
- Rifiuto dei pacchetti riprodotti

- Confidenzialità (crittografia)
- Riservatezza del flusso di traffico limitato

Transport and Tunnel Modes

Sia AH che ESP supportano due modalità di utilizzo: modalità trasporto e modalità tunnel. Il funzionamento di queste due modalità si comprende meglio nel contesto della descrizione dell'ESP, trattata successivamente. Qui forniamo una breve panoramica.

Modalità di trasporto

La modalità di trasporto fornisce protezione principalmente per i protocolli di livello superiore. Cioè, la protezione della modalità di trasporto si estende al **payload** di un pacchetto IP. In genere, la modalità di trasporto viene utilizzata per la comunicazione end-to-end tra due host (es, client-server o due workstation). Quando un host esegue AH o ESP su IPv4, il payload sono i dati che normalmente seguono l'intestazione IP. Per IPv6, il payload sono i dati che normalmente seguono sia l'intestazione IP che eventuali intestazioni di estensioni IPv6 presenti, con la possibile eccezione dell'intestazione delle opzioni di destinazione, che può essere inclusa nella protezione.

L'**ESP** in modalità trasporto cripta, e facoltativamente autentica, il payload IP ma non l'intestazione IP.

AH in modalità trasporto autentica il payload IP e parti selezionate dell'intestazione IP.

Modalità tunnel

La modalità tunnel fornisce **protezione all'intero pacchetto IP**. Per ottenere ciò, dopo che i campi AH o ESP sono stati aggiunti al pacchetto IP, l'intero pacchetto più i campi di sicurezza vengono trattati come il payload del nuovo pacchetto IP esterno con una nuova intestazione IP esterna. L'intero pacchetto originale, interno, viaggia attraverso un tunnel da un punto all'altro di una rete IP; nessun router lungo il percorso è in grado di esaminare l'intestazione IP interna. Poiché il pacchetto originale è incapsulato, il nuovo pacchetto più grande potrebbe avere indirizzi di origine e destinazione completamente diversi, il che aumenta la sicurezza. La modalità tunnel viene utilizzata quando una o entrambe le estremità di un'associazione di sicurezza (SA) sono un gateway di sicurezza, ad esempio un firewall o un router che implementa IPsec. Con la modalità tunnel, diversi host su reti protette da firewall possono impegnarsi in comunicazioni sicure senza implementare IPsec. I pacchetti non protetti generati da tali host vengono incanalati attraverso reti esterne tramite SA in modalità tunnel impostate dal software IPsec nel firewall o nel router sicuro al confine della rete locale.

ES: L'host A su una rete genera un pacchetto IP con l'indirizzo di destinazione dell'host B su un'altra rete. Questo pacchetto viene instradato dall'host di origine a un firewall o router sicuro al confine della rete di A. Il firewall filtra tutti i pacchetti in uscita per determinare la

necessità di elaborazione IPsec. Se questo pacchetto da A a B richiede IPsec, il firewall esegue l'elaborazione IPsec e incapsula il pacchetto con un'intestazione IP esterna. L'indirizzo IP di origine di questo pacchetto IP esterno è questo firewall e l'indirizzo di destinazione potrebbe essere un firewall che costituisce il confine con la rete locale di B. Questo pacchetto viene ora instradato al firewall di B, con i router intermedi che esaminano solo l'intestazione IP esterna. Nel firewall di B, l'intestazione IP esterna viene rimossa e il pacchetto interno viene consegnato a B.

L'**ESP** in modalità tunnel crittografa, e facoltativamente autentica, l'intero pacchetto IP interno, **inclusa l'intestazione IP interna**.

AH in modalità tunnel autentica l'intero pacchetto IP interno e parti selezionate dell'intestazione IP esterna.

Politiche di Sicurezza IP

Fondamentale per il funzionamento di IPsec è il concetto di **politica di sicurezza** applicata a ciascun pacchetto IP che transita da una sorgente a una destinazione. La politica IPsec è determinata principalmente dall'interazione di due database, il **security association database (SAD)** e il **security policy database (SPD)**. In questa sezione viene fornita una panoramica di questi due database e quindi viene riepilogato il loro utilizzo durante l'operazione IPsec. La Figura 9.2 illustra le relazioni rilevanti.

Associazioni di sicurezza

Un concetto chiave che appare sia nei meccanismi di autenticazione che di confidenzialità per l'IP è l'**associazione di sicurezza (SA)**. Un'associazione è una connessione logica unidirezionale tra un mittente e un destinatario che fornisce servizi di sicurezza al traffico trasportato su di essa. Se è necessaria una relazione peer per lo scambio sicuro bidirezionale, sono necessarie due associazioni di sicurezza.

Una SA è identificata in modo univoco da tre parametri:

1. **Security Parameters Index (SPI)**: un numero intero senza segno a 32 bit assegnato a questa SA e avente solo significato locale. L'SPI viene trasportato nelle intestazioni AH ed ESP per consentire al sistema ricevente di selezionare la SA in base alla quale verrà elaborato il pacchetto ricevuto.
2. **IP Destination Address**: questo è l'indirizzo dell'endpoint di destinazione della SA, che può essere un sistema dell'utente finale o un sistema di rete come un firewall o un router.
3. **Security Protocol Identifier**: questo campo dell'intestazione IP esterna indica se l'associazione è un'associazione di sicurezza AH o ESP.

Pertanto, in un pacchetto IP, la **SA** è identificata in modo univoco dall'indirizzo di destinazione nell'intestazione IPv4/6 e dall'**SPI** nell'intestazione dell'estensione racchiusa (AH o ESP).

Security Association Database

In ciascuna implementazione IPsec è presente un Security Association Database nominale che definisce i parametri associati a ciascuna SA. Un'associazione di sicurezza è normalmente definita dai seguenti parametri in una voce SAD:

1. **Security Parameter Index:** un valore a 32 bit selezionato dal lato ricevente di una SA per identificare in modo univoco la SA. In una voce SAD per una SA in uscita, l'SPI viene utilizzato per costruire l'intestazione AH o ESP del pacchetto. In una voce SAD per una SA in entrata, l'SPI viene utilizzato per mappare il traffico alla SA appropriata.
2. **Sequence Number Counter:** un valore a 32 bit utilizzato per generare il campo del numero di sequenza nelle intestazioni AH o ESP, descritto nella Sezione 9.3 (richiesto per tutte le implementazioni).
3. **Sequence Counter Overflow:** un flag che indica se l'overflow del contatore del numero di sequenza deve generare un evento verificabile e impedire l'ulteriore trasmissione di pacchetti su questa SA (richiesto per tutte le implementazioni).
4. **Anti-Replay Window:** utilizzata per determinare se un pacchetto AH o ESP in entrata è un replay, descritto nella Sezione 9.3 (richiesto per tutte le implementazioni).
5. **AH Information:** algoritmo di autenticazione, chiavi, durate delle chiavi e parametri correlati utilizzati con AH (richiesti per le implementazioni AH).
6. **ESP Information:** algoritmo di crittografia e autenticazione, chiavi, valori di inizializzazione, durata delle chiavi e parametri correlati utilizzati con ESP (richiesti per le implementazioni ESP).
7. **Lifetime of this Security Association:** un intervallo di tempo o conteggio di byte dopo il quale una SA deve essere sostituita con una nuova SA (e una nuova SPI) o terminata, oltre a un'indicazione di quale di queste azioni dovrebbe verificarsi (richiesto per tutte le implementazioni).
8. **IPsec Protocol Mode:** tunnel, trasporto o carattere jolly.
9. **Path MTU:** qualsiasi unità di trasmissione massima del percorso osservato (dimensione massima di un pacchetto che può essere trasmesso senza frammentazione) e variabili di invecchiamento (richieste per tutte le implementazioni).

Security Policy Database

Il mezzo con cui il traffico IP è correlato a SA specifiche (o nessuna SA nel caso di traffico autorizzato a bypassare IPsec) è il Security Policy Database (SPD) nominale. Nella sua forma più semplice, un SPD contiene voci, ciascuna delle quali definisce un sottoinsieme di

traffico IP e punta a una SA per quel traffico. In ambienti più complessi, potrebbero esserci più voci potenzialmente correlate a una singola SA o più SA associate a una singola voce SPD. Si rimanda il lettore ai documenti IPsec rilevanti per una discussione completa. Ciascuna voce SPD è definita da una serie di valori di campo IP e di protocollo di livello superiore, chiamati selettori. In effetti, questi selettori vengono utilizzati per filtrare il traffico in uscita al fine di mapparlo in una particolare SA. L'elaborazione in uscita obbedisce alla seguente sequenza generale per ciascun pacchetto IP.

1. Confrontare i valori dei campi appropriati nel pacchetto (i campi del selettore) con quelli dell'SPD per trovare una voce SPD corrispondente, che punterà a zero o più SA.
2. Determinare l'eventuale SA per questo pacchetto e la relativa SPI.
3. Eseguire l'elaborazione IPsec richiesta (ad esempio, elaborazione AH o ESP).

I seguenti selettori determinano una voce SPD:

1. **Indirizzo IP remoto:** può essere un singolo indirizzo IP, un elenco enumerato o un intervallo di indirizzi oppure un indirizzo con caratteri jolly (maschera). Gli ultimi due sono tenuti a supportare più di un sistema di destinazione che condivide la stessa SA (ad esempio, dietro un firewall).
2. **Indirizzo IP locale:** può essere un singolo indirizzo IP, un elenco enumerato o un intervallo di indirizzi oppure un indirizzo con carattere jolly (maschera). Gli ultimi due sono tenuti a supportare più di un sistema sorgente che condivide la stessa SA (ad esempio, dietro un firewall).
3. **Next Layer Protocol:** l'intestazione del protocollo IP (IPv4, IPv6 o estensione IPv6) include un campo (Protocollo per IPv4, Intestazione successiva per IPv6 o Estensione IPv6) che designa il protocollo che opera su IP. Si tratta di un numero di protocollo individuale, ANY, o solo per IPv6, OPAQUE. Se viene utilizzato AH o ESP, questa intestazione del protocollo IP precede immediatamente l'intestazione AH o ESP nel pacchetto.
4. **Nome:** un identificatore utente dal sistema operativo. Questo non è un campo nelle intestazioni IP o di livello superiore, ma è disponibile se IPsec è in esecuzione sullo stesso sistema operativo dell'utente.
5. **Porte locali e remote:** possono essere singoli valori di porta TCP o UDP, un elenco enumerato di porte o una porta con caratteri jolly.

Elaborazione del Traffico IP

IPsec viene eseguito pacchetto per pacchetto. Quando viene implementato IPsec, ogni pacchetto IP in uscita viene elaborato dalla logica IPsec prima della trasmissione e ogni pacchetto in entrata viene elaborato dalla logica IPsec dopo la ricezione e prima di passare

il contenuto del pacchetto al livello successivo superiore (ad esempio, TCP o UDP). Esaminiamo alternativamente la logica di queste due situazioni.

Pacchetti in uscita

Un blocco di dati da uno layer superiore, come TCP, viene passato allo strato IP e viene formato un pacchetto IP, costituito da un'intestazione IP e da un body IP. Quindi si verificano i seguenti passaggi:

1. IPsec cerca nell'SPD una corrispondenza con questo pacchetto.
2. Se non viene trovata alcuna corrispondenza, il pacchetto viene scartato e viene generato un messaggio di errore.
3. Se viene trovata una corrispondenza, l'ulteriore elaborazione è determinata dalla prima voce corrispondente nell'SPD. Se la politica per questo pacchetto è DISCARD, il pacchetto verrà scartato. Se la policy è BYPASS, non vi è alcuna ulteriore elaborazione IPsec; il pacchetto viene inoltrato alla rete per la trasmissione.
4. Se la policy è PROTECT, viene effettuata una ricerca nel SAD per una voce corrispondente. Se non viene trovata alcuna voce, viene richiamato IKE per creare una SA con le chiavi appropriate e viene creata una voce nella SA.
5. La voce corrispondente nel SAD determina l'elaborazione di questo pacchetto. È possibile eseguire la crittografia, l'autenticazione o entrambe e utilizzare la modalità trasporto o tunnel. Il pacchetto viene quindi inoltrato alla rete per la trasmissione.

Pacchetti in entrata

Un pacchetto IP in entrata attiva l'elaborazione IPsec. Si verificano i seguenti passaggi:

1. IPsec determina se si tratta di un pacchetto IP non protetto o di un pacchetto con intestazioni/trailer ESP o AH, esaminando il campo Protocollo IP (IPv4) o il campo Intestazione successiva (IPv6).
2. Se il pacchetto non è protetto, IPsec cerca nell'SPD una corrispondenza con questo pacchetto. Se la prima voce corrispondente ha una politica di BYPASS, l'intestazione IP viene elaborata e rimossa e il corpo del pacchetto viene consegnato al livello successivo più alto, come TCP. Se la prima voce corrispondente ha una politica di PROTECT o DISCARD, o se non esiste alcuna voce corrispondente, il pacchetto viene scartato.
3. Per un pacchetto protetto, IPsec cerca nel SAD. Se non viene trovata alcuna corrispondenza, il pacchetto viene scartato. In caso contrario, IPsec applica l'elaborazione ESP o AH appropriata. Quindi, l'intestazione IP viene elaborata e rimossa e il corpo del pacchetto viene consegnato al livello successivo superiore, come TCP.

Encapsulating Security Payload

ESP può essere utilizzato per fornire confidenzialità, autenticazione dell'origine dei dati, integrità senza connessione, un servizio anti-replay (una forma di integrità della sequenza parziale) e riservatezza (limitata) del flusso di traffico. L'insieme dei servizi forniti dipende dalle opzioni selezionate al momento della costituzione della *Security Association (SA)* e dalla posizione dell'implementazione nella topologia di rete. ESP può funzionare con una varietà di algoritmi di crittografia e autenticazione, inclusi algoritmi di crittografia autenticati come GCM.

Formato ESP

ESP può essere utilizzato per fornire confidenzialità, autenticazione dell'origine dei dati, integrità senza connessione, un servizio anti-replay (una forma di integrità della sequenza parziale) e riservatezza (limitata) del flusso di traffico. L'insieme dei servizi forniti dipende dalle opzioni selezionate al momento della costituzione della Security Association (SA) e dalla posizione dell'implementazione nella topologia di rete. ESP può funzionare con una varietà di algoritmi di crittografia e autenticazione, inclusi algoritmi di crittografia autenticati come GCM.

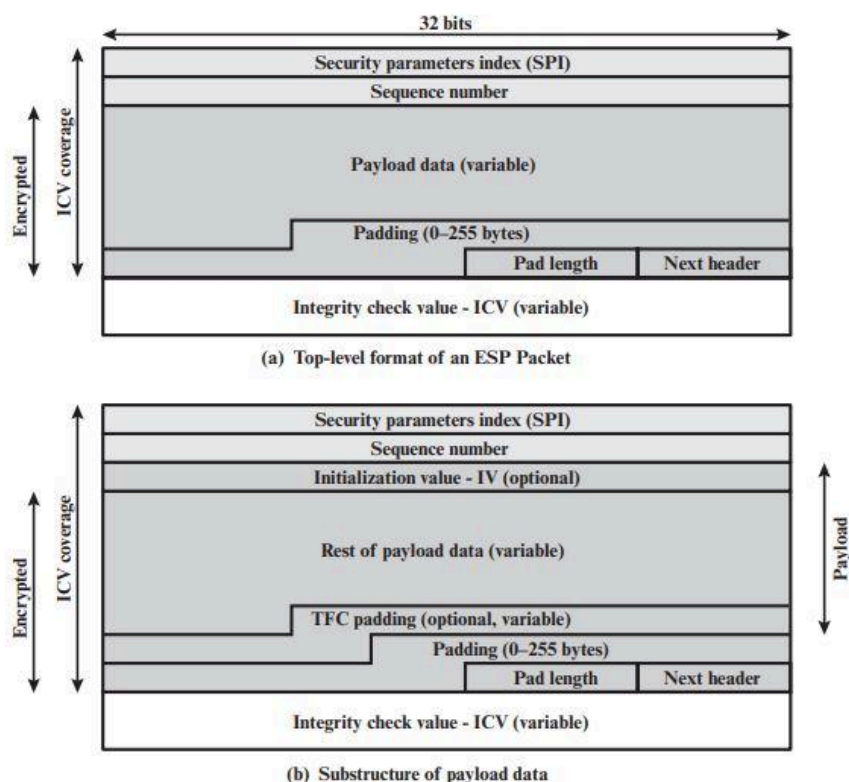


Figure 9.5 ESP Packet Format

La figura mostra il formato di primo livello di un pacchetto ESP. Contiene i seguenti campi.

1. **Security parameters index (32 bit)**: identifica un'associazione di sicurezza.
2. **Sequence Number (32 bit)**: un valore del contatore che aumenta in modo monotono; questo fornisce una funzione anti-replay, come discusso per AH.
3. **Payload Data (variabile)**: si tratta di un segmento a livello di trasporto (modalità trasporto) o di un pacchetto IP (modalità tunnel) protetto mediante crittografia.
4. **Padding (0-255 byte)**: lo scopo di questo campo verrà discusso più avanti.
5. **Pad Length (8 bit)**: indica il numero di byte pad immediatamente precedenti questo campo.
6. **Next Header (8 bit)**: identifica il tipo di dati contenuti nel campo dati del payload identificando la prima intestazione in quel payload (ad esempio, un'intestazione di estensione in IPv6 o un protocollo di livello superiore come TCP).
7. **Integrity Check Value (variabile)**: un campo di lunghezza variabile (deve essere un numero intero di parole a 32 bit) che contiene il valore di controllo di integrità calcolato sul pacchetto ESP meno il campo Dati di autenticazione.

Quando viene impiegato un qualsiasi algoritmo in modalità combinata, ci si aspetta che l'algoritmo stesso restituisca sia il testo in chiaro decriptato che un'indicazione di *pass/fail* per il controllo di integrità. Per gli algoritmi in modalità combinata, l'**ICV** che normalmente apparirebbe alla fine del pacchetto **ESP** (*quando è selezionata l'integrità*) può essere omesso. Quando l'ICV viene omesso e viene selezionata l'integrità, è responsabilità dell'algoritmo della modalità combinata codificare all'interno dei dati del **payload** un mezzo equivalente all'ICV per verificare l'integrità del pacchetto.

Nel **payload** possono essere presenti due campi aggiuntivi (Figura 9.5b). Un **initialization value (IV)**, o **nonce**, è presente se richiesto dalla crittografia o dall'algoritmo di crittografia autenticato utilizzato per ESP. Se viene utilizzata la *modalità tunnel*, l'implementazione IPsec può aggiungere il **traffic flow confidentiality (TFC)** dopo il **payload** e prima del campo Padding, come spiegato di seguito.

Algoritmi di Crittografia e Autenticazione

I campi **Payload Data**, **Padding**, **Pad Length** e **Next Header** sono crittografati dal servizio **ESP**. Se l'algoritmo utilizzato per crittografare il **payload** richiede dati di sincronizzazione crittografica, come un vettore di inizializzazione (IV), allora questi dati possono essere riportati esplicitamente all'inizio del campo **Payload Data**. Se incluso, un **IV** solitamente non viene crittografato, sebbene venga spesso indicato come parte del testo cifrato. Il campo ICV è facoltativo. È presente solo se è selezionato il servizio di integrità ed è fornito da un algoritmo di integrità separato o da un algoritmo in modalità combinata che utilizza un ICV. L'ICV viene calcolato dopo l'esecuzione della crittografia. Questo ordine di elaborazione facilita il rilevamento rapido e il rifiuto dei pacchetti riprodotti o fasulli da parte del

destinatario prima di decrittografare il pacchetto, riducendo quindi potenzialmente l'impatto degli attacchi Denial of Service (DoS). Consente inoltre la possibilità di elaborazione parallela dei pacchetti presso il ricevitore, ovvero la decrittografia può avvenire parallelamente al controllo dell'integrità. Si noti che poiché l'ICV non è protetto dalla crittografia, per calcolare l'ICV è necessario utilizzare un algoritmo di integrità con chiave.

Padding

Il campo Padding ha diversi scopi:

1. Se un algoritmo di crittografia richiede che il testo in chiaro sia un multiplo di un certo numero di byte (ad esempio, il multiplo di un singolo blocco per una cifratura a blocchi), il campo Padding viene utilizzato per espandere il testo in chiaro (costituito da Payload Data, Padding, Pad Lunghezza e Intestazione successiva) alla lunghezza richiesta.
2. Il formato ESP richiede che i campi Pad Length e Next Header siano allineati a destra all'interno di una parola a 32 bit. Allo stesso modo, il testo cifrato deve essere un multiplo intero di 32 bit. Il campo Padding viene utilizzato per garantire questo allineamento.
3. È possibile aggiungere ulteriore imbottitura per fornire una parziale riservatezza del flusso di traffico nascondendo la lunghezza effettiva del carico utile.

Anti-Replay Service

Un attacco replay è quello in cui un utente malintenzionato ottiene una copia di un pacchetto autentificato e successivamente lo trasmette alla destinazione prevista. La ricezione di pacchetti IP duplicati e autentificati potrebbe interrompere il servizio in qualche modo o avere altre conseguenze indesiderate. Il campo **Sequence Number** è progettato per contrastare tali attacchi. Innanzitutto, discutiamo la generazione del **Sequence Number** da parte del mittente, poi esaminiamo come viene elaborato dal destinatario. Quando viene stabilita una nuova SA, il mittente inizializza un contatore del **Sequence Number** a 0. Ogni volta che un pacchetto viene inviato su questa SA, il mittente incrementa il contatore e inserisce il valore nel campo **Sequence Number**. Pertanto, il primo valore da utilizzare è 1. Se l'anti-replay è abilitato (impostazione predefinita), il mittente non deve consentire al **Sequence Number** di passare da $2^{32} - 1$ e tornare a zero. Altrimenti ci sarebbero più pacchetti validi con lo stesso numero di sequenza. Se viene raggiunto il limite di $2^{32} - 1$, il mittente deve terminare questa SA e negoziare una nuova SA con una nuova chiave. Poiché IP è un servizio senza connessione e inaffidabile, il protocollo non garantisce che i pacchetti vengano consegnati in ordine e non garantisce che tutti i pacchetti vengano consegnati.

Pertanto, il documento di autenticazione IPsec impone che il destinatario debba implementare una finestra di dimensione W , con un valore predefinito di $W = 64$. Il bordo destro della finestra rappresenta il **Sequence Number** più alto, N , finora ricevuto per un pacchetto valido. Per ogni pacchetto con un **Sequence Number** compreso tra $N - W + 1$ e N che è stato ricevuto correttamente (cioè correttamente autenticato), viene contrassegnato lo slot corrispondente nella finestra (Figura 9.6). L'elaborazione in entrata procede come segue quando viene ricevuto un pacchetto:

1. Se il pacchetto ricevuto rientra nella finestra ed è nuovo, viene controllato il **MAC**. Se il pacchetto è autenticato, viene contrassegnato lo slot corrispondente nella finestra.
2. Se il pacchetto ricevuto si trova a destra della finestra ed è nuovo, viene controllato il **MAC**. Se il pacchetto è autenticato, la finestra viene avanzata in modo che questo **Sequence Number** sia il bordo destro della finestra e lo slot corrispondente nella finestra sia contrassegnato.
3. Se il pacchetto ricevuto si trova a sinistra della finestra o se l'autenticazione fallisce, il pacchetto viene scartato; questo è un evento controllabile.

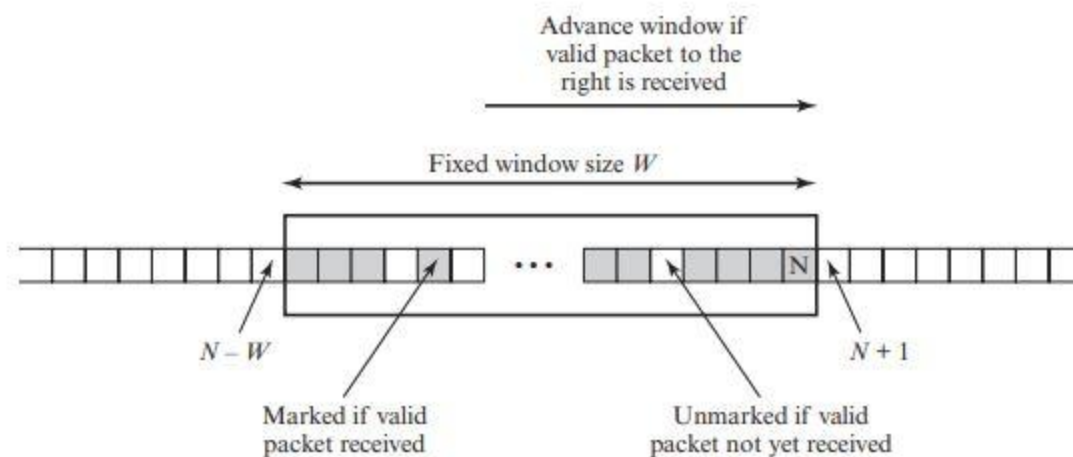


Figure 9.6 Anti-replay Mechanism

Transport e Tunnel Modes

La figura sottostante mostra due modi in cui è possibile utilizzare il **servizio ESP IPsec**. Nella parte superiore della figura, la crittografia (e facoltativamente l'autenticazione) viene fornita direttamente tra due host. La Figura 9.7b mostra come è possibile utilizzare il funzionamento in modalità tunnel per configurare una rete privata virtuale. In questo esempio, un'organizzazione dispone di quattro reti private interconnesse tramite Internet. Gli host sulle reti interne utilizzano Internet per il trasporto di dati ma non interagiscono con altri host basati su Internet. Terminando i tunnel sul gateway di sicurezza di ciascuna rete interna, la configurazione consente agli host di evitare di implementare la funzionalità di sicurezza. La prima tecnica è supportata da una SA in modalità trasporto, mentre la

seconda tecnica utilizza una SA in modalità tunnel. In questa sezione esamineremo l'ambito dell'ESP per le due modalità. Le considerazioni sono leggermente diverse per IPv4 e IPv6. Usiamo i formati dei pacchetti della Figura 9.8a come punto di partenza.

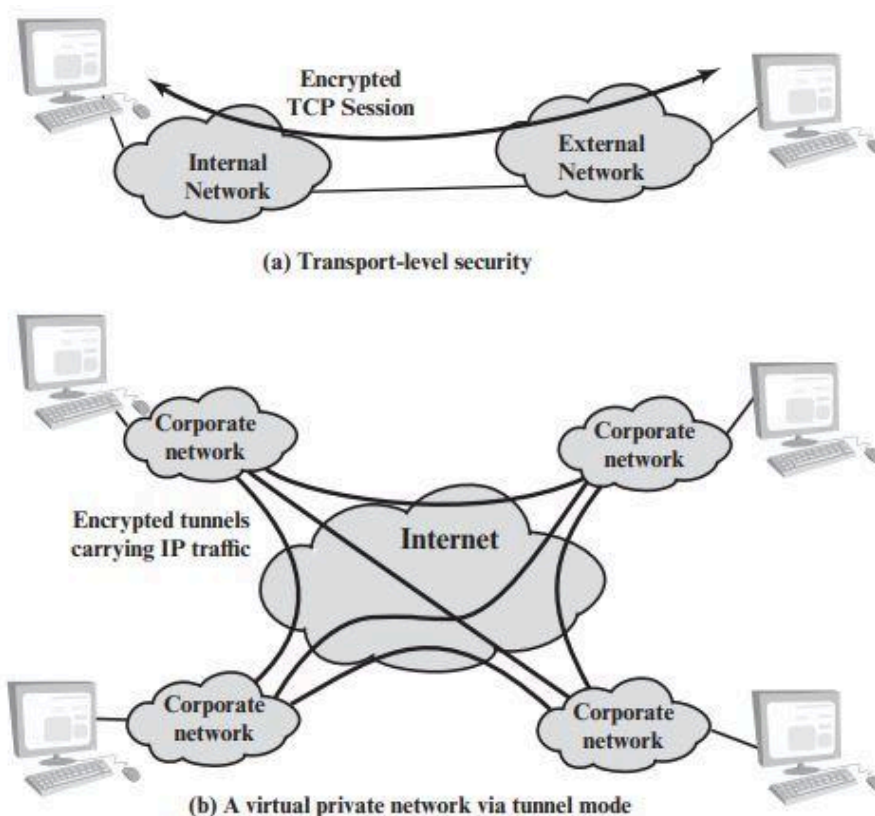


Figure 9.7 Transport-Mode versus Tunnel-Mode Encryption

Modalità di Trasporto ESP

La **modalità di trasporto ESP** viene utilizzata per crittografare e facoltativamente autenticare i dati trasportati da IP (ad esempio, un segmento TCP), come mostrato nella **Figura 9.8b**. Per questa modalità che utilizza **IPv4**, l'**header ESP** viene inserita nel pacchetto IP immediatamente prima dell'intestazione del livello di trasporto (ad esempio, TCP, UDP, ICMP) e viene inserito un **trailer ESP** (*campi Padding, Pad Length e Next Header*) dopo il pacchetto IP. Se è selezionata l'autenticazione, il campo **ESP Authentication Data** viene aggiunto dopo il **trailer ESP**. L'intero segmento a livello di trasporto più il trailer ESP sono **crittografati**. L'**autenticazione** copre tutto il testo cifrato più l'intestazione ESP. Nel contesto di **IPv6**, ESP è visto come un *payload end-to-end*; cioè non viene esaminato o elaborato dai router intermedi. Pertanto, l'**header ESP** viene visualizzato dopo l'**header di base IPv6** e l'**header di estensione hop-by-hop, routing e frammento**. L'**header dell'estensione delle opzioni di destinazione** potrebbe apparire prima o dopo l'**header ESP**, a seconda della semantica desiderata. Per IPv6, la **crittografia** copre l'intero segmento a livello di trasporto

più il trailer ESP più l'header dell'estensione delle opzioni di destinazione se si trova dopo l'**header ESP**. Ancora una volta, l'**autenticazione** copre il testo cifrato più l'intestazione ESP.

Il funzionamento in modalità trasporto può essere riassunto come segue:

1. Alla fonte, il blocco di dati costituito dal **trailer ESP** più l'intero segmento dello strato di trasporto viene crittografato e il testo in chiaro di questo blocco viene sostituito con il suo testo cifrato per formare il pacchetto IP per la trasmissione. L'autenticazione viene aggiunta se questa opzione è selezionata.
2. Il pacchetto viene quindi instradato verso la destinazione. Ciascun router intermedio deve esaminare ed elaborare l'intestazione IP più eventuali intestazioni di estensione IP in testo normale, ma non è necessario che esamini il testo cifrato.
3. Il nodo di destinazione esamina ed elabora l'intestazione IP più eventuali intestazioni di estensione IP in testo normale. Quindi, sulla base dell'SPI nell'intestazione ESP, il nodo di destinazione decodifica il resto del pacchetto per recuperare il segmento del livello di trasporto in chiaro.

Il funzionamento in modalità trasporto garantisce confidenzialità per qualsiasi applicazione che la utilizza, evitando così la necessità di implementare la confidenzialità in ogni singola applicazione. Uno svantaggio di questa modalità è che è possibile effettuare analisi del traffico sui pacchetti trasmessi.

Modalità Tunnel ESP

La modalità tunnel ESP viene utilizzata per crittografare un intero pacchetto IP **Figura 9.8c**. Per questa modalità, l'**header ESP** viene preceduto dal pacchetto, quindi il **pacchetto** e il **trailer ESP** vengono **crittografati**. Questo metodo può essere utilizzato per contrastare l'analisi del traffico. Poiché l'header IP contiene l'indirizzo di destinazione ed eventualmente le direttive di routing di origine e le informazioni sulle opzioni hop-by-hop, non è possibile trasmettere semplicemente il pacchetto IP crittografato preceduto dall'intestazione ESP. I router intermedi non sarebbero in grado di elaborare un pacchetto di questo tipo. Pertanto, è necessario incapsulare l'intero blocco (intestazione ESP più testo cifrato più dati di autenticazione, se presenti) con una **nuovo header IP** che conterrà informazioni sufficienti per l'instradamento ma non per l'analisi del traffico. Mentre la modalità trasporto è adatta per proteggere le connessioni tra host che supportano la funzionalità ESP, la modalità tunnel è utile in una configurazione che include un firewall o un altro tipo di gateway di sicurezza che protegge una rete affidabile dalle reti esterne. In quest'ultimo caso, la crittografia avviene solo tra un host esterno e il gateway di sicurezza o tra due gateway di sicurezza. Ciò solleva gli host sulla rete interna dal carico di elaborazione della crittografia e semplifica l'attività di distribuzione delle chiavi riducendo il numero di chiavi necessarie. Inoltre, ostacola l'analisi del traffico basata sulla destinazione finale.

Consideriamo il caso in cui un host esterno desidera comunicare con un host su una rete interna protetta da un firewall e in cui ESP è implementato nell'host esterno e nel firewall. Si verificano i seguenti passaggi per il trasferimento di un segmento del livello di trasporto dall'host esterno all'host interno:

1. La sorgente prepara un pacchetto IP interno con un indirizzo di destinazione dell'host interno di destinazione. Questo pacchetto è preceduto da un'intestazione ESP; quindi il pacchetto e il trailer ESP vengono crittografati e possono essere aggiunti i dati di autenticazione. ***Il blocco risultante viene incapsulato con un nuovo header IP*** (intestazione di base più estensioni opzionali come opzioni di routing e hop-by-hop per IPv6) il cui indirizzo di destinazione è il firewall; questo costituisce il pacchetto IP esterno.
2. Il pacchetto esterno viene instradato al firewall di destinazione. Ciascun router intermedio deve esaminare ed elaborare l'header IP esterno più eventuali headers di estensione IP esterni, ma non è necessario che esamini il testo cifrato.
3. Il firewall di destinazione esamina ed elabora l'header IP esterno più eventuali intestazioni di estensione IP esterne. Quindi, ***sulla base dell'SPI nell'intestazione ESP***, il nodo di destinazione decodifica il resto del pacchetto per recuperare il pacchetto IP interno in testo semplice. Questo pacchetto viene poi trasmesso nella rete interna.
4. Il pacchetto interno viene instradato attraverso zero o più router nella rete interna fino all'host di destinazione.

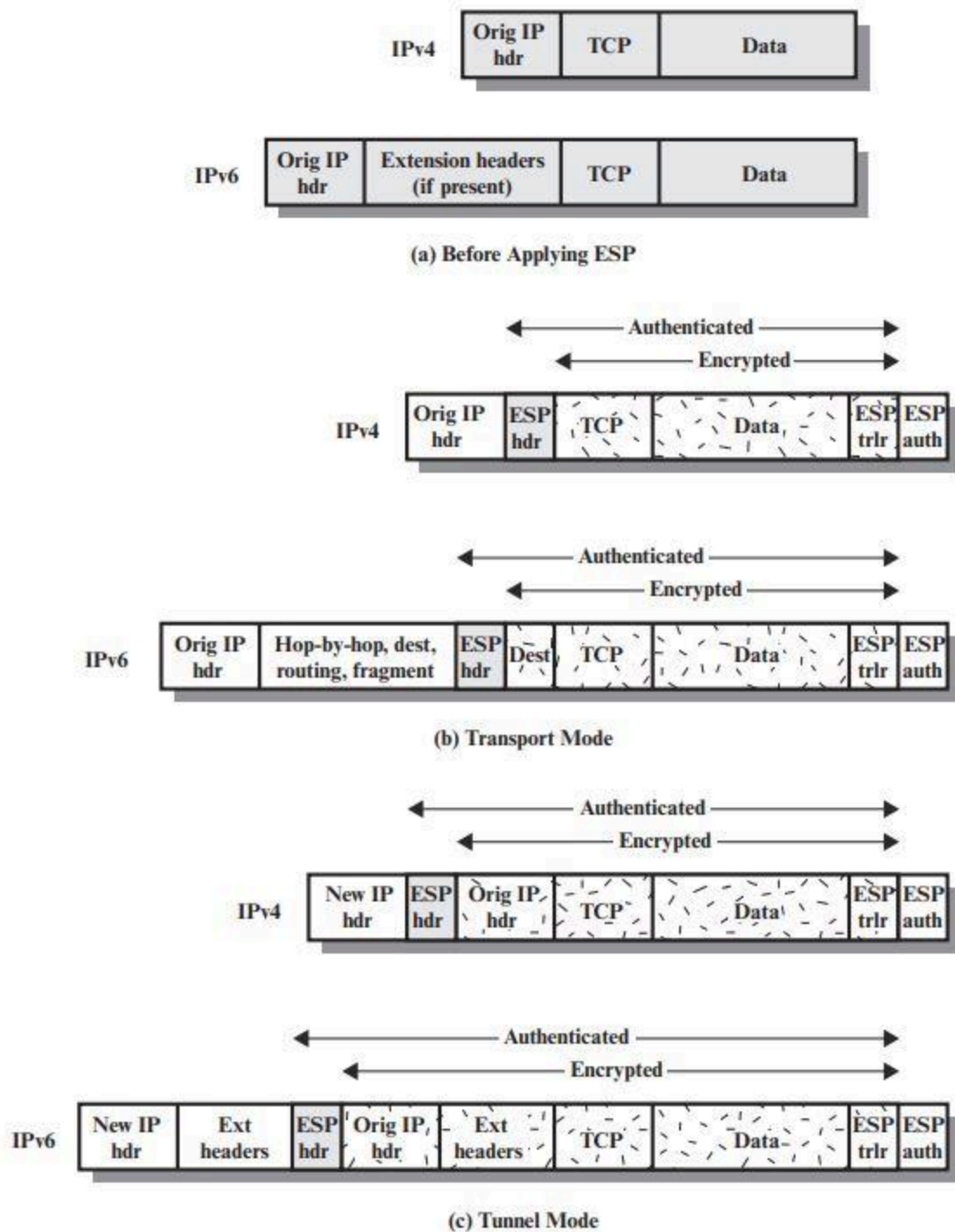


Figure 9.8 Scope of ESP Encryption and Authentication

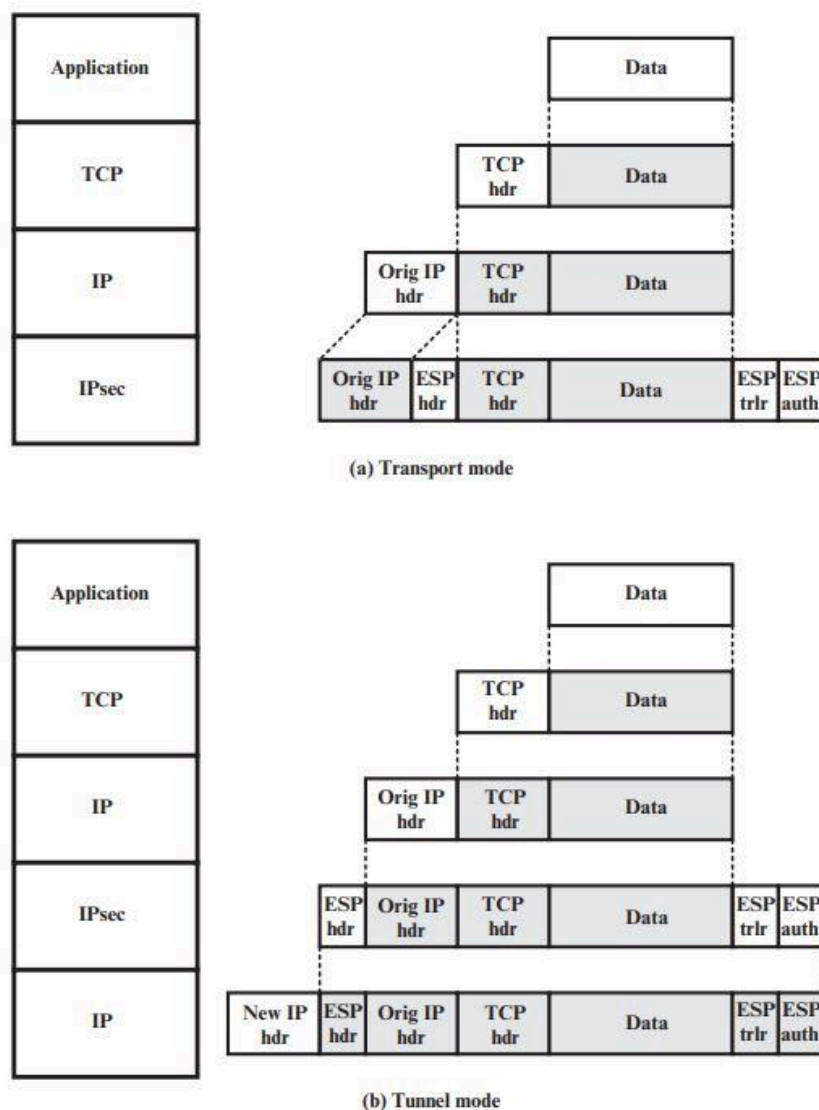


Figure 9.9 Protocol Operation for ESP

Combining Security Associations

Una singola SA può implementare il protocollo AH o ESP ma non entrambi. A volte un particolare flusso di traffico richiederà i servizi forniti sia da AH che da ESP. Inoltre, un particolare flusso di traffico può richiedere servizi IPsec tra host e, per lo stesso flusso, servizi separati tra gateway di sicurezza, come i firewall. In tutti questi casi, è necessario impiegare più SA per lo stesso flusso di traffico per ottenere i servizi IPsec desiderati. Il termine pacchetto di associazioni di sicurezza si riferisce a una sequenza di SA attraverso le quali il traffico deve essere elaborato per fornire l'insieme desiderato di servizi IPsec. Le SA in un bundle possono terminare su endpoint diversi o sugli stessi endpoint.

Le associazioni di sicurezza possono essere combinate in bundle in due modi:

1. **Transport adjacency:** si riferisce all'applicazione di più di un protocollo di sicurezza allo stesso pacchetto IP senza richiamare il tunneling. Questo approccio alla combinazione di AH ed ESP consente un solo livello di combinazione; un'ulteriore nidificazione non produce alcun vantaggio aggiuntivo poiché l'elaborazione viene eseguita in un'istanza IPsec: la destinazione (ultima).
2. **Iterated Tunneling:** si riferisce all'applicazione di più livelli di protocolli di sicurezza effettuata tramite tunneling IP. Questo approccio consente più livelli di annidamento, poiché ogni tunnel può avere origine o terminare in un sito IPsec diverso lungo il percorso.

I due approcci possono essere combinati, ad esempio, facendo in modo che una SA di trasporto tra host viaggi parte del percorso attraverso una SA tunnel tra gateway di sicurezza. Un problema interessante che emerge quando si considerano i bundle SA è l'ordine in cui l'autenticazione e la crittografia possono essere applicate tra una determinata coppia di endpoint e le modalità per farlo. Esamineremo la questione in seguito. Poi esamineremo le combinazioni di SA che coinvolgono almeno un tunnel.

Autenticazione più Confidenzialità

La crittografia e l'autenticazione possono essere combinate per trasmettere un pacchetto IP che abbia sia confidenzialità che autenticazione tra host. Esaminiamo diversi approcci.

ESP con opzione di Autenticazione

Questo approccio è illustrato nella Figura 9.8. In questo approccio, l'utente applica prima l'ESP ai dati da proteggere e poi aggiunge il campo dei dati di autenticazione. In realtà ci sono due sottocasi:

1. **Transport mode ESP:** l'autenticazione e la crittografia si applicano al payload IP consegnato all'host, ma l'intestazione IP non è protetta.
2. **Tunnel mode ESP:** l'autenticazione si applica all'intero pacchetto IP consegnato all'indirizzo IP di destinazione esterno (ad esempio, un firewall) e l'autenticazione viene eseguita a tale destinazione. L'intero pacchetto IP interno è protetto dal meccanismo di privacy per la consegna alla destinazione IP interna.

In entrambi i casi, l'autenticazione si applica al testo cifrato piuttosto che al testo in chiaro.

Transport Adjacency

Un altro modo per applicare l'autenticazione dopo la crittografia consiste nell'**utilizzare due SA di trasporto in bundle**, di cui quella interna è una SA ESP e quella esterna è una SA AH. In questo caso, ESP viene utilizzato senza la sua opzione di autenticazione. Poiché la SA interna è una SA di trasporto, al payload IP viene applicata la crittografia. Il pacchetto risultante è costituito da un'**header IP** (e possibilmente estensioni dell'intestazione IPv6) seguita da un **ESP. AH** viene quindi applicato in modalità trasporto, in modo che **l'autenticazione** copra l'ESP più l'intestazione IP originale (e le estensioni) ad eccezione dei campi modificabili. Il vantaggio di questo approccio rispetto al semplice utilizzo di una singola ESP SA con l'opzione di autenticazione ESP è che l'autenticazione copre più campi, inclusi gli indirizzi IP di origine e di destinazione. Lo svantaggio è il sovraccarico di due SA rispetto a una SA.

Transport-Tunnel Bundle

L'uso dell'autenticazione prima della crittografia potrebbe essere preferibile per diversi motivi. Innanzitutto, poiché i dati di autenticazione sono protetti dalla crittografia, è impossibile per chiunque intercettare il messaggio e alterare i dati di autenticazione senza essere scoperto. In secondo luogo, potrebbe essere desiderabile memorizzare le informazioni di autenticazione con il messaggio a destinazione per riferimento futuro. È più conveniente farlo se le informazioni di autenticazione si applicano al messaggio non crittografato; altrimenti il messaggio dovrebbe essere crittografato nuovamente per verificare le informazioni di autenticazione. Un approccio per applicare l'autenticazione prima della crittografia tra due host consiste nell'utilizzare un pacchetto costituito da una SA di trasporto AH interna e da una SA di tunnel ESP esterna. In questo caso, l'autenticazione viene applicata al payload IP più all'intestazione IP (e alle estensioni) ad eccezione dei campi modificabili. Il pacchetto IP risultante viene quindi elaborato in modalità tunnel dall'ESP; il risultato è che l'intero pacchetto interno autenticato viene crittografato e viene aggiunta una nuova intestazione IP esterna (e le estensioni).

Combinazioni di base delle Security Associations

Il documento Architettura IPsec elenca quattro esempi di combinazioni di SA che devono essere supportate da host IPsec conformi (ad esempio workstation, server) o gateway di sicurezza (ad esempio firewall, router). Questi sono illustrati nella **Figura 9.10**. La parte inferiore di ciascuna custodia nella figura rappresenta la connettività fisica degli elementi; la parte superiore rappresenta la connettività logica tramite una o più SA nidificate. Ogni SA può essere AH o ESP. Per le SA host-to-host, la modalità può essere trasporto o tunnel; altrimenti deve essere la modalità tunnel.

CASO1. Tutta la sicurezza viene fornita tra i sistemi finali che implementano IPsec. Affinché due sistemi finali possano comunicare tramite una SA, devono condividere le chiavi segrete appropriate. Tra le possibili combinazioni ci sono:

- a. AH in modalità trasporto
- b. ESP in modalità trasporto
- c. ESP seguito da AH in modalità trasporto (un ESP SA all'interno di un AH SA)
- d. Uno qualsiasi tra a, b o c all'interno di un AH o ESP in modalità tunnel

Abbiamo già discusso di come queste varie combinazioni possano essere utilizzate per supportare l'autenticazione, la crittografia, l'autenticazione prima della crittografia e l'autenticazione dopo la crittografia.

CASO2. La sicurezza è fornita solo tra gateway (router, firewall, ecc.) e nessun host implementa IPsec. Questo caso illustra il semplice supporto della rete privata virtuale. Il documento sull'architettura di sicurezza specifica che per questo caso è necessaria una sola SA tunnel. Il tunnel potrebbe supportare AH, ESP o ESP con l'opzione di autenticazione. I tunnel nidificati non sono necessari perché i servizi IPsec si applicano all'intero pacchetto interno.

CASO3. Si basa sul caso 2 aggiungendo sicurezza end-to-end. Qui sono consentite le stesse combinazioni discusse per i casi 1 e 2. Il tunnel gateway-to-gateway fornisce autenticazione, riservatezza o entrambe per tutto il traffico tra i sistemi finali. Quando il tunnel da gateway a gateway è ESP, fornisce anche una forma limitata di riservatezza del traffico. I singoli host possono implementare eventuali servizi IPsec aggiuntivi richiesti per determinate applicazioni o determinati utenti tramite SA end-to-end.

CASO4. Fornisce supporto per un host remoto che utilizza Internet per raggiungere il firewall di un'organizzazione e quindi per ottenere l'accesso ad alcuni server o workstation dietro il firewall. È richiesta solo la modalità tunnel tra l'host remoto e il firewall. Come nel caso 1, è possibile utilizzare una o due SA tra l'host remoto e l'host locale.

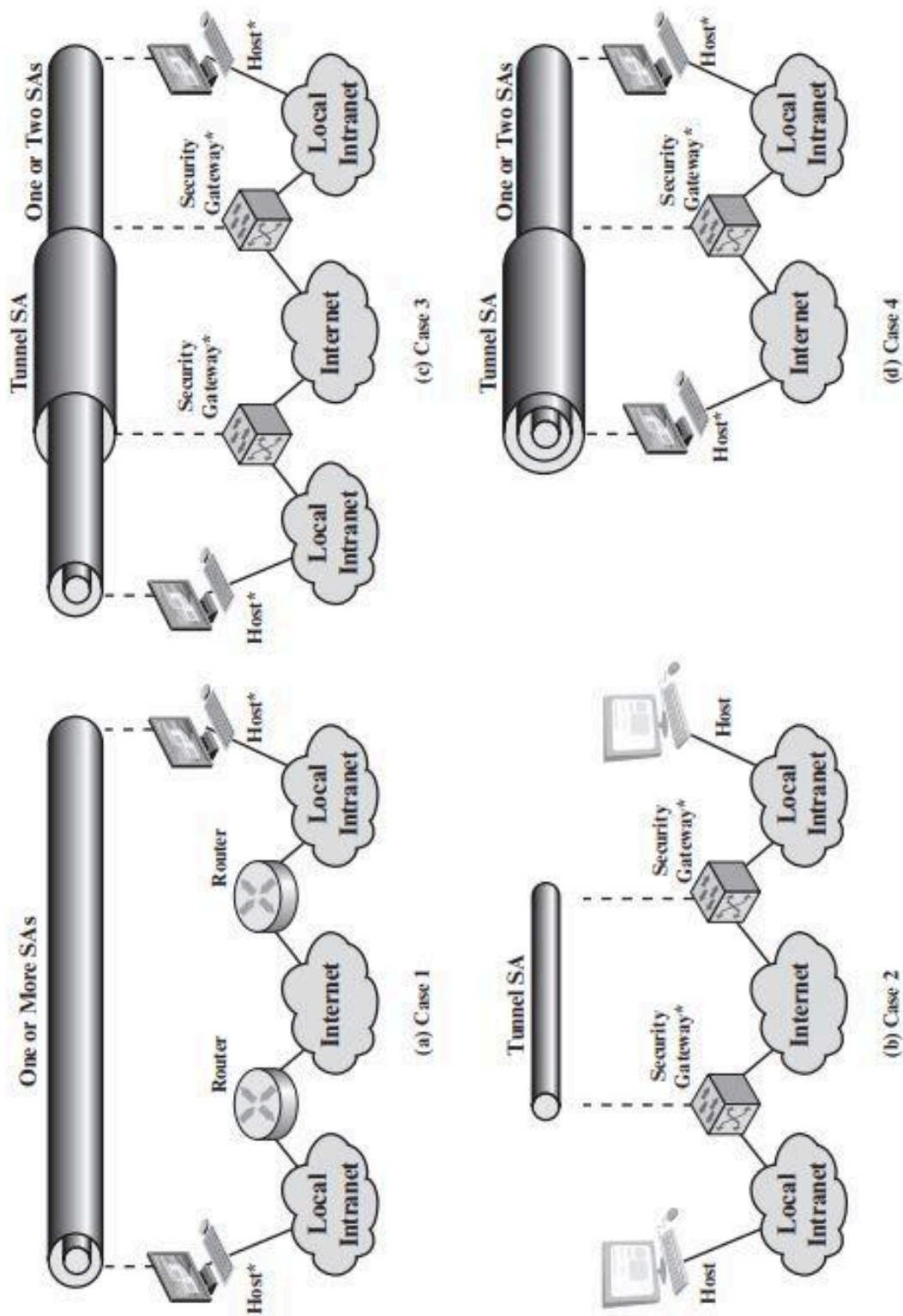


Figure 9.10 Basic Combinations of Security Associations

Scambio delle Chiavi in Internet - Internet Key Exchange

La parte di gestione delle chiavi di IPsec prevede la determinazione e la distribuzione delle chiavi segrete. Un requisito tipico sono quattro chiavi per la comunicazione tra due applicazioni: coppie di trasmissione e ricezione sia per l'integrità che per la riservatezza. Il documento Architettura IPsec impone il supporto per due tipi di gestione delle chiavi:

1. **Manuale:** un amministratore di sistema configura manualmente ciascun sistema con le proprie chiavi e con le chiavi di altri sistemi comunicanti. Ciò è pratico per ambienti piccoli e relativamente statici.
2. **Automatizzato:** un sistema automatizzato consente la creazione, su richiesta, di chiavi per le SA e facilita l'utilizzo delle chiavi in un ampio sistema distribuito con una configurazione in evoluzione.

Il protocollo di gestione automatizzata delle chiavi predefinito per IPsec è denominato **ISAKMP/Oakley** ed è costituito dai seguenti elementi:

1. **Oakley Key Determination Protocol:** Oakley è un protocollo di scambio di chiavi basato sull'algoritmo *Diffie-Hellman* ma che fornisce maggiore sicurezza. Oakley è generico nel senso che non impone formati specifici.
2. **ISAKMP (Internet Security Association and Key Management Protocol):** ISAKMP fornisce un framework per la gestione delle chiavi Internet e fornisce il supporto del protocollo specifico, inclusi i formati, per la negoziazione degli attributi di sicurezza.

ISAKMP di per sé non impone uno specifico algoritmo di scambio di chiavi; piuttosto, ISAKMP consiste in un insieme di tipi di messaggi che consentono l'uso di una varietà di algoritmi di scambio di chiavi. Oakley è l'algoritmo specifico di scambio di chiavi obbligatorio per l'uso con la versione iniziale di ISAKMP. In IKEv2 i termini Oakley e ISAKMP non vengono più utilizzati e esistono differenze significative rispetto all'uso di Oakley e ISAKMP in IKEv1. Tuttavia, la funzionalità di base è la stessa. In questa sezione descriviamo la specifica IKEv2.

Protocollo per la determinazione delle chiavi

La determinazione delle chiavi **IKE** è un perfezionamento dell'algoritmo di scambio delle chiavi Diffie-Hellman. Ricordiamo che Diffie-Hellman implica la seguente interazione tra gli utenti **A** e **B**. Esiste un accordo preliminare su due parametri globali: **q**, un numero primo grande; e **a**, radice primitiva di **q**. **A** seleziona un intero casuale **X_A** come chiave privata e trasmette a **B** la sua chiave pubblica **Y_A = a^{X_A} mod q**. Allo stesso modo, **B** seleziona un intero casuale **X_B** come chiave privata e trasmette ad **A** la sua chiave pubblica **Y_B = a^{X_B} mod q**. Ciascuna parte può ora calcolare la chiave di sessione segreta:

$$K = (Y_B)^{X_A} \bmod q = (Y_A)^{X_B} \bmod q = a^{X_A X_B} \bmod q$$

L'algoritmo Diffie-Hellman ha due caratteristiche interessanti:

1. Le chiavi segrete vengono create solo quando necessario. Non è necessario conservare le chiavi segrete per un lungo periodo di tempo, esponendole a una maggiore vulnerabilità.
2. Lo scambio non richiede infrastrutture preesistenti oltre a un accordo sui parametri globali.

Tuttavia, ci sono una serie di punti deboli nel Diffie-Hellman, come sottolineato in [HUIT98]:

1. Non fornisce alcuna informazione sull'identità delle parti.
2. È soggetto ad un attacco *man-in-the-middle*, in cui una terza parte C impersona B mentre comunica con A e impersona A mentre comunica con B. Sia A che B finiscono per negoziare una chiave con C, che può quindi ascoltare e trasferire il traffico.
3. È computazionalmente oneroso. Di conseguenza, è vulnerabile a un attacco di intasamento, in cui un avversario richiede un numero elevato di chiavi. La vittima spende considerevoli risorse informatiche eseguendo inutili esponenziazioni modulari anziché un lavoro reale.

La determinazione delle chiavi IKE è progettata per mantenere i vantaggi di Diffie-Hellman, contrastando i suoi punti deboli.

Caratteristiche della determinazione della chiave IKE

L'algoritmo di determinazione della chiave IKE è caratterizzato da cinque importanti caratteristiche:

1. Utilizza un meccanismo noto come *cookie* per contrastare gli attacchi di intasamento.
2. Permette alle due parti di negoziare un *gruppo*; questo, in sostanza, specifica i parametri globali dello scambio di chiavi Diffie-Hellman.
3. Utilizza i nonce per proteggersi dagli attacchi di replay.
4. Consente lo scambio di valori di chiave pubblica Diffie-Hellman.
5. Autentica lo scambio Diffie-Hellman per contrastare gli attacchi man-in-the-middle.

Abbiamo già discusso di Diffie-Hellman. Consideriamo ora il resto di questi elementi. Innanzitutto, considera il problema degli attacchi di intasamento. In questo attacco, un avversario falsifica l'indirizzo di origine di un utente legittimo e invia una chiave Diffie-Hellman pubblica alla vittima. La vittima esegue quindi un'esponenziazione modulare per calcolare la chiave segreta. Messaggi ripetuti di questo tipo possono intasare il sistema della vittima con lavori inutili. Lo scambio di cookie richiede che ciascuna parte invii un numero pseudocasuale, il cookie, nel messaggio iniziale, che l'altra parte riconosce. Questo riconoscimento deve essere ripetuto nel primo messaggio dello scambio di chiavi

Diffie-Hellman. Se l'indirizzo di origine è stato falsificato, l'avversario non riceve risposta. Pertanto, un avversario può solo forzare un utente a generare riconoscimenti e non a eseguire il calcolo Diffie-Hellman.

IKE impone che la generazione dei cookie soddisfi tre requisiti fondamentali:

1. Il cookie deve dipendere da soggetti specifici. Ciò impedisce a un utente malintenzionato di ottenere un cookie utilizzando un indirizzo IP e una porta UDP reali e quindi di utilizzarlo per sommergere la vittima con richieste provenienti da indirizzi IP o porte scelti casualmente.
2. Non deve essere possibile per nessun altro oltre all'entità emittente generare cookie che saranno accettati da tale entità. Ciò implica che l'entità emittente utilizzerà informazioni segrete locali nella generazione e successiva verifica di un cookie. Non deve essere possibile dedurre queste informazioni segrete da nessun particolare cookie. Il punto di questo requisito è che l'entità emittente non ha bisogno di salvare copie dei suoi cookie, che sono quindi più vulnerabili al rilevamento, ma può verificare il riconoscimento di un cookie in entrata quando necessario.
3. I metodi di generazione e verifica dei cookie devono essere rapidi per contrastare gli attacchi volti a sabotare le risorse del processore.

Il metodo consigliato per creare il cookie consiste nell'eseguire un hash veloce (ad esempio MD5) sugli indirizzi IP di origine e di destinazione, sulle porte UDP di origine e di destinazione e su un valore segreto generato localmente. La determinazione della chiave IKE supporta l'uso di gruppi diversi per lo scambio di chiavi Diffie-Hellman. Ciascun gruppo comprende la definizione dei due parametri globali e l'identità dell'algoritmo.

Scambi IKEv2

Il protocollo **IKEv2** prevede lo scambio di messaggi a coppie. Le prime due coppie di scambi sono chiamate scambi iniziali (**Figura 9.11a**). Nel primo scambio, i due peer si scambiano informazioni riguardanti gli algoritmi crittografici e altri parametri di sicurezza che desiderano utilizzare insieme ai valori *nonces* e *Diffie-Hellman (DH)*. Il risultato di questo scambio è la creazione di una **SA speciale** chiamata **IKE SA** (vedi Figura 9.2). Questa SA definisce i parametri per un canale sicuro tra i peer su cui hanno luogo i successivi scambi di messaggi. Pertanto, tutti i successivi scambi di messaggi IKE sono protetti dalla crittografia e dall'autenticazione dei messaggi. Nel secondo scambio, le due parti si autenticano a vicenda e configurano una prima **SA IPsec** da inserire nel **SADB** e utilizzata per proteggere le comunicazioni ordinarie (cioè non IKE) tra i peer. Pertanto, sono necessari quattro messaggi per stabilire la prima SA per uso generale. Lo scambio **CREATE_CHILD_SA** può essere utilizzato per stabilire ulteriori SA per proteggere il traffico. Lo scambio di informazioni viene utilizzato per scambiare informazioni di gestione, messaggi di errore IKEv2 e altre notifiche.

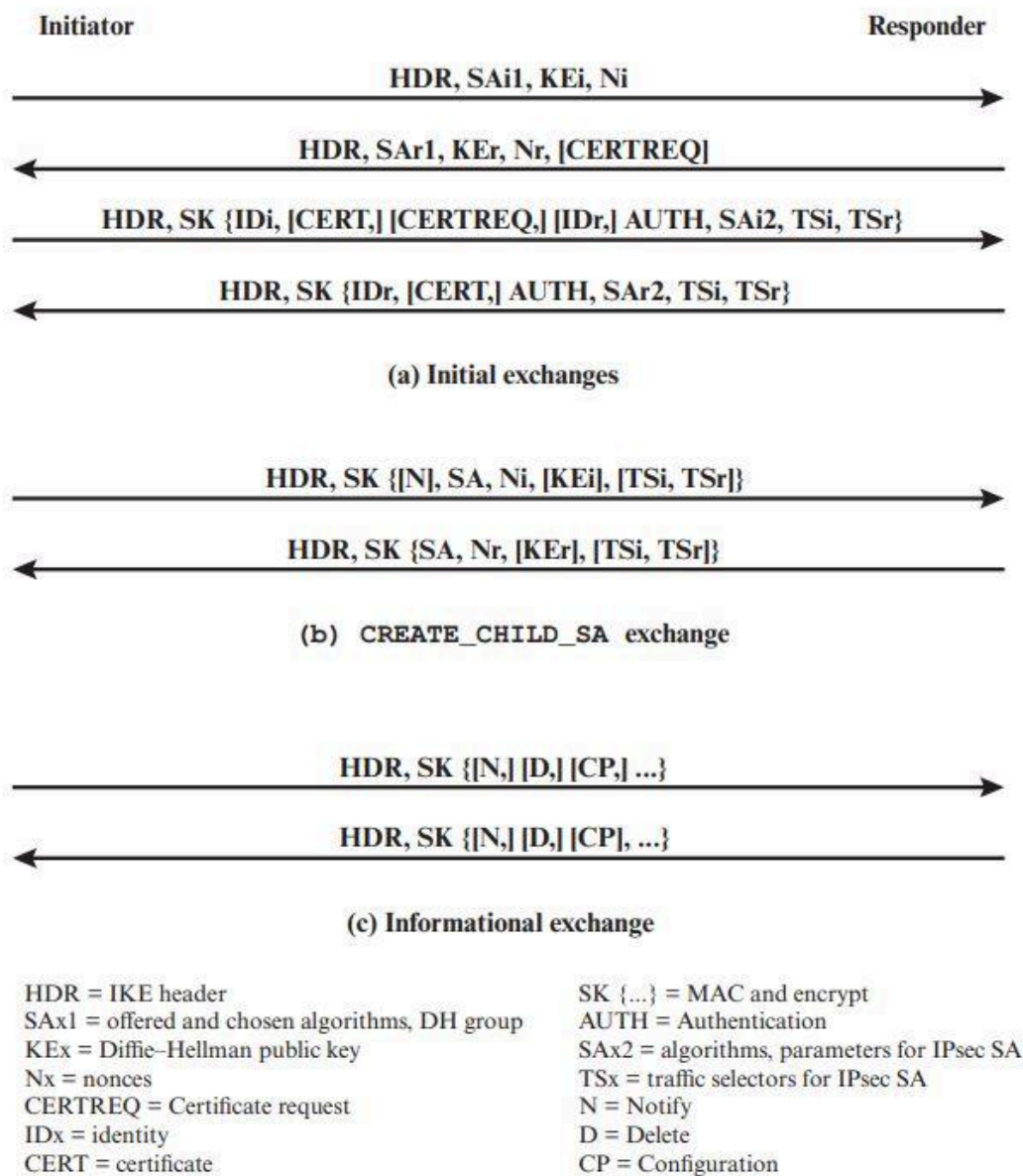


Figure 9.11 IKEv2 Exchanges

Altri argomenti da leggere nelle stampate:

- Formati di intestazione e Payload (329-333)
- Suite Crittografiche (333-335)

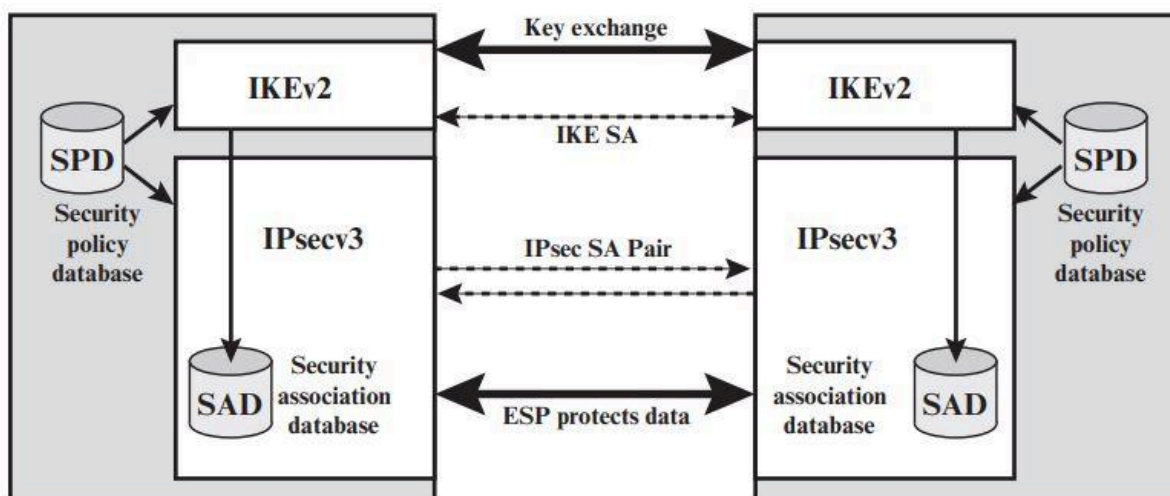


Figure 9.2 IPsec Architecture

CAPITOLO 11

Un problema di sicurezza significativo per i sistemi in rete è l'accesso ostile, o almeno indesiderato, da parte di utenti o software. La violazione da parte dell'utente può assumere la forma di **accesso non autorizzato** a una macchina o, nel caso di un utente autorizzato, di **acquisizione di privilegi o di esecuzione di azioni oltre quelle autorizzate**. La violazione del software può assumere la forma di **virus, worm o Trojan**. Tutti questi attacchi riguardano la sicurezza della rete perché l'accesso al sistema può essere ottenuto attraverso una rete. Tuttavia, questi attacchi non si limitano agli attacchi basati sulla rete. Un utente con accesso a un terminale locale può tentare la violazione senza utilizzare una rete intermedia. Un virus o un Trojan può essere introdotto in un sistema tramite un disco ottico. Solo il worm è un fenomeno esclusivamente di rete. Pertanto, la violazione del sistema è un'area in cui le preoccupazioni relative alla sicurezza della rete e alla sicurezza informatica si sovrappongono. Poiché il focus di questo libro è la sicurezza della rete, non tenteremo un'analisi completa né degli attacchi né delle contromisure di sicurezza relative alla violazione del sistema. Invece, in questa parte presentiamo un'ampia panoramica di queste preoccupazioni. Questo capitolo tratta l'argomento degli **INTRUDERS**. Innanzitutto, esaminiamo la natura dell'attacco e poi esaminiamo le strategie volte alla prevenzione e, in caso contrario, al rilevamento. Successivamente esaminiamo l'argomento correlato della gestione delle password.

Intruders

Una delle due minacce alla sicurezza più pubblicizzate è l'**INTRUDER** (l'altro sono i **VIRUS**), spesso definito hacker o cracker. In un importante studio iniziale sull'intrusione, Anderson [ANDE80] ha identificato tre classi di intrusi:

1. **Masquerader**: un individuo non autorizzato a utilizzare il computer e che penetra nei controlli di accesso di un sistema per sfruttare l'account di un utente legittimo.
2. **Misfeasor/Trasgressore**: un utente legittimo che accede a dati, programmi o risorse per i quali tale accesso non è autorizzato o che è autorizzato a tale accesso ma abusa dei propri privilegi.
3. **Clandestine user/Utente clandestino**: un individuo che assume il controllo di supervisione del sistema e utilizza questo controllo per eludere l'auditing e i controlli di accesso o per sopprimere la raccolta di audit. (audit → verifica)

E' probabile che il masquerader sia un outsider, il trasgressore generalmente è un insider e l'utente clandestino può essere un outsider o un insider. Gli attacchi degli intrusi vanno da quelli benigni a quelli gravi. All'estremità positiva della scala, ci sono molte persone che desiderano semplicemente esplorare Internet, nella parte più grave ci sono gli individui che tentano di leggere dati privilegiati, eseguire modifiche non autorizzate ai dati o interrompere il sistema.

Modelli di comportamento degli Intruders

Le tecniche e i modelli di comportamento degli intrusi cambiano costantemente, per sfruttare i punti deboli scoperti di recente e per eludere il rilevamento e le contromisure. Anche così, gli intrusi in genere seguono uno dei numerosi modelli di comportamento riconoscibili e questi modelli in genere differiscono da quelli degli utenti ordinari. Di seguito, esamineremo tre esempi generali di modelli di comportamento degli intrusi, per dare al lettore un'idea della sfida che deve affrontare l'amministratore della sicurezza.

Hacker

Tradizionalmente, coloro che hackerano i computer lo fanno per puro gusto o per motivi di status. La comunità degli hacker è una forte meritocrazia in cui lo status è determinato dal livello di competenza. Pertanto, gli aggressori spesso cercano bersagli opportunistici e quindi condividono le informazioni con altri. Gli intrusi benigni potrebbero essere tollerabili, anche se consumano risorse e potrebbero rallentare le prestazioni degli utenti legittimi. Tuttavia, non è possibile sapere in anticipo se un intruso sarà benigno o maligno. Di conseguenza, anche per i sistemi senza risorse particolarmente sensibili, esiste una motivazione a controllare questo problema. I sistemi di rilevamento delle intrusioni (**IDS**) e i

sistemi di prevenzione delle intrusioni (**IPS**) sono progettati per contrastare questo tipo di minaccia hacker. Oltre a utilizzare tali sistemi, le organizzazioni possono valutare la possibilità di limitare gli accessi remoti a specifici indirizzi IP e/o utilizzare la tecnologia di rete privata virtuale. Uno dei risultati della crescente consapevolezza del problema degli intrusi è stata la creazione di una serie di squadre di risposta alle emergenze informatiche (CERT). Queste iniziative cooperative raccolgono informazioni sulle vulnerabilità del sistema e le divulgano ai gestori dei sistemi. Gli hacker leggono regolarmente anche i rapporti del CERT. Pertanto, è importante che gli amministratori di sistema inseriscano rapidamente tutte le **patch software** per le vulnerabilità scoperte. Sfortunatamente, data la complessità di molti sistemi IT e la velocità con cui vengono rilasciate le patch, ciò è sempre più difficile da raggiungere senza l'aggiornamento automatizzato. Anche in questo caso possono verificarsi problemi causati da incompatibilità risultanti dal software aggiornato. Da qui la necessità di più livelli di difesa nella gestione delle minacce alla sicurezza dei sistemi IT.

Criminali

I gruppi organizzati di hacker sono diventati una minaccia diffusa e comune per i sistemi basati su Internet. Questi gruppi possono essere alle dipendenze di una società o di un governo, ma spesso sono bande di hacker vagamente affiliate. In genere, queste bande sono composte da giovani, spesso hacker dell'Europa orientale, della Russia o del sud-est asiatico che fanno affari sul Web. Si incontrano in forum clandestini con nomi come DarkMarket.org e theftservices.com per scambiare suggerimenti e dati, e coordinare gli attacchi. **Un obiettivo comune** è il **file di una carta di credito su un server di e-commerce**. Gli aggressori tentano di ottenere l'accesso root. I numeri delle carte vengono utilizzati dagli hacker per acquistare articoli costosi e vengono poi pubblicati sui siti dei cardatori, dove altri possono accedere e utilizzare i numeri di conto; questo oscura i modelli di utilizzo e complica le indagini. Mentre gli hacker tradizionali cercano bersagli opportunistici, gli hacker criminali di solito hanno in mente obiettivi specifici, o almeno classi di obiettivi. Una volta penetrato un sito, l'aggressore agisce rapidamente, raccogliendo quante più informazioni preziose possibile e uscendo. Anche gli IDS e gli IPS possono essere utilizzati per questi tipi di aggressori, ma potrebbero essere meno efficaci a causa della natura rapida dell'attacco in entrata e in uscita. Per i siti di e-commerce, la crittografia del database dovrebbe essere utilizzata per le informazioni sensibili dei clienti, in particolare le carte di credito. Per i siti di e-commerce ospitati (forniti da un servizio esterno), l'organizzazione di e-commerce dovrebbe utilizzare un server dedicato (non utilizzato per supportare più clienti) e monitorare da vicino i servizi di sicurezza del fornitore.

Insider Attacks

Gli attacchi interni sono tra i più difficili da rilevare e prevenire. I dipendenti hanno già accesso e conoscenza della struttura e del contenuto dei database aziendali. Gli attacchi interni possono essere motivati da vendetta o semplicemente da un sentimento di diritto.

Sebbene le strutture IDS e IPS possano essere utili per contrastare gli attacchi interni, altri approcci più diretti hanno una priorità maggiore. Gli esempi includono quanto segue:

1. Applicare i privilegi minimi, consentendo l'accesso solo alle risorse necessarie ai dipendenti per svolgere il proprio lavoro.
2. Impostare i registri per vedere quali utenti accedono e quali comandi stanno inserendo.
3. Proteggere le risorse sensibili con un'autenticazione forte.
4. Al termine del rapporto di lavoro, eliminare il computer e l'accesso alla rete del dipendente.
5. Al momento del licenziamento, creare un'immagine speculare del disco rigido del dipendente prima di rilasciarlo nuovamente. Tali prove potrebbero essere necessarie se le informazioni sulla tua azienda vengono scoperte da un concorrente.

IDS e IPS

IDS - Intrusion Detection System

Un **sistema di rilevamento delle intrusioni (IDS)** è un software che automatizza il processo di monitoraggio degli eventi che si verificano in un sistema informatico o in una rete. Un IDS analizza tali eventi al fine di rilevare segni di possibili incidenti, che possono essere violazioni o minacce imminenti di violazione delle:

1. politiche di sicurezza informatica;
2. politiche di utilizzo accettabili;
3. pratiche di sicurezza standard.

L'**IDS** si concentra su:

1. Identificazione e analisi dei possibili incidenti.
2. Logging/Recording delle informazioni relative agli eventi osservati:
 - a livello locale,
 - inviati a sistemi separati come server di registrazione centralizzati, soluzioni SIEM (Security Information and Event Management) e sistemi di gestione aziendale.
3. Notifica agli amministratori della sicurezza di importanti eventi osservati (**ALERT**) tramite:
 - e-mail;
 - pagine;
 - messaggi sull'interfaccia utente IDPS;
 - Trappole SNMP (Simple Network Management Protocol);
 - messaggi syslog e programmi e script definiti dall'utente.

Un messaggio di notifica in genere include solo informazioni di base relative a un evento; gli amministratori devono accedere all'IDPS per ulteriori informazioni.

4. Produzione di report, in particolare si occupa di riassumere gli eventi monitorati o fornire dettagli su particolari eventi di interesse.

IPS - Intrusion Prevention System

Un sistema di **prevenzione delle intrusioni (IPS)** è un software che ha tutte le funzionalità di un sistema di rilevamento delle intrusioni e può anche tentare di fermare possibili incidenti.

L'**IPS** si concentra sul **tentativo di fermare gli attacchi informatici**; questo si può suddividere in 3 obiettivi differenti:

1. **FERMARE L'ATTACCO STESSO**: e quindi Interrompere la connessione di rete o la sessione utente utilizzata per l'attacco; oppure bloccare l'accesso alla destinazione (o eventualmente ad altre probabili destinazioni) dall'account utente offensivo, dall'indirizzo IP o da altri attributi dell'aggressore; oppure bloccare tutti gli accessi all'host, al servizio, all'applicazione o ad altre risorse di destinazione.
2. **CAMBIARE L'AMBIENTE DI SICUREZZA**: un IPS può anche cambiare la configurazione di altri dispositivi di sicurezza per contrastare un attacco. Un esempio comune è quello di riconfigurare dispositivi di rete (firewall, router, switch) per bloccare l'accesso all'attaccante. Alcuni IPS possono anche causare l'applicazione di patch a un host se l'IPS rileva che l'host presenta delle vulnerabilità.
3. **MODIFICARE IL CONTENUTO DELL'ATTACCO**: Alcune tecnologie IPS possono rimuovere o sostituire parti dannose di un attacco per renderlo benigno. Un semplice esempio è un IPS che rimuove un file allegato infetto da un'e-mail e consente quindi all'e-mail ripulita di raggiungere il destinatario. Un esempio più complesso è un IPS che funge da proxy e normalizza le richieste in entrata, il che significa che il proxy riconfeziona i payload delle richieste, scartando le informazioni sull'intestazione. Ciò potrebbe far sì che alcuni attacchi vengano scartati come parte del processo di normalizzazione.

IDPS - Intrusion Detection and Prevention System

Un **IDPS** combina entrambe le funzionalità sia dell'IDS che dell'IPS, fornendo quindi una protezione più completa. L'obiettivo principale di un IDPS è quello di fornire una difesa attiva e in tempo reale contro le minacce informatiche; un IDPS è in grado di:

1. Identificare problemi legati alla politica di sicurezza: un IDPS può fornire un certo grado di controllo di qualità per l'implementazione delle politiche di sicurezza, come la duplicazione delle regole del firewall e l'avviso quando rileva traffico di rete che avrebbe dovuto essere bloccato dal firewall ma non lo è stato a causa di un errore di configurazione del firewall.

2. Documentazione delle minacce: gli IDPS registrano le informazioni sulle minacce che rilevano; compresa la frequenza e le caratteristiche degli attacchi contro le risorse informatiche dell'organizzazione. Questa attività è molto utile, in quanto permette di individuare le opportune misure di sicurezza per la protezione delle risorse stesse e permette di educare il management sulle minacce che l'organizzazione deve affrontare.
3. Scoraggiare gli individui a violare le politiche di sicurezza: se gli individui sono consapevoli che le loro azioni vengono monitorate dalle tecnologie IDPS, potrebbero evitare di commettere tali violazioni a causa del rischio di rilevamento.

Gli IDPS hanno bisogno di essere messi a punto

Le tecnologie IDPS non possono fornire un rilevamento completamente accurato.

1. Quando un IDPS identifica erroneamente un'attività benigna come dannosa, si è verificato un **falso positivo**.
2. Quando un IDPS non riesce a identificare un'attività dannosa, si è verificato un **falso negativo**.

Non è possibile eliminare tutti i falsi positivi e negativi; nella maggior parte dei casi, la riduzione delle occorrenze dell'uno aumenta le occorrenze dell'altro. Molte organizzazioni scelgono di ridurre i falsi negativi al costo di aumentare i **falsi positivi**, il che significa che vengono rilevati più eventi dannosi ma sono necessarie più risorse di analisi per differenziare i falsi positivi dai veri eventi dannosi.

La modifica della configurazione di un IDPS per migliorarne la precisione di rilevamento è nota come **TUNING** (messa a punto).

IDPS soffre la possibilità di Evasione

La maggior parte delle tecnologie IDPS offrono anche funzionalità che compensano l'uso di tecniche di evasione comuni. L'**evasione** consiste nel modificare il formato o la tempistica dell'attività dannosa in modo che il suo aspetto cambi ma il suo effetto sia lo stesso. Gli aggressori utilizzano tali tecniche di evasione per cercare di impedire alle tecnologie IDPS di rilevare i loro attacchi. La maggior parte delle tecnologie IDPS possono **sopraspedere** le tecniche di evasione comuni duplicando l'elaborazione speciale eseguita dai **target**. Se gli IDPS riescono a "vedere" l'attività nello stesso modo in cui la vedrebbe il target, allora le tecniche di evasione generalmente non riusciranno a nascondere gli attacchi.

Raccomandazioni IDPS

Raccomandazione 1: Proteggi i componenti IDPS!

Proteggere i componenti degli IDPS è molto importante perché gli IDPS sono spesso presi di mira da aggressori che vogliono impedire loro di rilevare attacchi o vogliono ottenere l'accesso a informazioni sensibili negli IDPS, come configurazioni host e vulnerabilità note. Gli IDPS sono composti da diversi tipi di componenti, inclusi **sensori** o **agenti**, **server di gestione**, **server di database**, **console utente e d'amministratore** e **reti di gestione**. I sistemi operativi e le applicazioni di tutti i componenti dovrebbero essere mantenuti completamente aggiornati e tutti i componenti IDPS basati su software dovrebbero essere protetti dalle minacce.

Azioni protettive specifiche di particolare importanza includono:

1. creazione di account separati per ciascun utente e amministratore IDPS;
2. limitare l'accesso alla rete ai componenti IDPS e garantire che le comunicazioni di gestione degli IDPS siano adeguatamente protette, ad esempio crittografandole o trasmettendole su una rete fisicamente o logicamente separata.

Gli amministratori dovrebbero mantenere la sicurezza dei componenti IDPS su base continuativa, tra cui:

1. verificare che i componenti funzionino come desiderato;
2. monitorare i componenti per problemi di sicurezza;
3. eseguire valutazioni periodiche delle vulnerabilità;
4. rispondere adeguatamente alle vulnerabilità nei componenti IDPS, testare e distribuire gli aggiornamenti IDPS.

Gli amministratori devono inoltre eseguire periodicamente il backup delle impostazioni di configurazione e applicando prima gli aggiornamenti per garantire che le impostazioni esistenti non vadano perse inavvertitamente.

Raccomandazione 2: Utilizzare più tecnologie IDPS!

E' importante utilizzare più tipi di tecnologie IDPS per ottenere un rilevamento e una prevenzione più completi e accurati di attività dannose. I quattro tipi principali di tecnologie IDPS (basate su rete, wireless, NBA e basate su host) offrono ciascuno capacità di raccolta, registrazione, rilevamento e prevenzione delle informazioni fondamentalmente diverse. In molti ambienti, non è possibile ottenere una solida soluzione IDPS senza utilizzare più tipi di tecnologie IDPS. Per la maggior parte degli ambienti, per una soluzione IDPS efficace è necessaria una combinazione di tecnologie host-based IDPS e network-based IDPS.

Raccomandazione 3: Integrazione di più sistemi IDPS!

Quando si pianifica l'utilizzo di più tipi di tecnologie IDPS o di più prodotti dello stesso tipo di tecnologia IDPS, è necessario considerare se gli IDPS debbano essere integrati o meno. Possiamo distinguere l'integrazione delle tecnologie IDPS in **diretta** e **indiretta**:

1. L'integrazione diretta IDPS si verifica molto spesso quando un'organizzazione utilizza più prodotti IDPS di un singolo fornitore, disponendo di un'unica console che può essere utilizzata per gestire e monitorare più prodotti.
2. L'integrazione indiretta IDPS viene solitamente eseguita con il software SIEM (Security Information and Event Management), ovvero un intermediario progettato per importare informazioni da vari registri relativi alla sicurezza e correlare gli eventi tra loro.

L'integrazione di più IDPS tra loro si riferisce alla pratica di connettere e coordinare più sistemi IDPS all'interno di un'infrastruttura di sicurezza per migliorare la copertura, l'efficacia e la capacità di rilevamento delle intrusioni.

Raccomandazione 4: Definire i requisiti che i prodotti IDPS devono soddisfare!

Prima di valutare i prodotti IDPS, le organizzazioni dovrebbero **definire i requisiti** che i prodotti dovrebbero soddisfare. I valutatori devono comprendere le caratteristiche del sistema dell'organizzazione e degli ambienti di rete, in modo che possa essere selezionato un IDPS compatibile in grado di monitorare gli eventi di interesse sui sistemi e/o sulle reti. I valutatori dovrebbero definire gli obiettivi che desiderano perseguire utilizzando un IDPS. I valutatori devono inoltre definire serie specifiche di requisiti per quanto segue:

1. Funzionalità di sicurezza, tra cui raccolta, registrazione, rilevamento e prevenzione delle informazioni;
2. Prestazioni, inclusa la capacità massima e le caratteristiche prestazionali;
3. Gestione, inclusa progettazione e implementazione, funzionamento e manutenzione, formazione, documentazione e supporto tecnico;
4. Costi del ciclo di vita, sia costi iniziali che di manutenzione.

Raccomandazione 5: Accertare la qualità del prodotto IDPS da diverse fonti!

Nel valutare i prodotti IDPS, le organizzazioni dovrebbero prendere in considerazione l'utilizzo di una combinazione di diverse fonti di dati sulle caratteristiche e capacità dei prodotti. Le fonti comuni di dati sui prodotti includono test di laboratorio o test di prodotti reali, informazioni fornite dai fornitori, recensioni di prodotti di terze parti e precedenti esperienze IDPS da parte di individui all'interno dell'organizzazione e di persone fidate in altre organizzazioni.

Metodologie di rilevamento degli IDPS

Esistono 3 principali metodologie utilizzate per il rilevamento di eventi dannosi da parte degli IDPS:

1. ***Signature-Based Detection***;
2. ***Anomaly-Based Detection***;
3. ***Stateful Protocol Analysis***.

Signature-Based Detection

La metodologia di rilevamento IDPS basata su firme, ***Signature-Based Detection***, è un approccio comune utilizzato per identificare e mitigare le minacce informatiche all'interno di una rete o di un sistema informatico. Il rilevamento basato sulle firme è il processo di confronto delle firme con gli eventi osservati per identificare possibili incidenti.

Funziona attraverso la seguente logica:

1. *Creazione di firme*: gli sviluppatori di sicurezza informatica creano firme digitali o pattern che rappresentano in modo univoco un particolare tipo di attacco o di malware. Queste firme sono essenzialmente sequenze di byte o regole che definiscono come appare o si comporta una minaccia specifica.
2. *Database di firme*: il sistema IDPS mantiene un database di tali firme. Questo DB viene costantemente aggiornato poiché nuove minacce vengono scoperte e identificate.
3. *Scansione del traffico*: l'IDPS monitora il traffico di rete o le attività del sistema in tempo reale. Può farlo in vari modi, tra cui l'analisi dei pacchetti di rete, la scansione dei file o l'ispezione del comportamento delle applicazioni.
4. *Confronto con le firme*: quando il sistema rileva il traffico o l'attività sospetta, confronta ciò che osserva con le firme nel DB. Questo è un processo di ricerca delle corrispondenze tra il traffico osservato e le firme note.
5. *Segnalazione o mitigazione*: se viene trovata una corrispondenza tra il traffico osservato e una firma nel database, l'IDPS può intraprendere azioni specifiche in base alla sua configurazione; come l'invio di avvisi agli amministratori di sicurezza, la registrazione degli eventi o la mitigazione automatica dell'attacco, come il blocco del traffico dannoso.

Vantaggi della ***Signature-Based Detection***:

- ***Efficace nell'identificare minacce conosciute***: le firme sono basate su modelli noti di attacchi e malware, quindi sono molto efficaci nell'identificare minacce già conosciute.

- *Bassa probabilità di falsi positivi*: poiché le firme sono specifiche, ciò riduce la probabilità di segnalare erroneamente il traffico legittimo come minaccia.

Svantaggi della **Signature-Based Detection**:

- *Inefficace contro minacce nuove*: non può rilevare minacce sconosciute poiché le firme per tali minacce non sono presenti nel DB.
- *Richiede aggiornamenti costanti*: il DB delle firme deve essere costantemente aggiornato per rimanere rilevante contro le nuove minacce.
- *Facile da eludere*: gli aggressori possono modificare leggermente il loro malware o i loro attacchi per sfuggire al rilevamento basato su firme.

Anomaly-Based Detection

La metodologia di rilevamento IDPS **Anomaly-Based Detection**, è un approccio diverso rispetto al rilevamento basato su firme. Invece di cercare firme specifiche di minacce conosciute, questa metodologia si concentra sul rilevamento di comportamenti anomali o inusuali all'interno di una rete o di un sistema informatico. Ecco come funziona:

1. Creazione di un modello di comportamento normale: per prima cosa, l>IDPS deve avere una comprensione approfondita del comportamento normale della rete o del sistema. Questo processo comporta la raccolta di dati storici e l'analisi dei modelli di traffico, delle attività degli utenti e del comportamento delle applicazioni quando tutto è in uno stato sicuro.
2. Apprendimento automatico: una volta acquisito un modello di comportamento normale, l>IDPS utilizza algoritmi di apprendimento automatico per creare un modello di riferimento basato su questi dati. Questo modello rappresenta ciò che è "normale" per la rete o il sistema.
3. Monitoraggio in tempo reale: l>IDPS continua a monitorare il traffico di rete o le attività del sistema in tempo reale e confronta costantemente ciò che osserva con il modello di riferimento creato.
4. Rilevamento di anomalie: quando l>IDPS identifica un comportamento che si discosta significativamente dal modello di comportamento normale, lo considera un'**anomalia**. Queste anomalie possono essere indicatori di possibili minacce o problemi di sicurezza.
5. Segnalazione o azioni preventive: una volta rilevata un'anomalia, l>IDPS può intraprendere azioni come l'invio di avvisi agli amministratori di sicurezza o l'attivazione di misure preventive, come il blocco del traffico sospetto.

Vantaggi della **Anomaly-Based Detection**:

- *Rileva minacce sconosciute*: è efficace nel rilevare minacce sconosciute o comportamenti anomali che potrebbero non essere coperti dalle firme.

- *Adattabile alle nuove minacce*: non richiede aggiornamenti continui delle firme, il che lo rende più adattabile alle minacce emergenti.
- *Minima dipendenza da conoscenze pregresse*: non richiede una conoscenza preventiva delle minacce.

Svantaggi della **Anomaly-Based Detection**:

- *Possibilità di falsi positivi*: il rilevamento basato su anomalie può generare falsi positivi quando il comportamento normale subisce variazioni legittime.
- *Complessità*: la creazione di modelli di comportamento normale può essere complessa e richiede una raccolta di dati rappresentativi.

In generale, molte soluzioni IDPS moderne utilizzano un approccio ibrido che combina sia il rilevamento basato su firme che quello basato su anomalie per migliorare l'efficacia complessiva nella rilevazione delle minacce informatiche. Questo approccio consente di sfruttare le forze di entrambe le metodologie e mitigare le relative debolezze.

Stateful Protocol Analysis

La metodologia di rilevamento IDPS basata sull'analisi di protocolli con stato **Stateful Protocol Analysis**, è un approccio avanzato per il rilevamento delle minacce informatiche che si concentra sulla comprensione completa dello stato delle connessioni di rete e dei protocolli utilizzati per la comunicazione. Ecco come funziona:

1. Monitoraggio delle connessioni di rete: l>IDPS monitora attentamente tutte le connessioni di rete in ingresso e in uscita. Questo monitoraggio è effettuato in tempo reale e coinvolge il **tracciamento dello stato** di ogni connessione.
2. Comprendere i protocolli di comunicazione: l>IDPS è a conoscenza dei protocolli di comunicazione utilizzati in una rete o sistema specifico. Questo significa che ha una comprensione dettagliata di come dovrebbe funzionare ogni tipo di comunicazione, inclusi i protocolli di rete come TCP/IP, HTTP, SMTP, FTP e molti altri.
3. Tracciamento dello stato: l>IDPS traccia lo stato di ogni connessione di rete, ad esempio, se una connessione TCP è in uno stato di "handshake," "dati in transito," o "chiusura." Questo tracciamento dello stato consente all>IDPS di identificare comportamenti anomali all'interno di una connessione.
4. Analisi del traffico: l>IDPS esegue un'analisi approfondita del traffico di rete per cercare comportamenti inusuali o sospetti. Questo può includere controlli come la sequenza corretta di messaggi di protocollo, la quantità di dati trasmessi o ricevuti, la velocità delle connessioni e altri parametri rilevanti.
5. Rilevamento delle anomalie di stato: quando l>IDPS identifica un comportamento che non corrisponde allo stato atteso di una connessione o di un protocollo, lo segnala come un'anomalia. Queste anomalie possono essere indicatori di minacce informatiche o di problemi di sicurezza.

6. *Segnalazione o azioni preventive*: una volta rilevata un'anomalia, l'IDPS può intraprendere azioni come l'invio di avvisi agli amministratori di sicurezza o l'attivazione di misure preventive, come il blocco del traffico sospetto.

Vantaggi della **Stateful Protocol Analysis**:

1. *Rilevamento accurato delle minacce avanzate*: l'analisi dei protocolli con stato è in grado di rilevare minacce avanzate che sfruttano vulnerabilità o manipolazioni complesse nei protocolli di comunicazione.
2. *Minimizzazione dei falsi positivi*: poiché l'IDPS ha una comprensione dettagliata dei protocolli di comunicazione e dello stato delle connessioni, è meno incline a generare falsi positivi rispetto ad altre metodologie di rilevamento.
3. *Monitoraggio approfondito del traffico*: questo approccio offre una visione completa e dettagliata del traffico di rete, consentendo una migliore comprensione del comportamento della rete e la possibilità di individuare anomalie.
4. *Adattabilità a vari protocolli*.

Svantaggi della **Stateful Protocol Analysis**:

1. *Complessità di configurazione*: configurare un IDPS basato sull'analisi di protocolli con stato richiede una conoscenza approfondita dei protocolli di rete e delle connessioni.
2. *Risorse di elaborazione*: questo tipo di IDPS richiede risorse di elaborazione significative per monitorare e analizzare il traffico di rete in tempo reale.
3. *Possibile ritardo nell'elaborazione*: a causa della necessità di monitorare attentamente il traffico di rete e analizzarlo, potrebbe verificarsi un ritardo nell'elaborazione delle informazioni.
4. *Limitazioni nel rilevamento di minacce sconosciute*: l'analisi dei protocolli con stato è efficace nel rilevare minacce conosciute o comportamenti anomali all'interno dei protocolli noti, ma può non essere altrettanto efficace nel rilevare minacce sconosciute o zero-day.

Tecnologie IDPS

Componenti

Un IDPS è composto da diverse componenti chiave che lavorano insieme per identificare e mitigare le minacce informatiche. Ecco le componenti principali di un IDPS:

1. **Sensori o Agenti**: i sensori sono la parte del sistema che monitora il traffico di rete o le attività del sistema. Possono essere *sensori di rete (Sensore)* o *sensori host (Agente)*, a seconda del punto in cui vengono implementati. I sensori raccolgono dati dal traffico in ingresso e in uscita e li passano all'analizzatore per l'elaborazione.

2. **Management Server/Analizzatore:** il management server è responsabile dell'elaborazione dei dati provenienti dai sensori. Utilizza algoritmi e regole preconfigurate per analizzare il traffico e determinare se ci sono comportamenti sospetti o violazioni di politiche di sicurezza. Se rileva un evento rilevante, genera un allarme o una segnalazione.
3. **Database Server:** un database server è un archivio di informazioni sugli eventi registrati dai Sensori, Agenti e/o Management Server.
4. **Console di gestione:** una console di gestione fornisce un'interfaccia utente per gli amministratori di sicurezza per configurare, gestire e monitorare il sistema IDPS. Il software della console di gestione viene generalmente installato su computer desktop o portatili. Alcune console vengono utilizzate solo per l'amministrazione IDPS, come la configurazione di sensori o agenti e l'applicazione di aggiornamenti software, altre console vengono utilizzate esclusivamente per il monitoraggio e l'analisi.

Network Architectures - Architetture di rete

I componenti IDPS possono essere collegati tra loro attraverso le reti standard di un'organizzazione o attraverso una rete separata strettamente progettata per la gestione del software di sicurezza nota come rete di gestione (**management network**). Se viene utilizzata una rete di gestione, ciascun host di *sensori* o *agenti* dispone di un'interfaccia di rete aggiuntiva nota come **interfaccia di gestione** che si collega alla rete di gestione. Inoltre, ciascun host di sensori o agenti non è in grado di far passare il traffico tra la propria interfaccia di gestione e le altre interfacce di rete. I **management server**, i **database server** e le **console** sono collegati solo alla rete di gestione. Questa architettura isola di fatto la rete di gestione dalle reti di produzione.

I **vantaggi** di questa architettura sono: nascondere l'esistenza e l'identità degli IDPS agli aggressori, proteggere gli IDPS dagli attacchi, e garantire che l'IDPS disponga di una larghezza di banda adeguata per funzionare in condizioni avverse (ad esempio, attacco di worm o DDoS sulle reti monitorate).

Gli **svantaggi** dell'utilizzo di una rete di gestione includono: i costi aggiuntivi per le apparecchiature di rete e altro hardware e l'inconveniente per gli utenti e gli amministratori degli IDPS di utilizzare computer separati per la gestione e il monitoraggio degli IDPS.

Se un IDPS viene distribuito senza una rete di gestione separata, un altro modo per migliorare la sicurezza dell'IDPS è creare una rete di gestione virtuale utilizzando una rete locale virtuale (VLAN) all'interno delle reti standard. L'uso di una VLAN fornisce protezione per le comunicazioni IDPS, ma non tanto quanto una rete di gestione separata.

Funzionalità di sicurezza

La maggior parte delle tecnologie IDPS possono fornire un'ampia varietà di funzionalità di sicurezza: ***raccolta di informazioni, registrazione, rilevamento e prevenzione***.

Information Gathering - Raccolta di Informazioni

Alcune tecnologie IDPS hanno la capacità di raccogliere e analizzare dati e informazioni pertinenti per identificare minacce informatiche e comportamenti sospetti all'interno di una rete o di un sistema. Questa raccolta di informazioni è fondamentale per il corretto funzionamento dell'IDPS e per garantire una risposta efficace alle minacce. Gli esempi includono l'identificazione degli host, dei sistemi operativi e delle applicazioni che utilizzano e l'identificazione delle caratteristiche generali della rete.

Logging - Registrazione

Gli IDPS in genere eseguono ***registrazioni/Logging di dati relativi agli eventi rilevati***. Questi dati possono essere utilizzati per: confermare la validità degli avvisi, indagare sugli incidenti e correlare gli eventi tra l'IDPS e altre fonti di registrazione.

I campi dati comunemente utilizzati dagli IDPS includono: data e ora dell'evento, tipo di evento, valutazione della gravità e azioni di prevenzione eseguite (se presenti).

Le tecnologie IDPS in genere consentono agli amministratori di archiviare i registri localmente e inviare copie dei registri ai server di Logging centralizzati. In generale, i registri dovrebbero essere archiviati sia localmente che centralmente per supportare l'integrità e la disponibilità dei dati. Inoltre, gli IDPS dovrebbero sincronizzare i loro orologi utilizzando il Network Time Protocol (NTP) o attraverso frequenti aggiustamenti manuali in modo che le loro voci di registro abbiano timestamp accurati.

Detection - Rilevamento

Le tecnologie IDPS offrono in genere ampie ed estese capacità di rilevamento. La maggior parte dei prodotti utilizza una combinazione di tecniche di rilevamento, per ottimizzare e personalizzare il rilevamento. La maggior parte degli IDPS richiede qualche TUNING e personalizzazione per migliorare la precisione, l'usabilità e l'efficacia del rilevamento. In genere, quanto più potenti sono le funzionalità di ottimizzazione e personalizzazione di un prodotto, tanto più è possibile migliorare la precisione del rilevamento rispetto alla configurazione predefinita. Esempi di tali capacità sono i seguenti:

1. ***Soglie/Thresholds***: una soglia è un valore che stabilisce il limite tra comportamento normale e anormale.
2. ***Blacklist e Whitelist***: una lista nera è un elenco di entità discrete, come host, numeri di porta TCP o UDP, tipi e codici ICMP, applicazioni, nomi utente, URL, nomi di file o estensioni di file, che sono stati precedentemente determinati come associati ad

attività dannose. Una whitelist è un elenco di entità discrete note per essere benigne.

3. **Impostazioni degli avvisi/Alert Settings:** la maggior parte delle tecnologie IDPS consentono agli amministratori di personalizzare ciascun tipo di avviso.
4. **Visualizzazione e modifica del codice/Code viewing and editing:** alcune tecnologie IDPS consentono agli amministratori di vedere parte o tutto il codice relativo al rilevamento. La visualizzazione del codice può aiutare gli analisti a determinare il motivo per cui sono stati generati particolari avvisi. La possibilità di modificare tutto il codice relativo al rilevamento e scrivere nuovo codice è necessaria per personalizzare completamente alcuni tipi di funzionalità di rilevamento. La modifica del codice richiede competenze di programmazione e rilevamento delle intrusioni e i bug introdotti nel codice durante il processo di personalizzazione potrebbero causare il malfunzionamento o il fallimento totale dell'IDPS.

Gli amministratori dovrebbero rivedere periodicamente l'ottimizzazione e le personalizzazioni per assicurarsi che siano ancora accurate.

Prevention - Prevenzione

La maggior parte degli IDPS offre molteplici capacità di prevenzione; le capacità specifiche variano in base al tipo di tecnologia IDPS. Gli IDPS solitamente consentono agli amministratori di specificare la configurazione della capacità di prevenzione per ciascun tipo di avviso. Alcuni sensori IDPS hanno una modalità di apprendimento o simulazione che sopprime tutte le azioni di prevenzione e indica invece quando sarebbe stata eseguita un'azione di prevenzione. Ciò consente agli amministratori di monitorare e ottimizzare la configurazione delle funzionalità di prevenzione prima di abilitare azioni di prevenzione, riducendo così il rischio di bloccare inavvertitamente attività benigne.

Management - Gestione dei IDPS

La maggior parte dei prodotti IDPS offrono funzionalità di gestione simili: **implementazione, funzionamento e manutenzione.**

Implementazione

Una volta selezionato un prodotto IDPS, gli amministratori devono:

1. Il primo passo nell'implementazione di un IDPS è la **progettazione di un'architettura**. Le considerazioni architettoniche includono quanto segue: dove devono essere posizionati i sensori o gli agenti, stabilire il livello di affidabilità che si vuole ottenere, dove posizionare gli altri componenti dell'IDPS, stabilire con quali altri sistemi dovrà interfacciarsi l'IDPS (es. firewall, router, altri sistemi di gestione, sistemi a cui fornisce dati..), stabilire se verrà utilizzata una rete di gestione dedicata e stabilire nuovi controlli di sicurezza al fine di implementare correttamente l'IDPS.

2. Il secondo passo è quello di eseguire test sui componenti IDPS, e le organizzazioni dovrebbero prendere in considerazione di implementare le componenti prima in una **rete di test**, per ridurre la probabilità che problemi di implementazione interrompano la rete produttiva. Una volta eseguiti i test in una rete di test le componenti IDPS possono essere implementate nella rete di produzione ed è importante non sovraccaricare da subito la rete di sensori e agenti, ma di riempirla in modo graduale, così da evitare sovraccarichi e malfunzionamenti.
3. Il terzo passo è quello di proteggere i componenti degli IDPS perché sono spesso presi di mira dagli aggressori, poiché spesso contengono informazioni sensibili e vulnerabilità note che potrebbero essere utili nella pianificazione di ulteriori attacchi. È importante che gli admin configurino firewall, router e altri dispositivi di filtraggio dei pacchetti per limitare l'accesso diretto a tutti i componenti IDPS solo agli host che ne hanno il permesso.

Funzionamento e Manutenzione

Quasi tutti i prodotti IDPS sono progettati per essere gestiti e mantenuti tramite un'interfaccia utente grafica (GUI), nota anche come **console**. La console in genere consente agli amministratori di: configurare e aggiornare i sensori e i server di gestione, nonché monitorarne lo stato. Anche gli amministratori possono gestire gli account utente, personalizzare i report e eseguire molte altre funzioni utilizzando la console.

La maggior parte degli IDPS consente agli amministratori di concedere a ciascun account solo i privilegi necessari per il ruolo di ciascuna persona. La console spesso riflette ciò mostrando menu e opzioni diversi in base al ruolo designato dell'account attualmente autenticato.

Alcuni prodotti IDPS offrono anche interfacce della riga di comando (**CLI**) che vengono generalmente utilizzate per la gestione locale di sensori e agenti. A volte è possibile raggiungere una CLI in remoto tramite SSH.

Tipologie di IDPS

Gli IDPS possono essere suddivisi in diverse tipologie in base al loro scopo, alla loro posizione nell'architettura di rete e alle metodologie di rilevamento. Ecco alcune delle principali tipologie di IDPS:

1. **Network-Based IDPS (NIDPS)**
2. **Host-Based IDPS (HIDPS)**
3. **Network-Behavior Analysis (NBA)**
4. **Wireless**

Network-Based IDPS (NIDPS)

Un Network-Based IDPS **monitora** il traffico di rete per particolari segmenti o dispositivi di rete, e **analizza** i protocolli di rete, trasporto e applicazione per identificare attività sospette.

Componenti tipiche

Tutte le componenti di un network-based IDPS sono simili ad altri tipi di tecnologie, *ad eccezione dei sensori*. Un **sensore NIDPS** monitora e analizza l'attività di rete su uno o più segmenti di rete. Le schede di interfaccia di rete che eseguiranno il monitoraggio accetteranno tutti i pacchetti in arrivo che vedono indipendentemente dalle destinazioni previste. La maggior parte delle implementazioni IDPS utilizzano più sensori, con implementazioni di grandi dimensioni che hanno centinaia di sensori. I sensori sono disponibili in due formati:

Apparecchio/Appliance: un sensore basato su un dispositivo è composto da hardware specializzato e software del sensore. L'hardware è in genere ottimizzato per l'uso dei sensori, inclusi NIC specializzati e driver NIC per l'acquisizione efficiente dei pacchetti e processori specializzati o altri componenti hardware che aiutano nell'analisi. Parti o l'intero software IDPS potrebbero risiedere nel firmware per una maggiore efficienza. Le apparecchiature utilizzano spesso un sistema operativo personalizzato e potenziato a cui gli amministratori non sono destinati ad accedere direttamente.

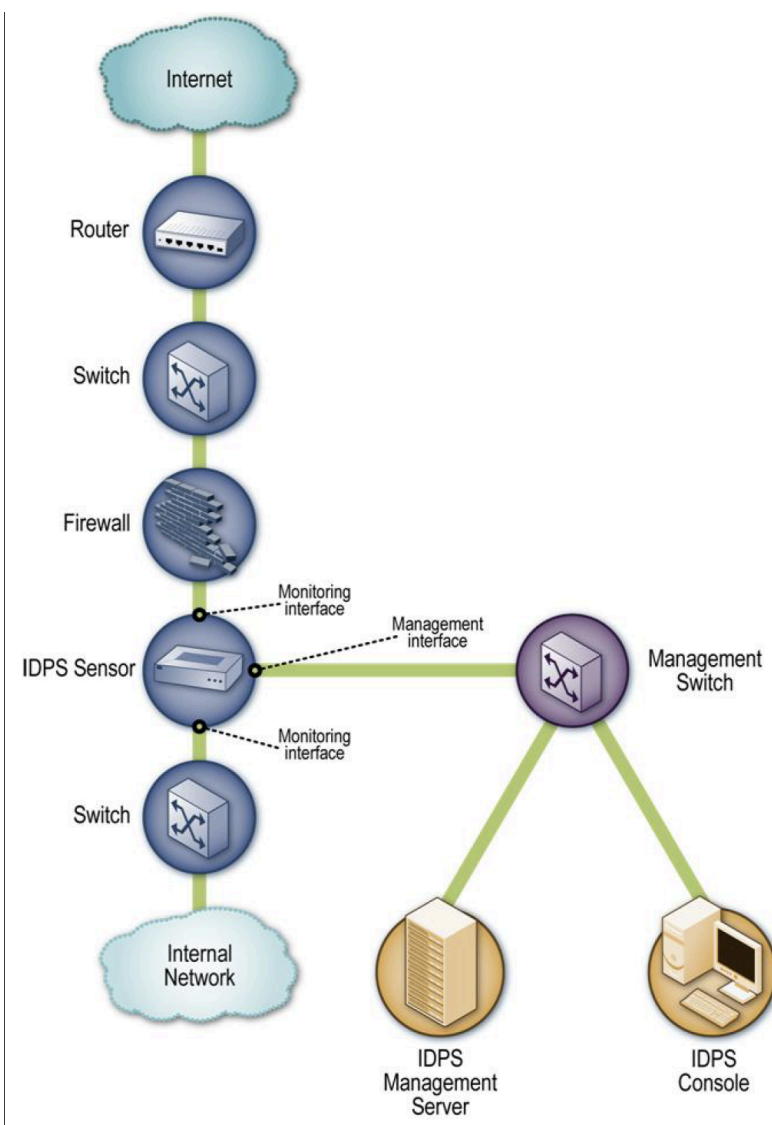
Solo software/Only Software: alcuni venditori vendono software per sensori senza apparecchio. Gli amministratori possono installare il software su host che soddisfano determinate specifiche. Il software del sensore potrebbe includere un OS personalizzato oppure potrebbe essere installato su un OS standard proprio come farebbe qualsiasi altra applicazione.

Oltre a scegliere la rete appropriata per i componenti, gli amministratori devono anche decidere dove posizionare i sensori IDPS. I sensori possono essere implementati in due modalità: **InLines**, **Passive**.

InLine Sensors

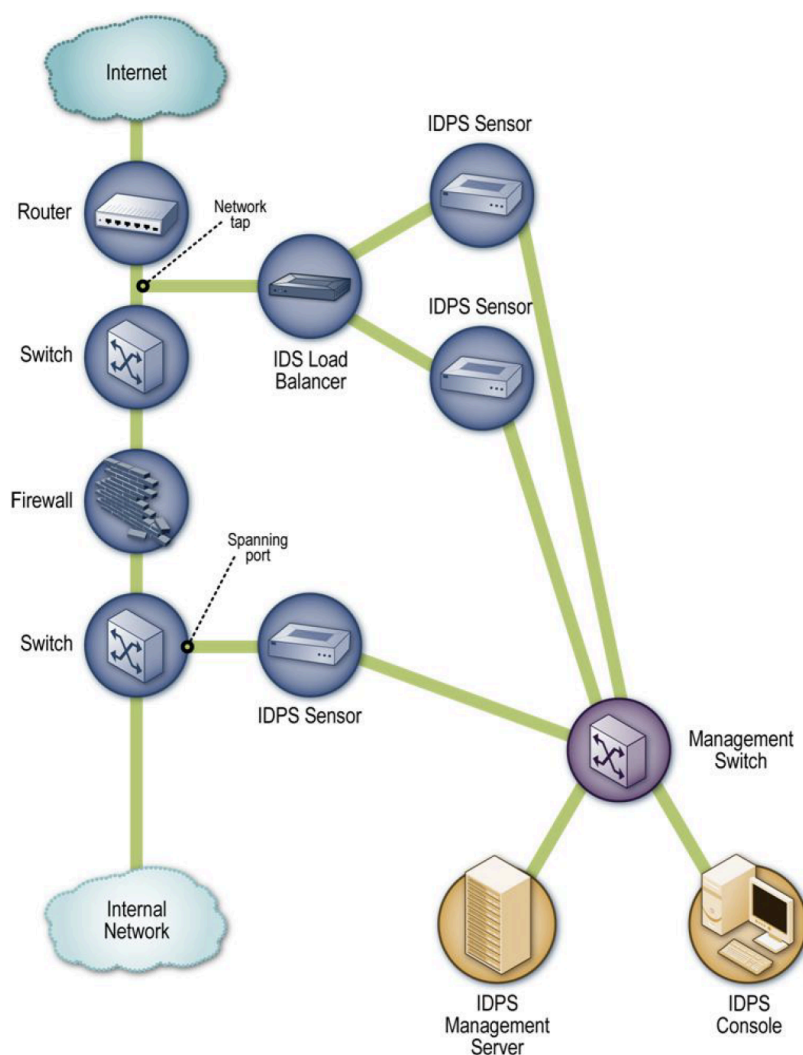
Viene distribuito un **InLine Sensor** in modo che il traffico di rete che sta monitorando lo attraversi, proprio come il flusso di traffico associato a un firewall. In effetti, alcuni **InLine Sensors** sono dispositivi ibridi **firewall/IDPS**, mentre altri sono semplicemente IDPS. La motivazione principale per l'implementazione dei sensori IDPS in linea è consentire loro di fermare gli attacchi bloccando il traffico di rete. Gli **InLine Sensors** vengono generalmente posizionati dove verrebbero posizionati i firewall di rete, ovvero nei punti di divisione tra le reti. I sensori in linea che **non** sono dispositivi ibridi firewall/IDPS vengono spesso distribuiti sul lato più sicuro di una divisione di rete in modo da **avere meno traffico da elaborare** (come

in figura). I sensori possono anche essere posizionati *sul lato meno sicuro* di una divisione di rete per *fornire protezione e ridurre il carico sul dispositivo di divisione*, come un firewall.



Passive Sensors

Viene utilizzato un **Passive Sensor** in modo da monitorare una copia del traffico di rete effettivo; nessun traffico effettivamente passa attraverso il sensore. I sensori passivi vengono generalmente distribuiti in modo da poter monitorare le posizioni chiave della rete, come le divisioni tra le reti, e i segmenti chiave della rete, come l'attività su una sottorete della zona demilitarizzata (DMZ).



Spanning Port: molti switch dispongono di una spanning port, ovvero una porta che può vedere tutto il traffico di rete che passa attraverso lo switch. Collegare un sensore a una spanning port può consentirgli di monitorare il traffico in entrata e in uscita da molti host. Sebbene questo metodo di monitoraggio sia relativamente semplice ed economico, può anche essere problematico.

Network Tap: una network tap è una connessione diretta tra un sensore e il supporto fisico della rete stessa, come un cavo in fibra ottica. Il Tap fornisce al sensore una copia di tutto il traffico di rete trasportato. Inoltre, a differenza delle porte spanning, che di solito sono già presenti in tutta un'organizzazione, i tap di rete devono essere acquistati come componenti aggiuntivi della rete.

IDS Load Balancer: un bilanciatore di carico IDS è un dispositivo che aggrega e indirizza il traffico di rete ai sistemi di monitoraggio, inclusi i sensori IDS. Un sistema di bilanciamento del carico può ricevere copie del traffico di rete da una o più porte spanning o tap di rete e



aggregare il traffico da reti diverse. Il sistema di bilanciamento del carico quindi distribuisce copie del traffico a uno o più dispositivi di ascolto, inclusi i sensori IDPS, in base a una serie di regole configurate da un amministratore.

Funzionalità di sicurezza degli IDPS Network-Based

Gli IDSP Network-Based permettono: **Information Gathering, Logging, Detection** e **Prevention**.

Host-Based IDPS (HIDPS)

Un Host-Based IDPS **monitora** le caratteristiche di **un singolo host** e gli eventi che si verificano all'interno di quest'ultimo, al fine di individuare attività sospette. Un HIDPS potrebbe monitorare il traffico di rete cablata e wireless (solo per quell'host), i registri di sistema, i processi in esecuzione, l'accesso e la modifica dei file e le modifiche alla configurazione del sistema e delle applicazioni.

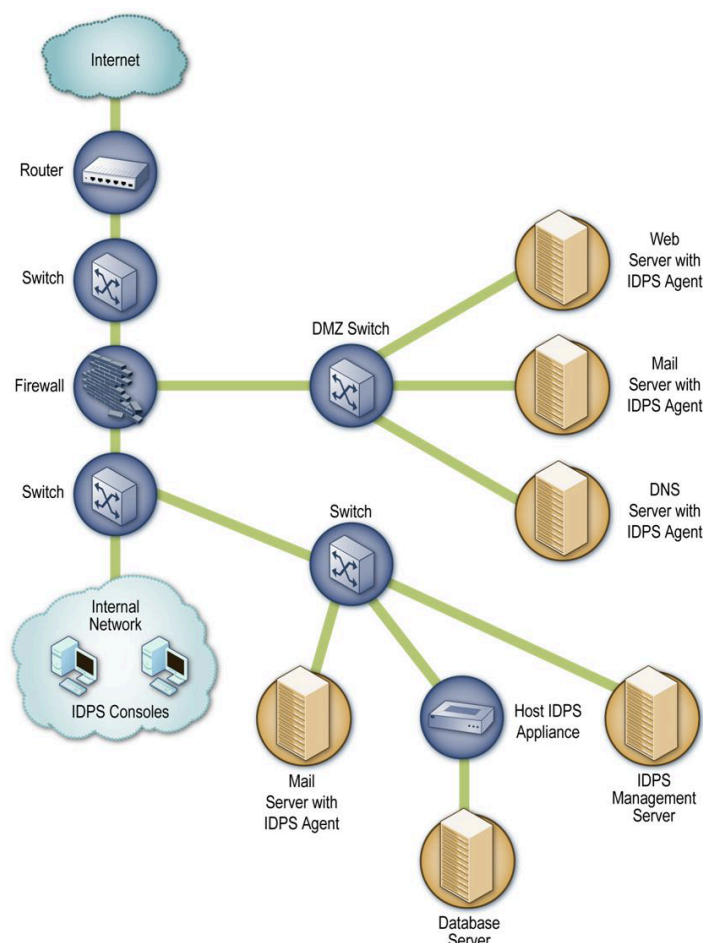
Componenti tipiche

La maggior parte degli HIDPS dispongono di software di rilevamento noti come **agenti** installati sugli host di interesse. Ciascun agente monitora l'attività su un singolo host e, se le funzionalità IDPS sono abilitate, esegue anche azioni di prevenzione. Gli agenti trasmettono i dati ai server di gestione, che possono facoltativamente utilizzare server di database per l'archiviazione. Le console vengono utilizzate per la gestione e il monitoraggio.

Ogni agente è in genere progettato per proteggere uno dei seguenti elementi: un server, un host o un servizio applicativo.

Architettura di rete

L'architettura di rete per le implementazioni HIDPS è in genere molto semplice. Poiché gli agenti vengono distribuiti su host esistenti sulle reti dell'organizzazione, i componenti solitamente comunicano su tali reti invece di utilizzare una rete di gestione separata. La maggior parte dei prodotti crittografa le comunicazioni, impedendo agli intercettatori di accedere a informazioni sensibili. Gli agenti basati su appliance vengono generalmente distribuiti in linea immediatamente davanti agli host che stanno proteggendo.



Posizioni degli agenti

Gli agenti IDPS basati su host vengono comunemente distribuiti su host critici come server accessibili pubblicamente e server contenenti informazioni sensibili. Tuttavia le organizzazioni potrebbero potenzialmente distribuire agenti sulla maggior parte dei propri server e desktop/laptop. Alcune organizzazioni utilizzano agenti HIDPS principalmente per analizzare attività che non possono essere monitorate da altri controlli di sicurezza. Ad esempio, i sensori NIDPS non possono analizzare l'attività all'interno delle comunicazioni di rete crittografate, ma gli agenti HIDPS installati sugli endpoint possono vedere l'attività non crittografata.

Architettura host

Per fornire funzionalità di prevenzione delle intrusioni, la maggior parte degli agenti HIDPS alterano l'architettura interna degli host su cui sono installati. Ciò avviene in genere tramite uno SHIM (spessore), ovvero uno strato di codice posizionato tra strati di codice esistenti. Uno shim intercetta i dati in un punto in cui normalmente verrebbero passati da un pezzo di codice a un altro, e può quindi analizzare i dati e determinare se debbano essere consentiti o negati. Alcuni agenti HIDPS non alterano l'architettura dell'host ma effettuano

monitoraggio e non utilizzano *shim*. Sono meno invasivi per l'host, riducendo la possibilità di interferenza con attività utente ma sono generalmente meno efficaci nel rilevare le minacce e spesso non possono eseguire alcuna azione di prevenzione.

Una delle decisioni importanti nella scelta di una soluzione HIDPS è se installare agenti sugli host o utilizzare dispositivi basati su agenti. L'installazione di agenti sugli host è generalmente preferibile perché gli agenti hanno accesso diretto alle caratteristiche degli host, spesso consentendo loro di eseguire un rilevamento e una prevenzione più completi e accurati. Tuttavia, gli agenti spesso supportano solo pochi sistemi operativi comuni; se un host non utilizza un sistema operativo supportato, è possibile invece distribuire un'appliance (apparecchio). Un altro motivo per utilizzare un'appliance sono le prestazioni; se un agente avesse un impatto negativo eccessivo sulle prestazioni dell'host monitorato, potrebbe essere necessario scaricare le funzioni dell'agente su un'appliance.

Wireless IDPS

Un IDPS wireless monitora il traffico della rete wireless e analizza i suoi protocolli di rete wireless per identificare attività sospette che coinvolgono i protocolli stessi. La rete wireless consente ai dispositivi con funzionalità wireless di utilizzare le risorse di elaborazione senza essere fisicamente connessi a una rete. I dispositivi devono semplicemente trovarsi entro una certa distanza (detta **portata**) dall'infrastruttura di rete wireless. Una rete locale wireless (WLAN) è un gruppo di nodi di rete wireless all'interno di un'area geografica **limitata** in grado di scambiare dati tramite comunicazioni radio.

Componenti tipiche

I componenti tipici di un IDPS wireless sono gli stessi di un IDPS basato su rete: **console**, **database server** (opzionali), **server di gestione** e **sensori**. Tutti i componenti, ad eccezione dei sensori, hanno essenzialmente la stessa funzionalità per entrambi i tipi di IDPS.

I **sensori wireless** svolgono lo stesso ruolo fondamentale dei sensori NIDPS, ma funzionano in modo molto diverso a causa della complessità del monitoraggio delle comunicazioni wireless.

Un IDPS wireless funziona campionando il traffico. Esistono due bande di frequenza da monitorare (2,4 GHz e 5 GHz, bande più comuni per le reti WI-FI) e ciascuna banda è separata in canali. Attualmente non è possibile per un sensore monitorare simultaneamente tutto il traffico su una banda; un sensore deve monitorare un singolo canale alla volta. Quando il sensore è pronto per monitorare un canale diverso, il sensore deve spegnere la radio, cambiare canale, e poi accendere la radio. Quanto più a lungo viene monitorato un singolo canale, tanto più è probabile che il sensore non rilevi attività dannose che si verificano su altri canali. Per evitare ciò, i **sensori in genere cambiano**

frequentemente canale, operazione nota come **channel scanning**, in modo da poter monitorare ciascun canale alcune volte al secondo.

Forme di sensori wireless

Dedicated: un sensore dedicato è un dispositivo che esegue funzioni IDPS wireless ma non trasmette il traffico di rete dall'origine alla destinazione. I sensori dedicati sono spesso completamente **passivi** e funzionano in modalità di monitoraggio a radiofrequenza (RF) per sniffare il traffico della rete wireless. Alcuni sensori dedicati eseguono l'analisi del traffico che monitorano, mentre altri sensori inoltrano il traffico di rete a un server di gestione per l'analisi. Il sensore è generalmente collegato alla rete cablata. I sensori dedicati sono generalmente progettati per uno dei due tipi di distribuzione:

1. **Fixed/Fissi:** il sensore viene distribuito in una posizione particolare. Tali sensori dipendono in genere dall'infrastruttura dell'organizzazione. I sensori fissi sono generalmente basati su apparecchi.
2. **Mobile:** il sensore è progettato per essere utilizzato durante il movimento. I sensori mobili sono basati su dispositivi o basati su software.

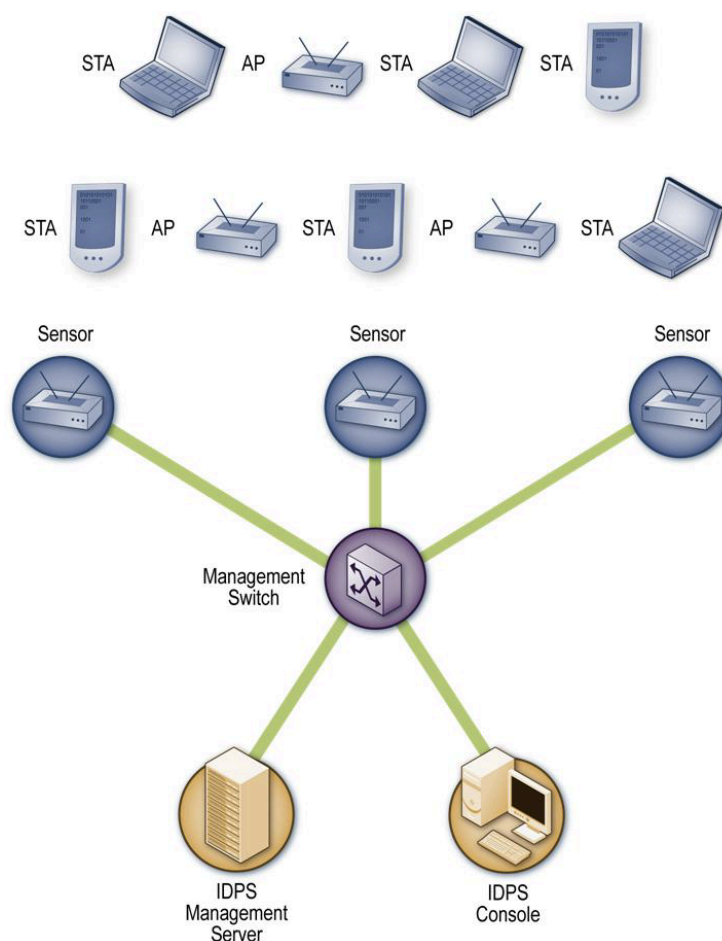
I sensori dedicati in genere offrono capacità di rilevamento più potenti rispetto ai sensori wireless forniti in bundle con AP o switch wireless.

Bundle con un AP: diversi fornitori hanno aggiunto funzionalità IDPS agli AP, un AP in bundle fornisce in genere una capacità di rilevamento meno rigorosa rispetto a un sensore dedicato perché l'AP deve dividere il proprio tempo tra fornire l'accesso alla rete e monitorare più canali o bande per attività dannose.

Bundle con uno switch wireless: gli switch wireless sono destinati ad assistere gli amministratori nella gestione e nel monitoraggio dei dispositivi wireless; alcuni di questi switch offrono anche alcune funzionalità IDPS wireless come funzione secondaria. Gli switch wireless in genere non offrono funzionalità di rilevamento potenti quanto gli AP in bundle o i sensori dedicati.

Architettura della rete wireless

I componenti IDPS wireless sono generalmente collegati tra loro tramite una rete cablata. Come con un NIDPS, è possibile utilizzare una rete di gestione separata o le reti standard dell'organizzazione per le comunicazioni dei componenti IDPS wireless. Poiché dovrebbe già esistere una separazione rigorosamente controllata tra le reti wireless e cablate, l'utilizzo di una rete di gestione o di una rete standard dovrebbe essere accettabile per i componenti IDPS wireless. Inoltre, alcuni sensori IDPS wireless (in particolare quelli mobili) vengono utilizzati in modo autonomo e non necessitano di connettività di rete cablata.



Posizioni dei sensori

La scelta della posizione dei sensori per una distribuzione IDPS wireless è un problema fondamentalmente diverso dalla scelta della posizione per qualsiasi altro tipo di sensore IDPS. Se l'organizzazione utilizza WLAN, **dovrebbero essere installati sensori wireless in modo da monitorare la portata RF delle WLAN dell'organizzazione**. Molte organizzazioni desiderano inoltre implementare sensori per monitorare le regioni fisiche delle proprie strutture dove non dovrebbe esserci attività WLAN, nonché canali e bande che le WLAN dell'organizzazione non dovrebbero utilizzare, come modo per **rilevare AP non autorizzati**. Altre considerazioni per la selezione delle posizioni dei sensori wireless includono quanto segue:

1. **Sicurezza fisica:** I sensori vengono spesso installati in luoghi aperti perché la loro portata è molto maggiore lì che in luoghi chiusi. In genere, i sensori in luoghi interni ed esterni aperti sono più suscettibili alle minacce fisiche rispetto ad altri sensori. Se le minacce fisiche sono significative, le organizzazioni potrebbero dover selezionare sensori con **funzionalità anti-manomissione** o implementare sensori controllati (es da una telecamera di sicurezza).

2. *Sensor range*: la portata effettiva di un sensore varia in base alle strutture circostanti (ad esempio muri, porte).
3. *Connessioni di rete cablate*: in genere i sensori devono essere collegati alla rete cablata, per cui deve essere possibile il collegamento alla rete cablata.
4. *Costo*: ponderare il beneficio-costo con il rischio.
5. *Posizioni degli AP e degli switch wireless*: se una soluzione in bundle soddisfacesse gli altri requisiti dell'organizzazione, allora le posizioni degli AP e degli switch wireless sono particolarmente importanti perché il software IDPS wireless potrebbe essere potenzialmente distribuito su tali dispositivi.

Network-Behavior Analysis System (NBA)

Un NBA system esamina il traffico di rete o le statistiche sul traffico di rete per **identificare flussi di traffico insoliti**, come attacchi **DDoS** (Distributed Denial of Service), alcune forme di **malware** (es worm, backdoor) e **violazioni delle policy** (es, un sistema client che fornisce servizi di rete ad altri sistemi).

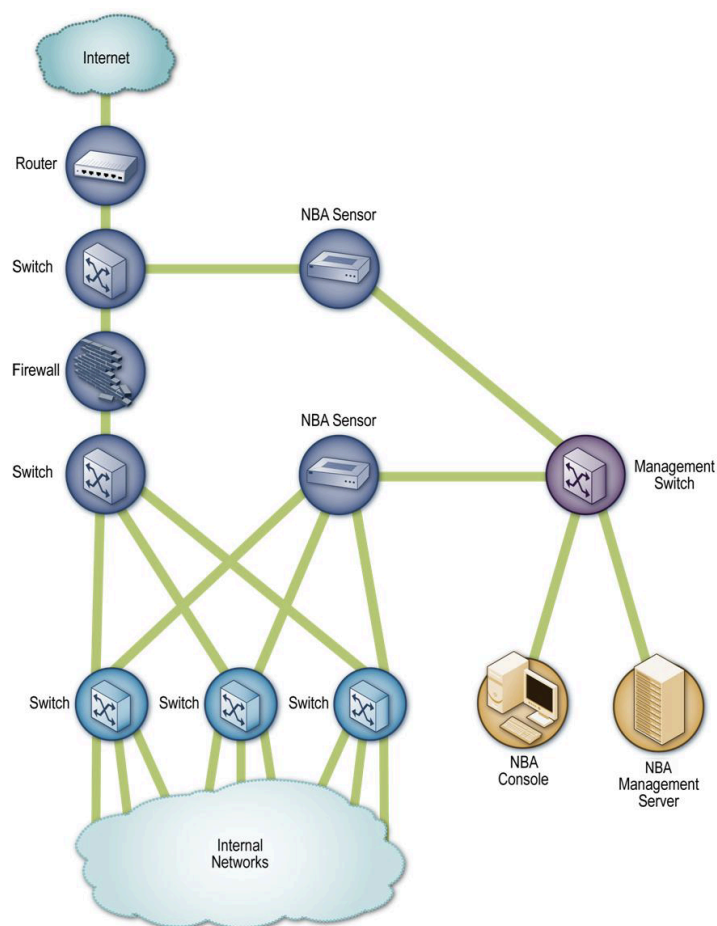
Componenti tipiche

Le soluzioni NBA solitamente dispongono di **sensori** e **console**, e alcuni prodotti offrono anche **server di gestione** (a volte chiamati analizzatori). I sensori NBA sono generalmente disponibili solo come appliance/apparecchi. Alcuni sensori sono simili ai sensori NIDPS in quanto sniffano i pacchetti per monitorare l'attività di rete su uno o pochi segmenti di rete. Altri sensori NBA non monitorano direttamente le reti, ma si basano invece sulle informazioni sul flusso di rete fornite dai router e da altri dispositivi di rete. Il flusso si riferisce a una particolare sessione di comunicazione che si verifica tra host. I dati di flusso tipici particolarmente rilevanti per il rilevamento e la prevenzione delle intrusioni includono quanto segue:

- Indirizzi IP di origine e destinazione
- Porte TCP o UDP di origine e destinazione o tipi e codici ICMP
- Numero di pacchetti e numero di byte trasmessi nella sessione
- Timestamp per l'inizio e la fine della sessione.

Architettura rete

Come con un NIDPS, per le comunicazioni dei componenti NBA è possibile utilizzare una **rete di gestione separata** o le reti standard dell'organizzazione. Se vengono utilizzati sensori che raccolgono dati sul flusso di rete da altri dispositivi, l'intera soluzione NBA può essere logicamente separata dalle reti standard.



Posizioni dei sensori

La maggior parte dei sensori NBA può essere implementata solo in modalità passiva, utilizzando gli stessi metodi di connessione (es: tap di rete, switch spanning port) degli NIDPS. I sensori passivi che eseguono il monitoraggio diretto della rete devono essere posizionati in modo da poter monitorare le posizioni chiave della rete, come le divisioni tra reti, e i segmenti chiave della rete, come le sottoreti della zona demilitarizzata (DMZ). Gli InLine Sensors sono generalmente destinati all'uso perimetrale della rete, quindi verrebbero distribuiti in prossimità dei firewall perimetrali, spesso tra il firewall e il router di confine Internet per limitare gli attacchi in entrata che potrebbero sopraffare il firewall.

CAPITOLO 12

Sebbene l'accesso a Internet offra vantaggi alle organizzazioni, consente al mondo esterno di raggiungere e interagire con le risorse della rete locale. Ciò crea una minaccia per le organizzazioni. Anche se è possibile dotare ogni workstation e server della rete locale di efficaci funzionalità di sicurezza, come la IDPS, ciò potrebbe non essere sufficiente e in alcuni casi non è conveniente. Un'alternativa ampiamente accettata o almeno un complemento ai servizi di sicurezza basati su host è il **firewall**. Il firewall viene inserito tra la rete della sede e Internet per stabilire un collegamento controllato ed erigere un muro o perimetro di sicurezza esterno. Lo scopo di questo perimetro è **proteggere la rete locale dagli attacchi basati su Internet e fornire un unico punto di strozzatura in cui è possibile imporre sicurezza e controllo**. Il firewall può essere un singolo sistema informatico o un insieme di due o più sistemi che cooperano per eseguire la funzione firewall. Il firewall, quindi, fornisce un ulteriore livello di difesa, isolando i sistemi interni dalle reti esterne.

Definizione Firewall di Rete → I firewall di rete sono dispositivi o sistemi che controllano il flusso del traffico di rete tra reti che utilizzano diversi livelli di sicurezza. (NIST)

Caratteristiche dei Firewall e Politiche di accesso

I Firewall perseguono i seguenti obiettivi:

1. **Tutto il traffico** dall'interno verso l'esterno e viceversa deve **passare attraverso il firewall**, ciò si ottiene bloccando fisicamente tutti gli accessi alla rete locale tranne che tramite il firewall.
2. Viene **consentito il passaggio solo al traffico autorizzato**, secondo quanto definito dalla politica di sicurezza locale. Possono essere utilizzati vari tipi di firewall, che implementano vari tipi di politiche di sicurezza.
3. Il firewall stesso è **immune alla penetrazione**. Ciò implica l'uso di un sistema rafforzato con un sistema operativo protetto. I sistemi informatici affidabili sono adatti per ospitare un firewall e spesso richiesti nelle applicazioni governative.

Una componente critica nella pianificazione e implementazione di un firewall è la definizione di una politica di accesso adeguata. Elenca i tipi di traffico autorizzati a passare attraverso il firewall, inclusi intervalli di indirizzi, protocolli, applicazioni e tipi di contenuto. Una policy di accesso del firewall potrebbe utilizzare diverse caratteristiche/filtri per filtrare il traffico, tra cui:

- **Indirizzi IP e valori del protocollo**: controlla l'accesso in base agli indirizzi di origine o di destinazione e ai numeri di porta, alla direzione del flusso in entrata o in uscita e ad altre caratteristiche della rete e del livello di trasporto.

- **Application protocol:** controlla l'accesso sulla base dei dati del protocollo applicativo autorizzato. Questo tipo di filtro viene utilizzato da un gateway a livello di applicazione che inoltra e monitora lo scambio di informazioni per protocolli applicativi specifici, ad esempio controllando la posta elettronica SMTP per lo spam o le richieste Web HTTP solo ai siti autorizzati.
- **User Identity:** controlla l'accesso in base all'identità dell'utente, in genere per gli utenti interni che si identificano utilizzando una qualche forma di tecnologia di autenticazione sicura, come IPSec.
- **Network Activity:** controlla l'accesso in base a considerazioni quali l'ora o la richiesta, (ad es: solo durante l'orario lavorativo); tasso di richieste, (ad es: per rilevare i tentativi di scansione); o altri modelli di attività.

Cosa può fare un Firewall - PRO

1. Un firewall definisce un singolo punto di strozzatura che mantiene gli utenti non autorizzati fuori dalla rete protetta, impedisce ai servizi potenzialmente vulnerabili di entrare o uscire dalla rete e fornisce protezione da vari tipi di spoofing IP e attacchi di routing.
2. Un firewall fornisce una posizione per monitorare gli eventi relativi alla sicurezza. **Log Function** molto importante.
3. Schermare alcune reti interne e nasconderele all'esterno.
4. Bloccare alcuni accessi a servizi o ad utenti.
5. Proteggere le risorse della rete privata da attacchi esterni.
6. Prevenire l'esportazione di dati dall'interno verso l'esterno.

Cosa non riesce a fare un Firewall - CONTRO

1. Un firewall non può proteggere da attacchi che aggirano il firewall stesso.
2. Un firewall potrebbe non proteggere completamente da minacce interne (es. dipendente scontento).
3. Un firewall non difende contro nuovi bachi non ancora documentati nei protocolli.
4. I filtri del firewall sono difficili da settare e da mantenere perché va trovato un difficile compromesso tra libertà e sicurezza.
5. Può degradare le performance della rete.

Tipi di Firewall

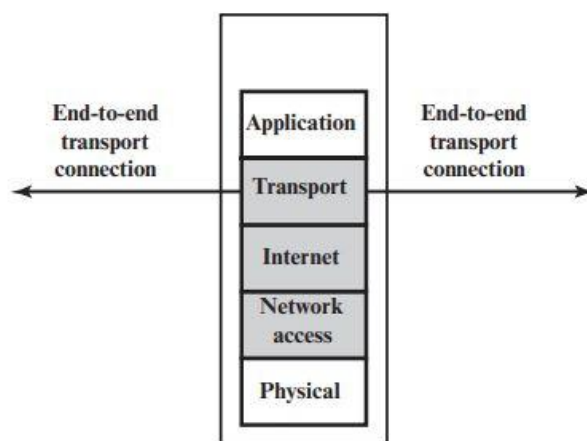
Un firewall può monitorare il traffico di rete a diversi livelli, dai pacchetti di rete di basso livello individualmente o come parte di un flusso, a tutto il traffico all'interno di una connessione di trasporto, fino all'ispezione dei dettagli dei protocolli applicativi. La scelta del livello appropriato è determinata dalla politica di accesso al firewall desiderata. Può

funzionare come **filtro positivo**, consentendo di far passare solo i pacchetti che soddisfano criteri specifici, o come **filtro negativo**, rifiutando qualsiasi pacchetto che soddisfi determinati criteri. I criteri implementano la politica di accesso, di cui abbiamo parlato nella sezione precedente. A seconda del tipo di firewall, può esaminare una o più intestazioni di protocollo in ciascun pacchetto, il payload di ciascun pacchetto o il modello generato da una sequenza di pacchetti. In questa sezione esamineremo i principali tipi di firewall.

Packet Filtering Firewalls

Un firewall di filtraggio dei pacchetti applica una serie di regole a ciascun pacchetto IP in entrata e in uscita e quindi **inoltra** o **scarta** il pacchetto. Il firewall è generalmente configurato per filtrare i pacchetti che vanno in entrambe le direzioni (da e verso la rete interna). Le regole di filtraggio si basano sulle informazioni contenute in un pacchetto di rete: **IP sorgente**, **IP destinazione**, **indirizzo a livello di trasporto di origine e destinazione** ovvero il numero di porta a livello di trasporto che definisce applicazioni come SNMP o TELNET(es. numero porta TCP o UDP), **campo protocollo ip** che definisce il protocollo a livello di trasporto e **interfaccia** ovvero da quale interfaccia del firewall proviene il pacchetto o a quale interfaccia del firewall è destinato. Il **packet filter** è generalmente impostato come un **elenco di regole basate sulle corrispondenze con i campi nell'intestazione IP o TCP**. Se esiste una corrispondenza con una delle regole, tale regola viene richiamata per determinare se inoltrare o scartare il pacchetto. Se non c'è corrispondenza con alcuna regola, viene eseguita un'azione predefinita. Sono possibili due politiche predefinite:

1. Default = **DENY**/scarta: ciò che non è espressamente consentito è proibito.
2. Default = **PERMIT**/avanti: ciò che non è espressamente vietato è consentito.



(b) Packet filtering firewall

I **vantaggi** di un packet filtering firewall sono: la sua semplicità, trasparenza per l'utente e alta velocità.

Gli **svantaggi** di un packet filtering firewall sono: poiché tali firewall non esaminano i dati a livello superiore, non possono impedire attacchi che sfruttano vulnerabilità a livello applicativo; la maggior parte di tali firewall non supporta schemi avanzati di autenticazione; a causa delle limitate informazioni disponibili per tali firewall la funzionalità di Logging è molto limitata e inefficace; infine c'è difficoltà nel realizzare buone/corrette regole di filtraggio.

Alcuni degli attacchi che possono essere sferrati ai firewall di filtraggio dei pacchetti e le contromisure appropriate sono i seguenti:

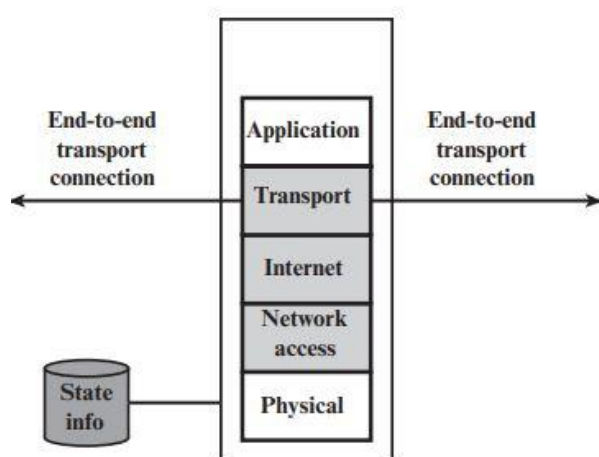
- **Spoofing dell'indirizzo IP**: l'intruso trasmette pacchetti dall'esterno con un campo dell'indirizzo IP di origine contenente l'indirizzo di un host interno. L'aggressore spera che l'uso di un indirizzo falsificato consenta la penetrazione di sistemi che utilizzano la semplice sicurezza dell'indirizzo di origine, in cui vengono accettati pacchetti provenienti da specifici host interni fidati. La contromisura consiste nello scartare i pacchetti con un indirizzo di origine interno se il pacchetto arriva su un'interfaccia esterna. In effetti, questa contromisura viene spesso implementata sul router esterno al firewall.
- **Source routing attacks**: la stazione di origine specifica il percorso che un pacchetto dovrebbe seguire mentre attraversa Internet, nella speranza che ciò aggiri le misure di sicurezza che non analizzano le informazioni di routing di origine. La contromisura è scartare tutti i pacchetti che utilizzano questa opzione.
- **Tiny fragment attacks/Attacchi con frammenti minuscoli**: l'intruso utilizza l'opzione di frammentazione IP per creare frammenti estremamente piccoli e forzare le informazioni dell'intestazione TCP in un frammento di pacchetto separato. Questo attacco è progettato per eludere le regole di filtraggio che dipendono dalle informazioni dell'intestazione TCP. Tipicamente, un filtro di pacchetto prenderà una decisione di filtraggio sul primo frammento di un pacchetto. Tutti i frammenti successivi di quel pacchetto vengono filtrati esclusivamente sulla base del fatto che fanno parte del pacchetto il cui primo frammento è stato rifiutato. L'aggressore spera che il firewall di filtraggio esamini solo il primo frammento e che i frammenti rimanenti vengano attraversati. La contromisura si ottiene applicando una regola secondo la quale il primo frammento di un pacchetto deve contenere una quantità minima predefinita di intestazione di trasporto. Se il primo frammento viene rifiutato, il filtro può ricordare il pacchetto e scartare tutti i frammenti successivi.

Stateful Inspection Firewalls

Un tradizionale packet filter prende decisioni di filtraggio sulla base del **singolo pacchetto** e non prende in considerazione alcun contesto di livello superiore. Per comprendere cosa si intende per contesto e perché un filtro di pacchetti tradizionale è limitato rispetto al contesto, è necessaria un po' di conoscenza generale. La maggior parte delle applicazioni standardizzate eseguite su TCP seguono un modello client/server. Ad esempio, per SMTP, la posta elettronica viene trasmessa da un client a un server. Il client genera nuovi messaggi di posta elettronica, in genere dall'input dell'utente. Il server accetta i messaggi di posta elettronica in arrivo e li inserisce nelle cassette postali degli utenti appropriati. SMTP funziona impostando una connessione TCP tra client e server, in cui il **numero di porta del server TCP**, che identifica l'applicazione del server SMTP, è **25**. Il **numero di porta TCP per il client SMTP** è un numero compreso tra **1024 e 65535** generato dal client SMTP. In generale, quando un'applicazione che utilizza TCP crea una sessione con un host remoto, crea una connessione TCP in cui il numero di porta TCP per l'applicazione remota (server) è un numero **inferiore a 1024** e il numero di porta TCP per l'applicazione locale (client) è un numero compreso **tra 1024 e 65535**. I numeri inferiori a 1024 sono i numeri di porta **"noti"** e vengono assegnati in modo permanente a particolari applicazioni. I numeri compresi tra 1024 e 65535 vengono generati dinamicamente e hanno un significato temporaneo solo per la durata di una connessione TCP. Uno *stateful packet firewall* rafforza le regole per il traffico TCP **creando una directory di connessioni TCP in uscita**, come mostrato nella Tabella 12.2. Per ogni connessione attualmente stabilita è presente una voce. Il packet filter ora consentirà il traffico in entrata verso le porte con numero alto, solo per quei pacchetti che corrispondono al profilo di una delle voci in questa directory. Un *stateful packet firewall* esamina le stesse informazioni sui pacchetti di un packet filtering firewall, ma **registra anche informazioni sulle connessioni TCP** (Figura c).

Table 12.2 Example Stateful Firewall Connection State Table (SP 800-41-1)

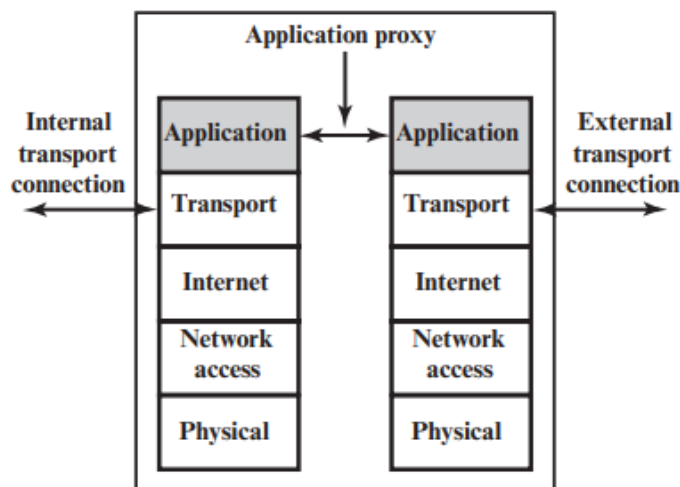
Source Address	Source Port	Destination Address	Destination Port	Connection State
192.168.1.100	1030	210.22.88.29	80	Established
192.168.1.102	1031	216.32.42.123	80	Established
192.168.1.101	1033	173.66.32.122	25	Established
192.168.1.106	1035	177.231.32.12	79	Established
223.43.21.231	1990	192.168.1.6	80	Established
2122.22.123.32	2112	192.168.1.6	80	Established
210.922.212.18	3321	192.168.1.6	80	Established
24.102.32.23	1025	192.168.1.6	80	Established
223.21.22.12	1046	192.168.1.6	80	Established



(c) Stateful inspection firewall

Application-Level Gateway

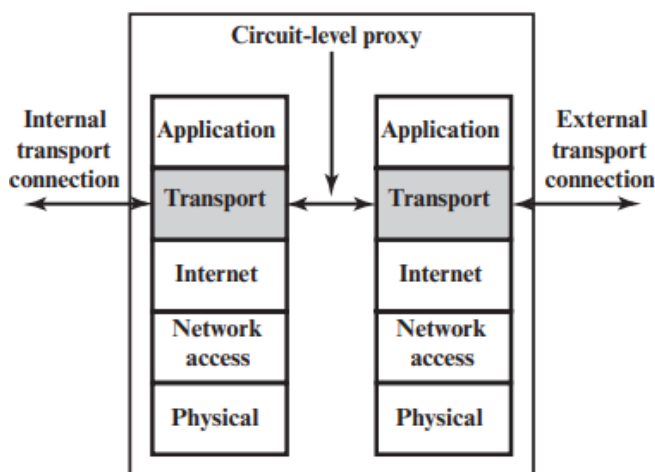
Un gateway a livello di applicazione, chiamato anche **application proxy gateway**, funge da relè del traffico a livello di applicazione (Figura d). L'utente contatta il gateway utilizzando un'applicazione TCP/IP, come Telnet o FTP, e il gateway chiede all'utente il nome dell'host remoto a cui accedere. Quando l'utente risponde e fornisce un ID utente valido e informazioni di autenticazione, il gateway *contatta l'applicazione sull'host remoto* e *inoltra i segmenti TCP contenenti i dati dell'applicazione tra i due endpoint*. Se il gateway non implementa il codice proxy per un'applicazione specifica, il servizio non è supportato e non può essere inoltrato attraverso il firewall. Inoltre, il gateway può essere configurato per supportare solo funzionalità specifiche di un'applicazione che l'amministratore di rete considera accettabili negando tutte le altre funzionalità. I gateway a livello di applicazione tendono ad essere più sicuri dei packet filter. Piuttosto che cercare di gestire le numerose possibili combinazioni che devono essere consentite e vietate a livello TCP e IP, il gateway a livello di applicazione deve solo esaminare alcune applicazioni consentite. Inoltre, è facile registrare e controllare tutto il traffico in entrata a livello di applicazione. Uno svantaggio principale di questo tipo di gateway è il sovraccarico di elaborazione aggiuntivo su ciascuna connessione. In effetti, ci sono due connessioni di giunzione tra gli utenti finali, con il gateway nel punto di giunzione e il gateway deve esaminare e inoltrare tutto il traffico in entrambe le direzioni.



(d) Application proxy firewall

Circuit-Level Gateway

Un quarto tipo di firewall è circuit-level gateway o circuit-level proxy (Figura e). Può trattarsi di un sistema autonomo o di una funzione specializzata eseguita da un gateway a livello di applicazione per determinate applicazioni. Come nel caso di un gateway applicativo, un gateway a livello di circuito non consente una connessione TCP end-to-end; piuttosto, il gateway stabilisce due connessioni TCP, una ***tra sé e un utente TCP su un host interno*** e una ***tra sé e un utente TCP su un host esterno***. Una volta stabilite le due connessioni, il gateway in genere inoltra i segmenti TCP da una connessione all'altra senza esaminare il contenuto. La funzione di sicurezza consiste nel determinare quali connessioni saranno consentite. Un utilizzo tipico dei gateway a livello di circuito è una situazione in cui l'amministratore di sistema si fida degli utenti interni. Il gateway può essere configurato per supportare il servizio proxy sulle connessioni in entrata e funzioni a livello di circuito per le connessioni in uscita. In questa configurazione, il gateway può sostenere il sovraccarico di elaborazione derivante dall'esame dei dati dell'applicazione in ingresso per funzioni vietate, ma non sostiene tale sovraccarico sui dati in uscita.



(e) Circuit-level proxy firewall

Firewall Basing

È prassi comune basare un firewall su una macchina autonoma che esegue un sistema operativo comune, come UNIX o Linux. La funzionalità firewall può essere implementata anche come modulo software in un router o switch LAN. In questa sezione verranno esaminate alcune considerazioni aggiuntive relative al firewall.

Bastion Host

Un bastion host (computer bastione) è un computer specializzato nell'isolare una rete locale da una connessione pubblica, creando uno **scudo** che permette di proteggere la rete locale. Una caratteristica fondamentale del bastion host è il possesso di ottime capacità di difesa, risultando teoricamente inattaccabile. In genere, il bastion host funge da piattaforma per un gateway a livello di applicazione o di circuito. Le caratteristiche tipiche di un bastion host sono le seguenti:

- Presenza dei soli servizi essenziali;
- Accesso limitato solo ai servizi specifici;
- Composizione modulare, con moduli software di piccole dimensioni, facilmente manutenibili e ispezionabili.

Host-Based Firewalls

Un Host-Based Firewall è un modulo software utilizzato per proteggere un singolo host. Tali moduli sono disponibili in molti sistemi operativi o possono essere forniti come pacchetto aggiuntivo. Come i firewall autonomi convenzionali, i firewall residenti sull'host filtrano e

limitano il flusso di pacchetti. Una posizione comune per tali firewall è un server. Esistono diversi vantaggi nell'utilizzo di un firewall basato su server o workstation:

- Le regole di filtraggio possono essere personalizzate in base all'ambiente host.
- La protezione è fornita indipendentemente dalla topologia, pertanto sia gli attacchi interni che quelli esterni devono passare attraverso il firewall.
- Utilizzato insieme ai firewall autonomi, l'host-based firewall fornisce un ulteriore livello di protezione.

Personal Firewalls

Un personal firewall controlla il traffico tra un personal computer o una workstation da un lato e Internet o la rete aziendale dall'altro. La funzionalità del personal firewall può essere utilizzata nell'ambiente domestico e nelle intranet aziendali. In genere, il personal firewall è un modulo software sul personal computer. In un ambiente domestico con più computer connessi a Internet, la funzionalità firewall può anche essere ospitata in un router che collega tutti i computer domestici a una DSL, un modem via cavo o un'altra interfaccia Internet. I personal firewall sono in genere molto meno complessi dei firewall basati su server o dei firewall autonomi. Il ruolo principale del personal firewall è negare l'accesso remoto non autorizzato al computer, può anche monitorare l'attività in uscita nel tentativo di rilevare e bloccare worm e altro malware.

Ubicazione e configurazione del Firewall

Un firewall è posizionato in modo da risultare una barriera protettiva tra una fonte di traffico esterna, potenzialmente non attendibile, e una rete interna. Tenendo ciò a mente, un amministratore della sicurezza deve decidere l'ubicazione e il numero di firewall necessari, in seguito esaminiamo alcune opzioni comuni:

DMZ - Reti demilitarizzate

La Figura 12.2 suggerisce la distinzione più comune, quella tra un firewall interno ed uno esterno. Un firewall esterno viene posizionato ai margini di una rete locale/aziendale, appena all'interno del router di confine che si connette a Internet. Uno o più firewall interni proteggono la maggior parte della rete aziendale. Tra questi due tipi di firewall ci sono uno o più dispositivi collegati in rete *in una regione denominata rete DMZ (zona demilitarizzata)*. I sistemi accessibili dall'esterno ma che necessitano di alcune protezioni si trovano solitamente su reti DMZ. In genere, *i sistemi nella DMZ richiedono o favoriscono la connettività esterna*, come un server WEB, un server SMTP o un server DNS. Il firewall esterno fornisce una misura di controllo e protezione degli accessi per i sistemi DMZ coerente con la loro necessità di connettività esterna. Il firewall esterno fornisce inoltre un

livello di protezione di base per il resto della rete aziendale. In questo tipo di configurazione, i firewall interni hanno tre scopi:

1. aggiunge capacità di filtraggio più rigorose rispetto al firewall esterno al fine di proteggere i server e le workstation aziendali da attacchi esterni.
2. fornisce protezione bidirezionale rispetto alla DMZ, ovvero protegge il resto della rete interna da possibili attacchi lanciati dai sistemi nella DMZ ma protegge anche la DMZ da possibili attacchi provenienti dalla rete interna.
3. è possibile utilizzare più firewall interni al fine di proteggere parti della rete interna le une dalle altre.

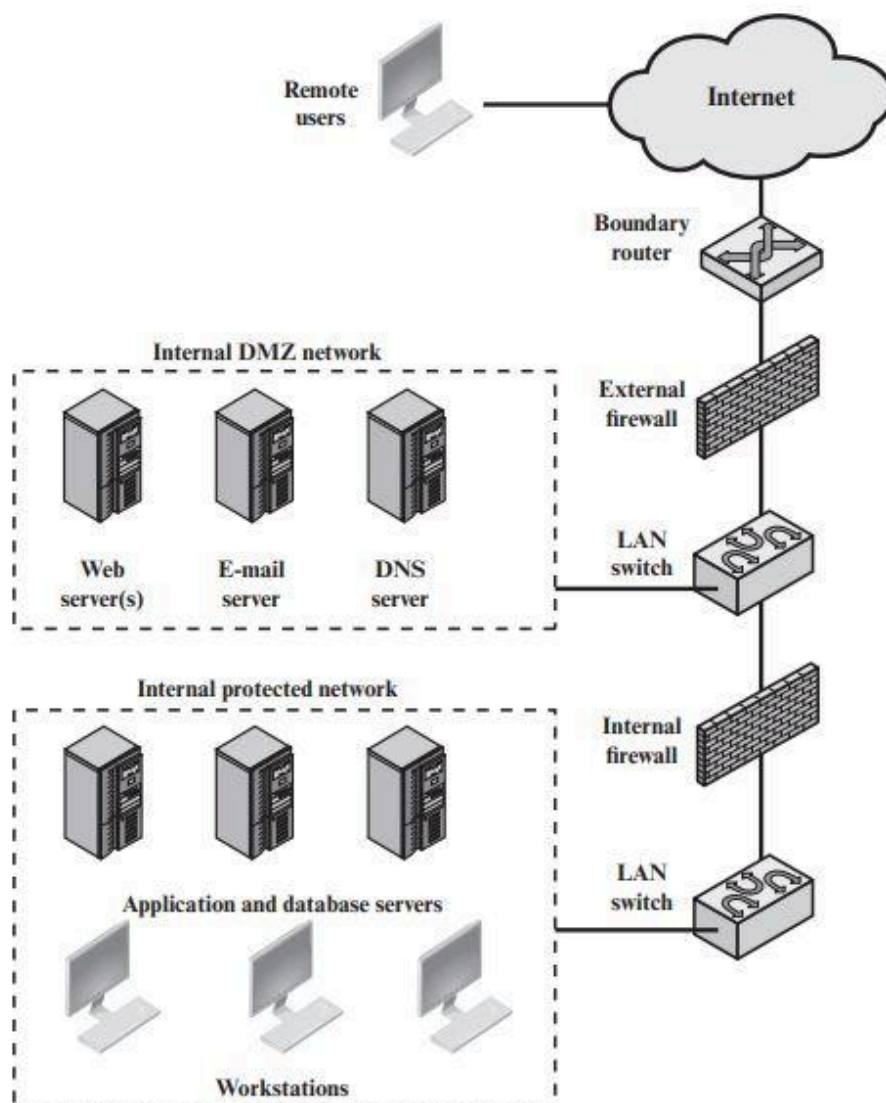


Figure 12.2 Example Firewall Configuration

Virtual Private Networks

Una VPN è costituita da un insieme di computer che si interconnettono tramite una rete relativamente non sicura e che utilizzano crittografia e protocolli speciali per garantire la sicurezza. In ogni sede aziendale, workstation, server e database sono collegati da una o più reti locali (LAN). Può essere utilizzata Internet o qualche altra rete pubblica per interconnettere le sedi, garantendo un risparmio sui costi rispetto all'uso di una rete privata e scaricando il compito di gestione della rete geografica al fornitore della rete pubblica. *Ma il manager si trova di fronte ad un'esigenza fondamentale: la sicurezza.* L'uso di una rete pubblica espone il traffico aziendale alle intercettazioni e fornisce un punto di ingresso per utenti non autorizzati. *Per contrastare questo problema, è necessaria una VPN.* In sostanza, una VPN *utilizza la crittografia e l'autenticazione nei livelli di protocollo inferiori per fornire una connessione sicura attraverso una rete altrimenti non sicura*, in genere Internet. Le VPN sono generalmente più economiche delle reti private reali che utilizzano linee private, ma si basano sullo stesso sistema di crittografia e autenticazione su entrambe le estremità. La crittografia può essere eseguita da un software firewall o eventualmente da router. Il meccanismo di protocollo più comune utilizzato a questo scopo è a livello IP: IPsec. La Figura 12.3 è uno scenario tipico di utilizzo di IPsec:

Un'organizzazione mantiene le LAN in località disperse. Su ciascuna LAN viene condotto traffico IP non protetto. Per il traffico fuori sede, attraverso una sorta di WAN privata o pubblica, vengono utilizzati i protocolli **IPSec**. Questi protocolli operano in dispositivi di rete, come router o firewall, che collegano ciascuna LAN al mondo esterno. Il dispositivo di rete IPSec in genere cripta e comprime tutto il traffico in ingresso nella WAN e decripta e decomprime il traffico proveniente dalla WAN; può essere fornita anche l'autenticazione. Queste operazioni sono trasparenti per le workstation e i server sulla LAN. La trasmissione sicura è possibile anche con singoli utenti che si collegano alla WAN. Tali workstation utente devono implementare i protocolli IPSec per garantire la sicurezza. Devono inoltre implementare elevati livelli di sicurezza dell'host, poiché sono direttamente connessi alla rete Internet più ampia. Ciò li rende un bersaglio attraente per gli aggressori che tentano di accedere alla rete aziendale.

Un mezzo logico per implementare un IPSec è in un **firewall**, come mostrato nella Figura 12.3. *Se IPSec è implementato in un box separato dietro (interno) al firewall, il traffico VPN che passa attraverso il firewall in entrambe le direzioni viene crittografato.* In questo caso, *il firewall non è in grado di eseguire la sua funzione di filtro o altre funzioni di sicurezza*, come il controllo degli accessi, la registrazione o la scansione antivirus. *IPSec potrebbe essere implementato nel router di confine, all'esterno del firewall.* Tuttavia, è probabile che questo dispositivo sia meno sicuro del firewall e quindi meno desiderabile come piattaforma IPSec.

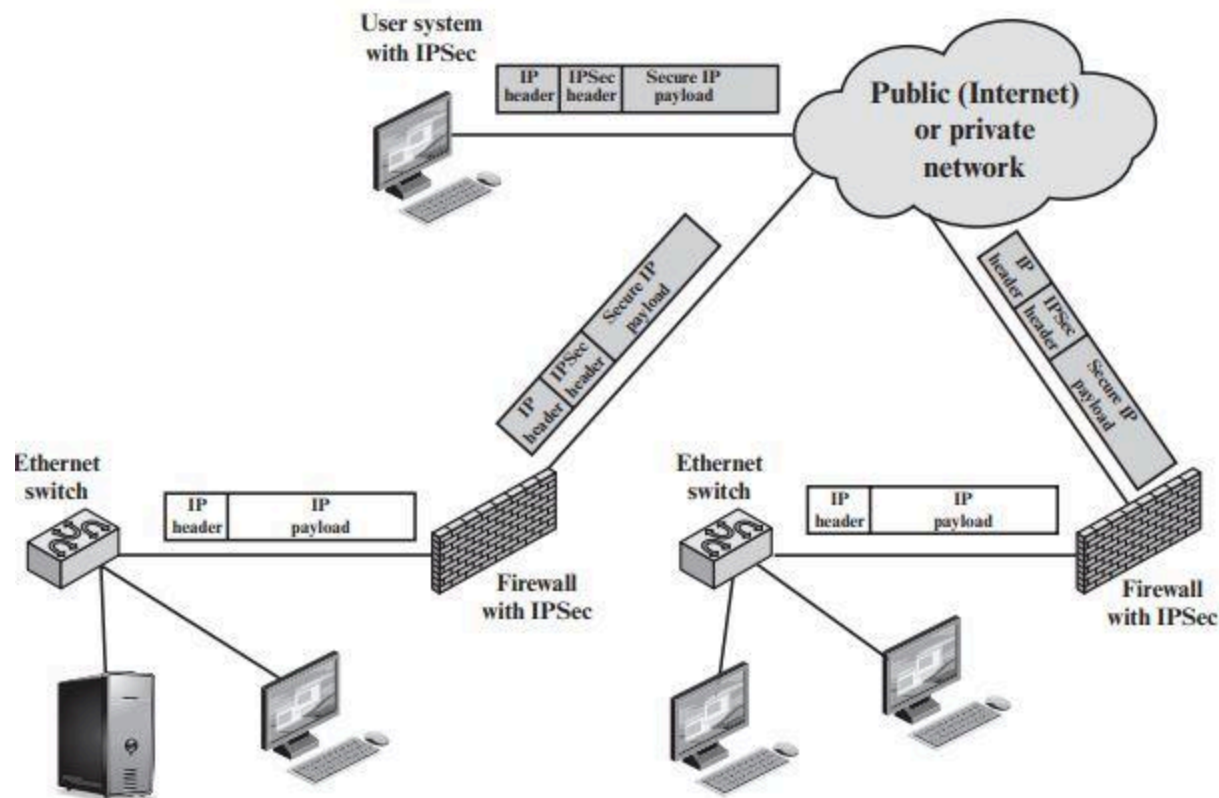


Figure 12.3 A VPN Security Scenario

Distributed Firewalls

Una configurazione firewall distribuita prevede dispositivi **firewall autonomi** più **host-based firewall** che lavorano insieme sotto un controllo amministrativo centrale. Gli amministratori possono configurare firewall residenti sull'host su centinaia di server e workstation, nonché configurare firewall personali su sistemi di utenti locali e remoti. Gli strumenti consentono all'amministratore di rete di impostare policy e monitorare la sicurezza dell'intera rete. Questi firewall proteggono dagli attacchi interni e forniscono una protezione su misura per macchine e applicazioni specifiche. I **firewall autonomi** forniscono protezione globale, inclusi firewall interni e un firewall esterno, come discusso in precedenza. Con i firewall distribuiti, può avere senso creare una DMZ sia interna che esterna. I server Web che necessitano di minore protezione perché contengono informazioni meno critiche potrebbero essere collocati in una DMZ esterna, al di fuori del firewall esterno. La protezione necessaria è fornita dai firewall basati su host su questi server. Un aspetto importante di una configurazione firewall distribuita è il monitoraggio della sicurezza. Tale monitoraggio include in genere l'aggregazione e l'analisi dei registri, le statistiche del firewall e il monitoraggio remoto capillare dei singoli host, se necessario.

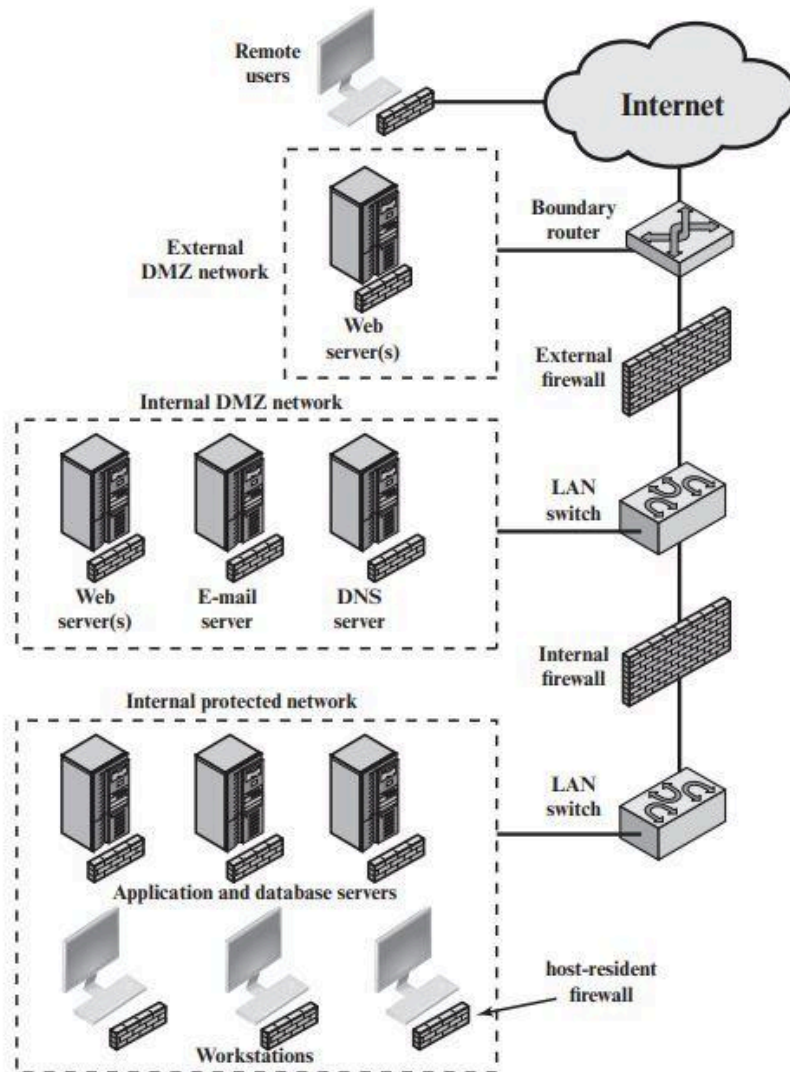
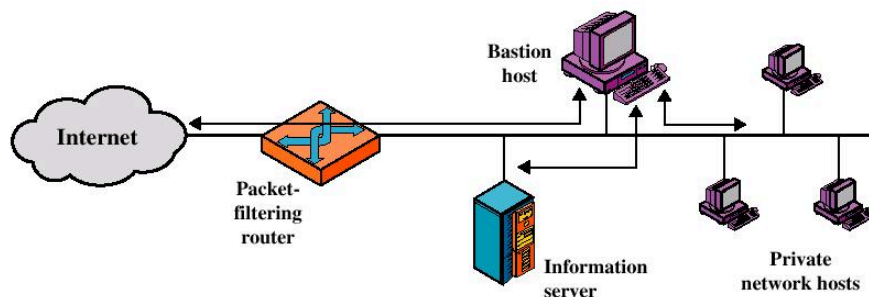


Figure 12.4 Example Distributed Firewall Configuration

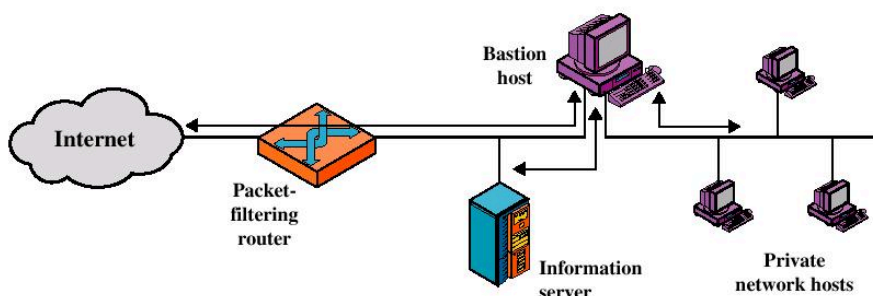
Riepilogo ubicazioni e tipologie di Firewall

Si individuano le seguenti alternative:

1. **Screened Host Firewall System (single-homed bastion host):** è un'architettura di sicurezza di rete che utilizza un bastion host come punto principale di difesa per proteggere una rete interna da minacce esterne. Si compone di:
 - **Bastion Host:** il bastion host in questa architettura è un server con una sola interfaccia di rete (*single-homed*) che si trova tra la rete interna e la rete esterna, di solito Internet; funziona come un ponte tra le due reti.
 - **Packet Filtering Router:** Il router (o firewall) di filtering è posizionato tra il bastion host e la rete esterna. Il suo ruolo principale è filtrare e controllare il traffico in ingresso e in uscita; accetta solo pacchetti **da** e **per** il bastion host.



2. **Screened Host Firewall System (dual-homed bastion host):** è un'architettura di sicurezza di rete che utilizza un bastion host come punto principale di difesa per proteggere una rete interna da minacce esterne. Si compone di: un **Bastion Host** e un **Packet Filtering Router**, si differenzia dal *single-homed* perchè il bastion host ha due interfacce di rete (una verso la rete interna e una verso la rete esterna); e il traffico tra Internet e gli hosts sulla rete privata deve fisicamente passare attraverso il bastion host.



3. **Screened-subnet Firewall System:** è un'architettura di sicurezza di rete che coinvolge l'uso di una subnet schermata o **DMZ** tra il firewall principale e la rete esterna. Questo tipo di configurazione è comunemente utilizzato per proteggere i servizi pubblici o esposti all'esterno, da potenziali minacce; mentre si preserva la sicurezza della rete interna. Ecco i componenti:
- **Firewall Esterno:** Il firewall esterno è il primo livello di protezione tra la rete interna e la rete esterna. Questo firewall controlla e filtra il traffico tra le due reti, permettendo solo il traffico autorizzato a passare attraverso.
 - **Subnet Schermata (DMZ):** la subnet schermata è una zona intermedia tra il firewall esterno e la rete interna. In questa subnet vengono ospitati i servizi pubblici o i server accessibili dall'esterno. La subnet schermata è configurata in modo che sia più sicura rispetto alla rete esterna, ma meno protetta rispetto alla rete interna.
 - **Firewall Principale (Interno):** il firewall principale (interno) è posizionato tra la subnet schermata e la rete interna. Questo firewall regola il traffico proveniente dalla subnet schermata e diretto verso la rete interna e

viceversa, in genere applica controlli molto rigidi poiché ultimo baluardo di sicurezza prima della sicurezza dei singoli host.

